Data Structures and Algorithms in Python, Hands-On

Data Structures and Algorithms in Python, Hands-On

Every Structure Built, Tested and Timed, with Interview Patterns, Exercises and Solutions

Aadhavan Muthurengan

Data Structures and Algorithms in Python, Hands-On
Every Structure Built, Tested and Timed, with Interview Patterns, Exercises and Solutions

Copyright © 2026 Aadhavan Muthurengan. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and certain other noncommercial uses permitted by copyright law. Readers may freely use and adapt the example programs, tests, benchmarks, exercises and solutions in this book for their own work. All the code is available at github.com/aadhavanshines/data-structures-and-algorithms-in-python-hands-on under the MIT License.

Disclaimer. This book is provided for educational and informational purposes only. AI products, features, and policies change frequently; verify current details with each provider. The author makes no guarantees about results and is not liable for any losses arising from the use of this information. Nothing in this book is legal, medical, financial, or other professional advice.

Trademarks. Python and the Python logos are trademarks or registered trademarks of the Python Software Foundation. Java is a registered trademark of Oracle and/or its affiliates. JavaScript is a trademark of Oracle Corporation in the United States. Chrome and Go are trademarks of Google LLC. Linux is a registered trademark of Linus Torvalds. Git is a trademark of the Software Freedom Conservancy. Redis is a registered trademark of Redis Ltd. PostgreSQL is a trademark of the PostgreSQL Community Association of Canada. Windows is a trademark of Microsoft Corporation. macOS is a trademark of Apple Inc. GitHub is a trademark of GitHub, Inc. Ubuntu is a registered trademark of Canonical Ltd. Intel and Xeon are trademarks of Intel Corporation. SQLite is in the public domain; pytest, matplotlib, uv and V8 are projects whose names belong to their owners. All people, towns, companies and data in the example programs are fictional. Product and company names are

used solely for identification. This book is independent and is not affiliated with, sponsored by, or endorsed by the Python Software Foundation or any of these companies or projects.

Examples. All people, towns, companies and data in the example programs are fictional. Any resemblance to real persons or businesses is coincidental.

First edition, 2026.

To my father and my mother,
for your love, your sacrifices, and every lesson you taught me.

To my wife,
for your patience, your encouragement, and your belief in me.

And to our three children,
may you always build the things you dream of.

This book is for you.

Contents

Introduction 11

Part I: Foundations 15

1. Why Build It Yourself 17
2. Setting Up 23
3. Measuring Speed: Big O You Can See 29
4. Python's Built-ins Under the Hood 43
5. Recursion and the Call Stack 53

Part II: Linear Structures 61

6. A Dynamic Array 63
7. Linked Lists 73
8. Stacks and Queues 87
9. Hash Tables 97

Part III: Searching and Sorting 109

10. Binary Search 111
11. Simple Sorts 121
12. Merge Sort and Quicksort 129
13. Heaps and Priority Queues 141
14. Sorting Without Comparing 151

Part IV: Trees **159**

15. Binary Trees 161
16. Binary Search Trees 169
17. Balanced Trees: AVL 179
18. Tries 189
19. Union-Find 195

Part V: Graphs **203**

20. Graphs 205
21. Breadth-First and Depth-First Search 213
22. Shortest Paths 223
23. Minimum Spanning Trees 231

Part VI: Interview Patterns **237**

24. Solving a Problem in an Interview 239
25. Two Pointers 247
26. Sliding Window 255
27. Hash Maps and Prefix Sums 263
28. BFS and DFS on Grids and Implicit Graphs 271
29. Dynamic Programming I: Memoization and Tables 281
30. Dynamic Programming II: Sequences, Strings and Knapsack 291
31. Greedy Algorithms and Backtracking 301

Part VII: Putting It Together **311**

32. Project: An LRU Cache 313
33. Is Python Getting Faster? 323
34. Next Steps 331

Appendix A: Exercise Solutions 335

Appendix B: Complexity Cheat Sheet 379

Appendix C: How the Benchmarks Were Run 385

Appendix D: Glossary 389

Appendix E: Sources 395

About the Author 403

More Books by the Author 405

Introduction

Every program you have ever used stands on a handful of ideas. Your phone's contacts app finds a name among thousands before you finish typing it. A map app finds the fastest route across a country in a fraction of a second. A database answers a question about millions of rows without reading them all. Python itself finds a key in a dictionary in the same time whether the dictionary holds ten items or ten million. None of this is magic. It is **data structures**, ways of arranging data, and **algorithms**, step-by-step methods for working on it, chosen well.

This book teaches both, in Python, by building them. You'll write a dynamic array, linked lists, a hash table, binary search trees and a self-balancing AVL tree, a heap, a trie, union-find, graphs and their searches, shortest paths and spanning trees, and an LRU cache. Then you'll learn the patterns behind most coding-interview problems: two pointers, sliding windows, hash maps and prefix sums, breadth-first and depth-first search, dynamic programming, greedy choices and backtracking.

What Makes This Book Different

You build every structure yourself. Not a diagram and a promise, but a working class, a few dozen lines long, that you can read, run and change. Building a hash table is the fastest way to understand why a Python dictionary is fast, and when it isn't.

Everything is tested. Each structure comes with a test file, and the tests do more than check a few examples: they run thousands of random operations against Python's own built-in structures and check that the answers agree.

You'll learn to test your own code this way, and it will catch bugs you would never think to look for. While this book was being written, it caught several, and you'll see them.

Everything is timed. Big O notation tells you how an algorithm's running time *grows*; this book also measures it. Every chapter has real benchmarks, with tables and charts, comparing your structure with Python's built-in one, a good algorithm with a naive one, and the best case with the worst. You'll see a quadratic algorithm's time multiply by four each time the input doubles, and a dictionary lookup that takes about the same tenth of a microsecond at every size. Numbers you have measured yourself are far more convincing than any formula, and they show something formulas hide: in Python, a simple algorithm running in fast built-in code often beats a cleverer one written in Python.

Every example is real, tested code. Each listing in this book is a file that was run on Python 3.12, 3.13, 3.14 and 3.15, and the output printed beneath it is the output it produced. Timings are the exception, because they differ on every computer and every run; they were recorded on one computer, described in Appendix C, and you should expect your numbers to differ while the patterns stay the same.

Interview patterns, with proof. Part VI covers the patterns behind most coding-interview questions. Each solution is checked against a slow, obviously correct brute-force version on thousands of random inputs, and timed against it.

Real-world uses. Boxes in every chapter show where each structure works in software you use: in Python itself, in databases, operating systems, Git, web browsers and the internet's routers. Each was checked against the project's source code or official documentation, and the sources are listed in Appendix E.

Who This Book Is For

You should know basic Python: variables, `if` and `for`, functions, lists and dictionaries, and simple classes. You don't need any math beyond school algebra; when logarithms appear, Chapter 3 explains what you need. The book suits:

- Self-taught programmers and bootcamp graduates who want the computer-science foundations
- Students taking a data structures course who want to see the ideas run
- Developers preparing for coding interviews
- Anyone who has used Python's lists and dictionaries for years and wants to know what happens inside them

How This Book Is Organized

Part I, Foundations, sets up your tools and teaches you to measure: Big O notation, how to time code properly, what Python's built-in structures cost, and recursion.

Part II, Linear Structures, builds a dynamic array, linked lists, stacks, queues and hash tables.

Part III, Searching and Sorting, covers binary search, the classic sorting algorithms, heaps, and sorts that don't compare.

Part IV, Trees, builds binary trees, binary search trees, a self-balancing AVL tree, tries and union-find.

Part V, Graphs, covers representations, breadth-first and depth-first search, shortest paths and minimum spanning trees.

Part VI, Interview Patterns, opens with a method for solving problems under pressure, then teaches one pattern per chapter, with practice problems.

Part VII, Putting It Together, builds an LRU cache from a hash map and a linked list, and runs the book's benchmarks on four versions of Python to see how much faster Python has become.

Every chapter ends with key takeaways and exercises. Appendix A has a tested solution for every exercise. Appendix B is a cheat sheet of costs, with the measurements from this book.

How to Use This Book

Type the code. Run it. Then change it and run it again: shrink a test until it fails, swap a data structure, double an input and predict the time before you measure it. Data structures are learned with the hands as much as the eyes.

All the code is free to download, arranged by chapter, from the book's repository at **github.com/aadhavanshines/data-structures-and-algorithms-in-python-hands-on**. Chapter 2 shows how to set it up.

> **Note:** All the data in the examples, including people, towns and companies, is fictional. Any resemblance to real persons or businesses is coincidental.

PART I

Foundations

CHAPTER 1 · BEGINNER

Why Build It Yourself

Python gives you excellent data structures for free. Lists, dictionaries, sets, tuples and strings cover most everyday needs, and the standard library adds `collections.deque`, `heapq` and `bisect`. So why build your own?

Because using a tool well means knowing what it costs. Two programs can give exactly the same answer while one finishes instantly and the other runs for an hour, and the difference is almost always the choice of data structure or algorithm. The only way to really know why one is fast and the other slow is to look inside, and the best way to look inside is to build it.

This chapter shows the method the whole book follows, using one small question.

One Question, Two Answers

Here is the question: is a value in a collection? In Python you write `x in collection`, and it works for lists and sets alike. The answer is the same. Is the time?

```
"""How long does `x in collection` take, for a list and for a set?"""

import bench


def make_list(n):
    return list(range(n)), -1  # -1 is never there: the worst case

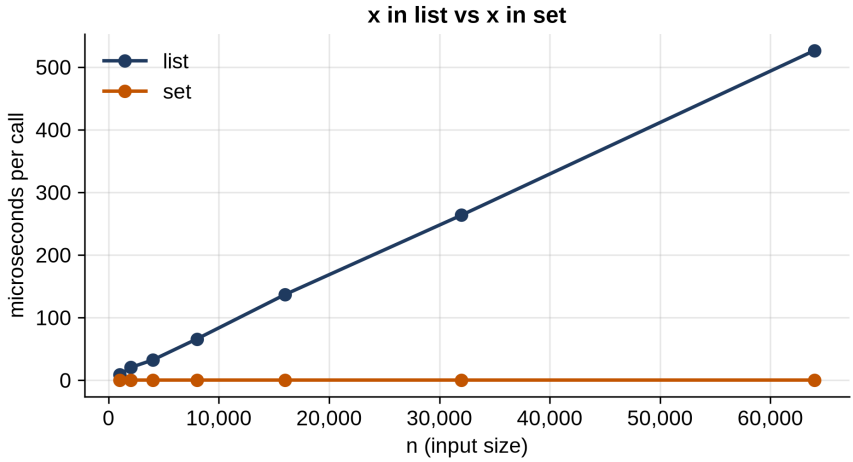

def make_set(n):
    return set(range(n)), -1
```

```
def contains(collection, x):
    return x in collection


sizes = [1_000, 2_000, 4_000, 8_000, 16_000, 32_000, 64_000]
results = {
    "list": bench.doubling(contains, make_list, sizes),
    "set": bench.doubling(contains, make_set, sizes),
}
bench.table(results)
bench.chart(results, "ch01_list_vs_set.png", "x in list vs x in set")
```

Don't worry about the `bench` module yet; Chapter 3 builds it line by line. For now, `bench.doubling` times a function for each input size in `sizes`, `bench.table` prints the results, and `bench.chart` draws them. The value searched for, `-1`, is never in the collection, so every search has to give its final answer, "no", the hard way. Here are the results:

```
         n            list             set
     1,000         8.61 us         85.7 ns
     2,000    20.2 us x2.3    86.9 ns x1.0
     4,000    32.1 us x1.6    85.8 ns x1.0
     8,000    65.7 us x2.0    85.6 ns x1.0
    16,000     137 us x2.1    87.5 ns x1.0
    32,000     263 us x1.9    87.1 ns x1.0
    64,000     526 us x2.0    87.4 ns x1.0
```



Searching a list takes longer as it grows; searching a set doesn't.

Read the table row by row. Each size is twice the one before, and the "x" number after each time says how many times longer that size took than the previous one. For the list, it's about 2: double the items, double the time. The list checks its items one by one until it finds the value or runs out, so a list with twice as many items takes twice as long to say "no".

For the set, the time hardly changes at all. A set with 64,000 items answers as fast as one with 1,000, in well under a microsecond. It doesn't look through the items; it computes where the value would be, and looks there. Chapter 9 builds a hash table, the structure behind sets and dictionaries, so you'll see exactly how.

At 64,000 items the set is thousands of times faster. At a million items, the gap would be bigger still. A program that checks membership in a loop, say, removing duplicates from a list of a million emails by checking each against what it has seen, would take hours with a list and well under a second with a set.

> **Tip:** When you write `if x in something` inside a loop, stop and check what `something` is. If it's a list that grows with the input, a set will almost always be dramatically faster.

Build, Test, Time

The list-versus-set comparison follows the three steps that every chapter of this book follows:

1. **Build.** Write the structure or algorithm yourself, in plain Python, short enough to read in one sitting. You'll understand it better than any description could make you, and you'll know exactly what each operation does.
2. **Test.** Check that it works, with a few hand-picked cases and then with thousands of random operations compared against something you trust, usually Python's built-in equivalent. If your hash table and a dictionary give the same answers after twenty thousand random insertions, lookups and deletions, it works.

3. **Time.** Measure how its running time grows as the input grows, and compare it with alternatives. Theory predicts the shape of the curve; a benchmark shows the real numbers, including the constant factors that theory leaves out.

Theory comes in too, of course. Chapter 3 introduces **Big O notation**, the standard way to describe how running time grows. But in this book every Big O claim is followed by a measurement, so you never have to take it on faith.

The Same Problem, Many Solutions

The first lesson of data structures is that the same task can be done in very different ways, and the right one depends on what you'll do most. A list keeps items in order and gives instant access by position, but finding a value means searching. A set finds values instantly, but keeps no order and can't hold duplicates. Neither is better; each is right for some jobs.

Throughout the book you'll keep asking the same three questions about every structure:

- **What operations does it make fast?** Adding at the end, finding by key, getting the smallest item, finding by prefix.
- **What does it make slow, or impossible?** And does your program need those operations?
- **What does it cost in memory?** Speed is often bought with space.

> **Real-World Use:** Python answers "is this name defined?" constantly while your program runs. A module's global variables live in a dictionary, the one `globals()` returns, and every object's attributes usually do too (`vars(obj)` shows it). So each time your code uses a global name or an attribute, Python does a hash-table lookup like the set lookup in this chapter's benchmark. If those lookups scanned a list, every Python program would slow down as it grew.

Python Is Not the Whole Story

One more thing the benchmarks will show again and again: in Python, *where* code runs matters as much as what it does. Python's built-in structures are written in C and compiled to machine code. Code you write in Python runs in the interpreter, typically ten to a hundred times slower per step. So your hand-built dynamic array will be slower than `list`, and your hash table slower than `dict`, even though they do the same work.

That's fine. You're not building these to replace the built-ins; you're building them to understand the built-ins, and to have the techniques ready for the problems the built-ins don't solve. And sometimes, as you'll see with linked lists, heaps and tries, the structure you build does something no built-in does.

Key Takeaways

- The same question can take microseconds or hours, depending on the data structure that answers it.
- Checking `x in list` takes time proportional to the list's length; `x in set` takes about the same time at any size.
- Every chapter follows the same method: build it, test it against something you trust, and time it.
- Ask three questions of every structure: what it makes fast, what it makes slow, and what it costs in memory.
- Python's built-ins run in compiled C, so they usually beat hand-written Python doing the same work. Build structures to understand them, then choose wisely.

Exercises

This chapter has no programming exercises; Chapter 2 sets up your tools. Instead, before you turn the page, write down your guess: how long would `x in a_list` take for a list of 10 million items on your computer? How about `x in a_set`? You'll be able to check both by the end of Chapter 3.

CHAPTER 2 · BEGINNER

Setting Up

This chapter gets your computer ready: Python, a virtual environment, the two packages the book uses, and the book's code. Then you'll write your first function and test it the way every structure in this book is tested.

Python 3.12 or Newer

The code in this book was run on Python 3.12, 3.13, 3.14 and 3.15, and needs at least 3.12. Check what you have in a terminal (Command Prompt or PowerShell on Windows):

```
$ python3 --version
```

On Windows, the command is usually `python` or `py` instead of `python3`; use whichever works throughout the book. If you have no Python, or one older than 3.12, install the latest from python.org. On a Mac, the installer from python.org is the simplest choice; on Linux, your distribution's package manager or python.org both work.

The Book's Code

All the code is in the book's repository, **github.com/aadhavanshines/data-structures-and-algorithms-in-python-hands-on**. Download it as a ZIP file from the green "Code" button, or, if you use Git:

```
$ git clone https://github.com/aadhavanshines/data-structures-and
  -algorithms-in-python-hands-on
$ cd data-structures-and-algorithms-in-python-hands-on
```

Inside there's one folder per chapter, `ch01` to `ch33`, each holding that chapter's listings. Every listing has a `.out` file next to it with the output printed in the book, so you can compare.

A Virtual Environment

A **virtual environment** is a private folder of Python packages for one project, so this book's packages can't clash with anything else on your computer. Create one in the code folder and activate it:

```
$ python3 -m venv .venv
$ source .venv/bin/activate
```

On Windows, the second command is `.venv\Scripts\activate`. Your prompt now starts with `(.venv)`. Activate it again whenever you open a new terminal to work on the book.

Now install the two packages the book uses: **pytest**, for testing, and **matplotlib**, for drawing benchmark charts. The data structures themselves use only Python's standard library.

```
$ pip install -r requirements.txt
```

`requirements.txt` pins the exact versions the book was tested with. Check that everything works:

```
"""Check that this computer is ready for the book's code."""

import sys

if sys.version_info < (3, 12):
    sys.exit("This book needs Python 3.12 or newer.")
```

```
print("Python 3.12 or newer: OK")

for package in ("pytest", "matplotlib"):
    try:
        __import__(package)
    except ImportError:
        sys.exit(f"Missing {package}: run  pip install -r requirements.txt")
    print(f"{package}: OK")
print("Everything is ready.")
```

```
Python 3.12 or newer: OK
pytest: OK
matplotlib: OK
Everything is ready.
```

`sys.version_info` is a tuple like `(3, 13, 5, ...)`, and tuples compare item by item, so `< (3, 12)` is true only for versions older than 3.12. `__import__` imports a module whose name is in a string, so one loop can check both packages.

Your First Algorithm

Here is a small algorithm: find the largest number in a list, without using Python's `max`.

```
"""Our first algorithm: find the largest number in a list."""


def largest(numbers):
    """Return the largest item. Raise ValueError if there are none."""
    if not numbers:
        raise ValueError("largest() of an empty list")
    best = numbers[0]
    for x in numbers[1:]:
        if x > best:
            best = x
    return best
```

Keep the best value seen so far, and look at each remaining item once, replacing the best when an item beats it. For a list of n items, it makes n - 1 comparisons. An empty list has no largest item, so the function says so with an exception rather than returning something misleading, as `max([])` does too.

Your First Tests

How do you know it works? You could run it on a few lists and look. Tests do the same thing, automatically, every time you change the code. pytest finds every function whose name starts with `test_` in files whose names start with `test_`, runs them, and reports any `assert` that fails:

```
"""Tests for largest(): a few chosen cases, then many random ones."""

import random

import pytest

from largest import largest


def test_one_item():
    assert largest([7]) == 7


def test_largest_first_middle_and_last():
    assert largest([9, 1, 2]) == 9
    assert largest([1, 9, 2]) == 9
    assert largest([1, 2, 9]) == 9


def test_negative_numbers_and_ties():
    assert largest([-5, -2, -9]) == -2
    assert largest([4, 4, 4]) == 4


def test_empty_list_is_an_error():
    with pytest.raises(ValueError):
        largest([])


def test_agrees_with_max_on_random_lists():
    rng = random.Random(2026)  # a fixed seed: the same lists every run
    for _ in range(1000):
        numbers = [
            rng.randint(-100, 100) for _ in range(rng.randint(1, 50))
        ]
        assert largest(numbers) == max(numbers)
```

The first four tests check chosen cases: one item, the largest at each position, negative numbers and ties, and the empty list, where `pytest.raises` checks that the right exception happens. Chosen cases are good at the edges that you know about.

The last test is different, and it's the most powerful kind in this book. It makes a thousand random lists, of random lengths between 1 and 50, and checks that `largest` agrees with Python's own `max` on every one. You don't have to think

of the tricky cases; random inputs find them for you. The `random.Random(2026)` generator is **seeded**: it produces the same "random" lists every time, so if a test fails, it fails the same way again and you can investigate.

Run the tests from the chapter's folder:

```
$ cd ch02
$ python -m pytest -q test_largest.py
```

```
.....                                                                    [100%]
5 passed in 0.01s
```

Each dot is one passing test. A failure prints an `F` instead, followed by a report that shows the failing `assert` with the values involved. Try it: change `if x > best` to `if x >= best` and run the tests again. They still pass, because the result is the same. Now change it to `if x < best`: every test that has more than one item fails.

> **Tip:** When you write a new data structure, write the random comparison test first. It takes five minutes, and it checks thousands of cases you'd never think to write by hand. Every structure in this book has one.

> **Interesting Fact:** Comparing a fast implementation against a slow, obviously correct one on random inputs has many names: differential testing, stress testing, or, when the inputs are generated from rules, property-based testing. Part VI uses it to check every interview solution, and Chapter 24 shows it catching a bug that looks perfectly fine.

Running the Book's Listings

Every listing runs from its chapter's folder, like this:

```
$ cd ch06
$ python grow_demo.py
```

Benchmarks also run this way, but take longer: up to a minute or two each, because they repeat every measurement several times. Their output won't

match the book's exactly, because your computer isn't the one the book's numbers came from. What should match is the pattern: which option is faster, and how each one's time grows.

Key Takeaways

- The book needs Python 3.12 or newer, a virtual environment, and two packages: pytest and matplotlib.
- Every listing is in the book's repository, by chapter, with its expected output.
- pytest runs every `test_` function and reports failed `asserts`.
- Test with chosen cases for the edges you know, and with random cases compared against a trusted implementation for the ones you don't.
- Seed random tests, so a failure can be repeated.

Exercises

1. **Smallest and largest.** Write `min_and_max(numbers)`, which returns both the smallest and the largest item in one pass. Then write a random test that compares it with Python's `min` and `max`. Can you do it in about 1.5 comparisons per item instead of 2? (Hint: look at the items in pairs.)
2. **A bug the tests catch.** Change `largest` so it starts from `best = 0` instead of `numbers[0]`. Which tests fail? If you deleted the test with negative numbers, would the random test still catch the bug, and why?

CHAPTER 3 · BEGINNER

Measuring Speed: Big O You Can See

How fast is a piece of code? "0.3 seconds" is an answer, but a fragile one: it depends on the computer, on what else is running, and above all on the input. A better question is: **how does the time grow as the input grows?** An algorithm whose time doubles when its input doubles will handle a million items happily; one whose time quadruples will grind to a halt somewhere around a hundred thousand. This chapter teaches the language for that question, Big O notation, and builds the small timing library, `bench.py`, that measures it throughout the book.

Counting Steps

Before timing anything, count. Here are two functions, instrumented to count their basic steps: a linear search that compares the target with each item, and a duplicate check that compares every pair of items.

```
"""Count the basic steps two functions take, as n grows."""


def find(items, target):
    """Linear search. Returns (index or -1, comparisons made)."""
    steps = 0
    for i, x in enumerate(items):
        steps += 1
        if x == target:
            return i, steps
    return -1, steps
```

```
def has_duplicate(items):
    """Compare every pair. Returns (answer, comparisons made)."""
    steps = 0
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            steps += 1
            if items[i] == items[j]:
                return True, steps
    return False, steps


print(f"{'n':>6} {'find: steps':>12} {'duplicates: steps':>18}")
for n in [10, 20, 40, 80, 160]:
    items = list(range(n))
    _, find_steps = find(items, -1)  # not there: the worst case
    _, pair_steps = has_duplicate(items)  # no duplicates: the worst case
    print(f"{n:>6} {find_steps:>12,} {pair_steps:>18,}")
```

```
     n  find: steps  duplicates: steps
    10           10                  45
    20           20                 190
    40           40                 780
    80           80               3,160
   160          160              12,720
```

Each row doubles n. The search's steps double too: it makes exactly n comparisons when the target is missing. The duplicate check's steps roughly quadruple: it compares every pair, and n items have n(n - 1)/2 pairs. For 160 items that's 12,720 comparisons; for 160,000 items it would be about 12.8 billion.

Notice that the counts don't depend on the computer at all. That's what makes step counts useful: they describe the algorithm, not the machine.

Big O Notation

Big O notation summarizes how the number of steps grows with the input size n, keeping only the part that matters for large n:

- Drop constant factors: 3n steps and n steps are both **O(n)**, because doubling n doubles both.
- Drop smaller terms: n(n - 1)/2 = $n^2/2 - n/2$ is **O(n^2)**, because for large n the n^2 part dwarfs the rest.

So linear search is O(n) and the all-pairs check is O(n^2). You read O(n^2) as "big O of n squared", or "order n squared", and it means "grows no faster than n^2 times some constant".

These are the growth rates you'll meet in this book, from fastest to slowest:

Notation	Name	Example in this book
O(1)	constant	reading `items[i]`; a dict lookup
O(log n)	logarithmic	binary search (Chapter 10)
O(n)	linear	searching a list; summing it
O(n log n)	"n log n"	merge sort; Python's `sorted()`
O(n^2)	quadratic	insertion sort; checking every pair
O(2^n)	exponential	trying every subset (Chapter 31)

To see how differently they grow:

```
"""How the common growth rates compare as n grows."""

import math

print(
    f"{'n':>9} {'log2 n':>6} {'n log2 n':>10} {'n squared':>17} "
    f"{'2 to the n':>14}"
)
for n in [10, 100, 1_000, 10_000, 100_000, 1_000_000]:
    if n <= 30:
        two_to_n = f"{2**n:,}"
    else:  # too long to print: count its digits instead
        two_to_n = f"{math.floor(n * math.log10(2)) + 1:,} digits"
    print(
        f"{n:>9,} {math.log2(n):>6.1f} {n * math.log2(n):>10,.0f} "
        f"{n * n:>17,} {two_to_n:>14}"
    )
```

```
        n log2 n   n log2 n         n squared     2 to the n
       10    3.3         33               100          1,024
      100    6.6        664            10,000      31 digits
    1,000   10.0      9,966         1,000,000     302 digits
   10,000   13.3    132,877       100,000,000   3,011 digits
  100,000   16.6  1,660,964    10,000,000,000  30,103 digits
1,000,000   19.9 19,931,569 1,000,000,000,000 301,030 digits
```

Logarithms in One Paragraph

$\log_2$ n (log base 2 of n) is the number of times you can halve n before reaching 1. Halve 1,000 ten times and you're at about 1, so $\log_2$ 1,000 ≈ 10. Halve a million twenty times; a billion, thirty times. Every time n doubles, $\log_2$ n goes up by just one. That's why an O(log n) algorithm, which halves its problem at each step, is almost as good as constant time: searching a billion sorted items takes about thirty steps. That is all the math the book needs about logarithms. (The base doesn't matter in Big O, because changing base only multiplies by a constant, so people just write O(log n).)

Timing Code: Harder Than It Looks

Counting steps is exact but tedious, and it hides the fact that some steps are far more expensive than others. So we also time code. The obvious way is a stopwatch:

```
"""Timing one call with a stopwatch: the numbers jump around."""

import time

numbers = list(range(100_000))
for attempt in range(5):
    start = time.perf_counter()
    sum(numbers)
    elapsed = time.perf_counter() - start
    print(f"attempt {attempt + 1}: {elapsed * 1e6:.0f} microseconds")
```

```
attempt 1: 760 microseconds
attempt 2: 605 microseconds
attempt 3: 538 microseconds
attempt 4: 485 microseconds
attempt 5: 442 microseconds
```

The same work, five times, five different answers. The first run is often slowest, while the processor's caches fill and its clock speeds up; after that, other programs, the operating system, and Python's garbage collector all steal a little time now and then. A single measurement of a fast operation tells you very little.

Good benchmarks fix this with three habits:

1. **Repeat until the measurement is long enough.** A single call of something that takes 50 nanoseconds can't be timed accurately; a million calls in a loop can.
2. **Take the fastest of several rounds.** Interruptions only ever add time. The fastest round is the one least disturbed, so it's the best estimate of the code's own speed. (This is also what Python's `timeit` module recommends.)
3. **Pause the garbage collector** while timing, so its occasional sweeps don't land in one measurement and not another.

bench.py: A Small Timing Library

Every benchmark in this book uses the same small module, `bench.py`. A copy sits in every chapter folder of the book's code, so each chapter runs on its own. It starts like this:

```
"""Timing helpers used by every benchmark in this book (Chapter 3).

measure()  times one operation: the fastest of several rounds
doubling() times an operation for a series of input sizes
table()    prints the results, with the ratio between neighboring sizes
chart()    draws the results as a line chart and saves it as a PNG file
"""

import gc
import os
import time

# BENCH_QUICK=1 makes a quick, rough run: the book's checks use it to
# confirm that every benchmark works, without waiting for exact timings
QUICK = os.environ.get("BENCH_QUICK") == "1"
```

`QUICK` exists for the book's own automated checks, which run every benchmark on four Python versions just to confirm it works; you can ignore it. Here is the heart of the module:

```
def measure(run, setup=None, *, rounds=5, min_time=0.02):
    """Seconds for one call of run(), the fastest of `rounds` rounds.

    Each round repeats the call until it has taken at least `min_time`
    seconds, so even very fast operations are timed accurately. If the
    operation changes its input (sorting, say), give setup(): it makes
    fresh input for every call, run(*setup()), and is not timed.
    """
```

```
    if QUICK:
        rounds, min_time = 1, 1e-9
    best = float("inf")
    for _ in range(rounds):
        gc.disable()  # garbage collection would add random pauses
        try:
            if setup is None:
                seconds = _batch(run, min_time)
            else:
                seconds = _fresh(run, setup, min_time)
        finally:
            gc.enable()
        best = min(best, seconds)
    return best


def _batch(run, min_time):
    """Time a loop of calls, doubling its length until it is long enough."""
    number = 1
    while True:
        start = time.perf_counter()
        for _ in range(number):
            run()
        total = time.perf_counter() - start
        if total >= min_time:
            return total / number
        number *= 2


def _fresh(run, setup, min_time):
    """Time one call at a time, each with fresh input from setup()."""
    calls, total = 0, 0.0
    while total < min_time:
        args = setup()
        start = time.perf_counter()
        run(*args)
        total += time.perf_counter() - start
        calls += 1
    return total / calls
```

`measure` takes a function to time, `run`, and returns the seconds per call, the fastest of five rounds. Each round runs `_batch` or `_fresh`:

- `_batch` calls `run` in a loop of `number` calls, doubling `number` until the loop takes at least `min_time` seconds (0.02 by default). Dividing the total by `number` gives the time per call. This is how the standard library's `timeit` decides how many times to run something.
- `_fresh` is for operations that change their input. Sorting a list sorts it, so a second sort would get an already-sorted list, a completely different (and easier) job. With `setup`, every call gets fresh input from `setup()`, and only `run` itself is inside the timer.

`time.perf_counter()` is Python's most precise clock for measuring short intervals. `gc.disable()` pauses the garbage collector, and the `try`/`finally` makes sure it's switched back on even if the timed code raises an error.

Most benchmarks time an operation at several input sizes:

```
def doubling(run, make_input, sizes, fresh=False, rounds=5, **options):
    """[(n, seconds), ...]: run(*make_input(n)) timed for each n.

    The input is made once per size and reused, unless fresh=True: then
    every call gets new input, for operations that change it. The rounds
    are interleaved: each round times every size once, and each size
    keeps its fastest round, so a burst of background activity on the
    computer can't spoil all the rounds of one size.
    """
    if QUICK:
        rounds = 1
    inputs = {} if fresh else {n: make_input(n) for n in sizes}
    best = dict.fromkeys(sizes, float("inf"))
    for _ in range(rounds):
        for n in sizes:
            if fresh:
                seconds = measure(
                    run, lambda n=n: make_input(n), rounds=1, **options
                )
            else:
                seconds = measure(
                    lambda n=n: run(*inputs[n]), rounds=1, **options
                )
            best[n] = min(best[n], seconds)
    return [(n, best[n]) for n in sizes]
```

`doubling` builds each size's input once (unless `fresh=True`), then times every size in turn, five times over. The rounds are **interleaved**: round one times every size, then round two times every size again, and so on, and each size keeps its fastest round. The first version of this book's benchmarks timed all five rounds of one size before moving to the next, and its tables had odd jumps: a burst of activity from another program would spoil all five rounds of whichever size happened to be running. Interleaving spreads each size's rounds over the whole run.

Finally, showing the results:

```
UNITS = [  # (short name, long name, seconds in one unit)
    ("s", "seconds", 1),
    ("ms", "milliseconds", 1e-3),
    ("us", "microseconds", 1e-6),
    ("ns", "nanoseconds", 1e-9),
]
```

```
def unit_for(seconds):
    """The largest unit that gives a number of at least 1."""
    return next((u for u in UNITS if seconds >= u[2]), UNITS[-1])


def show_time(seconds):
    """A time with a sensible unit: '512 ns', '3.21 us', '45.6 ms'."""
    short, _, scale = unit_for(seconds)
    return f"{seconds / scale:.3g} {short}"


def table(results):
    """Print a table: one row per size, one column per series.

    `results` maps a name to [(n, seconds), ...]. The ratio after each
    time shows how much longer that size took than the one before: about
    2 when the time is proportional to n, about 4 when it grows with n
    squared, and about 1 when it doesn't depend on n.
    """
    names = list(results)
    widths = [max(16, len(name) + 2) for name in names]
    header = "".join(f"{name:>{w}}" for name, w in zip(names, widths))
    print(f"{'n':>10}" + header)
    sizes = [n for n, _ in results[names[0]]]
    for i, n in enumerate(sizes):
        row = f"{n:>10,}"
        for name, width in zip(names, widths):
            seconds = results[name][i][1]
            cell = show_time(seconds)
            if i:
                cell += f" x{seconds / results[name][i - 1][1]:.1f}"
            row += f"{cell:>{width}}"
        print(row)
```

```
def chart(results, filename, title, log=False):
    """Save a line chart of the results: time against n."""
    import matplotlib.pyplot as plt
    from matplotlib.ticker import StrMethodFormatter

    longest = max(s for rows in results.values() for _, s in rows)
    _, unit, scale = unit_for(longest)
    fig, ax = plt.subplots()
    for name, rows in results.items():
        sizes = [n for n, _ in rows]
        times = [s / scale for _, s in rows]
        ax.plot(sizes, times, "o-", label=name)
    if log:  # equal steps on the axes are equal multiples
        ax.set_xscale("log")
        ax.set_yscale("log")
    else:
        ax.xaxis.set_major_formatter(StrMethodFormatter("{x:,.0f}"))
    ax.set_xlabel("n (input size)")
    ax.set_ylabel(f"{unit} per call")
    ax.set_title(title)
    ax.legend()
    fig.tight_layout()
    fig.savefig(filename)
    plt.close(fig)
```

`show_time` picks the unit that gives a number of at least 1, so you read "45.6 ms" rather than "0.0456". (`us` stands for microseconds, millionths of a second; the proper symbol is µs.) `table` prints one row per size and one column per series, and after each time, the ratio to the size before. `chart` draws the same data with matplotlib; with `log=True`, both axes use a logarithmic scale, where equal steps are equal multiples.

Reading the Ratios

The ratio column is the most useful number in every benchmark in this book, so it's worth practicing. When each size is double the one before:

- **×1**: the time doesn't depend on n. **O(1)**.
- **×2**: the time doubles with n. **O(n)**.
- **a little over ×2**: **O(n log n)**: doubling n doubles the n and adds one to log n.
- **×4**: doubling n quadruples the time. **O(n^2)**.
- **×8**: **O(n^3)**.

Here are three functions with three growth rates, measured:

```
"""Three functions, three growth rates, measured by doubling n."""

import bench


def first(items):  # constant: one step, whatever n is
    return items[0]


def total(items):  # linear: one step per item
    s = 0
    for x in items:
        s += x
    return s

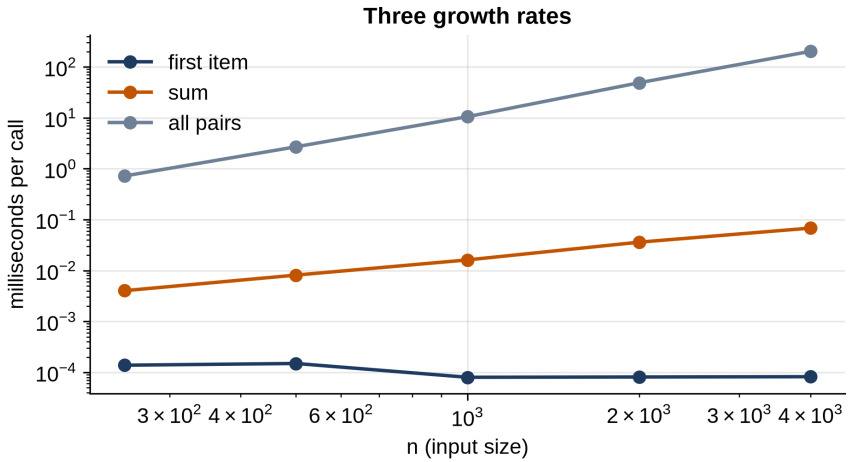

def has_duplicate(items):  # quadratic: one step per pair of items
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j]:
                return True
    return False


def make(n):
```

```
    return (list(range(n)),)


sizes = [250, 500, 1_000, 2_000, 4_000]
results = {
    "first item": bench.doubling(first, make, sizes),
    "sum": bench.doubling(total, make, sizes),
    "all pairs": bench.doubling(has_duplicate, make, sizes),
}
bench.table(results)
bench.chart(results, "ch03_growth.png", "Three growth rates", log=True)
```

```
         n      first item             sum       all pairs
       250          138 ns         4.02 us          720 us
       500     149 ns x1.1    8.13 us x2.0     2.7 ms x3.7
     1,000    79.9 ns x0.5    16.2 us x2.0    10.5 ms x3.9
     2,000    80.9 ns x1.0      36 us x2.2    48.8 ms x4.6
     4,000      82 ns x1.0    68.2 us x1.9     202 ms x4.1
```



On log-log axes, each growth rate is a straight line with its own slope.

Reading the first item takes the same tens of nanoseconds at every size: ratios near 1. The sum doubles. The all-pairs check roughly quadruples. Real measurements wobble: a ratio of 1.7 or 2.3 where theory says 2 is normal, especially for very fast operations where a few nanoseconds of noise is a large fraction. Look at the trend across the rows, not one ratio.

On the chart, with both axes logarithmic, each growth rate becomes a straight line, and the steepness of the line is the power of n: flat for O(1), slope 1 for

O(n), slope 2 for $O(n^2)$. It's the quickest way to read a growth rate off a picture.

> **Tip:** To find the growth rate of any code, time it at a few sizes that double, and look at the ratios. It takes a minute, and it's more reliable than reasoning about code you didn't write.

Best, Worst and Average Cases

The same algorithm can take very different times on different inputs of the same size. Linear search finds the first item immediately and a missing item only after checking everything:

```
"""Best, average and worst case: linear search for different targets."""

import bench

n = 100_000
items = list(range(n))
cases = {
    "best (first item)": 0,
    "middle item": n // 2,
    "worst (missing)": -1,
}
for name, target in cases.items():
    seconds = bench.measure(lambda: target in items)
    print(f"{name:>18}: {bench.show_time(seconds)}")
```

```
 best (first item): 38 ns
       middle item: 417 us
   worst (missing): 809 us
```

The **best case** is the fastest any input can be, the **worst case** the slowest, and the **average case** what to expect from typical inputs. When people give one Big O for an algorithm, they usually mean the worst case, because it's a promise: no input will be slower. Linear search is O(n) in the worst case, even though its best case is O(1).

Amortized Cost

One more idea you'll need in Chapter 6. Some operations are usually cheap but occasionally expensive. Appending to a Python list is like that: most

appends just write the new item into space the list already reserved, but now and then the list is full and must move everything to a bigger block of memory. If the expensive step is rare enough, the *average over many operations* is still constant. That average over a sequence of operations is the **amortized cost**, and list append is O(1) amortized. Chapter 6 builds a dynamic array and counts exactly how rare the expensive steps are.

Memory Counts Too

Everything in this chapter applies to memory as well as time. An algorithm that makes a copy of its input uses O(n) extra memory; one that keeps only a few variables uses O(1). The duplicate check above uses O(1) extra memory, while the faster approach from Chapter 1, putting items in a set, uses O(n). Trading memory for time like that is one of the most common moves in the whole subject.

Key Takeaways

- Big O describes how time (or memory) grows with input size, ignoring constant factors and smaller terms.
- O(1), O(log n), O(n), O(n log n), $O(n^2)$ and $O(2^n)$ cover most of what you'll meet; the gaps between them become enormous as n grows.
- $\log_2 n$ is the number of halvings to get from n to 1: about 30 for a billion.
- Time code by repeating it until the measurement is long, taking the fastest of several rounds, interleaving the rounds, and pausing garbage collection. `bench.py` does all four.
- When n doubles, a time ratio of about 1, 2 or 4 means O(1), O(n) or $O(n^2)$.
- Worst-case Big O is a promise; amortized cost is an average over a sequence of operations.

Exercises

1. **Count the steps.** This function counts pairs that add up to zero, but its inner loop only looks *before* position i:

```
for i in range(n): for j in range(i): steps += 1
```

How many steps does it take for n = 10, 100 and 1,000? What is its Big O? Write the code and check.

1. **Read a benchmark.** A function was timed at n = 1,000, 2,000, 4,000 and 8,000, taking 0.5, 2.1, 8.3 and 33 milliseconds. Another took 1.0, 2.2, 4.6 and 9.8 milliseconds. Classify each, and predict each one's time at n = 64,000. Write a short program that computes the ratios and the predictions.

CHAPTER 4 · BEGINNER

Python's Built-ins Under the Hood

Before building any structures of your own, it pays to know the ones you already use. This chapter looks inside Python's list, dictionary, set and string: how each stores its data, what each operation costs, and the measurements that prove it. Everything later in the book builds on these facts, and they'll change how you write everyday Python.

A List Is an Array of References

A Python list stores its items in one contiguous block of memory, as an **array**: slot 0, slot 1, slot 2, and so on, side by side. Each slot holds a **reference**, the memory address of the object, not the object itself. That's why a list can hold items of different types and sizes: every slot is the same size, 8 bytes on a 64-bit computer, whatever it points to.

Because the slots are side by side and all the same size, Python can find slot i by arithmetic: start of the block plus 8 × i. So `items[i]` takes the same time whether the list has ten items or ten million. That's **O(1) access by position**, the list's great strength.

Watching a List Grow

A block of memory can't simply grow; the memory right after it may belong to something else. So how does `append` work? Watch the list's size in memory as items go in:

```
"""Watch a list grow: its size in memory jumps, then stays put."""

import sys

items = []
last = sys.getsizeof(items)
print(f"empty list: {last} bytes")
for i in range(40):
    items.append(i)
    size = sys.getsizeof(items)
    if size != last:
        slots = (size - sys.getsizeof([])) // 8  # 8 bytes per reference
        print(f"length {len(items):>2}: {size:>3} bytes, room for {slots}")
        last = size
```

```
empty list: 56 bytes
length  1:  88 bytes, room for 4
length  5: 120 bytes, room for 8
length  9: 184 bytes, room for 16
length 17: 248 bytes, room for 24
length 25: 312 bytes, room for 32
length 33: 376 bytes, room for 40
```

`sys.getsizeof` reports the list object's own size: a 56-byte header plus 8 bytes per slot. The size doesn't change on every append. When the first item arrives, the list reserves room for 4. It stays at 88 bytes for the next three appends, then jumps to room for 8, then 16, 24, 32, 40. Each time the list is full, Python allocates a bigger block, copies the references across, and frees the old one. The extra room means most appends just write into a spare slot, which is why `append` is fast on average. Chapter 6 builds this mechanism yourself and counts exactly what it costs.

> **Interesting Fact:** The growth rule is one line of C in CPython's `listobject.c`: the new capacity is the needed size plus an eighth of it, plus 6, rounded down to a multiple of 4. The source code's own comment lists the pattern you just measured: "0, 4, 8, 16, 24, 32, 40, 52, 64, 76, ..." It grows by about 12.5% each time once lists get large, a more frugal choice than doubling.

What Each List Operation Costs

Adding or removing at the *end* touches one slot. Adding or removing at the *front* is another matter: every item after the change must shift over by one slot, so the time grows with the list's length. Measured:

```
"""What each list operation costs, as the list grows."""

import bench


def make(n):
    return (list(range(n)),)


def append_pop(items):  # add at the end, then take it off again
    items.append(0)
    items.pop()

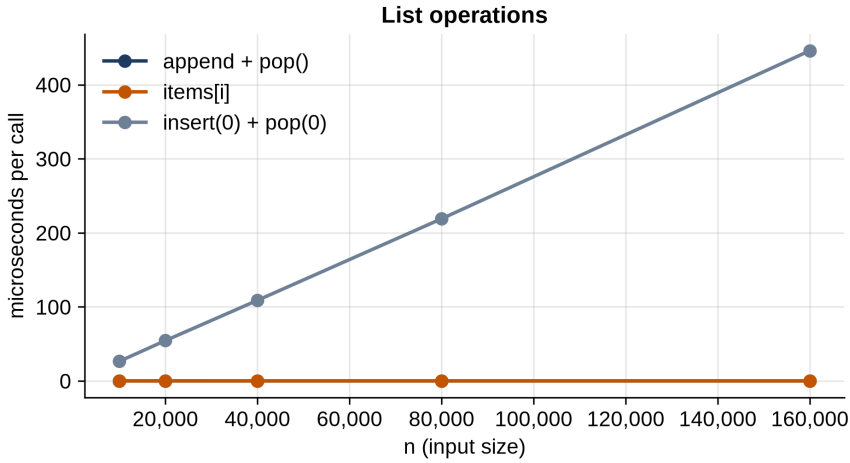

def insert_pop_front(items):  # add at the front, then take it off
    items.insert(0, 0)
    items.pop(0)


def read_middle(items):
    return items[len(items) // 2]


sizes = [10_000, 20_000, 40_000, 80_000, 160_000]
results = {
    "append + pop()": bench.doubling(append_pop, make, sizes),
    "items[i]": bench.doubling(read_middle, make, sizes),
    "insert(0) + pop(0)": bench.doubling(insert_pop_front, make, sizes),
}
bench.table(results)
bench.chart(results, "ch04_list_ops.png", "List operations")
```

```
         n  append + pop()        items[i]  insert(0) + pop(0)
    10,000          111 ns          116 ns             26.5 us
    20,000     132 ns x1.2     119 ns x1.0        54.5 us x2.1
    40,000     113 ns x0.9     121 ns x1.0         109 us x2.0
    80,000     109 ns x1.0     118 ns x1.0         219 us x2.0
   160,000     121 ns x1.1     130 ns x1.1         446 us x2.0
```

`append` plus `pop()` and reading `items[i]` stay flat at every size: O(1). Inserting at position 0 and popping it again doubles with the list's length: O(n). At 160,000 items it's thousands of times slower than working at the end. The shifting is done by a fast memory copy in C, which is why it's "only" hundreds of microseconds; but it's linear, and it adds up fast inside a loop.



Operations at the end and by index stay flat; inserting at the front grows with the list.

Here is the cost of each common list operation:

Operation	Cost	Why
`items[i]`, `items[i] = x`	O(1)	Arithmetic finds the slot
`append(x)`, `pop()`	O(1) amortized	Spare room at the end
`insert(i, x)`, `pop(i)`	O(n)	Later items shift
`x in items`, `index(x)`, `remove(x)`	O(n)	Checks items one by one
`items[a:b]`	O(b - a)	Copies the slice
`sort()`	O(n log n)	Chapter 12
`len(items)`	O(1)	The length is stored

> **Tip:** If your program takes items from the front of a list, `items.pop(0)` makes every removal O(n). Use `collections.deque`, whose `popleft()` is O(1). Chapter 8 measures the difference: thousands of times faster for a large queue.

Dictionaries and Sets: Hash Tables

A dictionary finds a key without looking through the others. It does that with a **hash table**: Python turns the key into a number, its **hash**, and uses that number to choose where in a table to store and later find the entry. Chapter 9 builds two kinds of hash table; for now, look at the hashes themselves:

```
"""Hash values: what dicts and sets use to find things."""

print(hash(42), hash(42) == hash(42.0))  # equal values, equal hashes
print(hash(-1), hash(-2))  # -1 is special in CPython: it becomes -2
print(hash((1, 2)) == hash((1, 2)))  # tuples hash by their contents

try:
    hash([1, 2])
except TypeError as error:
    print("TypeError:", error)  # a list can change, so it can't be a key

point = {(0, 0): "origin", (1, 2): "P"}
print(point[(1, 2)])
```

```
42 True
-2 -2
True
TypeError: unhashable type: 'list'
P
```

The rules that make hashing work:

- **Equal values must have equal hashes**, because a lookup with `42.0` must find the key `42`. So `hash(42) == hash(42.0)`.
- A small integer usually hashes to itself, except `-1`, which becomes `-2`, because CPython's C code uses `-1` to signal an error.
- **Only unchangeable values can be keys.** If a list could be a key and you appended to it, its hash would change and the dictionary could never find it again. So lists raise `TypeError`; tuples of hashable values are fine.

Because the table position comes from arithmetic on the hash, dictionary and set operations take about the same time at any size:

```
"""Dict and set operations take about the same time at any size."""

import bench


def make_dict(n):
    return ({i: i for i in range(n)},)
```

```
def make_set(n):
    return (set(range(n)),)


def dict_get(d):
    return d.get(12345)


def dict_set_delete(d):
    d[-1] = 0
    del d[-1]


def set_add_remove(s):
    s.add(-1)
    s.remove(-1)


sizes = [10_000, 100_000, 1_000_000]
results = {
    "dict get": bench.doubling(dict_get, make_dict, sizes),
    "dict set + del": bench.doubling(dict_set_delete, make_dict, sizes),
    "set add + remove": bench.doubling(set_add_remove, make_set, sizes),
}
bench.table(results)
```

```
         n        dict get  dict set + del  set add + remove
    10,000          102 ns          136 ns            128 ns
   100,000     119 ns x1.2     141 ns x1.0       131 ns x1.0
 1,000,000     117 ns x1.0     134 ns x1.0       131 ns x1.0
```

Around a tenth of a microsecond for a lookup, an insertion or a deletion, whether the table holds ten thousand entries or a million. That's what O(1) looks like. (Strictly, it's O(1) on average; Chapter 9 shows the rare inputs where hash tables slow down badly, and how Python defends against them.)

Operation	Cost	
`d[key]`, `d[key] = v`, `del d[key]`, `key in d`	O(1) average	
`s.add(x)`, `s.remove(x)`, `x in s`	O(1) average	
`a & b`, `a`	b` for sets	O(len(a) + len(b))
Looping over a dict or set	O(n)	

> **Note:** Since Python 3.7, dictionaries keep keys in the order they were inserted, as a guarantee of the language. Sets make no such promise: never rely on the order of a set.

Strings Can't Change

Strings are **immutable**: no operation changes a string; operations make new ones. So `text += word` builds a new string containing the old text plus the word, which means copying the old text. In a loop, that copying adds up: the first time copies a little, the last time copies almost everything, which is $O(n^2)$ in total. Here is the measurement, against `"".join()`, which computes the total length once and copies each piece once:

```
"""Building a long string: += in a loop or ''.join()?"""

import bench


def make(n):
    return (["word"] * n,)


def plus_equals(words):
    text = ""
    for w in words:
        text += w
    return text


def join(words):
    return "".join(words)


def plus_equals_two_names(words):
    text = ""
    for w in words:
        alias = text  # a second reference to the same string
        text += w
    return alias


sizes = [5_000, 10_000, 20_000, 40_000]
results = {
    "+=": bench.doubling(plus_equals, make, sizes),
    "join": bench.doubling(join, make, sizes),
    "+= (shared)": bench.doubling(plus_equals_two_names, make, sizes),
}
bench.table(results)
```

```
         n             +=            join     += (shared)
     5,000         194 us         25.3 us          895 us
    10,000     391 us x2.0    51.2 us x2.0    4.72 ms x5.3
    20,000     773 us x2.0     100 us x2.0    19.5 ms x4.1
    40,000    1.55 ms x2.0     199 us x2.0    77.7 ms x4.0
```

Surprise: the plain += loop grows linearly, not quadratically. CPython has an optimization: when the string has no other references, it can extend it in place instead of copying. But the third column shows how fragile that is. One extra name for the same string, `alias = text`, and the optimization can't apply, because changing the string in place would change `alias` too. The time jumps to quadratic: the ratio is about 4.

The optimization is a detail of CPython, not a promise of the Python language; other Pythons may not have it, and an innocent change to a loop can switch it off. Python's own style guide, PEP 8, says not to rely on it, and to use `"".join()` in code where speed matters.

> **Tip:** To build a string from many pieces, append the pieces to a list and call `"".join(pieces)` once at the end. It's always linear, in every version of Python.

Tuples

A tuple is like a list that can't change. It stores references side by side, so indexing is O(1), but there's no `append`, so no spare room is needed. Tuples are hashable when their items are, which makes them the natural dictionary key for several values together, as in `point[(1, 2)]` above.

Key Takeaways

- A list is an array of references: O(1) by position, O(n) to search, and O(n) to insert or remove anywhere but the end.
- A list keeps spare room and grows in jumps, so `append` is O(1) amortized. CPython's rule adds about 12.5% plus a little.
- Dictionaries and sets are hash tables: lookups, insertions and deletions are O(1) on average, at any size.

- Only hashable (unchangeable) values can be dictionary keys or set members; equal values have equal hashes.
- Strings are immutable. Build long strings with `"".join()`, not `+=` in a loop.

Exercises

1. **Membership in a loop.** Write two functions that return the items of list `a` that are also in list `b`: one with `x in b` on a list, one with a set made from `b`. Count the comparisons each makes for lists of 1,000 items (count the list version's comparisons with a loop; the set version makes about one hash lookup per item).
2. **When does the list grow?** Using `sys.getsizeof`, find the lengths at which a list grows between 1 and 200 appends, and compare the capacities with the rule in CPython's source: new capacity = n + n // 8 + 6, rounded down to a multiple of 4, where n is the length needed.

CHAPTER 5 · BEGINNER

Recursion and the Call Stack

Many structures in this book are defined in terms of smaller copies of themselves. A tree is a node with smaller trees below it. A list is a first item followed by a shorter list. Algorithms on such structures are most naturally written as functions that call themselves on the smaller copies: **recursion**. This chapter explains how recursion works in Python, what it costs, where it breaks, and the two techniques that fix its problems: memoization and explicit stacks.

A Function That Calls Itself

Every recursive function has two parts: a **base case**, small enough to answer directly, and a **recursive case**, which reduces the problem to a smaller one of the same kind.

```
"""Recursion: a function that calls itself on a smaller problem."""


def factorial(n):
    if n <= 1:  # base case: small enough to answer directly
        return 1
    return n * factorial(n - 1)  # recursive case: a smaller problem


def total(items):
    """Sum a list: the first item plus the sum of the rest."""
    if not items:
        return 0
    return items[0] + total(items[1:])


print(factorial(5))
```

```
print(factorial(20))
print(total([3, 1, 4, 1, 5, 9]))
```

```
120
2432902008176640000
23
```

`factorial(5)` is `5 * factorial(4)`, which is `4 * factorial(3)`, and so on down to `factorial(1)`, which is just 1. `total` sums a list as its first item plus the total of the rest. Each call works on a smaller problem, so eventually one reaches the base case, and the answers flow back up.

> **Common Mistake:** Forgetting the base case, or writing one the recursion never reaches (for example, testing `n == 0` when n can skip past it). The function then calls itself until Python stops it with a `RecursionError`.

`total` has a hidden cost: `items[1:]` copies the rest of the list on every call, so summing n items copies about $n^2/2$ items in all. Recursion can make the code beautiful and the algorithm slow at the same time. Chapter 12's merge sort is careful about exactly this.

The Call Stack

How does Python keep track of all the unfinished calls? Each call gets a **frame**, holding its local variables and where to continue when the call it made returns. Frames are kept on the **call stack**: a new call pushes a frame on top, and a return pops it off. This listing prints each call and return, indented by how deep the stack is:

```
"""Show each call and return, indented by the depth of the stack."""


def factorial(n, depth=0):
    pad = "  " * depth
    print(f"{pad}factorial({n}) called")
    result = 1 if n <= 1 else n * factorial(n - 1, depth + 1)
    print(f"{pad}factorial({n}) returns {result}")
    return result


factorial(4)
```

```
factorial(4) called
  factorial(3) called
    factorial(2) called
      factorial(1) called
      factorial(1) returns 1
    factorial(2) returns 2
  factorial(3) returns 6
factorial(4) returns 24
```

The calls go down four levels; then the returns come back up in reverse order: last called, first finished. That's exactly the behavior of the stack data structure you'll build in Chapter 8, and it's no coincidence. Any recursive function can be rewritten with an explicit stack, as you'll see at the end of this chapter.

The Recursion Limit

Each frame takes memory, and the stack isn't unlimited. Python stops runaway recursion at a fixed depth:

```
"""Python stops recursion that goes too deep."""

import sys


def depth(n):
    return 0 if n == 0 else 1 + depth(n - 1)


print("limit:", sys.getrecursionlimit())
print(depth(900))
try:
    depth(100_000)
except RecursionError:
    print("RecursionError: too deep")
```

```
limit: 1000
900
RecursionError: too deep
```

The default limit is 1,000 frames, counting the frames Python itself is using. Recursion 900 levels deep is fine; 100,000 levels raises `RecursionError`. You can raise the limit with `sys.setrecursionlimit`, and a few listings in this book do, but only for algorithms whose depth is known and moderate. Raising it far is risky: very deep recursion uses a lot of memory, and recursion that passes through C code (a `sorted` key function that recurses, say) can still

exhaust the system's stack and crash the interpreter instead of raising a clean error.

> **Tip:** In Python, use recursion when the depth grows like log n, as in balanced trees, merge sort and binary search: a billion items need only about 30 levels. When the depth can grow like n, as in walking a linked list or exploring a long path in a graph, use a loop.

Fibonacci and the Explosion of Calls

The Fibonacci numbers start 0, 1, and each one after is the sum of the two before: 0, 1, 1, 2, 3, 5, 8, 13, ... The definition translates directly into recursion, and the result is a famous disaster:

```
"""Fibonacci three ways, counting the calls each one makes."""

from functools import cache

calls = 0


def fib_naive(n):
    global calls
    calls += 1
    if n < 2:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)


@cache  # remembers every answer it has computed
def fib_memo(n):
    global calls
    calls += 1
    if n < 2:
        return n
    return fib_memo(n - 1) + fib_memo(n - 2)


def fib_loop(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a


for n in [10, 20, 25]:
    calls = 0
    answer = fib_naive(n)
    naive_calls = calls
    calls = 0
    fib_memo.cache_clear()
```

```
    assert fib_memo(n) == answer == fib_loop(n)
    print(
        f"fib({n}) = {answer:,}: naive {naive_calls:,} calls, "
        f"memoized {calls} calls"
    )
```

```
fib(10) = 55: naive 177 calls, memoized 11 calls
fib(20) = 6,765: naive 21,891 calls, memoized 21 calls
fib(25) = 75,025: naive 242,785 calls, memoized 26 calls
```

To compute `fib(25)`, the naive version makes 242,785 calls. It computes `fib(23)` twice, `fib(22)` three times, `fib(21)` five times, and so on; the repeated work grows exponentially, by a factor of about 1.6 for each step up in n.

The fix is **memoization**: remember each answer the first time it's computed, and look it up after that. The `@cache` decorator from `functools` does exactly that, storing results in a dictionary keyed by the arguments. With it, each `fib(k)` is computed once, so `fib(25)` takes 26 calls. The third version, `fib_loop`, computes upward from the start with two variables: no recursion and no memory beyond two numbers.

Timed:

```
"""Time the naive and the memoized Fibonacci."""

from functools import cache

import bench


def fib_naive(n):
    return n if n < 2 else fib_naive(n - 1) + fib_naive(n - 2)


def fib_memo(n):
    @cache
    def f(k):
        return k if k < 2 else f(k - 1) + f(k - 2)

    return f(n)  # a fresh cache every call, so each call does the work


sizes = [16, 18, 20, 22, 24]
results = {
    "naive": bench.doubling(fib_naive, lambda n: (n,), sizes),
    "memoized": bench.doubling(fib_memo, lambda n: (n,), sizes),
}
bench.table(results)
```

```
         n           naive        memoized
        16          104 us         4.93 us
        18     283 us x2.7    5.45 us x1.1
        20     725 us x2.6    5.89 us x1.1
        22     1.9 ms x2.6    6.74 us x1.1
        24    4.97 ms x2.6    7.45 us x1.1
```

Here n goes up by 2 at each row, not doubling, so read the ratios differently: each step of 2 multiplies the naive version's time by about 2.6, which is 1.6 squared, the growth factor per step. That's exponential growth: every few steps of n multiply the time by ten. The memoized version barely grows at all. Memoization is the seed of dynamic programming, which gets two whole chapters in Part VI.

> **Interesting Fact:** The ratio between consecutive Fibonacci numbers approaches the golden ratio, about 1.618, and so does the ratio of the naive function's call counts. The table's ratio of about 2.6 for each step of 2 is 1.618^2, about 2.618.

Recursion on Nested Data

Recursion shines on data that is nested to an unknown depth, like lists inside lists:

```
"""Recursion fits nested data: flatten lists inside lists."""


def flatten(items):
    out = []
    for x in items:
        if isinstance(x, list):
            out.extend(flatten(x))  # a smaller problem of the same kind
        else:
            out.append(x)
    return out


def flatten_loop(items):
    """The same with an explicit stack instead of recursion."""
    out, stack = [], [iter(items)]
    while stack:
        for x in stack[-1]:
            if isinstance(x, list):
                stack.append(iter(x))
                break
            out.append(x)
        else:  # this list is finished
```

```
            stack.pop()
    return out


nested = [1, [2, [3, 4], 5], [], [[6]], 7]
print(flatten(nested))
print(flatten_loop(nested))

deep = [0]
for i in range(1, 5000):
    deep = [deep, i]  # nested 5,000 levels deep
print(len(flatten_loop(deep)))  # the loop has no depth limit
```

```
[1, 2, 3, 4, 5, 6, 7]
[1, 2, 3, 4, 5, 6, 7]
5000
```

`flatten` handles each item: a list is flattened recursively, anything else is kept. It reads exactly like the definition of the problem.

`flatten_loop` does the same job with an explicit stack of iterators instead of the call stack. The top of the stack is the list being worked through. Meeting a nested list pushes an iterator over it; finishing a list pops it, and the `for` loop's `else` clause, which runs only when the loop wasn't stopped by `break`, does the pop. It's harder to read, but it has no depth limit: it flattens a list nested 5,000 levels deep, which would make the recursive version raise `RecursionError`. When depth can be large, this is the trade: a little clarity for unlimited depth.

Key Takeaways

- A recursive function needs a base case and a recursive case that makes the problem smaller.
- Each call adds a frame to the call stack; Python's default limit is 1,000 frames.
- Use recursion when the depth is logarithmic; use loops or an explicit stack when it can grow with n.
- Naive recursion can repeat work exponentially; memoization (`functools.cache`) computes each subproblem once.
- Any recursion can be rewritten with an explicit stack.

Exercises

1. **Power.** Write `power(x, n)` recursively using x^n = (x^(n/2))^2 for even n, and x × x^(n-1) for odd n. Count the multiplications for n = 1,000 and n = 1,000,000. Why is it O(log n)?
2. **Count the ways.** Write a recursive function counting the ways to make an amount of money from coins of 1, 2 and 5 (order doesn't matter). Use `functools.cache` and check your answer for amounts 0 to 10 by brute force. (Hint: either use the largest coin type at least once, or never use it.)

PART II

Linear Structures

CHAPTER 6 · BEGINNER

A Dynamic Array

Chapter 4 showed that a Python list is an array that grows in jumps. In this chapter you'll build one: a `DynamicArray` class with indexing, `append`, `insert` and `pop`, on top of storage that can't grow. Then you'll count exactly how much copying the growth costs, discover why doubling works and adding a fixed amount doesn't, and time your array against Python's list.

Fixed-Size Storage

Real arrays live in a block of memory of a fixed size. Python doesn't let us allocate raw memory, but we can imitate it with a list that we promise never to grow or shrink, only to read and write by index:

```
"""A dynamic array: a growable list built on fixed-size storage."""


def make_storage(capacity):
    """A fixed-size block of slots. We only ever read and write slots by
    index, never grow it, so it behaves like a raw block of memory."""
    return [None] * capacity
```

From now on, the only things `DynamicArray` does with its storage are `storage[i]` and `storage[i] = value`, exactly what real memory allows. Its job is to look like a list that can grow, while the storage beneath it never does.

The Class

The array keeps three things: the storage, its **capacity** (how many slots it has), and the number of items actually stored, `_n`. Only the first `_n` slots hold items; the rest are spare room.

```
class DynamicArray:
    def __init__(self, growth=2.0):
        self._n = 0  # how many items are stored
        self._capacity = 1  # how many slots the storage has
        self._data = make_storage(1)
        self._growth = growth
        self.copies = 0  # items copied by resizing, for experiments

    def __len__(self):
        return self._n

    def _check(self, i):
        """Turn a negative index into a positive one; reject bad ones."""
        if i < 0:
            i += self._n
        if not 0 <= i < self._n:
            raise IndexError("DynamicArray index out of range")
        return i

    def __getitem__(self, i):
        return self._data[self._check(i)]

    def __setitem__(self, i, value):
        self._data[self._check(i)] = value
```

`_check` handles negative indexes the way lists do, so `a[-1]` is the last item, and raises `IndexError` for anything outside the stored items. Note that it compares with `_n`, not the capacity: the spare slots exist, but they don't hold items, so reading them is an error.

`copies` isn't part of a real array. It counts how many items have been copied during resizing, so we can measure the cost of growing.

Growing

When the storage is full, `append` makes new storage, bigger by the growth factor (2 by default, so twice the size), and copies every item across:

```
    def _resize(self, capacity):
        """Move every item into new storage with `capacity` slots."""
        new = make_storage(capacity)
        for i in range(self._n):
            new[i] = self._data[i]
```

```
        self.copies += self._n
        self._data, self._capacity = new, capacity

    def append(self, value):
        if self._n == self._capacity:  # full: grow first
            self._resize(
                max(self._n + 1, int(self._capacity * self._growth))
            )
        self._data[self._n] = value
        self._n += 1
```

The `max(self._n + 1, ...)` guards against growth factors so small that `int()` would round the new capacity down to the old one. Watch it work:

```
"""Watch the capacity double as items are appended."""

from dynamic_array import DynamicArray

a = DynamicArray()
for i in range(17):
    before = a._capacity
    a.append(i)
    if a._capacity != before:
        print(
            f"append #{i + 1:>2}: capacity {before:>2} -> "
            f"{a._capacity:>2}, {a.copies:>2} items copied so far"
        )
```

```
append # 2: capacity  1 ->  2,  1 items copied so far
append # 3: capacity  2 ->  4,  3 items copied so far
append # 5: capacity  4 ->  8,  7 items copied so far
append # 9: capacity  8 -> 16, 15 items copied so far
append #17: capacity 16 -> 32, 31 items copied so far
```

The capacity doubles each time it runs out: 1, 2, 4, 8, 16, 32. Each resize copies every item stored so far, so the resizes copy 1, 2, 4, 8 and 16 items: 31 in total for 17 appends, fewer than two copies per append.

Why Doubling Works

That pattern holds at every size. With doubling, the copies done by all the resizes add up to 1 + 2 + 4 + ... + (the last capacity), which is less than twice the final size. So n appends cost n writes plus fewer than 2n copies: O(n) in total, and **O(1) amortized** per append. Most appends are cheap; the occasional expensive one is paid for by all the cheap ones before it.

What if the array grew by a fixed amount instead, say 100 slots at a time? Then a resize happens every 100 appends, and each copies everything so far: 100, 200, 300, ... items. The total grows with n^2, and the average cost per append grows with n. This program simulates three growth rules and reports the copies per appended item:

```
"""How many items does resizing copy in total? Three growth rules."""


def copies_for(n, grow):
    """Total items copied while appending n items, starting at 1 slot."""
    capacity, copies = 1, 0
    for size in range(n):
        if size == capacity:  # full: copy everything to new storage
            copies += size
            capacity = grow(capacity)
    return copies


rules = {
    "double": lambda c: 2 * c,
    "times 1.5": lambda c: max(c + 1, int(c * 1.5)),
    "add 100": lambda c: c + 100,
}
print(f"{'n':>9}" + "".join(f"{name:>16}" for name in rules))
for n in [1_000, 10_000, 100_000, 1_000_000]:
    row = "".join(f"{copies_for(n, g) / n:>16,.1f}" for g in rules.values())
    print(f"{n:>9,}{row}")
print("(copies per appended item)")
```

```
        n          double       times 1.5         add 100
    1,000             1.0             2.1             4.5
   10,000             1.6             2.4            49.5
  100,000             1.3             2.8           499.5
1,000,000             1.0             2.1         4,999.5
(copies per appended item)
```

Doubling stays below 2 copies per item at every size: where exactly it lands depends on how far n is past the last power of two. Growing by half (times 1.5) costs a few more copies but wastes less memory: at most a third of the slots are spare, instead of half. Adding 100 slots at a time is a disaster: 4,999.5 copies per item at a million items, and it keeps growing.

> **Real-World Use:** The growable arrays of popular languages all grow by a *factor*, but they choose different factors. CPython's list grows by about 1.125 plus a little (Chapter 4). Java's `ArrayList` grows by half, and Go's slices double while they're small, then move toward growing by a quarter once they pass 256 elements. Each is a different trade between copying time and wasted memory, all with O(1) amortized appends.

Inserting and Removing

Inserting anywhere but the end means making room: every item from the insertion point on shifts one slot to the right, starting from the end so nothing is overwritten. Removing shifts them back left:

```
    def insert(self, i, value):
        """Insert before position i, shifting later items right."""
        i = min(max(i + self._n if i < 0 else i, 0), self._n)
        if self._n == self._capacity:
            self._resize(
                max(self._n + 1, int(self._capacity * self._growth))
            )
        for j in range(self._n, i, -1):
            self._data[j] = self._data[j - 1]
        self._data[i] = value
        self._n += 1

    def pop(self, i=-1):
        """Remove and return the item at i, shifting later items left."""
        i = self._check(i)
        value = self._data[i]
        for j in range(i, self._n - 1):
            self._data[j] = self._data[j + 1]
        self._n -= 1
        self._data[self._n] = None  # don't keep a reference to it
        if 0 < self._n <= self._capacity // 4:  # mostly empty: shrink
            self._resize(self._capacity // 2)
        return value
```

Both are O(n): inserting at the front shifts every item. `pop` also sets the vacated slot back to `None`, so the array doesn't keep the removed object alive.

`pop` also **shrinks** the storage when it falls to a quarter full, halving the capacity. Why a quarter and not a half? Imagine an array that's exactly full at 8 items, then a program that alternately appends and pops. With "shrink at half full", the append grows it to 16, the pop leaves 8 items in 16 slots, exactly half, so it shrinks back to 8, and the next append grows it again: every single

operation copies the whole array. Exercise 2 measures it. Shrinking at a quarter leaves a gap between the grow point and the shrink point, so a run of operations near the boundary can't make the array bounce.

Finally, iteration and a readable `repr`:

```
    def __iter__(self):
        for i in range(self._n):
            yield self._data[i]

    def __repr__(self):
        return f"DynamicArray({list(self)})"
```

Defining `__iter__` makes the array work in `for` loops, `list(...)`, and everywhere else Python iterates.

Testing It

The tests check the basics, the errors, insertion and removal, growing and shrinking, and then run 5,000 random operations side by side on a `DynamicArray` and a Python list, checking after each that they agree:

```
"""Tests for DynamicArray, including random operations checked
against Python's own list."""

import random

import pytest

from dynamic_array import DynamicArray


def test_append_and_index():
    a = DynamicArray()
    for x in "abc":
        a.append(x)
    assert len(a) == 3
    assert a[0] == "a" and a[2] == "c" and a[-1] == "c"
    a[1] = "B"
    assert list(a) == ["a", "B", "c"]


def test_bad_index_raises():
    a = DynamicArray()
    a.append(1)
    for i in (1, -2):
        with pytest.raises(IndexError):
            a[i]
    with pytest.raises(IndexError):
        DynamicArray().pop()
```

```
def test_insert_and_pop_shift_items():
    a = DynamicArray()
    for x in [1, 2, 4]:
        a.append(x)
    a.insert(2, 3)
    a.insert(0, 0)
    assert list(a) == [0, 1, 2, 3, 4]
    assert a.pop(0) == 0 and a.pop() == 4
    assert list(a) == [1, 2, 3]


def test_capacity_grows_and_shrinks():
    a = DynamicArray()
    for i in range(100):
        a.append(i)
        assert a._capacity >= len(a)
    while len(a) > 1:
        a.pop()
    assert a._capacity <= 4  # shrank as it emptied


def test_random_operations_match_list():
    rng = random.Random(2026)
    a, expected = DynamicArray(), []
    for _ in range(5000):
        op = rng.choice(["append", "append", "insert", "pop", "set"])
        if op == "append":
            x = rng.randint(0, 99)
            a.append(x)
            expected.append(x)
        elif op == "insert":
            i, x = rng.randint(-5, len(expected) + 5), rng.randint(0, 99)
            a.insert(i, x)
            expected.insert(i, x)
        elif expected:
            i = rng.randrange(-len(expected), len(expected))
            if op == "pop":
                assert a.pop(i) == expected.pop(i)
            else:
                a[i] = expected[i] = rng.randint(0, 99)
        assert len(a) == len(expected)
    assert list(a) == expected
```

```
.....                                                                    [100%]
5 passed in 0.01s
```

The random test checks thousands of combinations, including inserts at negative positions and positions past the end, which `list.insert` clamps to the ends, and our `insert` must too.

Timing It

Now the benchmark: append n items to a Python list, to our doubling array, and to a version that grows by 100 slots:

```
"""Appending n items: our DynamicArray against Python's list."""

import bench
from dynamic_array import DynamicArray


class AddHundred(DynamicArray):
    """Grows by a fixed 100 slots instead of doubling."""

    def append(self, value):
        if self._n == self._capacity:
            self._resize(self._capacity + 100)
        self._data[self._n] = value
        self._n += 1

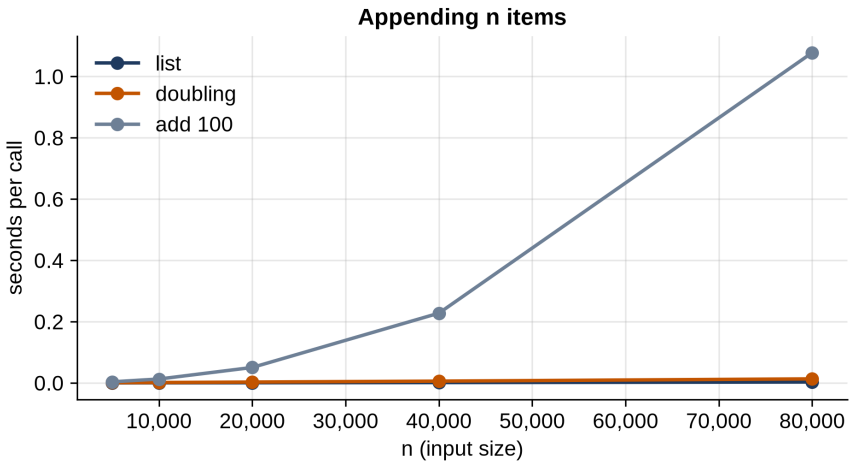

def fill(make):
    def run(n):
        a = make()
        for i in range(n):
            a.append(i)

    return run


sizes = [5_000, 10_000, 20_000, 40_000, 80_000]
results = {
    "list": bench.doubling(fill(list), lambda n: (n,), sizes),
    "doubling": bench.doubling(fill(DynamicArray), lambda n: (n,), sizes),
    "add 100": bench.doubling(fill(AddHundred), lambda n: (n,), sizes),
}
bench.table(results)
bench.chart(results, "ch06_append.png", "Appending n items")
```

```
         n            list        doubling         add 100
     5,000          120 us          654 us         3.53 ms
    10,000     246 us x2.1    1.28 ms x2.0    12.7 ms x3.6
    20,000     509 us x2.1     2.8 ms x2.2    50.4 ms x4.0
    40,000    1.03 ms x2.0    5.79 ms x2.1     228 ms x4.5
    80,000    2.92 ms x2.8    13.1 ms x2.3     1.08 s x4.7
```

Python's list and our doubling array both grow linearly: double the items, about double the time. Ours is several times slower, because every step of `append` runs as Python code (a method call, a comparison, two attribute updates) while `list.append` is a few machine instructions in C. The fixed-step version is in another league: its time roughly quadruples with each



Doubling grows linearly; adding 100 slots at a time grows quadratically.

doubling, and at 80,000 items it's on the order of a hundred times slower than our own doubling array. Same code, different growth rule.

> **Tip:** When you know how many items a list will hold, you can avoid growth entirely: `[None] * n` makes the whole list at once, and so does a list comprehension, which can size the list from its input.

Key Takeaways

- A dynamic array keeps items in fixed-size storage with spare room, and grows by moving everything to bigger storage.
- Growing by a constant factor makes append O(1) amortized: the total copying stays below a constant times n.
- Growing by a constant amount makes append O(n) on average and filling the array $O(n^2)$.
- Insertion and removal in the middle are O(n), because items must shift.
- Shrink at a quarter full, not at half, so a mix of appends and pops near the boundary can't trigger a copy every time.

Exercises

1. **extend.** Add an `extend(items)` method that appends every item of an iterable. When the number of new items is known, resize at most once, to a capacity big enough for all of them.
2. **Why a quarter?** Simulate an array that starts full at 8 items, then alternately appends and pops 1,000 times. Count the items copied when it shrinks at half full, and when it shrinks at a quarter full.
3. **remove.** Add `remove(value)`, which deletes the first item equal to `value` and raises `ValueError` if there is none, like `list.remove`.

CHAPTER 7 · BEGINNER

Linked Lists

An array keeps its items side by side, which makes reading by position instant and inserting at the front slow. A **linked list** makes the opposite trade. Each item lives in its own small object, a **node**, that points to the next node. Nothing is side by side, so nothing ever needs to shift: adding at the front is instant at any size. But finding the 1,000th item means following 1,000 pointers.

This chapter builds a singly linked list and a doubly linked list with a sentinel, solves three classic interview problems, and measures where linked lists win and where they lose.

Nodes

A node holds a value and a reference to the next node, or `None` at the end of the list:

```
"""A singly linked list: each node points to the next one."""


class Node:
    __slots__ = ("value", "next")  # saves memory: no per-node dict

    def __init__(self, value, next=None):
        self.value = value
        self.next = next
```

`__slots__` tells Python that a `Node` has exactly these two attributes. Normally every object carries a dictionary for its attributes; with `__slots__` it doesn't,

which saves memory and makes attribute access a little faster. That matters when a list has a million nodes.

A Singly Linked List

The list itself keeps a reference to the first node, the **head**, and to the last node, the **tail**, so that appending doesn't have to walk the whole list:

```
class LinkedList:
    def __init__(self, items=()):
        self.head = None  # first node
        self.tail = None  # last node, so append() needn't walk the list
        self._n = 0
        for x in items:
            self.append(x)

    def __len__(self):
        return self._n

    def __iter__(self):
        node = self.head
        while node:
            yield node.value
            node = node.next

    def __repr__(self):
        return " -> ".join(map(repr, self)) or "(empty)"

    def push_front(self, value):
        self.head = Node(value, self.head)
        if self.tail is None:
            self.tail = self.head
        self._n += 1

    def append(self, value):
        node = Node(value)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self._n += 1

    def pop_front(self):
        if self.head is None:
            raise IndexError("pop from an empty list")
        node = self.head
        self.head = node.next
        if self.head is None:
            self.tail = None
        self._n -= 1
        return node.value
```

`push_front` makes a new node pointing at the old head and makes it the new head: two assignments, whatever the length. That's O(1). `append` links the new node after the tail, also O(1). `pop_front` moves the head along one node. The empty list is the special case each method must handle: when the list is empty, the head and the tail are both `None`, and the first node added is both.

`__iter__` follows the `next` references from the head until it reaches `None`. Every way of reading the list, `for` loops, `list(s)`, `repr`, goes through it.

Finding and removing a value means walking from the head:

```
    def find(self, value):
        """The first node holding value, or None."""
        node = self.head
        while node and node.value != value:
            node = node.next
        return node

    def remove(self, value):
        """Unlink the first node holding value. Raise ValueError if none."""
        before, node = None, self.head
        while node and node.value != value:
            before, node = node, node.next
        if node is None:
            raise ValueError(f"{value!r} not in list")
        if before is None:
            self.head = node.next
        else:
            before.next = node.next
        if node is self.tail:
            self.tail = before
        self._n -= 1
```

`remove` keeps a second pointer, `before`, one node behind, because unlinking a node means changing the `next` of the node *before* it. Removing the head changes `self.head` instead, and removing the last node must move `self.tail` back. Each of these is a place where a linked list implementation typically goes wrong, and each is covered by the tests.

Reversing in Place

Reversing a linked list without making new nodes is the most famous linked-list interview question:

```
    def reverse(self):
        """Reverse the list in place by turning every arrow around."""
        before, node = None, self.head
        self.tail = node
        while node:
            node.next, before, node = before, node, node.next
        self.head = before
```

Walk the list once. At each node, turn its `next` arrow around to point at the node before, then move both pointers forward. Python's simultaneous assignment does all three steps in one line: it evaluates the whole right side first, so `node.next`'s old value is saved before it's overwritten. When the walk ends, `before` is the old last node, the new head. The old head becomes the tail. O(n) time and O(1) extra memory.

```
"""The singly linked list in action."""

from linked_list import LinkedList

s = LinkedList(["b", "c"])
s.push_front("a")
s.append("d")
print(s, f"(length {len(s)})")
s.remove("c")
print(s)
s.reverse()
print(s)
print("popped", s.pop_front(), "leaving", s)
```

```
'a' -> 'b' -> 'c' -> 'd' (length 4)
'a' -> 'b' -> 'd'
'd' -> 'b' -> 'a'
popped d leaving 'b' -> 'a'
```

A Doubly Linked List with a Sentinel

A singly linked list can't remove its last node in O(1): to unlink it, you need the node before it, and finding that means walking from the head. A **doubly linked list** gives every node a `prev` pointer as well, so you can step backward and unlink any node in O(1).

The code becomes much simpler with one trick, a **sentinel**: a dummy node that isn't part of the data. The sentinel sits before the first node and after the last, joining the list into a circle. An empty list is just the sentinel pointing at itself. Now every real node always has a real `prev` and `next`, and no method needs a special case for the ends:

```
"""A doubly linked list with a sentinel node.

The sentinel is a dummy node that sits before the first node and after
the last, so the list is a circle and no operation needs a special case
for an empty list or for the ends.
"""


class DNode:
    __slots__ = ("value", "prev", "next")

    def __init__(self, value=None):
        self.value = value
        self.prev = self.next = self


class DoublyLinkedList:
    def __init__(self, items=()):
        self._sentinel = DNode()
        self._n = 0
        for x in items:
            self.append(x)

    def __len__(self):
        return self._n

    def __iter__(self):
        node = self._sentinel.next
        while node is not self._sentinel:
            yield node.value
            node = node.next

    def _insert_after(self, node, value):
        new = DNode(value)
        new.prev, new.next = node, node.next
        node.next.prev = new
        node.next = new
        self._n += 1
        return new

    def append(self, value):
        return self._insert_after(self._sentinel.prev, value)

    def appendleft(self, value):
        return self._insert_after(self._sentinel, value)

    def unlink(self, node):
        """Remove a node we already hold, in constant time."""
        node.prev.next = node.next
        node.next.prev = node.prev
```

```
        self._n -= 1
        return node.value

    def pop(self):
        if not self._n:
            raise IndexError("pop from an empty list")
        return self.unlink(self._sentinel.prev)

    def popleft(self):
        if not self._n:
            raise IndexError("pop from an empty list")
        return self.unlink(self._sentinel.next)
```

Everything is built from two small operations. `_insert_after` links a new node between `node` and `node.next` by updating four pointers. `unlink` removes a node by connecting its neighbors to each other. `append` inserts after the last node (the sentinel's `prev`), and `appendleft` after the sentinel itself; `pop` and `popleft` unlink the node at either end.

`unlink` takes a node, not a value. Once you hold a reference to a node, removing it is O(1). Chapter 32 uses exactly this to build an LRU cache, where a dictionary maps each key to its node so any entry can be found and moved in constant time.

> **Real-World Use:** Python's own `collections.deque` is a doubly linked list, with a twist: each node holds a block of 64 items, not one. The blocks keep most operations as fast as an array's, and the links between blocks make adding and removing at both ends O(1). The Linux kernel uses circular doubly linked lists with a head node, exactly the sentinel design above, throughout its code.

Testing Both Lists

```
"""Tests for both linked lists, with random operations checked
against collections.deque."""

import random
from collections import deque

import pytest

from doubly_linked import DoublyLinkedList
from linked_list import LinkedList


def test_singly_basic_operations():
```

```
    s = LinkedList([1, 2, 3])
    s.push_front(0)
    s.append(4)
    assert list(s) == [0, 1, 2, 3, 4] and len(s) == 5
    assert s.pop_front() == 0
    s.remove(3)
    s.remove(4)  # the tail: tail must move back
    s.append(5)
    assert list(s) == [1, 2, 5]
    assert s.find(2).value == 2 and s.find(9) is None
    with pytest.raises(ValueError):
        s.remove(9)


def test_reverse():
    for n in range(5):
        s = LinkedList(range(n))
        s.reverse()
        assert list(s) == list(range(n))[::-1]
        s.append("end")  # the tail must be right after reversing
        assert list(s)[-1] == "end"


def test_empty_pops_raise():
    with pytest.raises(IndexError):
        LinkedList().pop_front()
    with pytest.raises(IndexError):
        DoublyLinkedList().pop()


def test_unlink_a_held_node():
    d = DoublyLinkedList("abc")
    middle = d._sentinel.next.next
    assert d.unlink(middle) == "b"
    assert list(d) == ["a", "c"]


def test_random_operations_match_deque():
    rng = random.Random(2026)
    s, d, expected = LinkedList(), DoublyLinkedList(), deque()
    for _ in range(5000):
        op = rng.choice(["front", "back", "popfront", "popback"])
        x = rng.randint(0, 99)
        if op == "front":
            s.push_front(x)
            d.appendleft(x)
            expected.appendleft(x)
        elif op == "back":
            s.append(x)
            d.append(x)
            expected.append(x)
        elif expected and op == "popfront":
            value = expected.popleft()
            assert s.pop_front() == value == d.popleft()
        elif expected:
            value = expected.pop()
            assert d.pop() == value
            s.reverse()  # the singly list can only pop at the front
            assert s.pop_front() == value
            s.reverse()
```

```
    assert list(s) == list(d) == list(expected)
    assert len(s) == len(d) == len(expected)
```

```
.....                                                                    [100%]
5 passed in 0.01s
```

The random test drives both lists and a `deque` with the same 5,000 operations. The singly linked list can only remove from the front, so to pop from the back the test reverses it, pops the front, and reverses it back, which also tests `reverse` thousands of times.

Three Interview Problems

Linked lists are a favorite in interviews because the pointer manipulation is easy to get wrong. Three classics:

```
"""Three classic linked-list interview problems."""

from linked_list import LinkedList, Node


def middle(head):
    """The middle node: a fast pointer moves two steps per slow step."""
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    return slow


def has_cycle(head):
    """Floyd's cycle detection: if the list loops, the fast pointer
    laps the slow one and they meet. Constant extra memory."""
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False


def merge_sorted(a, b):
    """Merge two sorted chains of nodes into one, reusing the nodes."""
    dummy = tail = Node(None)  # a dummy head avoids special cases
    while a and b:
        if a.value <= b.value:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

```
def values(head):
    out = []
    while head:
        out.append(head.value)
        head = head.next
    return out


print(middle(LinkedList([1, 2, 3, 4, 5]).head).value)
print(middle(LinkedList([1, 2, 3, 4]).head).value)

looped = LinkedList([1, 2, 3, 4])
print(has_cycle(looped.head))
looped.tail.next = looped.head.next  # 4 now points back to 2
print(has_cycle(looped.head))

merged = merge_sorted(
    LinkedList([1, 4, 6]).head, LinkedList([2, 3, 7]).head
)
print(values(merged))
```

```
3
3
False
True
[1, 2, 3, 4, 6, 7]
```

- **The middle node** uses two pointers moving at different speeds: `fast` moves two steps for each step of `slow`, so when `fast` reaches the end, `slow` is halfway. With an even number of nodes it returns the second of the two middle nodes. One pass, no counting.
- **Cycle detection** uses the same two pointers. If the list ends, `fast` reaches `None` and there's no cycle. If it loops, `fast` enters the loop and, moving one step faster per round than `slow`, eventually lands on it. This is **Floyd's cycle-finding algorithm**, also called "the tortoise and the hare", and it uses O(1) extra memory. Storing visited nodes in a set would also work, but with O(n) memory.
- **Merging two sorted lists** reuses the existing nodes instead of making new ones. A dummy head node, a sentinel again, means the first node needs no special case.

> **Interview Tip:** For almost any linked-list question, consider a dummy head node, and consider two pointers: one ahead of the other by k steps, or one moving twice as fast. Draw three or four nodes on paper and trace your pointers before writing code.

When Linked Lists Win, and When They Lose

Here's the case for linked lists: adding and removing at the front, against Python's list and `deque`:

```
"""Adding at the front: LinkedList against list and deque."""

from collections import deque

import bench
from linked_list import LinkedList


def make_list(n):
    return (list(range(n)),)


def make_linked(n):
    return (LinkedList(range(n)),)


def make_deque(n):
    return (deque(range(n)),)


def list_front(items):
    items.insert(0, 0)
    items.pop(0)

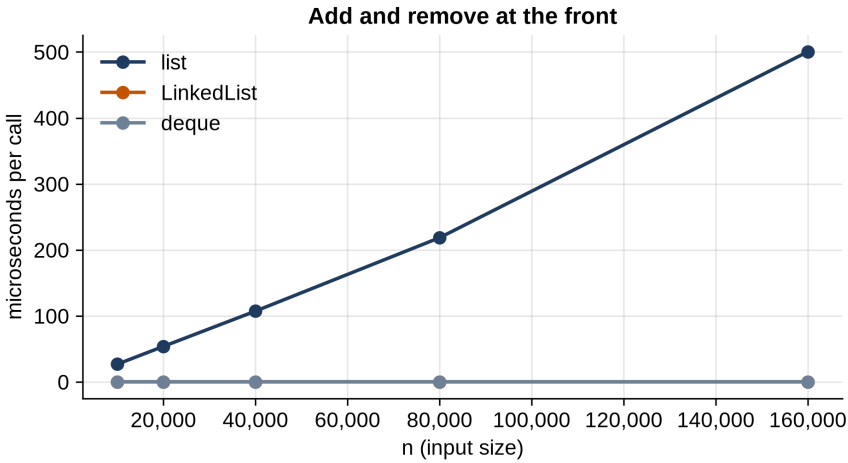

def linked_front(items):
    items.push_front(0)
    items.pop_front()


def deque_front(items):
    items.appendleft(0)
    items.popleft()


sizes = [10_000, 20_000, 40_000, 80_000, 160_000]
results = {
    "list": bench.doubling(list_front, make_list, sizes),
    "LinkedList": bench.doubling(linked_front, make_linked, sizes),
    "deque": bench.doubling(deque_front, make_deque, sizes),
}
bench.table(results)
```

```
bench.chart(results, "ch07_front.png", "Add and remove at the front")
```

```
         n            list      LinkedList           deque
    10,000         26.9 us          307 ns          115 ns
    20,000    53.9 us x2.0     310 ns x1.0     113 ns x1.0
    40,000     107 us x2.0     306 ns x1.0     114 ns x1.0
    80,000     219 us x2.0     311 ns x1.0     113 ns x1.0
   160,000     501 us x2.3     307 ns x1.0     114 ns x1.0
```



At the front, the list's time grows with its length; the linked list and deque stay flat.

The Python list's time doubles with its length, because every item shifts. The linked list stays flat at a few hundred nanoseconds at every size. And `deque`, a linked list of blocks written in C, is flat and faster still.

Now walking through every item:

```
"""Walking through every item: LinkedList against list."""

import bench
from linked_list import LinkedList


def walk(items):
    total = 0
    for x in items:
        total += x
    return total


sizes = [10_000, 20_000, 40_000, 80_000]
results = {
    "list": bench.doubling(walk, lambda n: (list(range(n)),), sizes),
```

```
    "LinkedList": bench.doubling(
        walk, lambda n: (LinkedList(range(n)),), sizes
    ),
}
bench.table(results)
```

```
         n            list      LinkedList
    10,000          182 us          394 us
    20,000     348 us x1.9     803 us x2.0
    40,000     694 us x2.0    1.64 ms x2.0
    80,000    1.97 ms x2.8    3.93 ms x2.4
```

Both are O(n), but the linked list is about twice as slow. Each step follows a pointer to a node object that could be anywhere in memory, and runs `__iter__`'s Python code, while iterating a list reads references stored side by side, in C. Modern processors are much faster at reading memory in order than at jumping around, so arrays win most whole-list operations, even in languages where both are compiled.

So: use a linked list (or `deque`) when you add and remove at the ends, or when you hold references to nodes and need to remove them in O(1). Use a list for almost everything else.

Key Takeaways

- A linked list stores each item in a node that points to the next; nothing ever shifts.
- Adding or removing at the head is O(1); finding an item or the k-th position is O(n).
- A doubly linked list with a sentinel can add and remove at both ends, and unlink any node it holds, in O(1), with no special cases.
- Two pointers, one faster or further ahead, solve many linked-list problems in one pass: the middle, cycles, the k-th node from the end.
- In practice arrays are faster for walking through items; `collections.deque` is the right choice for a queue or a double-ended list in Python.

Exercises

1. **k-th from the end.** Remove the k-th node from the end of a list in one pass. (Hint: start one pointer k nodes ahead of another.)
2. **Palindrome.** Check whether a linked list reads the same forward and backward, using O(1) extra memory: find the middle, reverse the second half, compare, and reverse it back.
3. **Where does the cycle start?** Extend Floyd's algorithm to return the first node of the cycle. (Hint: after the pointers meet, start one of them again from the head and move both one step at a time.)

CHAPTER 8 · BEGINNER

Stacks and Queues

A **stack** is a pile: you add to the top and take from the top, so the last item in is the first out (LIFO). A **queue** is a line: you join at the back and leave from the front, first in, first out (FIFO). They're the simplest structures in this book and among the most used. The call stack from Chapter 5 is a stack; so are "undo" in an editor and "back" in a browser. Print jobs, web requests waiting for a server and the breadth-first search of Chapter 21 are queues.

What makes them interesting is not the idea but the cost. A stack is trivially fast on a Python list. A queue on a Python list is a trap, and this chapter measures just how big a trap.

A Stack

A Python list is already a perfect stack, as long as the top is the *end* of the list, where `append` and `pop` are O(1):

```
"""A stack: last in, first out. A Python list does all the work."""


class Stack:
    def __init__(self):
        self._items = []

    def push(self, value):
        self._items.append(value)  # the end of the list is the top

    def pop(self):
        if not self._items:
            raise IndexError("pop from an empty stack")
        return self._items.pop()
```

```
    def peek(self):
        if not self._items:
            raise IndexError("peek at an empty stack")
        return self._items[-1]

    def __len__(self):
        return len(self._items)

    def __bool__(self):
        return bool(self._items)
```

Why wrap a list in a class at all? To give the structure a narrow interface: a `Stack` can only push, pop and peek, so code that uses it can't accidentally insert in the middle. It also gives clearer error messages and makes intent obvious to anyone reading the code. In quick scripts and interviews, a plain list with `append` and `pop` is perfectly idiomatic.

Balanced Brackets

The classic stack problem: are the brackets in a piece of text balanced? Every opener must be closed by the matching closer, in the right order:

```
"""Are the brackets balanced? A stack remembers what must close next."""

PAIRS = {")": "(", "]": "[", "}": "{"}


def balanced(text):
    stack = []
    for ch in text:
        if ch in "([{":
            stack.append(ch)
        elif ch in PAIRS:
            if not stack or stack.pop() != PAIRS[ch]:
                return False  # a closer with no matching opener
    return not stack  # anything left open is unbalanced


for text in ["(a[b]{c})", "([)]", "((", "f(x) = [1, {2: 3}]", ""]:
    print(f"{text!r:>22}: {balanced(text)}")
```

```
           '(a[b]{c})': True
                '([)]': False
                  '((': False
  'f(x) = [1, {2: 3}]': True
                    '': True
```

The stack holds the openers still waiting to be closed, innermost on top. A closer must match the top; `"([)]"` fails because `)` arrives while `[` is on top.

Text that ends with openers still on the stack, like `"(("`, is unbalanced too. Every code editor that highlights a matching bracket, and every parser that reads Python or JSON, does a version of this.

Monotonic Stacks

A more advanced use: for each day's temperature, how many days until a warmer one?

```
"""A monotonic stack: for each day, how many days until it is warmer?

The stack holds the days still waiting for a warmer one, coldest on
top. Each new day pops every waiting day it is warmer than. Every day is
pushed and popped at most once, so the whole thing takes linear time.
"""


def days_until_warmer(temps):
    answer = [0] * len(temps)
    waiting = []  # indexes of days, temperatures decreasing upward
    for day, t in enumerate(temps):
        while waiting and temps[waiting[-1]] < t:
            earlier = waiting.pop()
            answer[earlier] = day - earlier
        waiting.append(day)
    return answer  # days never followed by a warmer one stay 0


temps = [21, 19, 18, 22, 20, 25, 24, 24]
print(temps)
print(days_until_warmer(temps))
```

```
[21, 19, 18, 22, 20, 25, 24, 24]
[3, 2, 1, 2, 1, 0, 0, 0]
```

The obvious solution checks every later day for every day: $O(n^2)$. The stack version keeps the days still waiting for a warmer day, and their temperatures decrease from the bottom of the stack to the top: a **monotonic stack**. Each new day pops every waiting day that's colder, answering their question, then joins the stack itself. Each day is pushed once and popped at most once, so the whole thing is O(n). This pattern answers any "next greater" or "next smaller" question, and appears regularly in interviews.

Queues: The Trap

A queue on a Python list looks easy: `append` to join at the back, `pop(0)` to leave from the front. But `pop(0)` shifts every remaining item left, so it's O(n), and emptying a queue of n items costs $O(n^2)$. The rest of this chapter builds two queues that avoid the shifting, then measures all of them.

A Ring Buffer

The first is a **ring buffer**, also called a circular buffer. The items sit in fixed storage, and two positions move forward through it: the front (where items leave) and the back (where they join). When the back reaches the end of the storage, it wraps around to slot 0, if that slot is free. Nothing ever shifts.

```
"""A queue (first in, first out) in a ring buffer.

The items sit in fixed storage that wraps around: when the back reaches
the end, it continues at slot 0. Both ends move forward, so nothing is
ever shifted. When the storage is full, it doubles.
"""


class RingQueue:
    def __init__(self, capacity=8):
        self._data = [None] * capacity
        self._front = 0  # slot of the first item
        self._n = 0

    def __len__(self):
        return self._n

    def enqueue(self, value):
        if self._n == len(self._data):
            self._grow()
        back = (self._front + self._n) % len(self._data)
        self._data[back] = value
        self._n += 1

    def dequeue(self):
        if not self._n:
            raise IndexError("dequeue from an empty queue")
        value = self._data[self._front]
        self._data[self._front] = None
        self._front = (self._front + 1) % len(self._data)
        self._n -= 1
        return value

    def _grow(self):
        """Copy the items, in order, into storage twice the size."""
        old, size = self._data, len(self._data)
        self._data = [None] * (2 * size)
        for i in range(self._n):
```

```
            self._data[i] = old[(self._front + i) % size]
        self._front = 0
```

The back position is computed as `(front + n) % size`: the `%` (remainder) operator makes positions wrap around. `dequeue` moves the front forward the same way. When the storage is full, `_grow` copies the items, in queue order, to the start of storage twice the size, as the dynamic array did in Chapter 6, so enqueue is O(1) amortized and dequeue is O(1).

> **Real-World Use:** Ring buffers are everywhere that data streams through a fixed amount of memory: audio and video playback, network cards, keyboard input and logging systems that keep only the newest messages. The Linux kernel provides one as a general-purpose building block, `kfifo`.

A Queue from Two Stacks

The second is an interview favorite: build a queue using only two stacks.

```
"""A queue made of two stacks: an interview favorite.

New items go on the `inbox` stack. To take one out, pop from `outbox`;
when it is empty, pour the whole inbox into it, which reverses the
order so the oldest item is on top. Each item is moved at most once, so
the average cost per operation is constant.
"""


class TwoStackQueue:
    def __init__(self):
        self._inbox, self._outbox = [], []

    def __len__(self):
        return len(self._inbox) + len(self._outbox)

    def enqueue(self, value):
        self._inbox.append(value)

    def dequeue(self):
        if not self._outbox:
            while self._inbox:
                self._outbox.append(self._inbox.pop())
        if not self._outbox:
            raise IndexError("dequeue from an empty queue")
        return self._outbox.pop()
```

New items go onto the `inbox` stack. To take one out, pop from `outbox`; if it's empty, pour the whole inbox into it first. Pouring reverses the order, so the

oldest item ends up on top. A single `dequeue` can be slow (pouring n items is O(n)), but each item is poured exactly once in its lifetime, so the average over any sequence of operations is O(1): the same amortized argument as for the dynamic array.

Testing

```
"""Tests for the stack and both queues."""

import random
from collections import deque

import pytest

from ring_queue import RingQueue
from stack import Stack
from two_stack_queue import TwoStackQueue


def test_stack_is_last_in_first_out():
    s = Stack()
    for x in [1, 2, 3]:
        s.push(x)
    assert s.peek() == 3 and len(s) == 3
    assert [s.pop(), s.pop(), s.pop()] == [3, 2, 1]
    assert not s
    with pytest.raises(IndexError):
        s.pop()


@pytest.mark.parametrize("make", [RingQueue, TwoStackQueue])
def test_queue_is_first_in_first_out(make):
    q = make()
    for x in range(20):  # more than RingQueue's first 8 slots
        q.enqueue(x)
    assert [q.dequeue() for _ in range(20)] == list(range(20))
    with pytest.raises(IndexError):
        q.dequeue()


@pytest.mark.parametrize("make", [RingQueue, TwoStackQueue])
def test_random_operations_match_deque(make):
    rng = random.Random(2026)
    q, expected = make(), deque()
    for _ in range(5000):
        if rng.random() < 0.55 or not expected:
            x = rng.randint(0, 99)
            q.enqueue(x)
            expected.append(x)
        else:
            assert q.dequeue() == expected.popleft()
        assert len(q) == len(expected)


def test_ring_wraps_around_before_growing():
```

```
    q = RingQueue(4)
    for x in range(4):
        q.enqueue(x)
    q.dequeue()
    q.dequeue()
    q.enqueue(4)  # goes into slot 0: the ring wraps
    q.enqueue(5)
    assert len(q._data) == 4  # no growth was needed
    assert [q.dequeue() for _ in range(4)] == [2, 3, 4, 5]
```

```
......                                                                   [100%]
6 passed in 0.01s
```

`@pytest.mark.parametrize` runs a test once for each value given, here once for each queue class, so both queues face the same 5,000 random operations against a `deque`. The last test checks the ring really wraps around: after two dequeues, two enqueues fit into the freed slots without growing.

Timing Four Queues

Fill a queue with n items, then empty it, four ways:

```
"""A queue of n items filled, then emptied: four implementations."""

from collections import deque

import bench
from ring_queue import RingQueue
from two_stack_queue import TwoStackQueue


def with_list(n):
    q = []
    for i in range(n):
        q.append(i)
    while q:
        q.pop(0)  # every remaining item shifts left


def with_deque(n):
    q = deque()
    for i in range(n):
        q.append(i)
    while q:
        q.popleft()

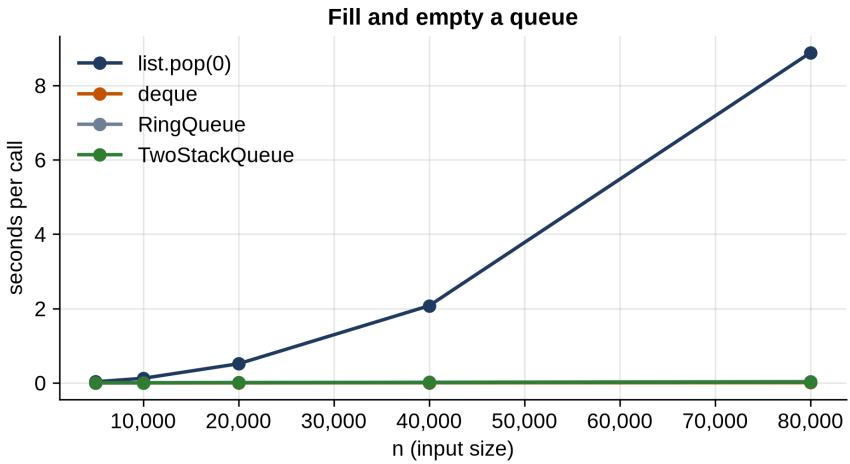

def with_class(make):
    def run(n):
        q = make()
        for i in range(n):
            q.enqueue(i)
        while len(q):
```

```
            q.dequeue()

    return run


sizes = [5_000, 10_000, 20_000, 40_000, 80_000]
results = {
    "list.pop(0)": bench.doubling(with_list, lambda n: (n,), sizes),
    "deque": bench.doubling(with_deque, lambda n: (n,), sizes),
    "RingQueue": bench.doubling(
        with_class(RingQueue), lambda n: (n,), sizes
    ),
    "TwoStackQueue": bench.doubling(
        with_class(TwoStackQueue), lambda n: (n,), sizes
    ),
}
bench.table(results)
bench.chart(results, "ch08_queues.png", "Fill and empty a queue")
```

```
         n     list.pop(0)           deque       RingQueue   TwoStackQueue
     5,000         29.9 ms          324 us         2.04 ms         1.17 ms
    10,000     123 ms x4.1     557 us x1.7    4.16 ms x2.0    2.35 ms x2.0
    20,000     514 ms x4.2    1.09 ms x2.0    8.51 ms x2.0    4.81 ms x2.0
    40,000     2.07 s x4.0    2.22 ms x2.0    17.3 ms x2.0    9.87 ms x2.1
    80,000     8.88 s x4.3    5.28 ms x2.4    35.2 ms x2.0    20.1 ms x2.0
```



Using list.pop(0) as a queue grows quadratically; the other three stay linear.

The `list.pop(0)` column quadruples at every row: O(n^2). At 80,000 items it takes about 9 seconds. The other three double: O(n). `deque` empties the same 80,000-item queue in about 5 milliseconds, well over a thousand times faster. Our two Python queues are slower than `deque` (Python code against C) but on

the same straight line, and the two-stack queue is faster than the ring buffer, because its work is mostly `list.append` and `list.pop`, which run in C.

> **Tip:** In Python, use `collections.deque` for any queue: `append` to join, `popleft` to leave. Reach for `queue.Queue` only when several threads share the queue, since it adds locking.

Key Takeaways

- A stack is last in, first out; a Python list is an excellent stack when you push and pop at the end.
- A queue is first in, first out; `list.pop(0)` makes a list a quadratic queue.
- A ring buffer stores a queue in fixed storage that wraps around, so nothing shifts.
- Two stacks make a queue with O(1) amortized operations, because each item moves between them once.
- A monotonic stack answers "next greater element" questions in O(n).
- In Python, `collections.deque` is the queue to use.

Exercises

1. **Min stack.** Make a stack that can also report its smallest item in O(1), at any moment. (Hint: a second stack.)
2. **Reverse Polish notation.** Evaluate expressions like `"3 4 + 2 *"`, where each operator follows its two operands, with a stack.
3. **Next greater element.** For each number in a list, find the first larger number to its right, or -1, in O(n).

CHAPTER 9 · INTERMEDIATE

Hash Tables

The dictionary is Python's most important data structure. Every module's global variables live in one, so do most objects' attributes, and so do the keyword arguments of every call. It finds a key among millions in about a tenth of a microsecond. This chapter builds the structure behind it, a **hash table**, in both of its classic forms, measures how full a table can get before it slows down, and stages the attack that made Python change how it hashes strings.

The Idea

A hash table keeps its entries in an array of **buckets** (or slots). To store a key, compute `hash(key)`, a number, and reduce it to a bucket number with `% len(buckets)`. To find the key later, do the same arithmetic and look in that bucket. No searching through the other keys.

Two different keys can land in the same bucket: that's a **collision**, and it's unavoidable, because there are more possible keys than buckets. A hash table is defined by how it handles collisions. The two classic answers are **separate chaining**, where each bucket holds a small list of all the keys that landed there, and **open addressing**, where a key that finds its slot taken tries other slots in the same table.

Separate Chaining

```
"""A hash map with separate chaining.

Each bucket is a small list of [key, value] pairs. A key's hash picks
its bucket; the bucket is searched with ==. When the average bucket
holds more than 0.75 pairs, the map doubles its buckets and rehashes.
"""


class ChainedMap:
    MAX_LOAD = 0.75

    def __init__(self, buckets=8):
        self._buckets = [[] for _ in range(buckets)]
        self._n = 0

    def __len__(self):
        return self._n

    def _bucket(self, key):
        return self._buckets[hash(key) % len(self._buckets)]

    def __getitem__(self, key):
        for pair in self._bucket(key):
            if pair[0] == key:
                return pair[1]
        raise KeyError(key)

    def __setitem__(self, key, value):
        bucket = self._bucket(key)
        for pair in bucket:
            if pair[0] == key:
                pair[1] = value  # an existing key: replace its value
                return
        bucket.append([key, value])
        self._n += 1
        if self._n > self.MAX_LOAD * len(self._buckets):
            self._rehash(2 * len(self._buckets))

    def __delitem__(self, key):
        bucket = self._bucket(key)
        for i, pair in enumerate(bucket):
            if pair[0] == key:
                bucket.pop(i)
                self._n -= 1
                return
        raise KeyError(key)

    def __contains__(self, key):
        return any(pair[0] == key for pair in self._bucket(key))

    def __iter__(self):
        for bucket in self._buckets:
            for key, _ in bucket:
                yield key

    def _rehash(self, size):
        """Move every pair into `size` new buckets."""
```

```
        pairs = [pair for bucket in self._buckets for pair in bucket]
        self._buckets = [[] for _ in range(size)]
        for key, value in pairs:
            self._bucket(key).append([key, value])
```

Each bucket is a list of `[key, value]` pairs. A lookup computes the bucket, then compares the key with each pair there using `==`. With a good hash function, keys spread evenly, so a bucket holds about `n / buckets` pairs on average. That number is the **load factor**. Keep it below a constant, here 0.75, and each bucket holds less than one pair on average, so every operation is O(1) on average.

To keep the load factor down, `__setitem__` doubles the number of buckets when it's exceeded, and `_rehash` moves every pair into the bucket its hash chooses in the bigger table. Each pair moves to a new bucket, because `hash % size` changes when the size does. As with the dynamic array, doubling makes this O(1) amortized.

The class defines the special methods `__getitem__`, `__setitem__`, `__delitem__`, `__contains__`, `__len__` and `__iter__`, so it works with Python's normal syntax: `m[key]`, `m[key] = value`, `del m[key]`, `key in m`, `len(m)` and `for key in m`. Here's a look inside:

```
"""Look inside: which bucket each key lands in."""

from chained_map import ChainedMap

m = ChainedMap(buckets=4)
for key in [10, 7, 14]:  # a fourth key would make it resize
    m[key] = f"value {key}"
    print(f"hash({key}) % 4 = {hash(key) % 4}")
for k, bucket in enumerate(m._buckets):
    print("bucket", k, bucket)
```

```
hash(10) % 4 = 2
hash(7) % 4 = 3
hash(14) % 4 = 2
bucket 0 []
bucket 1 []
bucket 2 [[10, 'value 10'], [14, 'value 14']]
bucket 3 [[7, 'value 7']]
```

10 and 14 both land in bucket 2 and share its list; 7 has bucket 3 to itself. (Small integers hash to themselves, which makes them easy to follow. Strings hash to large, scrambled numbers.)

> **Real-World Use:** Java's `HashMap` uses separate chaining with the same 0.75 load factor as `ChainedMap`, plus a defense: when one bucket collects 8 or more entries, it turns that bucket's list into a balanced search tree (Chapter 17), so even a flood of colliding keys costs O(log n) per lookup rather than O(n).

Open Addressing

Open addressing stores every pair directly in one table. A key starts at slot `hash(key) % size`; if another key is there, it tries the next slot, then the next, wrapping around at the end: **linear probing**. A search follows the same path until it finds the key or an empty slot.

```
"""A hash map with open addressing and linear probing.

All pairs live in one table. A key starts at slot hash(key) % size and,
if that slot is taken by another key, tries the next slot, and the next,
wrapping around. A deleted pair leaves a tombstone, so searches for keys
stored further along don't stop early. The table stays at most two
thirds full, counting tombstones, so there is always an empty slot.
"""

EMPTY = None
TOMBSTONE = object()  # marks a slot whose pair was deleted


class OpenMap:
    def __init__(self, size=8):
        self._slots = [EMPTY] * size  # each slot: EMPTY, TOMBSTONE or pair
        self._n = 0  # live pairs
        self._used = 0  # live pairs plus tombstones
        self.probes = 0  # slots examined, for experiments

    def __len__(self):
        return self._n

    def _find(self, key):
        """(slot index, found?) for key. If it isn't there, the index
        is where it should go: the first tombstone or empty slot seen."""
        size = len(self._slots)
        i = hash(key) % size
        free = None
        while True:
            self.probes += 1
            slot = self._slots[i]
            if slot is EMPTY:
                return (i if free is None else free), False
            if slot is TOMBSTONE:
                if free is None:
                    free = i
            elif slot[0] == key:
                return i, True
```

```
            i = (i + 1) % size  # linear probing: try the next slot

    def __getitem__(self, key):
        i, found = self._find(key)
        if not found:
            raise KeyError(key)
        return self._slots[i][1]

    def __setitem__(self, key, value):
        i, found = self._find(key)
        if found:
            self._slots[i] = (key, value)
            return
        if self._slots[i] is EMPTY:
            self._used += 1  # a tombstone reused doesn't add to the count
        self._slots[i] = (key, value)
        self._n += 1
        if self._used * 3 > len(self._slots) * 2:
            self._rehash()

    def __delitem__(self, key):
        i, found = self._find(key)
        if not found:
            raise KeyError(key)
        self._slots[i] = TOMBSTONE
        self._n -= 1

    def __contains__(self, key):
        return self._find(key)[1]

    def __iter__(self):
        for slot in self._slots:
            if slot is not EMPTY and slot is not TOMBSTONE:
                yield slot[0]

    def _rehash(self):
        """A fresh table for the live pairs, dropping the tombstones."""
        pairs = [
            s for s in self._slots if s is not EMPTY and s is not TOMBSTONE
        ]
        self._slots = [EMPTY] * max(8, 4 * len(pairs))
        self._n = self._used = 0
        for key, value in pairs:
            self[key] = value
```

Deleting needs care. If you simply emptied a deleted key's slot, a search for a key stored *after* it on the same probe path would stop at the gap and wrongly report "not found". So a deleted pair leaves a **tombstone**: a marker that says "keep looking past me". `_find` remembers the first tombstone it passes, so a new key can reuse that slot. Exercise 3 shows what goes wrong without tombstones.

The table rehashes when two thirds of its slots are used, counting tombstones, so there's always plenty of empty space to end searches quickly. Rehashing also clears out the tombstones.

Testing Both

```
"""Tests for both hash maps, with random operations checked against
Python's dict."""

import random

import pytest

from chained_map import ChainedMap
from open_map import OpenMap

MAPS = [ChainedMap, OpenMap]


@pytest.mark.parametrize("make", MAPS)
def test_set_get_replace_delete(make):
    m = make()
    m["apple"] = 1
    m["pear"] = 2
    m["apple"] = 3  # replace
    assert m["apple"] == 3 and len(m) == 2 and "pear" in m
    del m["pear"]
    assert "pear" not in m and len(m) == 1
    with pytest.raises(KeyError):
        m["pear"]
    with pytest.raises(KeyError):
        del m["pear"]


@pytest.mark.parametrize("make", MAPS)
def test_keys_that_collide(make):
    m = make()
    keys = [0, 8, 16, 24, 32]  # all in slot 0 of an 8-slot table
    for k in keys:
        m[k] = str(k)
    del m[8]  # the open map leaves a tombstone between the others
    assert [m[k] for k in (0, 16, 24, 32)] == ["0", "16", "24", "32"]
    assert 8 not in m


@pytest.mark.parametrize("make", MAPS)
def test_random_operations_match_dict(make):
    rng = random.Random(2026)
    m, expected = make(), {}
    for _ in range(20000):
        key = rng.randint(0, 300)
        op = rng.random()
        if op < 0.5:
            m[key] = expected[key] = rng.randint(0, 999)
        elif op < 0.8:
            if key in expected:
```

```
                del m[key]
                del expected[key]
            else:
                with pytest.raises(KeyError):
                    del m[key]
        else:
            assert (key in m) == (key in expected)
            if key in expected:
                assert m[key] == expected[key]
        assert len(m) == len(expected)
    assert sorted(m) == sorted(expected)
```

```
......                                                                   [100%]
6 passed in 0.01s
```

The collision test uses keys 0, 8, 16, 24 and 32, which all start at slot 0 of an 8-slot table, then deletes one from the middle of the probe chain. The random test runs 20,000 operations on a small key range against a `dict`, so the same keys are inserted, deleted and re-inserted over and over: exactly the situation where tombstone bugs hide.

How Full Is Too Full?

For linear probing, there's a classic formula for the average number of slots a successful search examines: about (1 + 1/(1 - a)) / 2, where a is the load factor. As the table fills, 1 - a shrinks toward zero and the searches get longer, slowly at first, then fast. Let's measure it against the formula with 100,000 slots:

```
"""How many slots does a search look at, as the table fills up?

Linear probing's average for a successful search is about
(1 + 1 / (1 - a)) / 2, where a is the load: the fraction of slots used.
"""

import random

from open_map import OpenMap

rng = random.Random(2026)
size = 100_000
print(f"{'load':>5} {'measured':>9} {'formula':>8}")
for load in [0.1, 0.3, 0.5, 0.6, 0.7, 0.8, 0.9]:
    m = OpenMap()
    m._slots = [None] * size  # a fixed size, so we control the load
    m._rehash = lambda: None  # and switch resizing off
    keys = [rng.getrandbits(60) for _ in range(int(load * size))]
    for k in keys:
        m[k] = True
```

```
    m.probes = 0
    for k in keys:
        m[k]  # look up every key once
    measured = m.probes / len(keys)
    formula = (1 + 1 / (1 - load)) / 2
    print(f"{load:>5} {measured:>9.2f} {formula:>8.2f}")
```

```
 load  measured  formula
  0.1      1.06     1.06
  0.3      1.21     1.21
  0.5      1.50     1.50
  0.6      1.77     1.75
  0.7      2.14     2.17
  0.8      2.98     3.00
  0.9      5.34     5.50
```

The measurements match the formula almost exactly. Half full, a search looks at 1.5 slots on average. Two thirds full, about 2. At 90%, over 5, and climbing steeply toward the end. That's why real hash tables resize long before they're full, trading memory for speed.

> **Real-World Use:** CPython's dict uses open addressing, not chaining, and resizes when two thirds full, exactly like `OpenMap`. Two refinements make it faster than linear probing. Its probe sequence mixes in more bits of the hash (a "perturbation") so keys with similar hashes don't follow the same path. And since Python 3.6 it's **compact**: the table holds small indexes into a separate array of entries kept in insertion order, which is why dictionaries remember the order of their keys and use less memory.

Timing Lookups

```
"""Lookups in our two maps and in Python's dict, as they grow."""

import bench
from chained_map import ChainedMap
from open_map import OpenMap


def filled(make):
    def make_input(n):
        m = make()
        for i in range(n):
            m[i * 7919] = i  # spread-out integer keys
        return m, (n // 2) * 7919

    return make_input
```

```
def lookup(m, key):
    return m[key]


sizes = [1_000, 10_000, 100_000, 1_000_000]
results = {
    "dict": bench.doubling(lookup, filled(dict), sizes),
    "ChainedMap": bench.doubling(lookup, filled(ChainedMap), sizes),
    "OpenMap": bench.doubling(lookup, filled(OpenMap), sizes),
}
bench.table(results)
```

```
         n            dict      ChainedMap         OpenMap
     1,000          108 ns          219 ns          328 ns
    10,000     107 ns x1.0     240 ns x1.1     339 ns x1.0
   100,000     108 ns x1.0     248 ns x1.0     333 ns x1.0
 1,000,000     114 ns x1.1     266 ns x1.1     362 ns x1.1
```

All three are flat from a thousand to a million keys: O(1), as promised. Python's `dict` takes about 110 nanoseconds per lookup, our maps two or three times that. A lookup in our maps runs several lines of Python; `dict`'s runs in C.

When Hashing Goes Wrong

Everything above assumes the hash function spreads keys out. What if it doesn't? This benchmark fills a dictionary with keys whose `__hash__` returns 42 for every key:

```
"""A hash flood: keys that all hash to the same value turn a dict's
constant-time operations into linear ones, and filling it quadratic."""

import bench


class Key:
    def __init__(self, x, flood):
        self.x = x
        self.flood = flood

    def __hash__(self):
        return 42 if self.flood else hash(self.x)

    def __eq__(self, other):
        return self.x == other.x


def fill(n, flood):
    d = {}
    for i in range(n):
```

```
        d[Key(i, flood)] = i


sizes = [500, 1_000, 2_000, 4_000]
results = {
    "good hashes": bench.doubling(fill, lambda n: (n, False), sizes),
    "all hash to 42": bench.doubling(fill, lambda n: (n, True), sizes),
}
bench.table(results)
bench.chart(results, "ch09_flood.png", "Filling a dict: a hash flood")
```

```
         n     good hashes  all hash to 42
       500         81.3 us         6.53 ms
     1,000     162 us x2.0    25.7 ms x3.9
     2,000     344 us x2.1     105 ms x4.1
     4,000     727 us x2.1     450 ms x4.3
```

With good hashes, filling the dictionary takes linear time. When every key has the same hash, every insertion probes past all the keys already there, comparing with each: O(n) per insertion, $O(n^2)$ to fill. The ratio column quadruples, and 4,000 keys take hundreds of times longer.

This isn't just a curiosity. If a web server puts request parameters into a dictionary, and an attacker can choose parameter names that collide, a single request full of crafted names can keep a processor busy for a long time. This attack, called **hash flooding**, was demonstrated at the Chaos Communication Congress in December 2011 against PHP, Python, Java, Ruby and other platforms at once.

> **Real-World Use:** Python's defense is **hash randomization**: since Python 3.3, `hash()` of a `str` or `bytes` mixes in a random value chosen when the interpreter starts, so attackers can't predict which strings will collide. That's why `hash("apple")` gives a different number every time you start Python, and why this chapter's demos use integer keys, whose hashes don't change. Setting the environment variable `PYTHONHASHSEED` fixes the seed when you need repeatable runs.

Making Your Own Classes Hashable

To use your own objects as dictionary keys, define `__eq__` and `__hash__` consistently: objects that are equal must have equal hashes. The easy, correct way is to hash a tuple of the same fields that `__eq__` compares, which

Exercise 1 does. Python's `dataclass` can generate both for you with `@dataclass(frozen=True)`.

> **Common Mistake:** Defining `__eq__` without `__hash__`. Python then sets `__hash__` to `None`, making the objects unhashable, which is a safe default. Worse is a `__hash__` that uses fields `__eq__` ignores, or fields that change after the object is stored: the dictionary then looks in the wrong bucket and can't find a key that's right there.

Key Takeaways

- A hash table turns each key into a bucket number with `hash(key) % size`, so it can find keys without searching.
- Collisions are handled by chaining (a list per bucket) or by open addressing (probe other slots in one table).
- Keep the load factor low by resizing (doubling) when it passes a threshold: 0.75 for `ChainedMap` and Java, two thirds for `OpenMap` and CPython.
- Open addressing needs tombstones for deletion; searches must skip past them.
- Average search length for linear probing grows like 1/(1 - load): fine at half full, bad near full. The measurements match the theory.
- Bad hashes make every operation O(n); Python randomizes string hashes to stop attackers from causing that deliberately.

Exercises

1. **A hashable Point.** Write a `Point` class with `x` and `y` that can be a dictionary key: two `Point` objects with the same coordinates must find the same entry.
2. **Word counts.** Count the words in a paragraph using your `ChainedMap`, then print the three most common.
3. **Why tombstones?** Subclass `OpenMap` so that deleting empties the slot instead of leaving a tombstone, and show a lookup that wrongly fails.

PART III

Searching and Sorting

CHAPTER 10 · INTERMEDIATE

Binary Search

Finding a value in an unsorted list means checking items one by one: O(n). If the list is sorted, you can do something much better. Look at the middle item. If it's too small, the target can only be in the right half; if too big, only in the left half. Either way, one comparison throws away half the list. Repeat, and a million items take twenty steps.

That's **binary search**, and it's both one of the simplest algorithms and one of the easiest to get subtly wrong. This chapter builds it carefully, with an invariant that makes it correct by construction, then its two more useful cousins, the standard library's `bisect`, and a powerful generalization: binary search on the answer.

The Algorithm and Its Invariant

```
def binary_search(items, target):
    """Index of target in sorted items, or -1.

    Invariant: if target is anywhere, it is in items[lo:hi]. Each step
    halves that range, so it takes about log2(n) steps.
    """
    lo, hi = 0, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if items[mid] == target:
            return mid
        if items[mid] < target:
            lo = mid + 1  # target can only be to the right of mid
        else:
            hi = mid  # target can only be to the left of mid
    return -1
```

The key to getting binary search right is to decide exactly what `lo` and `hi` mean, and keep that true at every step. Here the meaning, the **invariant**, is: *if the target is anywhere, it's in* `items[lo:hi]`. Note the half-open range: `lo` is included, `hi` is not.

- At the start, `lo = 0` and `hi = len(items)`, so the range is the whole list. True.
- If `items[mid]` is too small, the target, if present, is after `mid`, so `lo = mid + 1` keeps the invariant. If too big, it's before `mid`, so `hi = mid` (not `mid - 1`, because `hi` is excluded).
- The loop runs while the range `lo:hi` isn't empty. When it's empty, the invariant says the target isn't anywhere: return -1.

Each step at least halves the range, so the loop runs at most about $\log_2 n + 1$ times. Here's how many steps that is in practice. A `range` object can be indexed like a list without storing its items, so we can search a billion numbers for free:

```
"""How many steps does binary search take? range() can be indexed
like a list without storing it, so we can search a billion numbers."""


def steps_to_find(items, target):
    lo, hi, steps = 0, len(items), 0
    while lo < hi:
        steps += 1
        mid = (lo + hi) // 2
        if items[mid] == target:
            break
        if items[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return steps


for n in [1_000, 1_000_000, 1_000_000_000]:
    worst = max(steps_to_find(range(n), t) for t in (-1, 0, n - 1, n))
    print(f"{n:>13,} items: at most {worst} steps")
```

```
        1,000 items: at most 10 steps
    1,000,000 items: at most 20 steps
1,000,000,000 items: at most 30 steps
```

Thirty steps for a billion items. A linear search would need up to a billion.

> **Interesting Fact:** In 2006, Joshua Bloch reported that the binary search he had written for Java's standard library, and the one in Jon Bentley's classic book *Programming Pearls*, proved correct and tested there, shared a bug that went unnoticed for years (two decades, in the book's case): computing `(low + high) / 2` overflows when `low + high` exceeds the largest 32-bit integer, which can happen with arrays of about a billion elements. Python's integers never overflow, so `(lo + hi) // 2` is safe here. In languages with fixed-size integers, write `lo + (hi - lo) / 2`.

Lower Bound and Upper Bound

"Is it there?" is often the less useful question. With duplicates, you may want the first or the last occurrence; for a missing value, where it would go. Two variants answer these:

```
def lower_bound(items, target):
    """The first index whose item is >= target (len(items) if none).
    It is where target would be inserted to keep the order."""
    lo, hi = 0, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if items[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo


def upper_bound(items, target):
    """The first index whose item is > target (len(items) if none)."""
    lo, hi = 0, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if items[mid] <= target:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

`lower_bound` returns the first position whose item is at least the target: where you'd insert the target to keep the list sorted, before any equal items. `upper_bound` returns the first position whose item is *greater*, so after any equal items. They differ only in `<` against `<=`. Together they're very powerful: `upper_bound(x) - lower_bound(x)` counts the copies of x, and the items in a range of values sit between `lower_bound(low)` and `upper_bound(high)`.

Neither variant stops early when it finds the target; both always narrow the range down to a single position.

Testing Against bisect

```
"""Tests for binary search, checked against the bisect module."""

import bisect
import random

from binary_search import binary_search, lower_bound, upper_bound


def test_found_and_not_found():
    items = [2, 3, 5, 7, 11, 13]
    for i, x in enumerate(items):
        assert binary_search(items, x) == i
    for x in [0, 4, 12, 99]:
        assert binary_search(items, x) == -1
    assert binary_search([], 1) == -1


def test_bounds_with_duplicates():
    items = [1, 2, 2, 2, 5]
    assert lower_bound(items, 2) == 1
    assert upper_bound(items, 2) == 4
    assert upper_bound(items, 2) - lower_bound(items, 2) == 3  # count
    assert lower_bound(items, 6) == 5 and upper_bound(items, 0) == 0


def test_random_lists_match_bisect():
    rng = random.Random(2026)
    for _ in range(2000):
        n = rng.randint(0, 30)
        items = sorted(rng.randint(0, 20) for _ in range(n))
        target = rng.randint(-1, 21)
        assert lower_bound(items, target) == bisect.bisect_left(
            items, target
        )
        assert upper_bound(items, target) == bisect.bisect_right(
            items, target
        )
        found = binary_search(items, target)
        assert (found == -1) == (target not in items)
        if found != -1:
            assert items[found] == target
```

```
...                                                                      [100%]
3 passed in 0.01s
```

The random test builds 2,000 sorted lists of random lengths, full of duplicates (values from 0 to 20), and checks the bounds against the standard library's

`bisect_left` and `bisect_right`, which are exactly the same functions. Off-by-one errors, the usual binary-search bug, can't survive 2,000 random cases with empty lists, single items and long runs of duplicates.

The bisect Module

In real code, use `bisect`: it's written in C and well tested.

```
"""The bisect module: binary search in the standard library."""

import bisect

# Grades: the score where each grade starts, and the grade names
cutoffs = [60, 70, 80, 90]
grades = "FDCBA"
for score in [55, 60, 79, 90, 100]:
    print(score, grades[bisect.bisect_right(cutoffs, score)])

# Keep a list sorted as items arrive
scores = []
for s in [70, 40, 90, 55]:
    bisect.insort(scores, s)
print(scores)
```

```
55 F
60 D
79 C
90 A
100 A
[40, 55, 70, 90]
```

The grade lookup is a neat trick: `bisect_right(cutoffs, score)` counts how many cutoffs the score has reached, which indexes straight into the string of grades. `insort` inserts into a sorted list at the right place. Finding the position is O(log n), but inserting still shifts later items, so it's O(n) overall. For a list that changes often, Chapter 17's balanced tree is the better structure.

Binary Search on the Answer

Here's a problem that doesn't look like searching at all. Packages of given weights must be shipped, in order, within a number of days; each day a truck carries a run of consecutive packages up to its capacity. What is the smallest capacity that works?

```
"""Binary search on the answer: when you can check a guess quickly and
every guess above the answer passes, search the range of answers.

Problem: packages with these weights must ship in order within `days`
days. What is the smallest daily capacity that works?
"""


def days_needed(weights, capacity):
    days, load = 1, 0
    for w in weights:
        if load + w > capacity:  # this package starts a new day
            days, load = days + 1, 0
        load += w
    return days


def smallest_capacity(weights, days):
    lo, hi = max(weights), sum(weights)  # the answer is in lo..hi
    while lo < hi:
        mid = (lo + hi) // 2
        if days_needed(weights, mid) <= days:
            hi = mid  # mid works: maybe something smaller does too
        else:
            lo = mid + 1  # mid fails: the answer is bigger
    return lo


weights = [3, 2, 2, 4, 1, 4, 5, 6, 2, 3]
for days in [1, 3, 5, 10]:
    print(f"{days:>2} days: capacity {smallest_capacity(weights, days)}")
```

```
 1 days: capacity 32
 3 days: capacity 11
 5 days: capacity 9
10 days: capacity 6
```

The trick is to search the space of *answers*. The capacity must be at least the heaviest package and at most the total weight. And the question "does capacity c work?" is easy to check by simulation, `days_needed`, and has a special shape: if c works, every bigger capacity works too. Whenever a yes-or-no question flips from "no" to "yes" exactly once as the answer grows, binary search finds the flip point, here in about $\log_2$(total) checks of O(n) each. This pattern, **binary search on the answer**, solves a whole family of interview problems: minimum speed to finish on time, maximum minimum distance, smallest largest sum when splitting an array.

> **Real-World Use:** `git bisect` uses binary search to find the commit that introduced a bug. You mark one commit as good and a later one as bad; Git checks out the commit in the middle, you test it and say good or bad, and it halves the range again. A bug introduced somewhere in 1,000 commits is found in about 10 tests. It's binary search on the answer, where the answer is a commit and "is the bug present?" flips once.

Timing

```
"""Searching a sorted list: scanning, our binary search, and bisect."""

import bisect

import bench
from binary_search import binary_search


def make(n):
    return list(range(n)), n - 1  # the last item: the scan's worst case

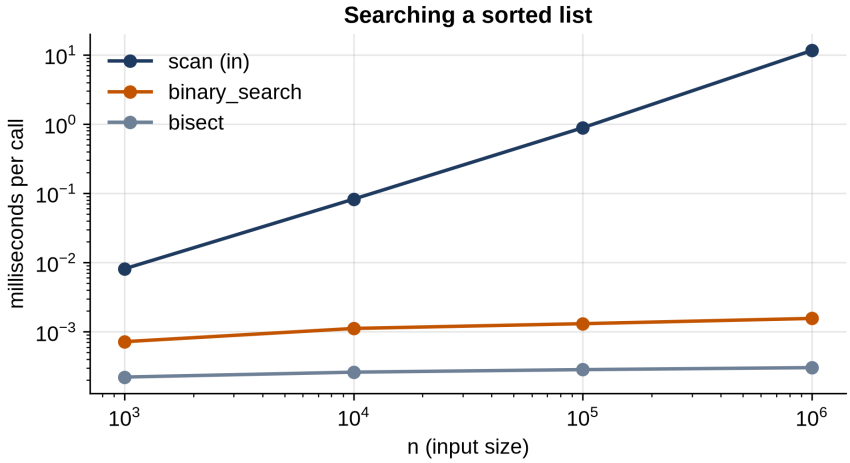

def scan(items, target):
    return target in items


def with_bisect(items, target):
    i = bisect.bisect_left(items, target)
    return i < len(items) and items[i] == target


sizes = [1_000, 10_000, 100_000, 1_000_000]
results = {
    "scan (in)": bench.doubling(scan, make, sizes),
    "binary_search": bench.doubling(binary_search, make, sizes),
    "bisect": bench.doubling(with_bisect, make, sizes),
}
bench.table(results)
bench.chart(results, "ch10_search.png", "Searching a sorted list", log=True)
```

```
         n       scan (in)   binary_search          bisect
     1,000         8.11 us          718 ns          221 ns
    10,000   82.3 us x10.2    1.11 us x1.6     260 ns x1.2
   100,000    885 us x10.7    1.31 us x1.2     283 ns x1.1
 1,000,000   11.6 ms x13.1    1.56 us x1.2     303 ns x1.1
```

The scan grows by about 10 times for each 10 times more items: O(n). Binary search grows by a fraction of a microsecond for each 10 times: O(log n), adding about 3.3 steps ($\log_2$ 10) each time. At a million items, our binary



On log-log axes, the scan rises steeply; binary search is nearly flat.

search is thousands of times faster than the scan, and `bisect`, in C, is about five times faster again.

> **Tip:** Sorting costs O(n log n), so it only pays for itself if you search many times. To search once, scan. To search a thousand times, sort once and binary search, or build a set.

Key Takeaways

- Binary search halves the remaining range with each comparison: O(log n), about 30 steps for a billion items.
- State the invariant ("the target is in `items[lo:hi]`") and keep it true; that's how to avoid off-by-one errors.
- `lower_bound` and `upper_bound` find the first position not less than, and greater than, a value; they count duplicates and find ranges.
- Use the `bisect` module in real code.
- If a yes-or-no question about a number flips once as the number grows, binary search on the answer finds the smallest number that works.

Exercises

1. **First and last.** Return the first and last positions of a target in a sorted list, or `[-1, -1]`, in O(log n).
2. **Rotated list.** A sorted list was rotated, like `[4, 5, 6, 7, 0, 1, 2]`. Find a target in O(log n). (Hint: one half around `mid` is always in order.)
3. **Integer square root.** Compute the largest r with $r \times r \le n$, by binary search on the answer, and check it against `math.isqrt` for very large n.

CHAPTER 11 · INTERMEDIATE

Simple Sorts

Sorting is the most studied problem in computer science, and for good reason: sorted data makes searching, merging, finding duplicates and finding ranges easy, and a surprising number of algorithms start by sorting. Python sorts with `sorted()` and `list.sort()`, and you should use them. But the classic sorting algorithms are the best possible training ground for thinking about algorithms: simple to state, easy to test, and different enough in their behavior to teach real lessons.

This chapter covers three simple sorts, all $O(n^2)$: selection sort, insertion sort and bubble sort. Chapter 12 covers the fast ones. The simple ones are worth knowing, because one of them, insertion sort, turns out to be excellent on small or nearly sorted lists, and it's part of how Python's own sort works.

Three Simple Sorts

```
"""Three simple sorts. Each sorts the list in place, in quadratic time."""


def selection_sort(items):
    """Find the smallest remaining item and swap it into place."""
    n = len(items)
    for i in range(n - 1):
        smallest = i
        for j in range(i + 1, n):
            if items[j] < items[smallest]:
                smallest = j
        items[i], items[smallest] = items[smallest], items[i]


def insertion_sort(items):
    """Grow a sorted part on the left, inserting each new item into it."""
```

```
    for i in range(1, len(items)):
        x = items[i]
        j = i - 1
        while j >= 0 and items[j] > x:  # shift bigger items right
            items[j + 1] = items[j]
            j -= 1
        items[j + 1] = x


def bubble_sort(items):
    """Swap neighbors that are out of order; stop when a pass swaps none."""
    n = len(items)
    for end in range(n - 1, 0, -1):
        swapped = False
        for j in range(end):
            if items[j] > items[j + 1]:
                items[j], items[j + 1] = items[j + 1], items[j]
                swapped = True
        if not swapped:
            return
```

All three sort the list **in place**: they rearrange the list they're given rather than returning a new one, using O(1) extra memory.

- **Selection sort** finds the smallest remaining item and swaps it to the front of the unsorted part, then repeats with the rest. It always makes the same number of comparisons, n(n - 1)/2, whatever the input looks like.
- **Insertion sort** grows a sorted part on the left. Each new item is shifted left past every larger item, like sorting a hand of cards. On a list that's already sorted, each new item is already in place after one comparison, so it does only n - 1 comparisons.
- **Bubble sort** compares neighbors and swaps them if they're out of order, so each pass "bubbles" the largest remaining item to the end. Stopping when a pass makes no swaps lets it finish early on sorted input.

Testing Sorts

Sorts are easy to test: the answer must equal `sorted()`'s.

```
"""Every sort must agree with sorted() on many random lists."""

import random

import pytest

from simple_sorts import bubble_sort, insertion_sort, selection_sort
```

```
@pytest.mark.parametrize(
    "sort", [selection_sort, insertion_sort, bubble_sort]
)
def test_matches_sorted(sort):
    rng = random.Random(2026)
    cases = [
        [],
        [1],
        [2, 1],
        [3, 3, 3],
        list(range(10)),
        list(range(9, -1, -1)),
    ]
    cases += [
        [rng.randint(-50, 50) for _ in range(rng.randint(0, 60))]
        for _ in range(300)
    ]
    for case in cases:
        items = list(case)
        sort(items)
        assert items == sorted(case)
```

```
...                                                                  [100%]
3 passed in 0.01s
```

The hand-picked cases cover the empty list, one item, two items, all items equal, and lists already sorted and reversed, which are the classic places for sorting bugs. Then 300 random lists of random lengths, with values that repeat often.

Counting the Work

How much work does each sort do, and does the input's arrangement matter? This program counts comparisons with a class whose comparison methods count each call, on 1,000 items arranged three ways:

```
"""Count the comparisons each sort makes on 1,000 items, arranged
three ways. Insertion sort adapts: nearly sorted input is cheap."""

import random

from simple_sorts import bubble_sort, insertion_sort, selection_sort


class Counted:
    """A number that counts every comparison made with it."""

    comparisons = 0

    def __init__(self, x):
        self.x = x
```

```
    def __lt__(self, other):
        Counted.comparisons += 1
        return self.x < other.x

    def __gt__(self, other):
        Counted.comparisons += 1
        return self.x > other.x


rng = random.Random(2026)
n = 1000
nearly = list(range(n))
for _ in range(10):  # swap ten random pairs of neighbors
    i = rng.randrange(n - 1)
    nearly[i], nearly[i + 1] = nearly[i + 1], nearly[i]
inputs = {
    "random": rng.sample(range(n), n),
    "nearly sorted": nearly,
    "reversed": list(range(n, 0, -1)),
}
print(f"{'':>15}" + "".join(f"{name:>15}" for name in inputs))
for sort in [selection_sort, insertion_sort, bubble_sort]:
    row = f"{sort.__name__:>15}"
    for data in inputs.values():
        items = [Counted(x) for x in data]
        Counted.comparisons = 0
        sort(items)
        row += f"{Counted.comparisons:>15,}"
    print(row)
```

```
                       random  nearly sorted       reversed
 selection_sort        499,500        499,500        499,500
 insertion_sort        258,269          1,009        499,500
    bubble_sort        496,799          2,994        499,500
```

`Counted` defines `__lt__` and `__gt__`, the methods Python calls for `<` and `>`, so each comparison adds one to a counter. Read the table by rows:

- Selection sort makes exactly 499,500 comparisons on every input, the n(n - 1)/2 pairs. It can't take advantage of order already present.
- Insertion sort makes about half as many on random input (each item travels about halfway back on average), the maximum on reversed input, and only 1,009 on a nearly sorted list: practically linear.
- Bubble sort with early stopping is also fast on nearly sorted input, but on random input it's as bad as selection sort, and it makes many more swaps than either.

A sort that runs faster on input that's already partly in order is called **adaptive**. Insertion sort is the champion here: its work is proportional to n

plus the number of **inversions**, the pairs of items that are out of order. Exercise 1 uses that fact to count inversions.

Stability

When two items compare equal, a **stable** sort keeps them in the order they had before sorting. That matters when you sort records by one field:

```
"""A stable sort keeps items with equal keys in their original order."""


def insertion_sort_by(items, key):
    for i in range(1, len(items)):
        x = items[i]
        j = i - 1
        while j >= 0 and key(items[j]) > key(x):  # > not >=: stable
            items[j + 1] = items[j]
            j -= 1
        items[j + 1] = x


def selection_sort_by(items, key):
    for i in range(len(items) - 1):
        m = min(range(i, len(items)), key=lambda j: key(items[j]))
        items[i], items[m] = items[m], items[i]  # this swap can jump


people = [("Ada", 30), ("Bo", 25), ("Cy", 30), ("Di", 25)]
for sort in (insertion_sort_by, selection_sort_by):
    items = list(people)
    sort(items, key=lambda p: p[1])  # by age
    print(f"{sort.__name__:>18}: {items}")
```

```
 insertion_sort_by: [('Bo', 25), ('Di', 25), ('Ada', 30), ('Cy', 30)]
 selection_sort_by: [('Bo', 25), ('Di', 25), ('Cy', 30), ('Ada', 30)]
```

Sorting people by age, insertion sort keeps Ada before Cy (they were in that order) because it only moves an item past *strictly* greater items. Selection sort's long-distance swap moves Ada behind Cy. Insertion sort and bubble sort are stable; selection sort isn't.

Stability is what lets you sort by several keys: sort by the least important key first, then by the most important, and items that tie on the second sort keep the order of the first. Python's `sorted()` is guaranteed stable, which Exercise 2 relies on.

Timing

```
"""The three simple sorts on random lists, and insertion sort on a
nearly sorted list."""

import random

import bench
from simple_sorts import bubble_sort, insertion_sort, selection_sort

rng = random.Random(2026)

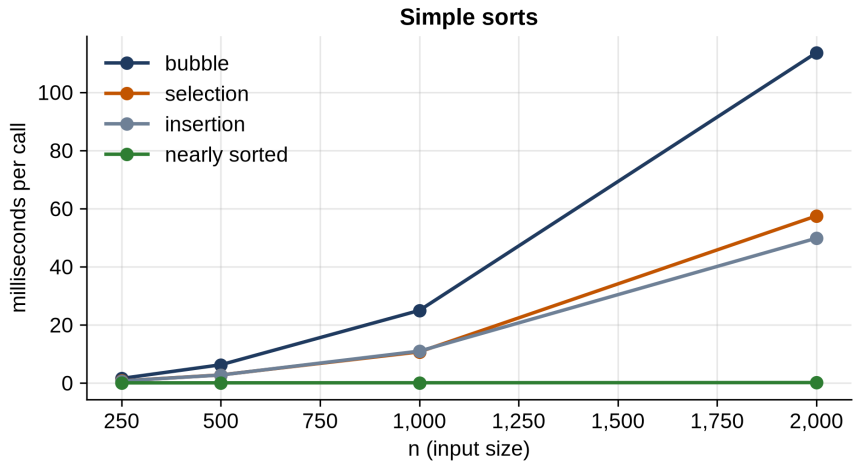

def shuffled(n):
    return (rng.sample(range(n), n),)


def nearly_sorted(n):
    items = list(range(n))
    for _ in range(n // 100):  # one neighbor swap per 100 items
        i = rng.randrange(n - 1)
        items[i], items[i + 1] = items[i + 1], items[i]
    return (items,)


sizes = [250, 500, 1_000, 2_000]
results = {
    "bubble": bench.doubling(bubble_sort, shuffled, sizes, fresh=True),
    "selection": bench.doubling(
        selection_sort, shuffled, sizes, fresh=True
    ),
    "insertion": bench.doubling(
        insertion_sort, shuffled, sizes, fresh=True
    ),
    "nearly sorted": bench.doubling(  # insertion sort
        insertion_sort, nearly_sorted, sizes, fresh=True
    ),
}
bench.table(results)
bench.chart(results, "ch11_simple.png", "Simple sorts")
```

```
         n          bubble       selection       insertion   nearly sorted
       250         1.54 ms          702 us          682 us         12.8 us
       500    6.24 ms x4.1    2.75 ms x3.9    2.71 ms x4.0    25.5 us x2.0
     1,000    24.9 ms x4.0    10.7 ms x3.9    10.9 ms x4.0    50.9 us x2.0
     2,000     114 ms x4.6    57.5 ms x5.4    49.8 ms x4.6     118 us x2.3
```

The `nearly sorted` column is insertion sort on lists with one neighbor swap per 100 items. On random lists all three quadruple when n doubles: $O(n^2)$. Bubble sort is the slowest by a factor of about two, because it does many more swaps. On nearly sorted input, insertion sort only doubles: linear, and at 2,000 items it's hundreds of times faster than on random data.



All three simple sorts grow quadratically on random lists; insertion sort on a nearly sorted list stays near the bottom.

Real-World Use: Many fast sorting implementations switch to insertion sort for small pieces, because for a few dozen items its low overhead beats their cleverness. Python's own sort, described in the next chapter, does: it first looks for runs of items already in order (reversing descending runs in place), and uses a binary insertion sort to extend short runs to a minimum length before merging them.

Interview Tip: You'll rarely be asked to write a simple sort, but you will be asked about their properties: which are stable, which are adaptive, which work in place. And "insertion sort on nearly sorted data is linear" is a fact worth remembering.

Key Takeaways

- Selection, insertion and bubble sort are all $O(n^2)$ on random input and sort in place with O(1) extra memory.
- Selection sort always does n(n - 1)/2 comparisons; insertion sort's work grows with the number of inversions, so it's linear on nearly sorted input.

- A stable sort keeps equal items in their original order; insertion and bubble sort are stable, selection sort isn't.
- Sort by several keys with a stable sort: least important key first.
- Test any sort against `sorted()` on random lists with many duplicates.

Exercises

1. **Count inversions.** Count the pairs `i < j` with `items[i] > items[j]` by counting the shifts insertion sort makes.
2. **Two keys.** Sort a list of (name, city, age) records by age, and by city within each age, using two calls to a stable sort. Check the result against a single sort with a tuple key.

CHAPTER 12 · INTERMEDIATE

Merge Sort and Quicksort

The simple sorts of Chapter 11 compare every item with many others, which is why they're O(n^2). The fast sorts use **divide and conquer**: split the problem into smaller pieces, solve each piece, and combine the answers. Splitting in half repeatedly gives about $\log_2 n$ levels, and if each level does O(n) work, the whole sort is O(n log n). For a million items that's about 20 million steps instead of 500 billion.

This chapter builds the two classic divide-and-conquer sorts, merge sort and quicksort. You'll see quicksort's famous worst case happen, fix it with a random pivot, make it fast on duplicates with a three-way partition, and then race everything against Python's `sorted()`.

Merging

Merge sort is built on one operation: merging two sorted lists into one.

```
def merge(left, right):
    """Merge two sorted lists into one sorted list."""
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:  # <= takes from the left on ties: stable
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out.extend(left[i:])  # one of these two is empty
    out.extend(right[j:])
    return out
```

Two positions, `i` and `j`, walk through the two lists. At each step the smaller of the two front items is the smallest left overall, so it goes next. When one list runs out, the rest of the other is already in order. Each item is looked at once: O(len(left) + len(right)). Taking from the left on ties (<=) keeps equal items in their original order, which makes merge sort stable.

Merge Sort

```
def merge_sort(items):
    """Return a new sorted list. Always about n log2 n comparisons."""
    if len(items) <= 1:
        return list(items)
    mid = len(items) // 2
    return merge(merge_sort(items[:mid]), merge_sort(items[mid:]))
```

Split the list in half, sort each half recursively, merge. A list of one item is already sorted. The recursion goes $\log_2 n$ levels deep, and the merges at each level together handle all n items, so merge sort is **O(n log n) on every input**: best, worst and average. The price is memory: it builds new lists as it goes, O(n) extra in all.

Quicksort

Quicksort divides differently. Pick one item, the **pivot**, and **partition** the list: move everything smaller than the pivot to its left and everything else to its right. The pivot is now in its final place. Sort the two sides recursively, and you're done; there's no merge step.

```
"""Quicksort: pick a pivot, put smaller items left of it and bigger
ones right of it, then sort each side. Sorts in place."""

import random
```

```
def partition(items, lo, hi):
    """Lomuto partition of items[lo..hi] around items[hi]. Returns the
    pivot's final index: everything left of it is smaller."""
    pivot = items[hi]
    store = lo
    for i in range(lo, hi):
        if items[i] < pivot:
            items[i], items[store] = items[store], items[i]
            store += 1
    items[store], items[hi] = items[hi], items[store]
```

```
    return store


def quicksort_naive(items, lo=0, hi=None):
    """Always uses the last item as the pivot."""
    if hi is None:
        hi = len(items) - 1
    if lo < hi:
        p = partition(items, lo, hi)
        quicksort_naive(items, lo, p - 1)
        quicksort_naive(items, p + 1, hi)
```

`partition` uses the **Lomuto** scheme: the pivot is the last item, `items[hi]`; `store` marks the end of the "smaller than pivot" region, and each smaller item found is swapped into it. Finally the pivot is swapped into place just after that region. `quicksort_naive` partitions and recurses on each side. It sorts in place, with no extra lists.

When the pivot splits the list roughly in half, quicksort is O(n log n). But nothing guarantees that. Watch what happens on a list that's already sorted:

```
"""Count quicksort's comparisons on random and on sorted input.
With the last item as pivot, sorted input splits off one item at a
time: n * (n - 1) / 2 comparisons, and n levels of recursion."""

import math
import random
import sys

import quicksort as q

calls = {"compared": 0}
original = q.partition


def counting_partition(items, lo, hi):
    calls["compared"] += hi - lo  # the loop compares each item once
    return original(items, lo, hi)


q.partition = counting_partition
sys.setrecursionlimit(5000)  # the naive version recurses n levels deep
random.seed(2026)
n = 2000
for name, sort in [
    ("naive", q.quicksort_naive),
    ("random pivot", q.quicksort),
]:
    for label, data in [
        ("random", random.sample(range(n), n)),
        ("sorted", list(range(n))),
    ]:
        calls["compared"] = 0
        sort(data)
        count = calls["compared"]
```

```
        print(f"{name:>12}, {label} input: {count:>9,} comparisons")
print(f"n log2 n = {n * math.log2(n):,.0f}")
print(f"n (n - 1) / 2 = {n * (n - 1) // 2:,}")
```

```
       naive, random input:    24,259 comparisons
       naive, sorted input: 1,999,000 comparisons
random pivot, random input:    25,072 comparisons
random pivot, sorted input:    23,604 comparisons
n log2 n = 21,932
n (n - 1) / 2 = 1,999,000
```

On random input, the naive quicksort makes about 24,000 comparisons for 2,000 items, close to $n \log_2 n$. On *sorted* input it makes 1,999,000, exactly $n(n - 1)/2$. The last item is always the largest, so each partition peels off just the pivot and leaves everything else on one side: n levels of recursion instead of $\log_2 n$, and $O(n^2)$ work. That's why the listing raises the recursion limit. A sorted list is not a rare input; it's one of the most common things anyone sorts.

`worst_case.py` counts comparisons by **monkey-patching**: it replaces the module's `partition` with a wrapper that counts, then calls the original. Because the sorting functions look up `partition` in the module each time they call it, they use the counting version. It's a handy trick for measuring code you don't want to edit.

Fixing the Worst Case

```
def quicksort(items, lo=0, hi=None):
    """A random pivot, and recursion only on the smaller side, so the
    stack stays about log2(n) deep even in an unlucky case."""
    if hi is None:
        hi = len(items) - 1
    while lo < hi:
        r = random.randint(lo, hi)
        items[r], items[hi] = items[hi], items[r]
        p = partition(items, lo, hi)
        if p - lo < hi - p:
            quicksort(items, lo, p - 1)
            lo = p + 1  # loop on the bigger side instead of recursing
        else:
            quicksort(items, p + 1, hi)
            hi = p - 1
```

Two changes. First, the pivot is chosen at **random** and swapped to the end. No particular input is bad anymore: a sorted list is as easy as any other, and the chance of a run of unlucky pivots on a large list is astronomically small. In

the table above, the random-pivot version makes about 24,000 comparisons on sorted input too.

Second, the function recurses only into the *smaller* side, and handles the larger side by looping. The smaller side is at most half the range, so the recursion is at most $\log_2 n$ deep, even if the pivots are unlucky.

Many Equal Items: Three-Way Partitioning

A list with lots of repeated values is another trap. Lomuto partitioning puts items equal to the pivot on one side, so a list of identical values degrades to $O(n^2)$ again. The fix is to partition three ways: smaller, equal and larger.

```
def quicksort3(items, lo=0, hi=None):
    """Three-way partition: items equal to the pivot are gathered in the
    middle and never touched again. Fast when there are many duplicates."""
    if hi is None:
        hi = len(items) - 1
    while lo < hi:
        pivot = items[random.randint(lo, hi)]
        lt, i, gt = (
            lo,
            lo,
            hi,
        )  # [lo,lt) < pivot, [lt,i) == pivot, (gt,hi] >
        while i <= gt:
            if items[i] < pivot:
                items[lt], items[i] = items[i], items[lt]
                lt, i = lt + 1, i + 1
            elif items[i] > pivot:
                items[i], items[gt] = items[gt], items[i]
                gt -= 1
            else:
                i += 1
        if lt - lo < hi - gt:
            quicksort3(items, lo, lt - 1)
            lo = gt + 1
        else:
            quicksort3(items, gt + 1, hi)
            hi = lt - 1
```

Three markers divide the range as it's scanned: `[lo, lt)` is less than the pivot, `[lt, i)` equals it, `(gt, hi]` is greater, and `[i, gt]` hasn't been looked at yet. Each step looks at `items[i]` and grows one region. When `i` passes `gt`, the equal items sit in the middle in their final positions and are never touched again. This is Dijkstra's **Dutch national flag** problem: sort three colors in one pass. Exercise 3 uses it directly.

Testing

```
"""Merge sort and the quicksorts must agree with sorted()."""

import random

import pytest

from merge_sort import merge, merge_sort
from quicksort import quicksort, quicksort3, quicksort_naive


def cases():
    rng = random.Random(2026)
    yield from (
        [],
        [1],
        [2, 1],
        [5] * 20,
        list(range(50)),
        list(range(50, 0, -1)),
    )
    for _ in range(300):
        n = rng.randint(0, 80)
        yield [rng.randint(0, rng.choice([3, 100])) for _ in range(n)]


def test_merge():
    assert merge([1, 4, 9], [2, 3, 10, 11]) == [1, 2, 3, 4, 9, 10, 11]
    assert merge([], [1]) == [1]


def test_merge_sort_is_stable():
    items = [(1, "b"), (0, "x"), (1, "a"), (0, "y")]
    keyed = [(k, i) for i, (k, _) in enumerate(items)]  # tag positions
    assert [i for _, i in merge_sort(keyed)] == [1, 3, 0, 2]


@pytest.mark.parametrize("sort", [quicksort_naive, quicksort, quicksort3])
def test_quicksorts_match_sorted(sort):
    random.seed(1)
    for case in cases():
        items = list(case)
        sort(items)
        assert items == sorted(case)


def test_merge_sort_matches_sorted():
    for case in cases():
        assert merge_sort(case) == sorted(case)
```

```
......                                                                   [100%]
6 passed in 0.01s
```

The cases include lists of a single repeated value and lists drawn from only 4 values, to exercise the duplicate handling. The stability test tags each item

with its original position and checks that equal keys come out in position order.

The Race

```
"""Merge sort and quicksort against Python's sorted(), on random data."""

import random

import bench
from merge_sort import merge_sort
from quicksort import quicksort, quicksort3

rng = random.Random(2026)


def shuffled(n):
    return (rng.sample(range(n), n),)


sizes = [10_000, 20_000, 40_000, 80_000]
results = {
    "merge_sort": bench.doubling(merge_sort, shuffled, sizes, fresh=True),
    "quicksort": bench.doubling(quicksort, shuffled, sizes, fresh=True),
    "quicksort3": bench.doubling(quicksort3, shuffled, sizes, fresh=True),
    "sorted()": bench.doubling(sorted, shuffled, sizes, fresh=True),
}
bench.table(results)
bench.chart(results, "ch12_sorts.png", "Sorting random lists")
```

```
         n      merge_sort       quicksort      quicksort3        sorted()
    10,000         16.9 ms         11.3 ms         15.8 ms         1.47 ms
    20,000    35.5 ms x2.1      26 ms x2.3    35.8 ms x2.3    3.22 ms x2.2
    40,000    77.2 ms x2.2    62.1 ms x2.4    78.4 ms x2.2    7.29 ms x2.3
    80,000     179 ms x2.3     130 ms x2.1     176 ms x2.2    17.5 ms x2.4
```

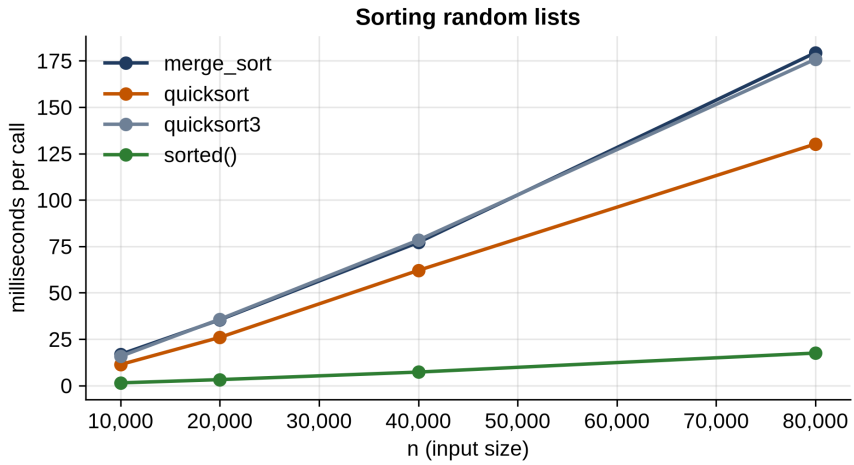
All four grow a little faster than double when n doubles: the "a little over ×2" of O(n log n). Our quicksort is the fastest of ours, as quicksort usually is in practice: it works in place and its inner loop is simple. Merge sort pays for building new lists. And Python's `sorted()`, written in C, is seven to ten times faster than any of them.

On many duplicates, three-way partitioning makes a dramatic difference:

```
"""Many equal items: the two-way and three-way quicksorts."""

import random
import sys

import bench
```



All four sorts grow at about the same rate; Python's sorted() is far faster.

```
from quicksort import quicksort, quicksort3

sys.setrecursionlimit(10_000)
rng = random.Random(2026)


def few_values(n):
    return ([rng.randint(0, 9) for _ in range(n)],)  # only 10 values


sizes = [2_000, 4_000, 8_000, 16_000]
results = {
    "quicksort": bench.doubling(quicksort, few_values, sizes, fresh=True),
    "quicksort3": bench.doubling(quicksort3, few_values, sizes, fresh=True),
}
bench.table(results)
```

```
         n       quicksort      quicksort3
     2,000         5.09 ms          435 us
     4,000    18.9 ms x3.7     917 us x2.1
     8,000      72 ms x3.8    1.82 ms x2.0
    16,000     280 ms x3.9    3.44 ms x1.9
```

With only ten distinct values, the two-way quicksort degrades to quadratic (the ratios approach 4), while the three-way version stays linear, about eighty times faster at 16,000 items.

How Python Sorts: Timsort

Python's `sorted()` and `list.sort()` use **Timsort**, written by Tim Peters for Python 2.3. It's a merge sort designed for real data, which is rarely random: it's often partly sorted, or sorted in reverse, or a sorted list with a few new items added. Timsort first scans for **runs**, stretches already in order (reversing descending ones), extends short runs with binary insertion sort, and merges runs in a careful order. On data with structure, it does far less work:

```
"""Python's sorted() (Timsort) finds runs that are already in order."""

import random

import bench

rng = random.Random(2026)
n = 1_000_000
ordered = list(range(n))
nearly = list(range(n))
for _ in range(100):
    i, j = rng.randrange(n), rng.randrange(n)
    nearly[i], nearly[j] = nearly[j], nearly[i]
cases = {
    "random": rng.sample(range(n), n),
    "sorted": ordered,
    "reversed": ordered[::-1],
    "100 swaps": nearly,
}
for name, data in cases.items():
    seconds = bench.measure(lambda: sorted(data))
    print(f"{name:>10}: {bench.show_time(seconds)}")
```

```
    random: 497 ms
    sorted: 18.9 ms
  reversed: 20.8 ms
 100 swaps: 24.4 ms
```

A million items in random order take about half a second; already sorted, reversed, or sorted with a hundred swaps, about 20 to 25 times less. A sorted list is just one run, so "sorting" it means checking it once: O(n).

Real-World Use: Timsort spread far beyond Python: V8, the JavaScript engine in Chrome and Node.js, switched `Array.prototype.sort` to it in 2018 (V8 7.0), which also made JavaScript's sort stable. Python itself refined it in 3.11, adopting the "powersort" rule of Ian Munro and Sebastian Wild for choosing which runs to merge. And Go's `sort` package uses yet another design, pattern-defeating quicksort, since Go 1.19: real-world sorting is still an active area.

Interesting Fact: Tony Hoare invented quicksort around 1960, as a visiting student in Moscow, where he needed to sort words for a machine-translation project. He later said it only became easy to explain once he learned recursion.

Key Takeaways

- Divide and conquer splits a problem into halves; $\log_2 n$ levels of O(n) work make O(n log n).
- Merge sort is O(n log n) on every input and stable, but needs O(n) extra memory.
- Quicksort sorts in place and is fast in practice, but a fixed pivot makes sorted input $O(n^2)$. Choose the pivot at random, and recurse on the smaller side to bound the stack.
- Three-way partitioning keeps quicksort linear-ish on data with many duplicates.
- Python's Timsort exploits existing order: already-sorted data sorts in O(n). Use `sorted()` in real code.

Exercises

1. **Count inversions fast.** Modify merge sort to count inversions in O(n log n). (Hint: when an item is taken from the right half, it's smaller than every item still waiting in the left half.)
2. **Quickselect.** Find the k-th smallest item in O(n) average time by partitioning and keeping only the side that contains position k.

3. **Sort colors.** Sort a list of 0s, 1s and 2s in one pass, with a three-way partition around 1.

CHAPTER 13 · INTERMEDIATE

Heaps and Priority Queues

A queue serves items in the order they arrived. Often that's not what you want: an emergency room treats the most urgent patient first, an operating system runs the most important task first, and a route planner explores the closest unexplored place first. What you need is a **priority queue**: add items in any order, and always take out the one with the highest priority (or the smallest key) next.

A sorted list makes "take the smallest" O(1), but adding is O(n). An unsorted list makes adding O(1), but finding the smallest is O(n). A **binary heap** does both in O(log n), using nothing but a plain list. This chapter builds one, shows how to build a heap in linear time, and then uses the standard library's `heapq` for four patterns that come up constantly.

A Tree Hidden in a List

A binary heap is a binary tree, a structure where every item has up to two children, with one rule, the **heap property**: every parent is less than or equal to its children. So the smallest item is always at the top, the **root**.

The clever part is the storage. The tree is kept complete, filled level by level from left to right, so it can be stored in a list with no pointers at all: the root at index 0, and for the item at index i, its children at 2i + 1 and 2i + 2 and its parent at (i - 1) // 2.

```
"""A binary min-heap stored in a list.

The item at index i has its children at 2i + 1 and 2i + 2, and its
parent at (i - 1) // 2. Every parent is <= its children, so the
smallest item is always at index 0.
"""


class MinHeap:
    def __init__(self):
        self._items = []

    @classmethod
    def heapify(cls, items):
        """Build a heap from a list in linear time: sift down every
        parent, from the last one up to the root."""
        heap = cls()
        heap._items = list(items)
        for i in range(len(items) // 2 - 1, -1, -1):
            heap._sift_down(i)
        return heap

    def __len__(self):
        return len(self._items)

    def peek(self):
        if not self._items:
            raise IndexError("peek at an empty heap")
        return self._items[0]

    def push(self, value):
        self._items.append(value)  # add at the bottom, then move it up
        self._sift_up(len(self._items) - 1)
```

`push` appends the new item at the bottom of the tree, the end of the list, then **sifts it up**: while it's smaller than its parent, swap them. The tree has about $\log_2 n$ levels, so that's O(log n) swaps at most.

```
    def pop(self):
        """Remove and return the smallest item."""
        items = self._items
        if not items:
            raise IndexError("pop from an empty heap")
        last = items.pop()
        if not items:
            return last
        smallest, items[0] = items[0], last  # the last item to the top
        self._sift_down(0)
        return smallest

    def _sift_up(self, i):
        items = self._items
        while i > 0:
            parent = (i - 1) // 2
            if items[i] >= items[parent]:
                break
```

```
            items[i], items[parent] = items[parent], items[i]
            i = parent

    def _sift_down(self, i):
        items, n = self._items, len(self._items)
        while True:
            smallest = i
            for child in (2 * i + 1, 2 * i + 2):
                if child < n and items[child] < items[smallest]:
                    smallest = child
            if smallest == i:
                return
            items[i], items[smallest] = items[smallest], items[i]
            i = smallest
```

`pop` must remove the root. It moves the *last* item into the root's place, which keeps the tree complete, and then **sifts it down**: while it's bigger than its smaller child, swap them. Again at most $\log_2 n$ levels.

Watch the list as items go in and out:

```
"""Watch the list behind the heap as items are pushed and popped."""

from binary_heap import MinHeap

h = MinHeap()
for x in [7, 4, 9, 1, 5]:
    h.push(x)
    print(f"push {x}: {h._items}")
while h:
    smallest = h.pop()
    print(f"pop -> {smallest}: {h._items}")
```

```
push 7: [7]
push 4: [4, 7]
push 9: [4, 7, 9]
push 1: [1, 4, 9, 7]
push 5: [1, 4, 9, 7, 5]
pop -> 1: [4, 5, 9, 7]
pop -> 4: [5, 7, 9]
pop -> 5: [7, 9]
pop -> 7: [9]
pop -> 9: []
```

After pushing 7, 4, 9, 1 and 5, the list is `[1, 4, 9, 7, 5]`. It isn't sorted, but it satisfies the heap property: 1 is above 4 and 9, and 4 is above 7 and 5. That's much less order than sorting, which is exactly why maintaining it is cheaper. Popping repeatedly gives the items in sorted order, which is **heapsort** (Exercise 1).

Building a Heap in Linear Time

To turn a list of n items into a heap, you could push them one by one: n pushes of O(log n) each. `heapify` is smarter: it sifts *down* every parent, starting from the last parent and moving up to the root. Most nodes are near the bottom, where sifting down is short (half the nodes are leaves and don't move at all), and the few near the top are the only ones that can travel far. Adding it all up gives fewer than n swaps: O(n).

Here's the count on the worst input for pushing, a list in decreasing order, where every new item sifts all the way to the top:

```
"""Building a heap of n items: n pushes or one heapify? Count the swaps
on the worst input for pushing, a list in decreasing order."""

from binary_heap import MinHeap


class Counting(MinHeap):
    swaps = 0

    def _sift_up(self, i):
        items = self._items
        while i > 0 and items[i] < items[(i - 1) // 2]:
            parent = (i - 1) // 2
            items[i], items[parent] = items[parent], items[i]
            Counting.swaps += 1
            i = parent

    def _sift_down(self, i):
        items, n = self._items, len(self._items)
        while True:
            small = min(
                (c for c in (i, 2 * i + 1, 2 * i + 2) if c < n),
                key=items.__getitem__,
            )
            if small == i:
                return
            items[i], items[small] = items[small], items[i]
            Counting.swaps += 1
            i = small


print(f"{'n':>9} {'n pushes':>10} {'heapify':>9}")
for n in [1_000, 10_000, 100_000]:
    data = list(range(n, 0, -1))
    Counting.swaps = 0
    h = Counting()
    for x in data:
        h.push(x)
    pushes = Counting.swaps
    Counting.swaps = 0
    Counting.heapify(data)
    print(f"{n:>9,} {pushes:>10,} {Counting.swaps:>9,}")
```

```
       n   n pushes   heapify
   1,000      7,987       992
  10,000    113,631     9,992
 100,000  1,468,946    99,990
```

`heapify` makes just under n swaps at every size; n pushes make about n $\log_2$ n. At 100,000 items, it's 99,990 swaps against nearly 1.5 million.

Testing

```
"""Tests for MinHeap, with random operations checked against heapq."""

import heapq
import random

import pytest

from binary_heap import MinHeap


def is_heap(items):
    return all(
        items[(i - 1) // 2] <= items[i] for i in range(1, len(items))
    )


def test_pops_come_out_sorted():
    h = MinHeap()
    for x in [5, 3, 8, 1, 9, 2]:
        h.push(x)
    assert [h.pop() for _ in range(6)] == [1, 2, 3, 5, 8, 9]
    with pytest.raises(IndexError):
        h.pop()


def test_heapify_makes_a_valid_heap():
    rng = random.Random(2026)
    for n in range(50):
        h = MinHeap.heapify([rng.randint(0, 20) for _ in range(n)])
        assert is_heap(h._items)


def test_random_operations_match_heapq():
    rng = random.Random(2026)
    h, expected = MinHeap(), []
    for _ in range(10000):
        if rng.random() < 0.6 or not expected:
            x = rng.randint(0, 999)
            h.push(x)
            heapq.heappush(expected, x)
        else:
            assert h.pop() == heapq.heappop(expected)
        assert len(h) == len(expected)
```

```
        assert is_heap(h._items)
```

```
...                                                                      [100%]
3 passed in 0.01s
```

`is_heap` checks the heap property directly: every item is at least its parent. The random test runs 10,000 pushes and pops against Python's `heapq` and checks the heap property after every single operation.

Timing

```
"""Building a heap: n pushes against heapify, ours and heapq's."""

import heapq
import random

import bench
from binary_heap import MinHeap

rng = random.Random(2026)


def data(n):
    return (rng.sample(range(n), n),)


def ours_push(items):
    h = MinHeap()
    for x in items:
        h.push(x)


def heapq_push(items):
    h = []
    for x in items:
        heapq.heappush(h, x)


sizes = [10_000, 20_000, 40_000, 80_000]
results = {
    "MinHeap pushes": bench.doubling(ours_push, data, sizes),
    "MinHeap.heapify": bench.doubling(MinHeap.heapify, data, sizes),
    "heappush": bench.doubling(heapq_push, data, sizes),
    "heapq.heapify": bench.doubling(heapq.heapify, data, sizes, fresh=True),
}
bench.table(results)
```

```
         n  MinHeap pushes  MinHeap.heapify        heappush   heapq.heapify
    10,000         2.79 ms          2.35 ms          727 us          295 us
    20,000    5.78 ms x2.1     5.37 ms x2.3    1.52 ms x2.1     726 us x2.5
    40,000    14.4 ms x2.5     10.8 ms x2.0     3.5 ms x2.3    2.23 ms x3.1
    80,000    38.3 ms x2.7     33.5 ms x3.1    7.54 ms x2.2    5.81 ms x2.6
```

On random data, n pushes take about as long as `heapify`. That's not a contradiction of the swap counts: a random new item usually belongs near the bottom of the tree, so it sifts up only a level or two, and n pushes of random data average O(1) each. The O(n log n) worst case needs adversarial input, like the decreasing list above. `heapq`, in C, is several times faster than our class, as usual.

The heapq Module: Four Patterns

`heapq` works on a plain list that you pass to its functions. It's a min-heap, the same as ours.

```
"""heapq, the standard library heap, in four common patterns."""

import heapq

scores = [88, 42, 97, 63, 71, 99, 54]

# 1. The k largest (or smallest) items
print(heapq.nlargest(3, scores), heapq.nsmallest(2, scores))

# 2. A priority queue of tasks; the counter breaks ties in arrival order
tasks, counter = [], 0
for priority, task in [(2, "email"), (1, "fix bug"), (2, "lunch")]:
    heapq.heappush(tasks, (priority, counter, task))
    counter += 1
print([heapq.heappop(tasks)[2] for _ in range(3)])

# 3. Merge sorted lists lazily, one item at a time
print(list(heapq.merge([1, 4, 7], [2, 5, 8], [3, 6, 9])))

# 4. A max-heap: store negated values
max_heap = [-s for s in scores]
heapq.heapify(max_heap)
print(-heapq.heappop(max_heap))
```

```
[99, 97, 88] [42, 54]
['fix bug', 'email', 'lunch']
[1, 2, 3, 4, 5, 6, 7, 8, 9]
99
```

1. **The k largest or smallest.** `nlargest(k, items)` keeps a heap of k items while it scans: O(n log k), much better than sorting when k is small.
2. **A priority queue.** Push `(priority, counter, task)` tuples. Tuples compare item by item, so the lowest priority number comes out first. The counter breaks ties in arrival order, and also stops Python from ever comparing two tasks directly, which would fail for objects that can't be compared.
3. **Merging sorted inputs.** `heapq.merge` keeps one item from each input in a heap and yields them in order, lazily: it works on files too large to fit in memory.
4. **A max-heap.** `heapq` only has a min-heap, so store negated values: the smallest negative is the largest original.

> **Real-World Use:** Python's `asyncio` keeps every scheduled callback, everything waiting for a timer (`asyncio.sleep`, `loop.call_later`), in a heap ordered by when it's due, pushing with `heapq.heappush`. The event loop only ever needs to know which timer fires *next*, exactly the question a heap answers in O(1).

A Running Median

A favorite interview problem: numbers arrive one at a time; after each one, report the median of all so far. Two heaps solve it: a max-heap holding the smaller half and a min-heap holding the larger half, kept the same size (or the small half one bigger):

```
"""The median of a stream of numbers, after each new one, with two
heaps: a max-heap for the smaller half and a min-heap for the larger."""

import heapq


class RunningMedian:
    def __init__(self):
        self.low = []  # max-heap of the smaller half (values negated)
        self.high = []  # min-heap of the larger half

    def add(self, x):
        heapq.heappush(self.low, -x)
        # move the biggest of the low half up, then rebalance
        heapq.heappush(self.high, -heapq.heappop(self.low))
        if len(self.high) > len(self.low):
```

```
            heapq.heappush(self.low, -heapq.heappop(self.high))

    def median(self):
        if len(self.low) > len(self.high):
            return -self.low[0]
        return (-self.low[0] + self.high[0]) / 2


m = RunningMedian()
for x in [5, 15, 1, 3, 8, 7]:
    m.add(x)
    print(f"add {x:>2}: median {m.median()}")
```

```
add  5: median 5
add 15: median 10.0
add  1: median 5
add  3: median 4.0
add  8: median 5
add  7: median 6.0
```

Each new number goes into the low half; the largest of the low half then moves to the high half; and if the high half has become bigger, its smallest moves back. Now the median is the top of the low half, or the average of both tops. Each number costs O(log n), where sorting all numbers so far after each arrival would cost O(n log n) every time.

> **Interview Tip:** "k largest", "k closest", "k most frequent", "merge k sorted", "schedule by deadline or priority", "median of a stream": any of these phrases means a heap.

Key Takeaways

- A binary heap keeps the smallest item at the root, with each parent ≤ its children.
- Stored in a list: children of i at 2i + 1 and 2i + 2, parent at (i - 1) // 2.
- `push` sifts up and `pop` sifts down: O(log n) each. Peeking at the smallest is O(1).
- `heapify` builds a heap in O(n) by sifting down from the last parent.
- Use `heapq`: for top-k, priority queues with a tie-breaking counter, lazy merging, and (with negation) max-heaps.
- Two heaps maintain a running median in O(log n) per number.

Exercises

1. **Heapsort.** Sort a list with `MinHeap`: heapify, then pop n times. What's its Big O?
2. **k closest points.** Find the k points closest to the origin in O(n log k), keeping a max-heap of the best k seen so far.
3. **Merge k sorted lists.** Write your own version of `heapq.merge` for k lists, with a heap of (value, list number, position) tuples.

CHAPTER 14 · INTERMEDIATE

Sorting Without Comparing

Every sort so far works by comparing two items at a time. This chapter shows why no comparison sort can beat O(n log n), and then breaks the barrier anyway, for the right kind of data: small integers, or anything that can be keyed by them. Counting sort and radix sort never compare two items. And then the benchmark delivers a lesson about Python.

The Speed Limit for Comparison Sorts

A list of n distinct items can be in n! ("n factorial", n × (n - 1) × ... × 1) different orders, and a sort must tell them all apart, since each needs different rearranging. Each comparison is a yes-or-no question, so at best it splits the remaining possibilities in half. To narrow n! possibilities down to one takes at least $\log_2(n!)$ comparisons, and $\log_2(n!)$ grows like n $\log_2$ n:

```
"""Why comparison sorts need about n log2 n comparisons: there are n!
possible orders, and each yes-or-no comparison can at best halve the
orders still possible, so it takes at least log2(n!) comparisons."""

import math

print(f"{'n':>7} {'log2(n!)':>11} {'n log2 n':>11}")
for n in [10, 100, 1_000, 10_000, 100_000]:
    log2_factorial = math.lgamma(n + 1) / math.log(2)  # log2 of n!
    print(f"{n:>7,} {log2_factorial:>11,.0f} {n * math.log2(n):>11,.0f}")
```

```
      n    log2(n!)    n log2 n
     10          22          33
    100         525         664
  1,000       8,529       9,966
 10,000     118,458     132,877
100,000   1,516,704   1,660,964
```

`math.lgamma(n + 1)` is the natural logarithm of n!, computed without making the huge number itself; dividing by `math.log(2)` converts it to $\log_2$. The bound is within about 10% of n $\log_2$ n at these sizes. So merge sort, heapsort and Timsort are, up to a constant factor, as good as comparison sorting can ever be. To go faster, a sort must learn something about the items without comparing them.

Counting Sort

If the items are integers from 0 to some small `largest`, just count them:

```
def counting_sort(numbers, largest):
    """Sort integers from 0 to largest by counting each value:
    time and memory proportional to n + largest."""
    counts = [0] * (largest + 1)
    for x in numbers:
        counts[x] += 1
    out = []
    for value, count in enumerate(counts):
        out.extend([value] * count)
    return out
```

One pass counts how many times each value appears; a second pass writes each value out that many times. That's O(n + k) time and O(k) extra memory, where k is the range of values. For a million test scores from 0 to 100, it's linear.

The catch is k. Sorting a few numbers between 0 and a billion would allocate a billion counters. Counting sort only makes sense when the range of values is not much bigger than the number of items.

The version above rebuilds the values from their counts, which works for plain numbers. To sort *records* by an integer key, keeping each record and keeping the sort stable, compute where each key's records start and drop each record into place:

```
def counting_sort_by(items, key, largest):
    """A stable counting sort of any items by an integer key."""
    counts = [0] * (largest + 1)
    for item in items:
        counts[key(item)] += 1
    starts, total = [0] * (largest + 1), 0
    for value in range(largest + 1):  # where each key's items begin
        starts[value], total = total, total + counts[value]
    out = [None] * len(items)
    for item in items:  # in input order: equal keys stay in order
        k = key(item)
        out[starts[k]] = item
        starts[k] += 1
    return out
```

After counting, `starts[v]` is the position where the first record with key v belongs: the total count of all smaller keys. Then each record, taken in input order, goes into the next free position for its key. Records with equal keys come out in the order they went in: stable.

Radix Sort

Radix sort extends counting sort to large integers. Sort the numbers by their last digit, then by the second-to-last digit, and so on, with a stable counting sort each time:

```
"""Radix sort in base 10, one digit at a time."""

from linear_sorts import counting_sort_by

numbers = [170, 45, 75, 90, 802, 24, 2, 66]
place = 1
while place <= max(numbers):
    numbers = counting_sort_by(numbers, lambda x: x // place % 10, 9)
    print(f"by the {place}s digit: {numbers}")
    place *= 10
```

```
by the 1s digit: [170, 90, 802, 2, 24, 45, 75, 66]
by the 10s digit: [802, 2, 24, 45, 66, 170, 75, 90]
by the 100s digit: [2, 24, 45, 66, 75, 90, 170, 802]
```

After sorting by the ones digit, then the tens digit, the numbers are sorted by their last two digits: stability keeps the ones-digit order among numbers with the same tens digit. After the last digit, they're fully sorted. With d digits, that's d passes of O(n + base) each. The library version uses base 256, one byte at a time, so 32-bit numbers take just four passes:

```
def radix_sort(numbers, base=256):
    """Sort non-negative integers digit by digit, least significant
    digit first, with a stable counting sort for each digit."""
    if not numbers:
        return []
    out, place = list(numbers), 1
    while place <= max(numbers):
        out = counting_sort_by(out, lambda x: x // place % base, base - 1)
        place *= base
    return out
```

> **Interesting Fact:** Radix sorting is older than electronic computers. Mechanical punched-card sorters, descendants of the machines Herman Hollerith built for the 1890 United States census, dropped each card into a pocket according to the digit punched in one column, one column per pass through the machine. To sort a deck by a multi-digit number, operators ran it through once per column, starting with the least significant column: exactly the least-significant-digit radix sort in this chapter.

Testing

```
"""The linear-time sorts against sorted()."""

import random

from linear_sorts import counting_sort, counting_sort_by, radix_sort


def test_counting_sort():
    rng = random.Random(2026)
    for _ in range(200):
        data = [rng.randint(0, 30) for _ in range(rng.randint(0, 60))]
        assert counting_sort(data, 30) == sorted(data)


def test_counting_sort_by_is_stable():
    words = ["pear", "fig", "kiwi", "apple", "plum", "yam"]
    by_length = counting_sort_by(words, len, 10)
    assert by_length == ["fig", "yam", "pear", "kiwi", "plum", "apple"]
```

```
def test_radix_sort_any_base():
    rng = random.Random(2026)
    for base in (2, 10, 256):
        for _ in range(100):
            data = [
                rng.randint(0, 10**9) for _ in range(rng.randint(0, 50))
            ]
            assert radix_sort(data, base) == sorted(data)
```

```
...                                                                  [100%]
3 passed in 0.01s
```

The stability test sorts words by length and checks that words of the same length keep their order. Radix sort is tested in bases 2, 10 and 256, on numbers up to a billion.

The Benchmark's Lesson

In theory, counting sort and radix sort are O(n), and `sorted()` is O(n log n). Here's what happens in Python, with n numbers from 0 to 999:

```
"""Our counting and radix sorts against sorted(), on n numbers from 0
to 999. Fewer steps in theory; but sorted() runs in C."""

import random

import bench
from linear_sorts import counting_sort, radix_sort

rng = random.Random(2026)


def data(n):
    return ([rng.randint(0, 999) for _ in range(n)],)


sizes = [100_000, 200_000, 400_000, 800_000]
results = {
    "counting_sort": bench.doubling(
        lambda d: counting_sort(d, 999), data, sizes
    ),
    "radix_sort": bench.doubling(radix_sort, data, sizes),
    "sorted()": bench.doubling(sorted, data, sizes),
}
bench.table(results)
```

```
         n   counting_sort      radix_sort        sorted()
   100,000         3.25 ms         42.3 ms         12.9 ms
   200,000    6.37 ms x2.0    85.8 ms x2.0    26.3 ms x2.0
   400,000    12.5 ms x2.0     179 ms x2.1    54.6 ms x2.1
   800,000    25.1 ms x2.0     357 ms x2.0     113 ms x2.1
```

All three double when n doubles: at these sizes, log n changes so slowly that O(n log n) looks linear too. Counting sort, with a range of only 1,000, is about four times faster than `sorted()`: its loops are simple, and `out.extend([value] * count)` does most of the work in C. But radix sort, despite its better Big O, is about three times *slower* than `sorted()`. Each of its passes runs a key function and several Python-level loops over every item, while Timsort runs entirely in C.

This is the theme of the book's benchmarks in its sharpest form: Big O tells you how time grows, but the constant factor, which in Python is often "Python code against C code", can be bigger than log n for any practical n. Measure before you decide.

> **Tip:** In Python, `sorted()` is almost always the fastest way to sort. Counting sort is worth it for large inputs with a small range of integer keys, like ages or scores. For heavy numeric sorting, NumPy's `numpy.sort` runs entirely in compiled code.

Key Takeaways

- Any comparison sort needs at least $\log_2(n!)$ comparisons, about $n \log_2 n$: merge sort and Timsort are optimal up to a constant.
- Counting sort sorts integers in a range of size k in O(n + k) time, without comparing.
- A stable counting sort by key can sort records, and is the building block of radix sort.
- Radix sort sorts d-digit numbers with d stable passes, least significant digit first.
- In Python, constant factors matter: a C-coded O(n log n) sort can beat a Python-coded O(n) one.

Exercises

1. **Sort by age.** Sort (name, age) records by age, from 0 to 120, with a stable counting sort; people of the same age must keep their order.
2. **Bucket sort.** For n floating-point numbers spread evenly between 0 and 1, put each into one of n buckets by its value, sort each bucket and join them. Why is this O(n) on average?

PART IV

Trees

CHAPTER 15 · INTERMEDIATE

Binary Trees

Lists and queues arrange items in a line. Much of the world's data is shaped like a tree instead: folders inside folders, the elements of a web page, the structure of a Python program, an organization chart, a family tree. And some of the fastest data structures, search trees and heaps, are trees by design.

This chapter covers the basics of **binary trees**, where each node has at most two children: how to build them, the four standard ways to visit every node, and how to measure their height. The next three chapters build on them.

Words for Trees

A tree is made of **nodes**. The top node is the **root**. Each node can have **children**, and every node except the root has exactly one **parent**. A node with no children is a **leaf**. In a binary tree, each node has at most two children, a **left** and a **right**. Every node, with everything below it, is itself a tree: its **subtree**. That's why recursion fits trees so well.

The **depth** of a node is the number of edges from the root down to it; the **height** of a tree is the greatest depth of any node. A tree of just a root has height 0, and an empty tree, by convention, height -1.

Nodes and a Builder

```
"""Binary trees: nodes with at most two children, and ways to visit them."""

from collections import deque


class TreeNode:
    __slots__ = ("value", "left", "right")

    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right


def from_levels(values):
    """Build a tree from values listed level by level, left to right,
    with None for a missing child: [1, 2, 3, None, 4] is
    1 with children 2 and 3, and 4 as the right child of 2."""
    if not values or values[0] is None:
        return None
    nodes = iter(values)
    root = TreeNode(next(nodes))
    waiting = deque([root])  # nodes whose children come next
    while waiting:
        node = waiting.popleft()
        for side in ("left", "right"):
            value = next(nodes, None)
            if value is not None:
                child = TreeNode(value)
                setattr(node, side, child)
                waiting.append(child)
    return root
```

`TreeNode` is a value with two child references, either of which can be `None`. Writing trees as nested constructor calls gets tedious, so `from_levels` builds one from a list of values level by level, left to right, with `None` for a missing child. It's the format that most interview problems use to describe trees. It hands out values to each node's two child positions in order, keeping a queue of nodes whose children come next: a breadth-first construction, a preview of Chapter 21.

Four Ways to Visit Every Node

To **traverse** a tree is to visit every node once. Three traversals are depth-first, differing only in when the node itself is visited relative to its two subtrees:

```
def preorder(node):  # the node, then its left subtree, then its right
    if node:
        yield node.value
        yield from preorder(node.left)
        yield from preorder(node.right)


def inorder(node):  # left subtree, the node, right subtree
    if node:
        yield from inorder(node.left)
        yield node.value
        yield from inorder(node.right)


def postorder(node):  # left subtree, right subtree, then the node
    if node:
        yield from postorder(node.left)
        yield from postorder(node.right)
        yield node.value
```

- **Preorder** visits a node before its subtrees: useful for copying a tree or writing it out, since every parent comes before its children.
- **Inorder** visits the left subtree, then the node, then the right. For a binary search tree (Chapter 16), this gives the keys in sorted order.
- **Postorder** visits a node after both subtrees: useful whenever a node's answer depends on its children's answers, like computing sizes, heights or deleting a tree.

Each is written as a recursive generator: `yield from` passes along everything the recursive call yields. The fourth traversal goes level by level, top to bottom:

```
def levels(root):
    """Breadth-first: a list of the values on each level."""
    out, level = [], [root] if root else []
    while level:
        out.append([n.value for n in level])
        level = [c for n in level for c in (n.left, n.right) if c]
    return out
```

The current level is a list of nodes; the next level is all their children. This is breadth-first search on a tree. And here they all are on one tree:

```
"""One tree, four orders of visiting it."""

from tree import draw, from_levels, inorder, levels, postorder, preorder

t = from_levels(["A", "B", "C", "D", "E", None, "F"])
draw(t)
print("preorder: ", *preorder(t))
print("inorder:  ", *inorder(t))
print("postorder:", *postorder(t))
print("levels:   ", levels(t))
```

```
        F
    C
A
        E
    B
        D
preorder:  A B D E C F
inorder:   D B E A C F
postorder: D E B F C A
levels:    [['A'], ['B', 'C'], ['D', 'E', 'F']]
```

`draw` prints the tree sideways, with the root at the left edge and the right subtree above it: tilt your head to the left and it looks like a tree. It's an in-order traversal in reverse, indented by depth:

```
def draw(node, depth=0):
    """Print the tree sideways: the root on the left, right side up."""
    if node:
        draw(node.right, depth + 1)
        print("    " * depth + str(node.value))
        draw(node.left, depth + 1)
```

Without Recursion

Recursive traversals are as deep as the tree is tall. For a balanced tree that's about $\log_2 n$, but a tree can be as tall as it has nodes, and Chapter 16 builds one. So it's worth knowing the loop version:

```
def inorder_loop(node):
    """In-order without recursion: a stack holds the nodes whose left
    side we are still exploring."""
    stack = []
    while stack or node:
        while node:  # go as far left as possible
            stack.append(node)
```

```
            node = node.left
        node = stack.pop()
        yield node.value
        node = node.right
```

The stack holds the nodes whose left subtrees are still being explored. Go as far left as possible, pushing each node; pop one, visit it, then do the same with its right subtree. It's exactly what the recursive version does with the call stack, made explicit.

Height

```
def height(node):
    """Edges on the longest path down from node; -1 for no tree."""
    if node is None:
        return -1
    return 1 + max(height(node.left), height(node.right))
```

Height is a postorder computation: a node's height is one more than the taller of its subtrees, with -1 for an empty subtree. A tree with n nodes has height at least about $\log_2 n$ (when it's perfectly balanced) and at most n - 1 (when it's a single chain). Almost every operation on a search tree costs O(height), which is why the next two chapters care so much about keeping it small.

Testing

```
"""Tests for the tree helpers."""

import random

from tree import (
    TreeNode,
    from_levels,
    height,
    inorder,
    inorder_loop,
    levels,
    postorder,
    preorder,
)


def sample():
    #       1
    #     2   3
    #    4 5    6
    return from_levels([1, 2, 3, 4, 5, None, 6])
```

```
def test_traversals():
    t = sample()
    assert list(preorder(t)) == [1, 2, 4, 5, 3, 6]
    assert list(inorder(t)) == [4, 2, 5, 1, 3, 6]
    assert list(postorder(t)) == [4, 5, 2, 6, 3, 1]
    assert levels(t) == [[1], [2, 3], [4, 5, 6]]


def test_height_and_empty_tree():
    assert height(sample()) == 2
    assert height(None) == -1 and levels(None) == []
    assert list(inorder(from_levels([]))) == []


def random_tree(rng, n):
    if n == 0:
        return None
    left = rng.randint(0, n - 1)
    return TreeNode(
        rng.randint(0, 99),
        random_tree(rng, left),
        random_tree(rng, n - 1 - left),
    )


def test_loop_matches_recursion_on_random_trees():
    rng = random.Random(2026)
    for _ in range(300):
        t = random_tree(rng, rng.randint(0, 40))
        assert list(inorder_loop(t)) == list(inorder(t))
```

```
...                                                                  [100%]
3 passed in 0.01s
```

`random_tree` builds random trees of any shape, by choosing at random how many nodes go into the left subtree. The test runs both in-order traversals on 300 of them and checks that they agree.

Expression Trees

Here's a tree that a program actually computes with. An arithmetic expression has a natural tree shape: operators inside, numbers at the leaves.

```
"""An expression as a tree: operators inside, numbers at the leaves.
Post-order evaluates it: both sides first, then the operator."""

import operator

from tree import TreeNode

OPS = {"+": operator.add, "-": operator.sub, "*": operator.mul}
```

```
def evaluate(node):
    if node.left is None:  # a leaf: a number
        return node.value
    left, right = evaluate(node.left), evaluate(node.right)
    return OPS[node.value](left, right)


def to_text(node):
    """In-order, with brackets around every operation."""
    if node.left is None:
        return str(node.value)
    return f"({to_text(node.left)} {node.value} {to_text(node.right)})"


# (3 + 4) * (5 - 2)
expr = TreeNode(
    "*",
    TreeNode("+", TreeNode(3), TreeNode(4)),
    TreeNode("-", TreeNode(5), TreeNode(2)),
)
print(to_text(expr), "=", evaluate(expr))
```

```
((3 + 4) * (5 - 2)) = 21
```

`evaluate` is a postorder traversal: it computes both subtrees, then applies the operator. `to_text` is an in-order traversal that wraps each operation in brackets, to show the structure.

> **Real-World Use:** This is how Python reads your code. Before running anything, Python parses the source into an **abstract syntax tree**, a tree of nodes for every expression and statement, and compiles it from there. You can look at it yourself: `import ast; print(ast.dump(ast.parse("(3 + 4) * (5 - 2)"), indent=2))` prints a tree of `BinOp` nodes with `Constant` leaves, the same shape as the tree above.

Key Takeaways

- A binary tree is a root node with up to two subtrees, each of which is itself a binary tree.
- Preorder, inorder and postorder visit a node before, between or after its subtrees; level order visits by depth.
- Postorder suits computations where a node depends on its children: height, size, evaluating expressions.

- Recursive traversals can be rewritten with an explicit stack, which matters for very tall trees.
- Operations on trees typically cost O(height): from about $\log_2 n$ for a balanced tree to n - 1 for a chain.

Exercises

1. **Mirror.** Swap the left and right children of every node in a tree.
2. **Diameter.** Find the number of edges on the longest path between any two nodes. (Hint: the longest path through a node is the height of its left subtree plus the height of its right subtree plus 2.)
3. **Lowest common ancestor.** Find the deepest node that has two given values in its subtree.

CHAPTER 16 · INTERMEDIATE

Binary Search Trees

A sorted list lets you binary search in O(log n), but inserting keeps costing O(n), because items shift. A hash table inserts and finds in O(1), but knows nothing about order: it can't tell you the smallest key, the keys between two values, or the next key after this one. A **binary search tree** (BST) aims to do everything in O(log n): find, insert, delete, and the ordered questions too. This chapter builds one, and then shows the input that destroys it.

The Rule

A BST is a binary tree with one rule: for every node, every key in its left subtree is **smaller** than the node's key, and every key in its right subtree is **bigger**. So to find a key, start at the root and compare: smaller, go left; bigger, go right; equal, found. Each step goes one level down, so a search costs O(height).

Searching and Inserting

```
"""A binary search tree: every key in a node's left subtree is smaller
than the node's key, and every key in its right subtree is bigger.

All loops, no recursion: a badly unbalanced tree can be thousands of
levels deep, deeper than Python's recursion limit.
"""


class Node:
    __slots__ = ("key", "value", "left", "right")
```

```
    def __init__(self, key, value):
        self.key, self.value = key, value
        self.left = self.right = None


class BST:
    def __init__(self):
        self.root = None
        self._n = 0

    def __len__(self):
        return self._n

    def _find(self, key):
        """(node with key or None, its parent)."""
        parent, node = None, self.root
        while node and node.key != key:
            parent = node
            node = node.left if key < node.key else node.right
        return node, parent

    def get(self, key, default=None):
        node, _ = self._find(key)
        return node.value if node else default

    def __contains__(self, key):
        return self._find(key)[0] is not None

    def put(self, key, value):
        node, parent = self._find(key)
        if node:
            node.value = value  # the key is there: replace its value
        elif parent is None:
            self.root = Node(key, value)
            self._n += 1
        else:
            child = Node(key, value)
            if key < parent.key:
                parent.left = child
            else:
                parent.right = child
            self._n += 1
```

Every method is a loop, not recursion. That's deliberate: as you'll see shortly, a BST can be thousands of levels deep, far past Python's recursion limit.

`_find` walks down from the root and returns two things: the node with the key (or `None`), and its parent, the last node visited. `get` and `__contains__` only need the first. `put` uses both: if the key exists, it replaces the value; if not, the search ended at exactly the place where the key belongs, as a child of `parent`, on the side the last comparison chose. A new key always becomes a leaf.

Deleting

Deleting is the tricky operation, with three cases:

```
    def delete(self, key):
        node, parent = self._find(key)
        if node is None:
            raise KeyError(key)
        if node.left and node.right:
            # Two children: copy in the next key (the smallest on the
            # right), then delete that node, which has no left child
            parent, successor = node, node.right
            while successor.left:
                parent, successor = successor, successor.left
            node.key, node.value = successor.key, successor.value
            node = successor
        child = node.left or node.right  # at most one child now
        if parent is None:
            self.root = child
        elif parent.left is node:
            parent.left = child
        else:
            parent.right = child
        self._n -= 1
```

- **A leaf** is simply cut off: its parent's link becomes `None`.
- **A node with one child** is replaced by that child, which takes its place in the parent.
- **A node with two children** can't be cut out so simply. Instead, find its **successor**, the next bigger key, which is the leftmost node of its right subtree. Copy the successor's key and value into the node, then delete the successor, which has no left child, so it's one of the first two cases.

Copying the successor keeps the rule intact: the successor is bigger than everything in the node's left subtree and smaller than everything else in its right subtree.

Ordered Operations

```
    def __iter__(self):
        """Keys in sorted order: an in-order walk with a stack."""
        stack, node = [], self.root
        while stack or node:
            while node:
                stack.append(node)
                node = node.left
            node = stack.pop()
```

```
            yield node.key
            node = node.right

    def min(self):
        node = self.root
        while node.left:
            node = node.left
        return node.key

    def floor(self, key):
        """The largest key <= key, or None."""
        node, best = self.root, None
        while node:
            if node.key == key:
                return key
            if node.key < key:
                best, node = node.key, node.right
            else:
                node = node.left
        return best

    def height(self):
        """Edges on the longest path down from the root, counted level
        by level."""
        level, h = [self.root] if self.root else [], -1
        while level:
            h += 1
            level = [c for n in level for c in (n.left, n.right) if c]
        return h
```

Iterating gives the keys **in sorted order**: it's the in-order traversal from Chapter 15, written with a stack. `min` follows left links to the bottom. `floor(key)` finds the largest key not bigger than `key`, remembering the best candidate whenever the walk goes right. Neither a dict nor a set can answer that without looking at every key. `height` counts levels with the level-by-level walk from Chapter 15.

```
"""Build a small BST, draw it, then delete the root."""

from bst import BST


def draw(node, depth=0):
    if node:
        draw(node.right, depth + 1)
        print("    " * depth + str(node.key))
        draw(node.left, depth + 1)


t = BST()
for k in [50, 30, 70, 20, 40, 60, 80]:
    t.put(k, None)
draw(t.root)
t.delete(50)  # 60, the next key, takes its place
print("after deleting 50:")
```

```
draw(t.root)
```

```
        80
    70
        60
50
        40
    30
        20
after deleting 50:
        80
    70
60
        40
    30
        20
```

Deleting the root, 50, which has two children: its successor 60, the leftmost node of the right subtree, takes its place.

Testing

```
"""Tests for BST, with random operations checked against a dict."""

import random

import pytest

from bst import BST


def test_put_get_and_order():
    t = BST()
    for k in [50, 30, 70, 20, 40, 60, 80]:
        t.put(k, str(k))
    assert list(t) == [20, 30, 40, 50, 60, 70, 80]
    assert t.get(40) == "40" and t.get(45) is None and 60 in t
    assert t.min() == 20 and t.height() == 2
    assert t.floor(65) == 60 and t.floor(10) is None


def test_delete_leaf_one_child_two_children_root():
    t = BST()
    for k in [50, 30, 70, 20, 40, 60, 80, 65]:
        t.put(k, k)
    t.delete(20)  # a leaf
    t.delete(60)  # one child (65)
    t.delete(30)  # one child left after the leaf went (40)
    t.delete(50)  # the root, with two children
    assert list(t) == [40, 65, 70, 80] and len(t) == 4
    with pytest.raises(KeyError):
        t.delete(50)
```

```
def test_random_operations_match_dict():
    rng = random.Random(2026)
    t, expected = BST(), {}
    for _ in range(20000):
        k = rng.randint(0, 200)
        if rng.random() < 0.6:
            t.put(k, -k)
            expected[k] = -k
        elif k in expected:
            t.delete(k)
            del expected[k]
        assert len(t) == len(expected)
    assert list(t) == sorted(expected)
    assert all(t.get(k) == v for k, v in expected.items())
```

```
...                                                                  [100%]
3 passed in 0.01s
```

The delete test removes a leaf, a node with one child, a node whose second child was just removed, and the root with two children. The random test runs 20,000 puts and deletes against a `dict`, then checks that iteration gives exactly the dict's keys in sorted order.

The Sorted-Input Disaster

Everything costs O(height). How tall does a BST get? It depends entirely on the order in which keys arrive:

```
"""The height of a BST built from random keys and from sorted keys."""

import math
import random

from bst import BST

rng = random.Random(2026)
print(f"{'n':>7} {'log2 n':>7} {'random order':>13} {'sorted order':>13}")
for n in [100, 1_000, 10_000]:
    shuffled, ordered = BST(), BST()
    for k in rng.sample(range(n), n):
        shuffled.put(k, None)
    for k in range(n):
        ordered.put(k, None)
    print(
        f"{n:>7,} {math.log2(n):>7.1f} {shuffled.height():>13,} "
        f"{ordered.height():>13,}"
    )
```

```
      n  log2 n  random order  sorted order
    100     6.6            14            99
  1,000    10.0            18           999
 10,000    13.3            26         9,999
```

Inserted in random order, 10,000 keys make a tree of height 26, about twice $\log_2$ n: random BSTs are reasonably balanced. Inserted in sorted order, every new key is bigger than all before it and goes to the far right: the tree is a single chain, height 9,999. It's a linked list with extra steps, and every operation is O(n). And sorted input is common: IDs that count up, timestamps, names read from a sorted file.

Timed, looking up the largest key:

```
"""Looking up a key: a BST built from random keys, one built from
sorted keys, and Python's dict."""

import random

import bench
from bst import BST

rng = random.Random(2026)

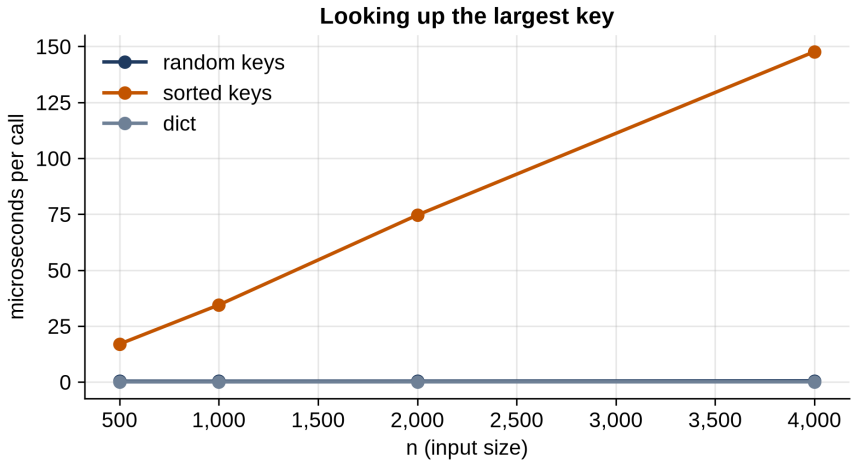

def built(order):
    def make(n):
        t = BST()
        for k in order(n):
            t.put(k, None)
        return t, n - 1  # the largest key: deepest in the sorted tree

    return make


def lookup(t, key):
    return key in t


sizes = [500, 1_000, 2_000, 4_000]
results = {
    "random keys": bench.doubling(
        lookup, built(lambda n: rng.sample(range(n), n)), sizes
    ),
    "sorted keys": bench.doubling(lookup, built(range), sizes),
    "dict": bench.doubling(
        lookup, lambda n: (dict.fromkeys(range(n)), n - 1), sizes
    ),
}
bench.table(results)
bench.chart(results, "ch16_bst.png", "Looking up the largest key")
```

```
        n     random keys     sorted keys            dict
      500          386 ns           17 us         92.4 ns
    1,000     380 ns x1.0    34.5 us x2.0    90.9 ns x1.0
    2,000     405 ns x1.1    74.7 us x2.2     111 ns x1.2
    4,000     527 ns x1.3     148 us x2.0     110 ns x1.0
```



A BST built from sorted keys gets slower in proportion to its size.

The tree built from random keys answers in about half a microsecond at every size here (O(log n) grows slowly). The tree built from sorted keys doubles its time with n: at 4,000 keys it's hundreds of times slower. The dict is flat and fastest, but can't answer any ordered question.

> **Tip:** If you need a sorted collection in Python and don't want to write a balanced tree, keep a sorted list and use `bisect` (Chapter 10): searches are O(log n), and inserts are O(n) but done by a fast memory copy, which is fine up to hundreds of thousands of items. Or use the third-party `sortedcontainers` package.

Key Takeaways

- In a BST, smaller keys go left and bigger keys go right, at every node.
- Search, insert and delete cost O(height); deleting a node with two children copies its successor.

- In-order traversal gives the keys sorted; BSTs answer ordered questions (min, floor, ranges) that hash tables can't.
- Random insertion order gives a height of about $2 \log_2 n$; sorted order gives a chain of height $n - 1$.
- Write BST operations as loops: an unbalanced tree can be deeper than the recursion limit.

Exercises

1. **Is it a BST?** Check whether a binary tree satisfies the BST rule. Careful: comparing each node only with its parent isn't enough; each node must lie between bounds set by all its ancestors.
2. **Range query.** Yield the keys between `low` and `high` in order, visiting only the parts of the tree that can contain them.
3. **k-th smallest.** Find the k-th smallest key with an in-order walk that stops after k keys.

CHAPTER 17 · INTERMEDIATE

Balanced Trees: AVL

The binary search tree of Chapter 16 is fast when it's bushy and useless when it's a chain, and the input decides which. A **self-balancing** tree removes that dependence: after every insertion and deletion, it rearranges itself just enough to stay bushy, guaranteeing O(log n) operations on any input. This chapter builds the oldest one, the **AVL tree**, and verifies its guarantee with tests and timings.

The Balance Rule

An AVL tree is a BST with one more rule: at every node, the heights of the left and right subtrees differ by **at most one**. That's enough to keep the whole tree's height below about 1.44 $\log_2$ n, as you'll check below.

Each node stores its own height, so the rule can be checked in O(1):

```
"""An AVL tree: a binary search tree that rebalances itself.

Every node records its height. After each insert or delete, any node
whose two subtrees differ in height by more than one is fixed with one
or two rotations, so the tree's height stays below 1.44 log2(n + 2).
"""


class Node:
    __slots__ = ("key", "value", "left", "right", "height")

    def __init__(self, key, value):
        self.key, self.value = key, value
        self.left = self.right = None
        self.height = 0
```

```
def height(node):
    return node.height if node else -1


def update(node):
    node.height = 1 + max(height(node.left), height(node.right))


def balance(node):
    """Positive when the left side is taller, negative when the right is."""
    return height(node.left) - height(node.right)
```

`balance` is positive when the left side is taller, negative when the right side is. After an insertion or deletion, any node whose balance becomes +2 or -2 must be fixed.

Rotations

The fix is a **rotation**: a small local rearrangement that changes heights while keeping the BST order.

```
def rotate_right(node):
    """Lift the left child above node; returns the new subtree root.

          node            left
          /  \\           /  \\
        left  c   ->     a   node
        /  \\                /  \\
       a    b              b    c
    """
    left = node.left
    node.left, left.right = left.right, node
    update(node)
    update(left)
    return left


def rotate_left(node):
    """The mirror image of rotate_right."""
    right = node.right
    node.right, right.left = right.left, node
    update(node)
    update(right)
    return right
```

`rotate_right` lifts a node's left child above it. Read the diagram in the docstring: the order a < left < b < node < c is the same before and after, so the

BST rule still holds, but the left side has moved up a level and the right side down. Only three links change. `rotate_left` is the mirror image.

```
def rebalance(node):
    update(node)
    if balance(node) > 1:  # left side too tall
        if balance(node.left) < 0:  # left-right case: two rotations
            node.left = rotate_left(node.left)
        return rotate_right(node)
    if balance(node) < -1:  # right side too tall
        if balance(node.right) > 0:  # right-left case
            node.right = rotate_right(node.right)
        return rotate_left(node)
    return node
```

`rebalance` handles four cases. If the left side is too tall and its taller part is on the outside (left of the left child), one right rotation fixes it. If its taller part is on the *inside* (right of the left child), one rotation would just move the problem; so first rotate the left child left, turning it into the outside case, then rotate right. The right side is the mirror image.

Inserting and Deleting

```
class AVLTree:
    def __init__(self):
        self.root = None
        self._n = 0

    def __len__(self):
        return self._n

    def get(self, key, default=None):
        node = self.root
        while node:
            if key == node.key:
                return node.value
            node = node.left if key < node.key else node.right
        return default

    def __contains__(self, key):
        node = self.root
        while node and node.key != key:
            node = node.left if key < node.key else node.right
        return node is not None

    def put(self, key, value):
        self.root = self._put(self.root, key, value)

    def _put(self, node, key, value):
        if node is None:
            self._n += 1
            return Node(key, value)
```

```
        if key == node.key:
            node.value = value
            return node
        if key < node.key:
            node.left = self._put(node.left, key, value)
        else:
            node.right = self._put(node.right, key, value)
        return rebalance(node)  # fix every node on the way back up

    def delete(self, key):
        self.root = self._delete(self.root, key)
        self._n -= 1

    def _delete(self, node, key):
        if node is None:
            raise KeyError(key)
        if key < node.key:
            node.left = self._delete(node.left, key)
        elif key > node.key:
            node.right = self._delete(node.right, key)
        elif node.left and node.right:
            successor = node.right  # the next key takes this node's place
            while successor.left:
                successor = successor.left
            node.key, node.value = successor.key, successor.value
            node.right = self._delete(node.right, successor.key)
        else:
            return node.left or node.right
        return rebalance(node)

    def __iter__(self):
        stack, node = [], self.root
        while stack or node:
            while node:
                stack.append(node)
                node = node.left
            node = stack.pop()
            yield node.key
            node = node.right

    def height(self):
        return height(self.root)
```

These are the BST operations from Chapter 16, written recursively, with one addition: on the way back up from the change, every node on the path is rebalanced: `return rebalance(node)`. Each call returns the root of its subtree, which may be a different node after a rotation, and the caller stores it.

Recursion is safe here. The height is at most about 1.44 $\log_2$ n, so even a billion keys would need fewer than 45 levels. That guarantee is exactly what the plain BST lacked. Lookups and iteration are loops, the same as in the plain BST.

Watch it stay balanced as sorted keys arrive, the input that turned the plain BST into a chain:

```
"""Insert 1 to 7 in order. A plain BST would become a chain; the AVL
tree rotates and ends perfectly balanced."""

from avl import AVLTree


def draw(node, depth=0):
    if node:
        draw(node.right, depth + 1)
        print("    " * depth + str(node.key))
        draw(node.left, depth + 1)


t = AVLTree()
for k in range(1, 8):
    t.put(k, None)
    if k in (3, 5, 7):
        print(f"after inserting 1 to {k}:")
        draw(t.root)
```

```
after inserting 1 to 3:
    3
2
    1
after inserting 1 to 5:
        5
    4
        3
2
    1
after inserting 1 to 7:
        7
    6
        5
4
        3
    2
        1
```

After 1, 2, 3 the tree rotates to put 2 at the root. By 7 keys it's a perfect tree of height 2.

Testing the Guarantee

```
"""AVL tests: after every random operation the tree must hold exactly
the keys a dict holds, in order, with every node balanced and every
stored height correct."""

import math
```

```
import random

import pytest

from avl import AVLTree, height


def check(node):
    """Return the subtree's height, asserting every AVL rule below."""
    if node is None:
        return -1
    left, right = check(node.left), check(node.right)
    assert abs(left - right) <= 1, "unbalanced"
    assert node.height == 1 + max(left, right), "stale height"
    if node.left:
        assert node.left.key < node.key
    if node.right:
        assert node.right.key > node.key
    return node.height


def test_sorted_inserts_stay_balanced():
    t = AVLTree()
    for k in range(1, 1024):
        t.put(k, k)
    assert t.height() == 9  # a perfect tree: 2**10 - 1 nodes
    check(t.root)


def test_random_operations_match_dict():
    rng = random.Random(2026)
    t, expected = AVLTree(), {}
    for _ in range(5000):
        k = rng.randint(0, 300)
        if rng.random() < 0.6:
            t.put(k, -k)
            expected[k] = -k
        elif k in expected:
            t.delete(k)
            del expected[k]
        else:
            with pytest.raises(KeyError):
                t.delete(k)
        check(t.root)
        assert len(t) == len(expected)
        n = len(expected)
        assert height(t.root) <= 1.44 * math.log2(n + 2)
    assert list(t) == sorted(expected)
    assert all(t.get(k) == v for k, v in expected.items())
```

```
..                                                                   [100%]
2 passed in 0.01s
```

`check` walks the whole tree and asserts everything an AVL tree promises: every node balanced, every stored height correct, and BST order between each node and its children. The first test inserts 1 to 1,023 in sorted order and

expects a perfect tree of height 9. The second runs 5,000 random puts and deletes, including deletes of missing keys, against a `dict`, and after every operation runs `check` and verifies the height bound.

While this chapter was being written, that test failed once. The failure was in the test, not the tree: the test assumed a failed delete changed the count and tried to "undo" it, but `delete` raises `KeyError` before touching the count. A test that checks after every single operation catches mistakes on both sides.

How Tall?

```
"""Heights after inserting the keys 0 to n - 1 in sorted order."""

import math

from avl import AVLTree

print(
    f"{'n':>8} {'plain BST':>10} {'AVL':>4} {'log2 n':>7} "
    f"{'1.44 log2 n':>12}"
)
for n in [100, 1_000, 10_000, 100_000]:
    t = AVLTree()
    for k in range(n):
        t.put(k, None)
    print(
        f"{n:>8,} {n - 1:>10,} {t.height():>4} {math.log2(n):>7.1f} "
        f"{1.44 * math.log2(n):>12.1f}"
    )
```

```
       n  plain BST  AVL  log2 n  1.44 log2 n
     100         99    6     6.6          9.6
   1,000        999    9    10.0         14.4
  10,000      9,999   13    13.3         19.1
 100,000     99,999   16    16.6         23.9
```

For 100,000 sorted keys, the plain BST is 99,999 levels tall; the AVL tree is 16, essentially $\log_2 n$, and nowhere near the $1.44 \log_2 n$ worst case. The worst case needs a carefully built tree with as few nodes as possible for its height, and Exercise 2 counts how few: it grows like the Fibonacci numbers.

Timing

Build a tree from n sorted keys, then look up every key:

```
"""Build a tree from n sorted keys, then look up every key: a plain
BST against an AVL tree."""

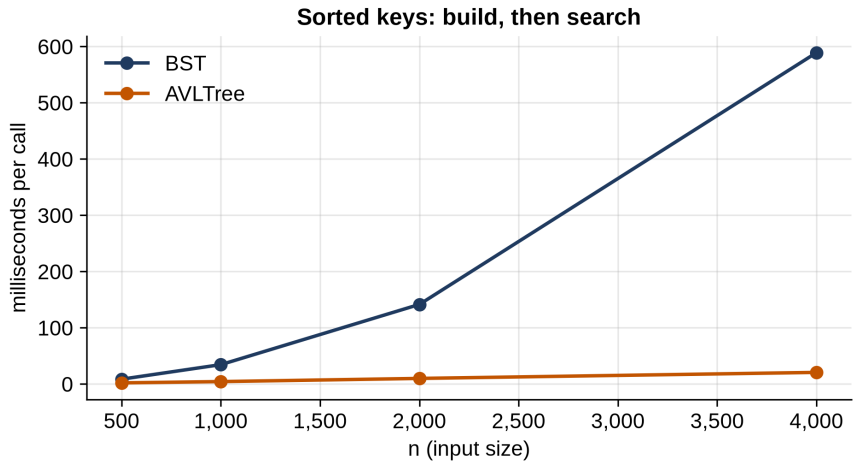
import bench
from avl import AVLTree
from bst import BST  # a copy of Chapter 16's bst.py


def build_and_search(make, n):
    t = make()
    for k in range(n):
        t.put(k, None)
    for k in range(n):
        k in t


sizes = [500, 1_000, 2_000, 4_000]
results = {
    "BST": bench.doubling(build_and_search, lambda n: (BST, n), sizes),
    "AVLTree": bench.doubling(
        build_and_search, lambda n: (AVLTree, n), sizes
    ),
}
bench.table(results)
bench.chart(results, "ch17_avl.png", "Sorted keys: build, then search")
```

```
         n             BST         AVLTree
       500         8.64 ms          2.1 ms
     1,000    34.5 ms x4.0    4.38 ms x2.1
     2,000     142 ms x4.1      10 ms x2.3
     4,000     589 ms x4.2    20.7 ms x2.1
```

The plain BST quadruples with each doubling: n insertions and n lookups, each O(n), make $O(n^2)$. The AVL tree roughly doubles: O(n log n). At 4,000 keys it's about 28 times faster, and the gap grows without limit. Balance isn't free: every insertion updates stored heights all the way up the path and sometimes rotates. You pay a constant factor for a guarantee that no input can break.



On sorted keys, the plain BST grows quadratically while the AVL tree grows like n log n.

Real-World Use: Self-balancing trees run inside software you use every day, usually as **red-black trees**, a cousin of AVL trees with a looser balance rule and fewer rotations. Java's `TreeMap` is "a Red-Black tree based NavigableMap implementation" with guaranteed log(n) operations. The Linux kernel uses red-black trees to track I/O requests, high-resolution timers, each process's memory areas and epoll's file descriptors. Databases use **B-trees**, which keep many keys per node so each node fills one disk page: SQLite stores tables and indexes in them, and PostgreSQL's `CREATE INDEX` makes a B-tree unless told otherwise.

Interesting Fact: The AVL tree is named after its inventors, Georgy Adelson-Velsky and Evgenii Landis, who published it in 1962. It was the first self-balancing binary search tree.

Key Takeaways

- A self-balancing tree restructures itself after each change to guarantee O(log n) height on any input.
- An AVL tree keeps the two subtree heights at every node within one of each other; its height stays below about $1.44 \log_2 n$.

- Rotations restore balance by changing three links while preserving BST order; the inside cases need two.
- Inserting and deleting rebalance every node on the path back up, O(log n) of them.
- Test structural invariants (balance, stored heights, order) after every operation, not just the results.

Exercises

1. **Counting rotations.** Count the rotations made while inserting 1,000 keys in sorted order and in random order. Which needs more, and why?
2. **The skinniest AVL tree.** The fewest nodes an AVL tree of height h can have satisfies $N(h) = N(h - 1) + N(h - 2) + 1$, with $N(-1) = 0$ and $N(0) = 1$. Compute N(h) up to h = 30 and compare $\log_2 N(h)$ with h.

CHAPTER 18 · INTERMEDIATE

Tries

Type "can" into a search box and it suggests "candle", "candy", "canvas". To do that, the program must find every stored word that starts with what you've typed, fast, on every keystroke. A hash table can't help: "candle" and "can" have unrelated hashes. A sorted list with binary search can. But there's a structure built exactly for questions about prefixes: the **trie**, or prefix tree.

The Idea

A trie stores words letter by letter along paths from the root. Each node maps a letter to a child node, so the path from the root to any node spells a prefix. Words that share a prefix share the path for it: "candle", "candy" and "cane" all go through the nodes for c, a, n. A flag on a node marks that a whole word ends there, since "can" is a word but "ca" isn't.

```
"""A trie (prefix tree) for words. Each node maps a letter to a child;
a path from the root spells a prefix, and a flag marks whole words."""


class TrieNode:
    __slots__ = ("children", "is_word")

    def __init__(self):
        self.children = {}
        self.is_word = False


class Trie:
    def __init__(self, words=()):
        self.root = TrieNode()
        self._n = 0
        for w in words:
```

```
            self.insert(w)

    def __len__(self):
        return self._n

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        if not node.is_word:
            node.is_word = True
            self._n += 1

    def _walk(self, prefix):
        """The node reached by spelling prefix, or None."""
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def __contains__(self, word):
        node = self._walk(word)
        return node is not None and node.is_word

    def has_prefix(self, prefix):
        return self._walk(prefix) is not None
```

`insert` walks down the word, creating any missing nodes with `dict.setdefault`, and marks the last node as a word end. `_walk` follows a prefix from the root and returns the node it reaches, or `None` if the path breaks off. Both `__contains__` and `has_prefix` are one walk: their cost is the length of the word, **O(L)**, however many words are stored. A million words or ten, looking up "candle" takes six steps.

Autocomplete

```
    def complete(self, prefix, limit=None):
        """Words starting with prefix, in alphabetical order."""
        start = self._walk(prefix)
        out = []
        stack = [(start, prefix)] if start else []
        while stack and (limit is None or len(out) < limit):
            node, word = stack.pop()
            if node.is_word:
                out.append(word)
            for ch in sorted(node.children, reverse=True):
                stack.append((node.children[ch], word + ch))
        return out
```

`complete` walks to the prefix's node, then explores everything below it with a stack, collecting the words. Pushing children in reverse alphabetical order means they come off the stack in alphabetical order, so the suggestions come out sorted, and `limit` stops the search after the first few.

```
"""Autocomplete: suggest words as the user types."""

from trie import Trie

words = """apple apply apricot banana band bandana bank banner
           can candle candy cane cannon canvas""".split()
t = Trie(words)
for typed in ["ap", "ban", "can", "cand", "cat"]:
    print(f"{typed!r:>7}: {t.complete(typed, limit=3)}")
```

```
   'ap': ['apple', 'apply', 'apricot']
  'ban': ['banana', 'band', 'bandana']
  'can': ['can', 'candle', 'candy']
 'cand': ['candle', 'candy']
  'cat': []
```

The cost of `complete` is the length of the prefix plus the size of the part of the trie below it, which is why autocomplete systems cap the number of suggestions.

Testing

```
"""Trie tests, with prefix queries checked against a plain scan."""

import random

from trie import Trie


def test_insert_and_lookup():
    t = Trie(["car", "cart", "care", "cat", "dog"])
    t.insert("car")  # a duplicate is not counted twice
    assert len(t) == 5
    assert "car" in t and "ca" not in t and "cars" not in t
    assert t.has_prefix("ca") and not t.has_prefix("cow")
    assert t.complete("car") == ["car", "care", "cart"]
    assert t.complete("ca", limit=2) == ["car", "care"]
    assert t.complete("x") == [] and t.complete("") == sorted(
        ["car", "cart", "care", "cat", "dog"]
    )


def test_random_words_match_a_scan():
    rng = random.Random(2026)
    words = {
        "".join(rng.choices("abc", k=rng.randint(1, 6))) for _ in range(500)
```

```
    }
    t = Trie(words)
    for _ in range(300):
        prefix = "".join(rng.choices("abc", k=rng.randint(0, 3)))
        expected = sorted(w for w in words if w.startswith(prefix))
        assert t.complete(prefix) == expected
    assert all(w in t for w in words)
    assert "d" not in t and len(t) == len(words)
```

```
..                                                                   [100%]
2 passed in 0.01s
```

The random test makes 500 words from just three letters, so they share prefixes heavily, then checks 300 random prefixes against a plain scan with `startswith`, including the empty prefix, which must return every word.

Timing

Does any of n words start with a given prefix? A scan, the trie, and binary search on a sorted list:

```
"""Is there any word starting with a prefix, among n words? A scan
with startswith(), our Trie, and binary search in a sorted list."""

import bisect
import random

import bench
from trie import Trie

rng = random.Random(2026)


def words(n):
    return sorted(
        {"".join(rng.choices("abcdefghij", k=8)) for _ in range(n)}
    )


def scan(ws, prefix):
    return any(w.startswith(prefix) for w in ws)


def sorted_search(ws, prefix):
    # the first word >= prefix is the only one that can start with it
    i = bisect.bisect_left(ws, prefix)
    return i < len(ws) and ws[i].startswith(prefix)


PREFIX = "jjjj"  # sorts last: a scan has to look at every word
sizes = [10_000, 20_000, 40_000, 80_000]
results = {
    "scan": bench.doubling(scan, lambda n: (words(n), PREFIX), sizes),
```

```
    "Trie": bench.doubling(
        Trie.has_prefix, lambda n: (Trie(words(n)), PREFIX), sizes
    ),
    "sorted + bisect": bench.doubling(
        sorted_search, lambda n: (words(n), PREFIX), sizes
    ),
}
bench.table(results)
```

```
         n            scan            Trie  sorted + bisect
    10,000          395 us          282 ns           316 ns
    20,000     888 us x2.2     285 ns x1.0      290 ns x0.9
    40,000    1.97 ms x2.2     284 ns x1.0      340 ns x1.2
    80,000    4.17 ms x2.1     283 ns x1.0      358 ns x1.1
```

The scan doubles with n. The trie takes the same time at every size: O(L), here four letters. Binary search on a sorted list is almost as fast, growing only by log n: the first word that sorts at or after the prefix is the only one that might start with it. In Python that makes a sorted list with `bisect` a very good alternative to a trie, with less memory: each trie node is a dictionary.

So when do tries win? When the work really is letter by letter: longest-prefix matching, wildcard search (Exercise 2), or walking many words that share prefixes at once, as a spell checker or a word game does.

> **Real-World Use:** When a router forwards a packet, it must find the most specific route that matches the destination address: the **longest prefix match**. Linux stores its IPv4 routing table in a compressed trie of address bits (`fib_trie`) and has done so since kernel 2.6.39, descending it bit by bit on every lookup.

> **Interesting Fact:** Edward Fredkin named the trie in 1960, from the middle of the word "retrieval", and pronounced it "tree". Many people say "try" instead, to tell the two apart.

Key Takeaways

- A trie stores strings letter by letter; words with a common prefix share the path for it.
- Insert, look up and prefix checks cost O(L), the length of the string, independent of how many strings are stored.

- Autocomplete is a walk to the prefix's node plus a traversal of the subtree below it.
- Tries use more memory than a sorted list; in Python, `bisect` on a sorted list is a strong alternative for simple prefix lookups.
- Tries shine for letter-by-letter work: longest prefix match, wildcards, word games.

Exercises

1. **Longest common prefix.** Find the longest prefix shared by all of a list of words, by walking down a trie of them while each node has exactly one child.
2. **Wildcards.** Search a trie with patterns where `.` matches any single letter, such as `".ad"` or `"b.."`.

CHAPTER 19 · INTERMEDIATE

Union-Find

Some questions are about groups. Which computers in a network can reach each other? Which people are connected through chains of friendships? Which pixels of an image belong to the same blob? Connections arrive one at a time, and at any moment you need to answer: are these two in the same group?

Union-find, also called a **disjoint-set** structure, answers exactly that, with two operations: `union(a, b)` merges the groups of a and b, and `find(a)` names a's group. With two simple tricks, both take nearly constant time, and this chapter measures the difference those tricks make.

Groups as Trees

Each group is stored as a tree, but an unusual one: every item points to its **parent**, not to its children, and the root of each tree names the group. Initially every item is its own group, its own root. `find(x)` follows parent pointers to the root. `union(a, b)` finds both roots and, if they differ, makes one root point to the other: one assignment merges two whole groups.

The items are numbered 0 to n - 1, so the parents fit in a plain list:

```
"""Union-find (disjoint sets): which items are connected?

Each set is a tree; its root names the set. find() follows parents to
the root and then points every node on the way straight at it (path
compression). union() hangs the smaller tree under the larger (union by
size). Together they make each operation nearly constant time.
"""
```

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))  # each item starts as its own set
        self.size = [1] * n
        self.sets = n

    def find(self, x):
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != root:  # compress the path
            self.parent[x], x = root, self.parent[x]
        return root

    def union(self, a, b):
        """Join the sets of a and b. False if already in the same set."""
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra  # the smaller tree goes under the larger
        self.size[ra] += self.size[rb]
        self.sets -= 1
        return True

    def connected(self, a, b):
        return self.find(a) == self.find(b)
```

The Two Tricks

Without care, the trees can grow into long chains, and `find` becomes O(n). Two tricks prevent that:

- **Union by size.** When merging, hang the *smaller* tree under the root of the larger one. An item's depth only increases when its group at least doubles in size, so no item can be more than $\log_2 n$ deep.
- **Path compression.** After `find` reaches the root, go along the path again and point every node on it directly at the root. The next `find` for any of them takes one step.

`find` is written as two loops rather than recursion: the first finds the root, the second compresses. `union` returns `False` when the two items were already in the same group, which turns out to be useful, and keeps a count of the groups, `sets`.

For comparison, the naive version has neither trick:

```
class NaiveUnionFind:
    """No compression, no sizes: trees can become long chains."""

    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        self.parent[ra] = rb
        return True
```

Friend Groups

```
"""Group people who are friends, directly or through others."""

from union_find import UnionFind

people = ["Ana", "Ben", "Cat", "Dan", "Eve", "Fay", "Gus"]
index = {name: i for i, name in enumerate(people)}
friendships = [
    ("Ana", "Ben"),
    ("Cat", "Dan"),
    ("Ben", "Cat"),
    ("Eve", "Fay"),
]

uf = UnionFind(len(people))
for a, b in friendships:
    uf.union(index[a], index[b])

groups = {}
for name in people:
    groups.setdefault(uf.find(index[name]), []).append(name)
print(f"{uf.sets} groups:", list(groups.values()))
print("Ana and Dan connected?", uf.connected(index["Ana"], index["Dan"]))
```

```
3 groups: [['Ana', 'Ben', 'Cat', 'Dan'], ['Eve', 'Fay'], ['Gus']]
Ana and Dan connected? True
```

Ana and Dan were never directly connected: Ana knows Ben, Ben knows Cat, Cat knows Dan. Union-find never stores those chains; it just merged the groups as each friendship arrived.

Testing Against a Search

```
"""Union-find checked against connected components found by search."""

import random

from union_find import NaiveUnionFind, UnionFind


def components(n, edges):
    """Label each item with its component, by depth-first search."""
    neighbors = {i: [] for i in range(n)}
    for a, b in edges:
        neighbors[a].append(b)
        neighbors[b].append(a)
    label = {}
    for start in range(n):
        if start in label:
            continue
        stack = [start]
        while stack:
            x = stack.pop()
            if x not in label:
                label[x] = start
                stack.extend(neighbors[x])
    return label


def test_small_example():
    uf = UnionFind(6)
    assert uf.union(0, 1) and uf.union(1, 2) and uf.union(3, 4)
    assert not uf.union(0, 2)  # already connected
    assert uf.connected(0, 2) and not uf.connected(2, 3)
    assert uf.sets == 3


def test_random_graphs_match_search():
    rng = random.Random(2026)
    for _ in range(200):
        n = rng.randint(1, 60)
        edges = [
            (rng.randrange(n), rng.randrange(n))
            for _ in range(rng.randint(0, n))
        ]
        label = components(n, edges)
        for make in (UnionFind, NaiveUnionFind):
            uf = make(n)
            for a, b in edges:
                uf.union(a, b)
            for a in range(n):
                for b in range(n):
                    same = uf.find(a) == uf.find(b)
                    assert same == (label[a] == label[b])
```

```
..                                                                   [100%]
2 passed in 0.01s
```

The random test builds 200 random graphs, labels their connected components with an independent method, a depth-first search (which Chapter 21 covers in detail), and checks that both union-find versions agree on every single pair of items.

Timing the Tricks

The worst case for the naive version: unions that build one long chain, then a `find` on every item.

```
"""n unions that build one long chain, then a find on every item:
naive union-find against path compression and union by size."""

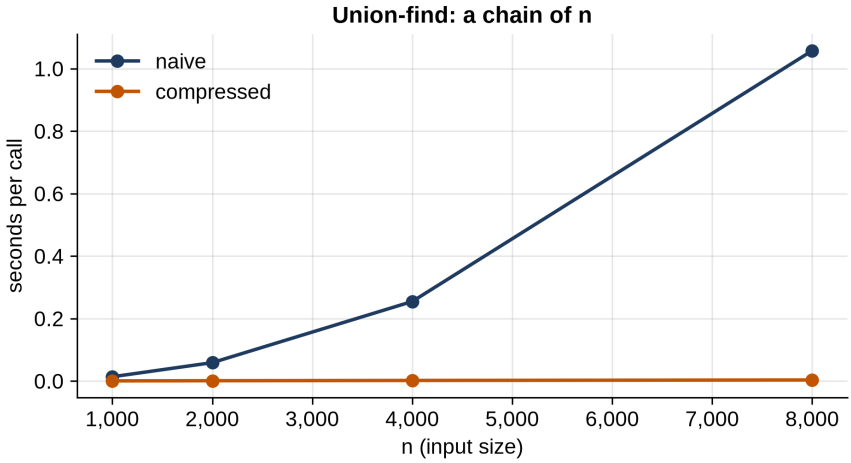
import bench
from union_find import NaiveUnionFind, UnionFind


def chain_then_find(make, n):
    uf = make(n)
    for i in range(n - 1):
        uf.union(i, i + 1)  # the naive version builds a chain
    for i in range(n):
        uf.find(i)


sizes = [1_000, 2_000, 4_000, 8_000]
results = {
    "naive": bench.doubling(
        chain_then_find, lambda n: (NaiveUnionFind, n), sizes
    ),
    "compressed": bench.doubling(
        chain_then_find, lambda n: (UnionFind, n), sizes
    ),
}
bench.table(results)
bench.chart(results, "ch19_union_find.png", "Union-find: a chain of n")
```

```
         n           naive      compressed
     1,000         13.6 ms          425 us
     2,000    58.8 ms x4.3     887 us x2.1
     4,000     255 ms x4.3    1.75 ms x2.0
     8,000     1.06 s x4.2    3.28 ms x1.9
```

The naive version quadruples with each doubling: n finds along a chain of length n are $O(n^2)$. With union by size and path compression, the work is linear, and at 8,000 items it's over 300 times faster. In theory, the cost per operation with both tricks is the **inverse Ackermann function** of n, a



The naive union-find grows quadratically on a chain; with the two tricks it stays linear.

function that grows so slowly it's below 5 for any n that could fit in the universe. In practice: constant.

> **Interesting Fact:** That astonishing bound, nearly constant time per operation for a sequence of unions and finds, was proved by Robert Tarjan in 1975, along with a matching lower bound: no union-find method of this kind can do better.

> **Interview Tip:** "Number of connected components", "are these connected?", "does adding this edge make a cycle?", "merge accounts that share an email": these are union-find questions. Kruskal's algorithm in Chapter 23 is built on it.

Key Takeaways

- Union-find keeps groups as trees of parent pointers, with each root naming its group.
- `find` follows parents to the root; `union` links one root under another.
- Union by size keeps trees at most $\log_2 n$ deep; path compression flattens them as they're used.

- With both, operations are effectively constant time; without them, a chain makes them O(n).
- `union` returning `False` for items already connected is how you detect cycles.

Exercises

1. **The redundant cable.** A network was a tree until one extra cable added a loop. Given the cables in the order they were added, find the first one that connected two computers that were already connected.
2. **When did everyone connect?** Friendships form at given times. Find the earliest time when everyone belongs to a single group.

PART V

Graphs

CHAPTER 20 · ADVANCED

Graphs

A **graph** is a set of things, called **vertices** (or nodes), and connections between pairs of them, called **edges**. That's all, and it's enough to describe a remarkable range of things: road networks and airline routes, friendships and follower lists, web pages and their links, computers and cables, courses and their prerequisites, tasks and their dependencies, the states of a puzzle and the moves between them. Trees and linked lists are graphs too, with extra rules.

This chapter introduces the vocabulary and the ways to store a graph in memory. The next three chapters are about what you can compute on them.

Words for Graphs

- In an **undirected** graph, an edge connects two vertices both ways, like a friendship or a two-way road. In a **directed** graph, each edge goes one way, like a follow, a link or a one-way street.
- In a **weighted** graph, each edge has a number: a distance, a cost, a travel time.
- Two vertices joined by an edge are **adjacent**, or **neighbors**. A vertex's **degree** is its number of edges; in a directed graph, its **out-degree** and **in-degree** count edges leaving and arriving.
- A **path** is a sequence of vertices, each joined to the next by an edge. A **cycle** is a path that returns to its start. A graph with no cycles that is directed is a **DAG**, a directed acyclic graph.

- A graph is **connected** if there's a path between every pair of vertices. The **connected components** are its maximal connected pieces.
- A graph with V vertices has at most about V^2 edges. One with far fewer, like a road network where each junction meets a handful of roads, is **sparse**; one close to the maximum is **dense**.

Three Ways to Store a Graph

The same small graph, written down three ways:

```
"""One small undirected graph, written down three ways."""

edges = [("A", "B"), ("A", "C"), ("B", "C"), ("C", "D")]
print("edge list:", edges)

adjacency = {}
for u, v in edges:
    adjacency.setdefault(u, []).append(v)
    adjacency.setdefault(v, []).append(u)
print("adjacency list:")
for v, nbrs in sorted(adjacency.items()):
    print(f"  {v}: {nbrs}")

names = sorted(adjacency)
position = {v: i for i, v in enumerate(names)}
matrix = [[0] * len(names) for _ in names]
for u, v in edges:
    matrix[position[u]][position[v]] = matrix[position[v]][position[u]] = 1
print("adjacency matrix:")
print("    " + " ".join(names))
for name, row in zip(names, matrix):
    print(f"  {name} " + " ".join(map(str, row)))
```

```
edge list: [('A', 'B'), ('A', 'C'), ('B', 'C'), ('C', 'D')]
adjacency list:
  A: ['B', 'C']
  B: ['A', 'C']
  C: ['A', 'B', 'D']
  D: ['C']
adjacency matrix:
    A B C D
  A 0 1 1 0
  B 1 0 1 0
  C 1 1 0 1
  D 0 0 1 0
```

- An **edge list** is just the edges. It's the simplest, and the natural input format, but answering "who are B's neighbors?" means scanning every edge.

- An **adjacency list** maps each vertex to a list of its neighbors. Finding a vertex's neighbors is immediate, and it uses memory proportional to V + E.
- An **adjacency matrix** is a V × V grid with a 1 where an edge exists. Checking one edge is a single lookup, but it uses V^2 memory however few edges there are, and finding a vertex's neighbors means scanning a whole row.

For sparse graphs, which are most real graphs, the difference in memory is enormous:

```
"""How many entries each representation stores for a sparse graph:
V vertices, each joined to about 4 others."""

print(
    f"{'vertices':>9} {'edges':>8} {'adjacency list':>15} "
    f"{'matrix cells':>16}"
)
for v in [100, 1_000, 10_000, 100_000]:
    e = 2 * v  # about 4 neighbors each: every edge has two ends
    print(f"{v:>9,} {e:>8,} {2 * e:>15,} {v * v:>16,}")
```

```
 vertices    edges  adjacency list     matrix cells
      100      200             400           10,000
    1,000    2,000           4,000        1,000,000
   10,000   20,000          40,000      100,000,000
  100,000  200,000         400,000   10,000,000,000
```

A road network with 100,000 junctions needs 400,000 adjacency-list entries, but a matrix of ten billion cells. Adjacency lists are the default; a matrix makes sense only for small or dense graphs.

A Graph Class

```
"""A graph as an adjacency map: each vertex maps to its neighbors, and
each neighbor to the weight of the edge (1 when weights don't matter)."""


class Graph:
    def __init__(self, directed=False):
        self.directed = directed
        self._adj = {}  # vertex -> {neighbor: weight}

    @classmethod
    def from_edges(cls, edges, directed=False):
        """Edges as (u, v) or (u, v, weight)."""
        g = cls(directed)
```

```
        for edge in edges:
            g.add_edge(*edge)
        return g

    def add_vertex(self, v):
        self._adj.setdefault(v, {})

    def add_edge(self, u, v, weight=1):
        self.add_vertex(u)
        self.add_vertex(v)
        self._adj[u][v] = weight
        if not self.directed:
            self._adj[v][u] = weight  # an undirected edge goes both ways

    def has_edge(self, u, v):
        return v in self._adj.get(u, {})

    def neighbors(self, v):
        return self._adj[v].keys()

    def weight(self, u, v):
        return self._adj[u][v]

    def vertices(self):
        return list(self._adj)

    def edges(self):
        """Every edge once, as (u, v, weight)."""
        seen = set()
        for u, nbrs in self._adj.items():
            for v, w in nbrs.items():
                if self.directed or (v, u) not in seen:
                    seen.add((u, v))
                    yield u, v, w

    def degree(self, v):
        return len(self._adj[v])

    def __len__(self):
        return len(self._adj)
```

This class stores an **adjacency map**: each vertex maps to a dictionary from neighbor to edge weight. It combines the best of both: neighbors are listed quickly, like an adjacency list, and `has_edge` is an O(1) dictionary lookup, like a matrix. An undirected edge is stored in both directions. `edges()` reports each undirected edge once, by remembering which ones it has already produced.

Many algorithms in the next chapters use a plain dictionary of lists, `{vertex: [neighbors]}`, or of dictionaries, `{vertex: {neighbor: weight}}`, instead of a class. It's the same structure without the methods, and it's what you'll write in an interview.

```
"""Tests for Graph."""

from graph import Graph


def test_undirected_edges_go_both_ways():
    g = Graph.from_edges([("A", "B"), ("B", "C", 5)])
    assert g.has_edge("B", "A") and g.weight("C", "B") == 5
    assert not g.has_edge("A", "C")
    assert sorted(g.neighbors("B")) == ["A", "C"]
    assert g.degree("B") == 2 and len(g) == 3
    assert sorted(g.edges()) == [("A", "B", 1), ("B", "C", 5)]


def test_directed_edges_go_one_way():
    g = Graph.from_edges([(1, 2), (2, 3), (3, 1)], directed=True)
    assert g.has_edge(1, 2) and not g.has_edge(2, 1)
    assert len(list(g.edges())) == 3


def test_lone_vertex():
    g = Graph()
    g.add_vertex("solo")
    assert g.vertices() == ["solo"] and list(g.neighbors("solo")) == []
```

```
...                                                                      [100%]
3 passed in 0.01s
```

Checking an Edge

How long does "is there an edge from u to v?" take with each representation? Each vertex here has ten neighbors:

```
"""Is there an edge from u to v? With neighbor lists, neighbor sets
and a matrix, as the graph grows (each vertex has 10 neighbors)."""

import random

import bench

rng = random.Random(2026)


def make(n):
    nbrs = [rng.sample(range(n), 10) for _ in range(n)]
    lists = nbrs
    sets = [set(x) for x in nbrs]
    matrix = [[False] * n for _ in range(n)]
    for u, row in enumerate(nbrs):
        for v in row:
            matrix[u][v] = True
    return lists, sets, matrix
```

```
sizes = [250, 500, 1_000, 2_000]
built = {n: make(n) for n in sizes}
results = {}
for i, name in enumerate(["lists", "sets", "matrix"]):
    if name == "matrix":

        def query(r):
            return r[7][-1]  # row 7, the last column
    else:

        def query(r):
            return -1 in r[7]  # not there: a list scans all 10

    results[name] = bench.doubling(
        query, lambda n, i=i: (built[n][i],), sizes
    )
bench.table(results)
```

```
         n           lists            sets          matrix
       250         177 ns         92.3 ns           96 ns
       500     179 ns x1.0    92.3 ns x1.0    94.7 ns x1.0
     1,000     179 ns x1.0    89.2 ns x1.0    96.3 ns x1.0
     2,000     184 ns x1.0    90.9 ns x1.0    97.2 ns x1.0
```

All three are flat as the graph grows, because the number of neighbors per vertex stays at ten. The list version scans those ten, so it's slower; with vertices of very high degree it would grow. Sets and the matrix are single lookups. Choosing the representation is choosing which questions are cheap.

> **Real-World Use:** Graphs are often stored without being called graphs. A web crawler's frontier, a package manager's dependency list (each package lists what it needs: an adjacency list), the links in a spreadsheet's formulas, and the tables of who-follows-whom in a social network are all graphs held as some form of adjacency list. Python's standard library even includes a graph algorithm, in the `graphlib` module (Python 3.9 and later), which Chapter 21 uses.

Key Takeaways

- A graph is vertices plus edges; edges can be directed or undirected, weighted or not.
- An adjacency list uses O(V + E) memory and lists neighbors quickly; it's the default for sparse graphs.

- An adjacency matrix checks any edge in O(1) but always uses $O(V^2)$ memory.
- A dictionary of neighbor dictionaries gives fast neighbor lists and O(1) edge checks.
- Most real graphs are sparse.

Exercises

1. **Degrees.** For a directed graph, print each vertex's in-degree and out-degree.
2. **The town judge.** In a town of n people, the judge trusts nobody and is trusted by everybody else. Given who trusts whom, find the judge, or -1 if there isn't one.
3. **Reverse the edges.** Build the transpose of a directed graph, with every edge reversed, and use it to list, for each page of a small website, the pages that link to it.

CHAPTER 21 · ADVANCED

Breadth-First and Depth-First Search

Almost every graph algorithm starts by exploring: starting from one vertex, visit everything reachable from it. There are two fundamental ways to explore, and the difference between them is the difference between a queue and a stack. **Breadth-first search** (BFS) visits vertices in rings: first the start, then everything one edge away, then everything two edges away. **Depth-first search** (DFS) plunges as far as it can down one path, then backs up and tries the next. This chapter builds both, and uses them to find shortest paths, connected components, an order for tasks with prerequisites, and whether a graph can be split in two.

The graphs here are plain dictionaries mapping each vertex to a list of its neighbors, as described in Chapter 20.

Breadth-First Search

```
"""Breadth-first and depth-first search, and what they're good for.

Graphs here are plain dicts mapping each vertex to a list of neighbors,
like the adjacency lists of Chapter 20.
"""

from collections import deque


def bfs(graph, start):
    """Fewest edges from start to every reachable vertex, and each
    vertex's parent on such a shortest path."""
```

```
    dist, parent = {start: 0}, {start: None}
    queue = deque([start])
    while queue:
        v = queue.popleft()
        for w in graph[v]:
            if w not in dist:  # first visit is by a shortest path
                dist[w] = dist[v] + 1
                parent[w] = v
                queue.append(w)
    return dist, parent


def shortest_path(graph, start, goal):
    """A path with the fewest edges, as a list, or None."""
    _, parent = bfs(graph, start)
    if goal not in parent:
        return None
    path = [goal]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return path[::-1]
```

BFS keeps a queue of discovered vertices waiting to be explored. Taking from the front of the queue means vertices are explored in the order they were discovered, so all vertices at distance 1 are explored before any at distance 2, and so on. The first time BFS reaches a vertex, it has found a path with the fewest possible edges, so it records the distance and the **parent**: the vertex it came from. Each vertex is queued once and each edge examined once (twice in an undirected graph): **O(V + E)**.

The parents form a tree, the **BFS tree**, and following parents back from any vertex traces a shortest path to the start. `shortest_path` does exactly that, then reverses the list.

```
"""Fewest stops between stations on a small (fictional) metro."""

from search import bfs, dfs, shortest_path

lines = {
    "Red": ["Harbor", "Market", "Central", "Park", "Zoo"],
    "Blue": ["Airport", "Central", "Museum", "Stadium"],
    "Green": ["Market", "Museum", "University"],
}
metro = {}
for stations in lines.values():
    for a, b in zip(stations, stations[1:]):
        metro.setdefault(a, []).append(b)
        metro.setdefault(b, []).append(a)

print(" -> ".join(shortest_path(metro, "Harbor", "Stadium")))
print(" -> ".join(shortest_path(metro, "Zoo", "University")))
dist, _ = bfs(metro, "Central")
for stops in sorted(set(dist.values())):
```

```
    names = [s for s, d in dist.items() if d == stops]
    print(f"{stops} away from Central: {', '.join(names)}")
print("depth-first from Harbor:")
print(", ".join(dfs(metro, "Harbor")))
```

```
Harbor -> Market -> Museum -> Stadium
Zoo -> Park -> Central -> Museum -> University
0 away from Central: Central
1 away from Central: Market, Park, Airport, Museum
2 away from Central: Harbor, Zoo, Stadium, University
depth-first from Harbor:
Harbor, Market, Central, Park, Zoo, Airport, Museum, Stadium, University
```

The route from Harbor to Stadium changes from the Red line to the Green line at Market, then to the Blue line at Museum. BFS counts stops, not distances; Chapter 22 handles weighted edges.

Depth-First Search

```
def dfs(graph, start):
    """Vertices in depth-first order, using an explicit stack."""
    seen, order, stack = set(), [], [start]
    while stack:
        v = stack.pop()
        if v in seen:
            continue
        seen.add(v)
        order.append(v)
        for w in reversed(graph[v]):  # so the first neighbor comes next
            if w not in seen:
                stack.append(w)
    return order


def components(graph):
    """The connected components of an undirected graph."""
    seen, out = set(), []
    for v in graph:
        if v not in seen:
            part = dfs(graph, v)
            seen.update(part)
            out.append(part)
    return out
```

DFS replaces the queue with a stack. Taking the most recently discovered vertex first means following one path as deep as possible before backing up. Neighbors are pushed in reverse so that the first neighbor is explored first, the same order a recursive DFS would use. A vertex can be pushed more than

once (by different neighbors) before it's visited, so `dfs` checks `seen` when it pops. Also O(V + E).

DFS is the natural tool for exploring everything: `components` runs a DFS from each vertex not yet seen, and each run collects one connected component. BFS would work just as well here; what matters is visiting everything reachable.

> **Tip:** DFS is often written recursively, and it's elegant. But a DFS can go as deep as the graph has vertices, which in Python means a `RecursionError` on any large graph. Chapter 28 shows it happen on a 100 × 100 grid. Use the explicit stack.

Ordering Tasks: Topological Sort

Some things must happen before others: you must take Data Structures before Algorithms, install a library before the program that needs it, compile a file before linking. In a directed graph where an edge A → B means "A comes before B", a **topological order** lists every vertex so that every edge points forward. One exists exactly when the graph has no cycles.

```
def topological_order(graph):
    """Order a directed graph so every edge points forward (Kahn's
    algorithm), or return None if there is a cycle."""
    indegree = dict.fromkeys(graph, 0)
    for v in graph:
        for w in graph[v]:
            indegree[w] += 1
    ready = deque(v for v in graph if indegree[v] == 0)
    order = []
    while ready:
        v = ready.popleft()
        order.append(v)
        for w in graph[v]:
            indegree[w] -= 1
            if indegree[w] == 0:
                ready.append(w)
    return order if len(order) == len(graph) else None
```

This is **Kahn's algorithm**. A vertex's **in-degree** counts the edges pointing into it: the things that must come before it. Vertices with in-degree 0 can go first. Each time one is placed, its outgoing edges are crossed off, lowering its neighbors' in-degrees; any that reach 0 are ready. If the vertices run out before

all are placed, the remaining ones wait on each other in a cycle, and there's no valid order.

```
"""In which order can the courses be taken? An edge A -> B means A must
come before B: a topological order of the prerequisite graph."""

from search import topological_order

before = {
    "Python": ["Data Structures", "Web"],
    "Math": ["Algorithms", "Statistics"],
    "Data Structures": ["Algorithms"],
    "Algorithms": ["Machine Learning"],
    "Statistics": ["Machine Learning"],
    "Web": [],
    "Machine Learning": [],
}
for i, course in enumerate(topological_order(before), 1):
    print(i, course)
before["Machine Learning"].append("Python")  # a cycle: impossible
print(topological_order(before))
```

```
1 Python
2 Math
3 Data Structures
4 Web
5 Statistics
6 Algorithms
7 Machine Learning
None
```

Once Machine Learning becomes a prerequisite of Python, the courses form a cycle (Python → Data Structures → Algorithms → Machine Learning → Python), and no order exists.

> **Real-World Use:** Python's standard library includes this algorithm as `graphlib.TopologicalSorter` (Python 3.9 and later). Note the direction: you give it, for each node, the nodes it *depends on*, and it returns an order where every node comes after its dependencies.

```
>>> from graphlib import TopologicalSorter
>>> deps = {"app": {"lib", "utils"}, "lib": {"utils"}, "utils":
  set()}
>>> list(TopologicalSorter(deps).static_order())
['utils', 'lib', 'app']
```

Build tools, package managers and task schedulers all order their work this way.

Two Colors

Can a graph's vertices be split into two groups so that every edge goes between the groups, never within one? Such a graph is **bipartite**. Two-color it with BFS: color the start 0, its neighbors 1, their neighbors 0, and so on; if an edge ever joins two vertices of the same color, it's impossible.

```
def two_colors(graph):
    """Color an undirected graph with two colors so no edge joins two
    of the same color (bipartite), or return None if impossible."""
    color = {}
    for start in graph:
        if start in color:
            continue
        color[start] = 0
        queue = deque([start])
        while queue:
            v = queue.popleft()
            for w in graph[v]:
                if w not in color:
                    color[w] = 1 - color[v]
                    queue.append(w)
                elif color[w] == color[v]:
                    return None
    return color
```

A square can be two-colored; a triangle can't: an odd cycle always forces a clash. Exercise 2 uses this to split people who dislike each other into two teams.

Testing

```
"""BFS distances checked by brute force, and topological orders checked
on random directed graphs."""

import itertools
import random

from search import (
    bfs,
    components,
    shortest_path,
    topological_order,
    two_colors,
)
```

```
def random_graph(rng, n, m, directed):
    g = {v: [] for v in range(n)}
    for _ in range(m):
        a, b = rng.randrange(n), rng.randrange(n)
        if a != b and b not in g[a]:
            g[a].append(b)
            if not directed:
                g[b].append(a)
    return g


def brute_distances(g, start):
    """Repeatedly relax every edge: slow but obviously right."""
    dist = {start: 0}
    for _ in range(len(g)):
        for v in list(dist):
            for w in g[v]:
                if dist[v] + 1 < dist.get(w, len(g) + 1):
                    dist[w] = dist[v] + 1
    return dist


def test_bfs_distances():
    rng = random.Random(2026)
    for _ in range(200):
        g = random_graph(rng, rng.randint(1, 15), rng.randint(0, 30), True)
        assert bfs(g, 0)[0] == brute_distances(g, 0)


def test_shortest_path_and_components():
    g = {"a": ["b"], "b": ["a", "c"], "c": ["b"], "x": ["y"], "y": ["x"]}
    assert shortest_path(g, "a", "c") == ["a", "b", "c"]
    assert shortest_path(g, "a", "x") is None
    assert sorted(map(sorted, components(g))) == [
        ["a", "b", "c"],
        ["x", "y"],
    ]


def test_topological_order():
    rng = random.Random(2026)
    for _ in range(300):
        g = random_graph(rng, rng.randint(1, 8), rng.randint(0, 12), True)
        order = topological_order(g)
        # A valid order exists exactly when some permutation works
        exists = any(
            all(p.index(v) < p.index(w) for v in g for w in g[v])
            for p in itertools.permutations(g)
        )
        assert (order is not None) == exists
        if order:
            assert all(
                order.index(v) < order.index(w) for v in g for w in g[v]
            )


def test_two_colors():
    square = {0: [1, 3], 1: [0, 2], 2: [1, 3], 3: [2, 0]}
    triangle = {0: [1, 2], 1: [0, 2], 2: [0, 1]}
```

```
    colors = two_colors(square)
    assert all(colors[v] != colors[w] for v in square for w in square[v])
    assert two_colors(triangle) is None
```

```
....                                                                     [100%]
4 passed in 0.01s
```

The tests compare with slow but obviously correct methods. BFS distances are checked against repeatedly relaxing every edge until nothing improves. Topological sort is checked against brute force: for small random directed graphs, try *every* ordering of the vertices; a valid one exists exactly when Kahn's algorithm finds one.

Timing BFS: When Does the Queue Matter?

BFS needs a queue. Chapter 8 showed that `list.pop(0)` makes a list a slow queue. Does it matter here?

```
"""BFS with a deque as the queue, and with a list using pop(0), on two
graphs: a square grid of n cells, and a star (one center vertex joined
to n others)."""

from collections import deque

import bench


def star(n):
    g = {(0, 0): [(i, 1) for i in range(n)]}
    for i in range(n):
        g[i, 1] = [(0, 0)]
    return (g,)


def grid(n):
    side = int(n**0.5)
    g = {}
    for r in range(side):
        for c in range(side):
            g[r, c] = [
                (r + dr, c + dc)
                for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1))
                if 0 <= r + dr < side and 0 <= c + dc < side
            ]
    return (g,)


def bfs_deque(g):
    seen, queue = {(0, 0)}, deque([(0, 0)])
    while queue:
        v = queue.popleft()
```

```
        for w in g[v]:
            if w not in seen:
                seen.add(w)
                queue.append(w)


def bfs_list(g):
    seen, queue = {(0, 0)}, [(0, 0)]
    while queue:
        v = queue.pop(0)
        for w in g[v]:
            if w not in seen:
                seen.add(w)
                queue.append(w)


print("grid of n cells:")
sizes = [10_000, 40_000, 160_000, 640_000]
bench.table(
    {
        "deque": bench.doubling(bfs_deque, grid, sizes),
        "list.pop(0)": bench.doubling(bfs_list, grid, sizes),
    }
)
print("star with n leaves:")
sizes = [10_000, 20_000, 40_000, 80_000]
bench.table(
    {
        "deque": bench.doubling(bfs_deque, star, sizes),
        "list.pop(0)": bench.doubling(bfs_list, star, sizes),
    }
)
```

```
grid of n cells:
         n           deque     list.pop(0)
    10,000         3.51 ms         4.06 ms
    40,000    31.9 ms x9.1    29.3 ms x7.2
   160,000     147 ms x4.6     156 ms x5.3
   640,000     845 ms x5.8     854 ms x5.5
star with n leaves:
         n           deque     list.pop(0)
    10,000         1.76 ms          126 ms
    20,000    4.75 ms x2.7     519 ms x4.1
    40,000    10.7 ms x2.3     2.17 s x4.2
    80,000    28.3 ms x2.6     8.58 s x4.0
```

On the grid, the sizes grow by four times per row and the times grow roughly in step: O(V + E). And surprisingly, `list.pop(0)` is no slower than `deque`. The queue only ever holds the current frontier of the search, a diagonal band of a few hundred cells, and shifting a few hundred references is nearly free.

On the star, where one center vertex is joined to every other, all n leaves enter the queue at once. Now each `pop(0)` shifts up to n items, and the list version is quadratic: at 80,000 leaves it takes about 300 times longer than `deque`. The

lesson is the one from Chapter 8, with a refinement: a slow operation only hurts when the structure it works on gets big. Since you rarely know in advance how big a BFS queue will get, use `deque`.

Key Takeaways

- BFS explores in rings using a queue; the first visit to each vertex is along a path with the fewest edges.
- DFS explores deep first using a stack; it's the natural way to explore everything reachable.
- Both are O(V + E). Write DFS with an explicit stack in Python to avoid the recursion limit.
- Kahn's algorithm finds a topological order of a DAG by repeatedly taking vertices with in-degree 0, and detects cycles.
- A graph is bipartite exactly when BFS can two-color it without a clash.

Exercises

1. **A cycle in an undirected graph.** Detect it with DFS: a visited neighbor that isn't the vertex you came from closes a loop.
2. **Two teams.** Given pairs of people who dislike each other, split everyone into two teams with no disliking pair on the same team, or report that it's impossible.

CHAPTER 22 · ADVANCED

Shortest Paths

BFS finds the path with the fewest edges. But on a road map, fewer roads isn't the goal; fewer kilometers is. When edges have weights, the shortest path is the one whose weights add up to the least, and BFS no longer finds it. This chapter builds the two classic algorithms for weighted shortest paths: **Dijkstra's algorithm**, fast but limited to non-negative weights, and **Bellman-Ford**, slower but able to handle negative weights and to detect when no shortest path exists.

Graphs here are dictionaries mapping each vertex to a dictionary of `{neighbor: weight}`.

Dijkstra's Algorithm

```
"""Shortest paths in weighted graphs.

Graphs are dicts: vertex -> {neighbor: edge weight}.
"""

import heapq
import math


def dijkstra(graph, source):
    """Shortest distances from source when no weight is negative, and
    each vertex's parent on its shortest path.

    A heap hands out the closest vertex not yet finished. When a vertex
    comes off the heap its distance is final: any other route would
    pass through a vertex at least as far away, plus a weight >= 0.
    """
    dist, parent = {source: 0}, {source: None}
    heap = [(0, source)]
```

```
    done = set()
    while heap:
        d, v = heapq.heappop(heap)
        if v in done:
            continue  # an old, longer entry for v: skip it
        done.add(v)
        for w, weight in graph[v].items():
            if d + weight < dist.get(w, math.inf):
                dist[w] = d + weight
                parent[w] = v
                heapq.heappush(heap, (dist[w], w))
    return dist, parent


def path_to(parent, goal):
    path = [goal]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return path[::-1]
```

Dijkstra's algorithm grows a set of vertices whose shortest distance is known for certain, starting from the source. At each step it takes the closest vertex not yet finished, finalizes it, and **relaxes** its edges: for each neighbor, if going through this vertex gives a shorter distance than the best known so far, record it.

Why is the closest unfinished vertex finished? Any other route to it would have to leave the finished set through some other unfinished vertex, which is at least as far away, and then add more edges, whose weights are all zero or positive. So no other route can be shorter. That argument is also exactly why Dijkstra needs non-negative weights.

A heap (Chapter 13) hands out the closest vertex. When a shorter distance to a vertex is found, a new entry is pushed instead of updating the old one, since heaps can't easily change an entry's priority; the old, longer entry is skipped when it's popped later, using the `done` set. This "lazy deletion" is the standard way to write Dijkstra in Python. Each edge can push one entry, so the cost is **O(E log E)**, often written O(E log V).

```
"""The shortest drive between towns (fictional, in kilometers)."""

from shortest import dijkstra, path_to

roads = [
    ("Ashford", "Brookton", 12),
    ("Ashford", "Cliffside", 35),
    ("Brookton", "Cliffside", 18),
    ("Brookton", "Dunmore", 40),
```

```
    ("Cliffside", "Dunmore", 9),
    ("Dunmore", "Eastvale", 14),
    ("Cliffside", "Eastvale", 30),
]
graph = {}
for a, b, km in roads:
    graph.setdefault(a, {})[b] = km
    graph.setdefault(b, {})[a] = km

dist, parent = dijkstra(graph, "Ashford")
for town in sorted(dist, key=dist.get):
    route = " -> ".join(path_to(parent, town))
    print(f"{town:>9} {dist[town]:>3} km: {route}")
```

```
  Ashford   0 km: Ashford
 Brookton  12 km: Ashford -> Brookton
Cliffside  30 km: Ashford -> Brookton -> Cliffside
  Dunmore  39 km: Ashford -> Brookton -> Cliffside -> Dunmore
 Eastvale  53 km: Ashford -> Brookton -> Cliffside -> Dunmore -> Eastvale
```

Ashford to Cliffside directly is 35 km, but via Brookton it's 12 + 18 = 30. Every shortest route here happens to go through Brookton and Cliffside, and `path_to` follows the parents back to show it.

> **Interesting Fact:** Edsger Dijkstra designed this algorithm in 1956, in about twenty minutes, sitting at a café terrace in Amsterdam with his fiancée, and without pencil or paper. He needed a demonstration for a new Dutch computer, the ARMAC, so he chose a problem anyone could understand: the shortest route between two cities on a map of the Netherlands. He published it in 1959, and it has been taught ever since.

When Weights Go Negative

Negative weights are rarer than positive ones, but they happen: a rebate on a route, a profit on a trade, a time offset. Here's what they do to Dijkstra:

```
"""Why Dijkstra needs weights >= 0. A negative edge (a rebate, say)
can make a route through a far vertex cheaper after Dijkstra has
already finalized a nearer one."""

from shortest import bellman_ford, dijkstra

g = {"s": {"a": 2, "b": 5}, "b": {"a": -4}, "a": {"t": 1}, "t": {}}
print("Dijkstra:    ", dijkstra(g, "s")[0])
print("Bellman-Ford:", bellman_ford(g, "s")[0])
```

```
Dijkstra:     {'s': 0, 'a': 1, 'b': 5, 't': 3}
Bellman-Ford: {'s': 0, 'a': 1, 'b': 5, 't': 2}
```

Dijkstra finalizes `a` at distance 2 and computes `t` = 3 from it. Later it finds the route s → b → a, which costs 5 - 4 = 1, so it updates `a`, but `a` is already finished, and `t` is never recomputed. The answer for `t`, 3, is wrong: the true distance is 2.

Bellman-Ford

```
def bellman_ford(graph, source):
    """Shortest distances even with negative weights: relax every edge
    V - 1 times. Raises ValueError if a negative cycle can be reached,
    because then there is no shortest path."""
    dist, parent = {source: 0}, {source: None}
    for _ in range(len(graph) - 1):
        changed = False
        for v in list(dist):
            for w, weight in graph[v].items():
                if dist[v] + weight < dist.get(w, math.inf):
                    dist[w] = dist[v] + weight
                    parent[w] = v
                    changed = True
        if not changed:
            break  # nothing improved: already finished
    for v in dist:
        for w, weight in graph[v].items():
            if dist[v] + weight < dist[w]:
                raise ValueError("negative cycle")
    return dist, parent
```

Bellman-Ford gives up on finalizing vertices one at a time. Instead it relaxes *every* edge, over and over. After one round, it has the shortest paths that use at most one edge; after k rounds, at most k edges. A shortest path never needs more than V - 1 edges (it would repeat a vertex), so V - 1 rounds are enough: **O(V × E)**. If a round changes nothing, it stops early.

One more round settles a special case. A **negative cycle**, a loop whose weights sum to less than zero, makes "shortest path" meaningless: going around it once more always makes the path shorter. If any edge can still be relaxed after V - 1 rounds, there must be such a cycle, and the function says so.

Testing Against Floyd-Warshall

```
"""Dijkstra and Bellman-Ford checked against Floyd-Warshall, which
tries every vertex as a stop between every pair."""

import math
import random

import pytest

from shortest import bellman_ford, dijkstra, path_to


def random_graph(rng, n, m, low=1):
    g = {v: {} for v in range(n)}
    for _ in range(m):
        a, b = rng.randrange(n), rng.randrange(n)
        if a != b:
            g[a][b] = rng.randint(low, 20)
    return g


def floyd_warshall(g):
    d = {
        (a, b): 0 if a == b else g[a].get(b, math.inf) for a in g for b in g
    }
    for k in g:
        for a in g:
            for b in g:
                d[a, b] = min(d[a, b], d[a, k] + d[k, b])
    return d


def test_both_match_floyd_warshall():
    rng = random.Random(2026)
    for _ in range(200):
        g = random_graph(rng, rng.randint(1, 12), rng.randint(0, 40))
        best = floyd_warshall(g)
        expected = {b: best[0, b] for b in g if best[0, b] < math.inf}
        dist, parent = dijkstra(g, 0)
        assert dist == expected
        assert bellman_ford(g, 0)[0] == expected
        for v in dist:  # the path's weights add up to the distance
            p = path_to(parent, v)
            assert sum(g[a][b] for a, b in zip(p, p[1:])) == dist[v]


def test_negative_edges_and_cycles():
    g = {"s": {"a": 4, "b": 1}, "a": {"c": 1}, "b": {"a": -2}, "c": {}}
    assert bellman_ford(g, "s")[0] == {"s": 0, "b": 1, "a": -1, "c": 0}
    g["c"]["b"] = -1  # b -> a -> c -> b now sums to -2
    with pytest.raises(ValueError):
        bellman_ford(g, "s")
```

```
..                                                                   [100%]
2 passed in 0.01s
```

The reference here is a third algorithm, **Floyd-Warshall**, which computes shortest distances between *all* pairs of vertices with three nested loops: for each vertex k, check whether going through k improves the route between every pair. It's $O(V^3)$, fine for the test's small graphs, and short enough to be obviously right. On 200 random graphs, both algorithms must match it, and every reported path must add up to its distance. A second test checks a negative edge and then a negative cycle.

Timing

```
"""Dijkstra against Bellman-Ford on random graphs with n vertices and
4n edges."""

import random

import bench
from shortest import bellman_ford, dijkstra

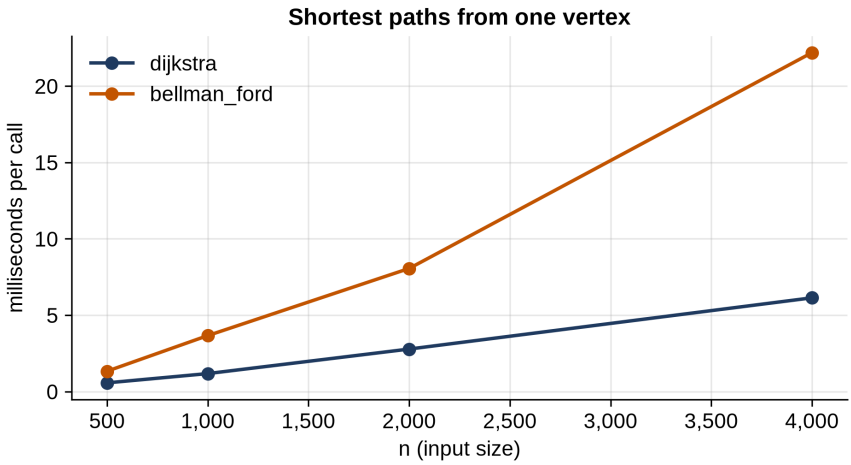
rng = random.Random(2026)


def make(n):
    g = {v: {} for v in range(n)}
    for v in range(n):
        g[v][(v + 1) % n] = rng.randint(1, 100)  # a ring: all reachable
        for _ in range(3):
            g[v][rng.randrange(n)] = rng.randint(1, 100)
    return g, 0


sizes = [500, 1_000, 2_000, 4_000]
results = {
    "dijkstra": bench.doubling(dijkstra, make, sizes),
    "bellman_ford": bench.doubling(bellman_ford, make, sizes),
}
bench.table(results)
bench.chart(results, "ch22_shortest.png", "Shortest paths from one vertex")
```

```
         n        dijkstra    bellman_ford
       500          570 us         1.33 ms
     1,000    1.18 ms x2.1    3.66 ms x2.7
     2,000    2.78 ms x2.4    8.06 ms x2.2
     4,000    6.14 ms x2.2    22.2 ms x2.8
```

Dijkstra grows a little faster than linearly, as O(E log V) predicts. Bellman-Ford is several times slower, but it doesn't show its O(V × E) worst case here: on random graphs, distances settle after a handful of rounds, and the early stop kicks in. Its worst case needs graphs where shortest paths have



Dijkstra and Bellman-Ford on random sparse graphs.

many edges, like a long chain listed in the wrong order. Use Dijkstra whenever weights are non-negative.

> **Real-World Use:** Both algorithms route the internet. In **OSPF** (Open Shortest Path First, RFC 2328), the routing protocol used inside many large networks, every router learns the full map of links and their costs, and computes its own shortest-path tree with Dijkstra's algorithm. The older **RIP** (RFC 2453) is a distance-vector protocol from the Bellman-Ford family: each router only learns its neighbors' distance estimates, and the network converges round by round, just like the relaxation rounds above.

Key Takeaways

- Dijkstra's algorithm finalizes the closest unfinished vertex each step; with a heap and lazy deletion it's O(E log V).
- Dijkstra requires non-negative weights; a negative edge can make it finalize a vertex too early.
- Bellman-Ford relaxes every edge V - 1 times: O(V × E), with negative weights allowed, and one extra round detects negative cycles.
- Keep each vertex's parent to reconstruct paths.

- For unweighted graphs, plain BFS is a shortest-path algorithm, and the fastest.

Exercises

1. **Network delay.** A signal starts at one server and travels along links with given delays. How long until every server has received it? (The largest shortest distance, or -1 if some server is unreachable.)
2. **Cheapest flight with at most k stops.** Find the cheapest route using at most k + 1 flights. (Hint: Bellman-Ford with exactly k + 1 rounds, where each round only builds on the previous round's prices.)

CHAPTER 23 · ADVANCED

Minimum Spanning Trees

A company wants to connect six offices with fiber. Laying cable between some pairs of offices is possible, each at a known cost. What's the cheapest set of cables that connects every office to every other, directly or through others?

The answer is a **minimum spanning tree** (MST): a subset of the edges that connects all the vertices (spanning), has no cycles (a tree, since any cycle has a redundant edge), and has the smallest possible total weight. With V vertices, a spanning tree has exactly V - 1 edges. This chapter builds the two classic algorithms, Kruskal's and Prim's, and checks them against brute force.

Kruskal's Algorithm

Kruskal's algorithm is greedy in the simplest possible way: go through the edges from lightest to heaviest, and take each one unless it would close a loop. Detecting loops is exactly what union-find (Chapter 19) does: an edge closes a loop when its two ends are already connected.

```
"""Minimum spanning trees: connect every vertex of a weighted,
undirected graph with the least total edge weight."""

import heapq

from union_find import UnionFind  # a copy of Chapter 19's


def kruskal(n, edges):
    """Vertices 0..n-1; edges as (weight, u, v). Take edges from the
    lightest up, skipping any that would close a loop."""
    uf, tree = UnionFind(n), []
    for weight, u, v in sorted(edges):
```

```
        if uf.union(u, v):
            tree.append((weight, u, v))
            if len(tree) == n - 1:
                break
    return tree
```

Sorting the edges dominates: **O(E log E)**. The union-find work is nearly linear. The chapter folder holds a copy of Chapter 19's `union_find.py`, so the code runs on its own.

Prim's Algorithm

Prim's algorithm grows a single tree from a starting vertex. At each step it adds the lightest edge that leads from the tree to a vertex not yet in it. A heap holds the candidate edges:

```
def prim(graph, start):
    """Graph: vertex -> {neighbor: weight}. Grow one tree from start,
    always adding the lightest edge that reaches a new vertex."""
    in_tree, tree = {start}, []
    heap = [(w, start, v) for v, w in graph[start].items()]
    heapq.heapify(heap)
    while heap and len(in_tree) < len(graph):
        weight, u, v = heapq.heappop(heap)
        if v in in_tree:
            continue
        in_tree.add(v)
        tree.append((weight, u, v))
        for x, w in graph[v].items():
            if x not in in_tree:
                heapq.heappush(heap, (w, v, x))
    return tree
```

It looks a lot like Dijkstra's algorithm, and it should: both grow a set of finished vertices using a heap. The difference is the priority. Dijkstra orders vertices by their total distance from the start; Prim orders them by the weight of the single edge that would attach them. As with Dijkstra, stale heap entries (edges to vertices already in the tree) are skipped when popped. **O(E log V)**.

Why Greedy Works Here

Most greedy algorithms are wrong (Chapter 31 shows one that is). Why are these right? Because of the **cut property**: split the vertices into any two groups; the lightest edge crossing between the groups belongs to some minimum spanning tree. If an MST didn't use it, swapping it in for a heavier

crossing edge on the same cycle would make a cheaper spanning tree. Kruskal's next edge and Prim's next edge are each the lightest edge across some split, so every choice is safe.

The Fiber Network

```
"""Lay fiber between six (fictional) offices as cheaply as possible.
Costs are in thousands of dollars."""

from mst import kruskal, prim

offices = ["HQ", "Depot", "Lab", "Shop", "Mill", "Port"]
costs = [
    (7, 0, 1),
    (5, 0, 3),
    (8, 1, 2),
    (9, 1, 3),
    (7, 1, 4),
    (5, 2, 4),
    (15, 3, 4),
    (6, 3, 5),
    (8, 4, 5),
    (11, 0, 5),
]
tree = kruskal(len(offices), costs)
for cost, a, b in tree:
    print(f"{offices[a]:>5} -- {offices[b]:<5} {cost}")
print(
    "total:", sum(c for c, _, _ in tree), "of", sum(c for c, _, _ in costs)
)

graph = {i: {} for i in range(len(offices))}
for c, a, b in costs:
    graph[a][b] = graph[b][a] = c
print("Prim's total:", sum(c for c, _, _ in prim(graph, 0)))
```

```
   HQ -- Shop  5
  Lab -- Mill  5
 Shop -- Port  6
   HQ -- Depot 7
Depot -- Mill  7
total: 30 of 81
Prim's total: 30
```

Five cables, the minimum for six offices, costing 30 of the 81 it would take to lay every possible cable. Prim's algorithm finds a tree with the same total. (When some edges have equal weights, there can be several different minimum spanning trees; the total weight is always the same.)

Testing Against Brute Force

```
"""Kruskal and Prim must find trees of the same, smallest total weight;
on small graphs, checked by trying every set of n - 1 edges."""

import itertools
import random

from mst import kruskal, prim
from union_find import UnionFind


def is_spanning_tree(n, chosen):
    uf = UnionFind(n)
    return len(chosen) == n - 1 and all(
        uf.union(u, v) for _, u, v in chosen
    )


def test_against_brute_force():
    rng = random.Random(2026)
    for _ in range(150):
        n = rng.randint(2, 6)
        edges = [
            (rng.randint(1, 9), i, i + 1) for i in range(n - 1)
        ]  # connected
        edges += [
            (rng.randint(1, 9), rng.randrange(n), rng.randrange(n))
            for _ in range(rng.randint(0, 6))
        ]
        edges = [e for e in edges if e[1] != e[2]]
        best = min(
            sum(w for w, _, _ in combo)
            for combo in itertools.combinations(edges, n - 1)
            if is_spanning_tree(n, combo)
        )
        graph = {v: {} for v in range(n)}
        for w, u, v in edges:
            if w < graph[u].get(v, 99):
                graph[u][v] = graph[v][u] = w
        k, p = kruskal(n, edges), prim(graph, 0)
        assert is_spanning_tree(n, k) and is_spanning_tree(n, p)
        assert sum(w for w, _, _ in k) == best == sum(w for w, _, _ in p)
```

```
.                                                                        [100%]
1 passed in 0.01s
```

For tiny graphs, try every set of V - 1 edges, keep those that form a spanning tree (union-find again: V - 1 edges with no loop), and find the minimum total. Both algorithms must produce valid spanning trees with exactly that total, on 150 random graphs, including graphs with several edges between the same pair of vertices.

Timing

```
"""Kruskal against Prim on random connected graphs: n vertices, 5n
edges."""

import random

import bench
from mst import kruskal, prim

rng = random.Random(2026)


def make(n):
    edges = [(rng.randint(1, 1000), i, i + 1) for i in range(n - 1)]
    edges += [
        (rng.randint(1, 1000), rng.randrange(n), rng.randrange(n))
        for _ in range(4 * n)
    ]
    graph = {v: {} for v in range(n)}
    for w, u, v in edges:
        if u != v:
            graph[u][v] = graph[v][u] = w
    return edges, graph


sizes = [2_000, 4_000, 8_000, 16_000]
built = {n: make(n) for n in sizes}
results = {
    "kruskal": bench.doubling(kruskal, lambda n: (n, built[n][0]), sizes),
    "prim": bench.doubling(prim, lambda n: (built[n][1], 0), sizes),
}
bench.table(results)
```

```
         n         kruskal            prim
     2,000         5.36 ms         7.75 ms
     4,000    12.7 ms x2.4    21.2 ms x2.7
     8,000      38 ms x3.0    54.3 ms x2.6
    16,000     101 ms x2.7     157 ms x2.9
```

Both grow a little faster than linearly, as O(E log E) and O(E log V) predict. Kruskal wins here, because its main cost is one call to `sorted()`, which runs in C; Prim does a heap operation in Python for every edge. On very dense graphs, a version of Prim's algorithm without a heap, which scans an array instead, takes $O(V^2)$, better than any edge-sorting approach (Exercise 1).

> **Interesting Fact:** The problem is older than computers. In 1926, Otakar Borůvka solved it to plan the most economical electrical network for the Moravia region of what's now the Czech Republic. Vojtěch Jarník published the algorithm now usually credited to Prim in 1930; Joseph Kruskal published his in 1956 and Robert Prim his in 1957, in a paper about telephone networks.

Key Takeaways

- A minimum spanning tree connects all V vertices with V - 1 edges of minimum total weight.
- Kruskal: sort the edges, take each one that doesn't close a loop (union-find). O(E log E).
- Prim: grow one tree, always adding the lightest edge to a new vertex (a heap). O(E log V).
- Both are greedy, and both are correct because of the cut property.
- An MST minimizes the total weight; it does not give shortest paths between vertices (that's Chapter 22).

Exercises

1. **Connect the points.** Points on a grid can be joined at a cost equal to their Manhattan distance, |x1 - x2| + |y1 - y2|. Find the minimum total cost to connect them all. Every pair can be joined, so the graph is dense: use Prim's algorithm with a plain array instead of a heap, in $O(V^2)$.
2. **Maximum spanning tree.** Find the spanning tree with the *largest* total weight, for example the strongest set of links in a network, by reusing `kruskal`.

PART VI

Interview Patterns

CHAPTER 24 · ADVANCED

Solving a Problem in an Interview

Coding interviews test something different from everyday programming: solving an unfamiliar problem, under time pressure, while explaining your thinking. The good news is that most interview problems are built on a small number of **patterns**, and the structures from Parts II to V are their building blocks. This part of the book teaches the patterns one per chapter. This chapter gives you the method that ties them together, and the testing habit that catches your bugs before the interviewer does.

A Five-Step Method

1. **Clarify.** Restate the problem in your own words. Ask about the input: how big can it be? Can it be empty? Can values be negative, or repeat? Is the input sorted? What should be returned if there's no answer? These questions often reveal the intended solution: "n can be up to 100,000" rules out $O(n^2)$.
2. **Work examples by hand.** Make a small example and solve it on paper. Then make an awkward one: empty input, one item, all items equal, the answer at the very end. You'll need these for testing anyway.
3. **Start with brute force.** Say the obvious solution, even if it's slow, and its Big O. It proves you understand the problem, it's a fallback, and it's often the starting point for the better solution: "the brute force checks every pair; can I avoid that?"

4. **Find the pattern.** Look for signals in the problem that point to a known technique (the table below), and say what the improved Big O would be before you write the code.
5. **Code, then test.** Write clean code with good names. Then trace your hand examples through it, line by line, out loud. Check the edge cases. State the final time and memory complexity.

One Problem, Three Solutions

The classic opener: given a list of numbers and a target, find two positions whose numbers add up to the target.

```
"""One problem, three solutions: find two positions whose numbers add
up to target. Return them as (i, j) with i < j, or None."""


def two_sum_brute(nums, target):
    """Try every pair: O(n^2) time, O(1) extra memory."""
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return i, j
    return None


def two_sum_sorted(nums, target):
    """Sort the positions by value, then move two pointers inward:
    O(n log n) time, O(n) memory."""
    order = sorted(range(len(nums)), key=nums.__getitem__)
    lo, hi = 0, len(order) - 1
    while lo < hi:
        total = nums[order[lo]] + nums[order[hi]]
        if total == target:
            return tuple(sorted((order[lo], order[hi])))
        if total < target:
            lo += 1
        else:
            hi -= 1
    return None


def two_sum_hash(nums, target):
    """Remember where each number was seen; look up its partner:
    O(n) time, O(n) memory."""
    seen = {}
    for j, x in enumerate(nums):
        if target - x in seen:
            return seen[target - x], j
        seen[x] = j
    return None
```

The brute force checks every pair: $O(n^2)$ time, O(1) memory. Step 4 asks: what is the brute force *really* searching for? For each number x, it looks for `target - x`. "Have I seen this value before?" is a hash-map question, and remembering each value's position as you go makes the whole thing one pass: O(n) time, O(n) memory. Sorting is the middle road: sort the positions by value, then move two pointers inward from the ends (Chapter 25's pattern). O(n log n) time.

```
"""The three two-sum solutions, when no pair exists (the worst case)."""

import random

import bench
from two_sum import two_sum_brute, two_sum_hash, two_sum_sorted

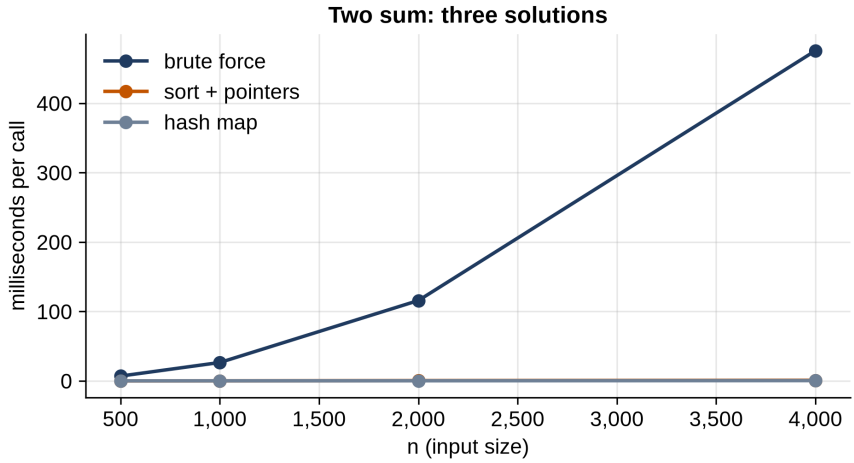
rng = random.Random(2026)


def make(n):
    return [rng.randrange(0, 10**9, 2) for _ in range(n)], 1  # all even


sizes = [500, 1_000, 2_000, 4_000]
results = {
    "brute force": bench.doubling(two_sum_brute, make, sizes),
    "sort + pointers": bench.doubling(two_sum_sorted, make, sizes),
    "hash map": bench.doubling(two_sum_hash, make, sizes),
}
bench.table(results)
bench.chart(results, "ch24_two_sum.png", "Two sum: three solutions")
```

```
         n     brute force  sort + pointers        hash map
       500         6.94 ms          59.3 us         37.2 us
     1,000    26.5 ms x3.8      153 us x2.6    73.9 us x2.0
     2,000     116 ms x4.4      410 us x2.7     174 us x2.3
     4,000     476 ms x4.1      908 us x2.2     383 us x2.2
```

At 4,000 numbers, brute force takes about half a second; the hash map, under half a millisecond, over a thousand times faster. That's the typical payoff of finding the pattern.



Brute force grows quadratically; the sorted and hash-map solutions grow almost linearly.

Patterns and Their Signals

Every pattern has telltale signs in the problem statement:

```
"""Words in a problem statement that point to a pattern."""

SIGNALS = [
    ("sorted array, pair or triple with a sum", "two pointers"),
    (
        "contiguous subarray or substring, longest or shortest",
        "sliding window",
    ),
    ("count subarrays with a sum; seen before; pairs", "hash map"),
    ("sums of ranges, asked many times", "prefix sums"),
    ("fewest steps, shortest path, levels, unweighted", "BFS"),
    ("connected regions, all paths, explore everything", "DFS"),
    ("order with prerequisites", "topological sort"),
    ("k largest, k smallest, k closest, merge k", "heap"),
    ("minimum or maximum value that works, monotone", "binary search"),
    ("count ways, best total, choices that overlap", "dynamic programming"),
    ("all combinations, permutations, placements", "backtracking"),
    ("next greater or smaller element", "monotonic stack"),
]
for signal, pattern in SIGNALS:
    print(f"{pattern:>19}: {signal}")
```

```
       two pointers: sorted array, pair or triple with a sum
     sliding window: contiguous subarray or substring, longest or shortest
           hash map: count subarrays with a sum; seen before; pairs
       prefix sums: sums of ranges, asked many times
               BFS: fewest steps, shortest path, levels, unweighted
               DFS: connected regions, all paths, explore everything
   topological sort: order with prerequisites
              heap: k largest, k smallest, k closest, merge k
      binary search: minimum or maximum value that works, monotone
dynamic programming: count ways, best total, choices that overlap
       backtracking: all combinations, permutations, placements
    monotonic stack: next greater or smaller element
```

The rest of Part VI covers them: two pointers (Chapter 25), sliding window (26), hash maps and prefix sums (27), BFS and DFS on grids and implicit graphs (28), dynamic programming (29 and 30), and greedy algorithms and backtracking (31). Binary search on the answer, heaps, topological sort and monotonic stacks were covered with their structures in Chapters 10, 13, 21 and 8.

What the Constraints Tell You

The input size is a strong hint about the expected complexity. Python runs on the order of ten million simple steps per second (the loops timed in Chapter 3 manage tens of millions). With a few seconds to spare:

Largest n	Fast enough	Typical pattern
about 10 to 20	$O(2^n)$, O(n!)	Backtracking, trying all subsets
about 500	$O(n^3)$	Triple loops, some DP on intervals
about 5,000	$O(n^2)$	All pairs, DP on two sequences
100,000 to 1,000,000	O(n log n), O(n)	Sorting, heaps, two pointers, windows, hashing
beyond that	O(log n), O(1)	Binary search, math

Test Like a Professional: Stress Testing

Every pattern solution in Part VI is checked the same way: against a slow, obviously correct brute force, on thousands of small random inputs. It's called **stress testing**, and it's worth doing whenever you write something clever,

including in an interview, where you can describe it even if there's no time to run it.

```
"""A stress test: compare a fast solution with a brute-force one on
thousands of small random inputs. The first difference is a bug, and
the input is small enough to debug by hand."""

import random

from two_sum import two_sum_brute, two_sum_hash, two_sum_sorted


def valid(answer, nums, target, brute_answer):
    """Is answer right? There may be several right pairs, so check the
    pair itself rather than comparing it with brute force's pair."""
    if answer is None:
        return brute_answer is None
    i, j = answer
    return 0 <= i < j < len(nums) and nums[i] + nums[j] == target


def stress(fast, slow, make_input, tries=5000, seed=2026):
    rng = random.Random(seed)
    for t in range(tries):
        nums, target = make_input(rng)
        answer = fast(nums, target)
        if not valid(answer, nums, target, slow(nums, target)):
            return f"FAILED on try {t}: {nums}, {target} -> {answer}"
    return f"passed {tries} random tests"


def small_case(rng):
    nums = [rng.randint(-5, 5) for _ in range(rng.randint(0, 6))]
    return nums, rng.randint(-6, 6)


def two_sum_buggy(nums, target):
    """Looks fine, but may use the same position twice."""
    seen = {x: i for i, x in enumerate(nums)}
    for i, x in enumerate(nums):
        if target - x in seen:
            return tuple(sorted((i, seen[target - x])))
    return None


print("hash:  ", stress(two_sum_hash, two_sum_brute, small_case))
print("sorted:", stress(two_sum_sorted, two_sum_brute, small_case))
print("buggy: ", stress(two_sum_buggy, two_sum_brute, small_case))
```

```
hash:   passed 5000 random tests
sorted: passed 5000 random tests
buggy:  FAILED on try 9: [-1, -1, 2, -3], 4 -> (2, 2)
```

`valid` checks an answer on its own terms: if the brute force found no pair, there must be none; otherwise the answer must be two different positions, in

order, whose numbers add up to the target. (There can be several right pairs, so comparing with brute force's pair directly would give false alarms.)

The third solution looks right. It builds a dictionary from each value to its position, then looks up each number's partner. But on the ninth random input, `[-1, -1, 2, -3]` with target 4, it returns `(2, 2)`: it paired the 2 with itself. Hand-picked tests would very likely have missed it, because the bug only appears when a number is exactly half the target. The stress test also hands you a tiny input to debug by hand, because it uses small lists.

> **Interview Tip:** Interviewers care about how you think as much as what you write. Talk through each step, state complexities without being asked, and when you finish, test your code yourself: "let me trace through the example... and the empty list... and a list where the answer uses the same value twice."

Key Takeaways

- Most interview problems are built on a few patterns, applied to the structures you already know.
- Clarify, work examples, state the brute force, find the pattern, then code and test.
- Words in the problem signal the pattern; the size of n signals the complexity you need.
- Stress test clever code against a brute force on many small random inputs; it finds bugs that hand-picked tests miss, with small inputs to debug.

Exercises

1. **Two sum on a stream.** Numbers arrive one at a time. After each one, say whether any two numbers seen so far add up to the target. What does each step cost?

CHAPTER 25 · ADVANCED

Two Pointers

Many problems on arrays and strings have a brute-force solution that looks at every pair of positions: $O(n^2)$. The **two-pointer** pattern replaces the pairs with two indexes that each move in one direction only, so the whole job is one pass: O(n). The trick is always the same: find a reason why, at each step, one of the pointers can safely move, because the positions it skips can't be part of the answer.

There are two main shapes. **Opposite ends**: one pointer starts at the left, one at the right, and they move toward each other. **Same direction**: a slow pointer and a fast pointer move left to right, with the fast one reading and the slow one writing. You've already met a third, fast and slow pointers on a linked list, in Chapter 7.

Pair Sum in a Sorted List

```
def pair_sum_sorted(nums, target):
    """Positions of two numbers in a sorted list adding up to target."""
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        total = nums[lo] + nums[hi]
        if total == target:
            return lo, hi
        if total < target:
            lo += 1  # need a bigger sum: only lo can give one
        else:
            hi -= 1  # need a smaller sum: only hi can give one
    return None
```

The list is sorted. Start with the smallest and largest numbers. If their sum is too small, the left number is too small to pair with *anything*, since the right number is already the biggest available, so move `lo` right. If it's too big, the right number is too big for anything, so move `hi` left. Each step rules out one number for good, so it takes at most n steps.

Writing in Place: Same-Direction Pointers

```
def dedupe_sorted(nums):
    """Remove duplicates from a sorted list in place; return the new
    length. `write` trails `read`, keeping each new value once."""
    write = 0
    for read in range(len(nums)):
        if read == 0 or nums[read] != nums[read - 1]:
            nums[write] = nums[read]
            write += 1
    del nums[write:]
    return write
```

To remove duplicates from a sorted list without a second list, `read` scans every position while `write` marks where the next kept value goes. A value is kept when it differs from the one before it. `write` never passes `read`, so nothing unread is ever overwritten. The same shape moves all the zeros to the end:

```
def move_zeros(nums):
    """Move every 0 to the end in place, keeping the order of the rest."""
    write = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write], nums[read] = nums[read], nums[write]
            write += 1
```

Palindromes, Ignoring Punctuation

```
def is_palindrome(text):
    """Ignoring case and anything that isn't a letter or digit."""
    lo, hi = 0, len(text) - 1
    while lo < hi:
        if not text[lo].isalnum():
            lo += 1
        elif not text[hi].isalnum():
            hi -= 1
        elif text[lo].lower() != text[hi].lower():
            return False
        else:
```

```
            lo, hi = lo + 1, hi - 1
    return True
```

Compare the outside characters and move inward, skipping anything that isn't a letter or digit. No need to build a cleaned-up copy of the string first.

The Container With the Most Water

Vertical lines of given heights stand at positions 0, 1, 2, ...; any two lines and the ground form a container, holding water up to the shorter line. Which pair holds the most?

```
def most_water(heights):
    """The most water held between two vertical lines. Moving the
    taller line inward can never help, so move the shorter one."""
    lo, hi, best = 0, len(heights) - 1, 0
    while lo < hi:
        width = hi - lo
        best = max(best, width * min(heights[lo], heights[hi]))
        if heights[lo] < heights[hi]:
            lo += 1
        else:
            hi -= 1
    return best
```

Start with the widest container, the two ends. Every other container is narrower, so it can only hold more if it's taller. Moving the *taller* line inward can't help: the height is still limited by the shorter line, and the width shrinks. So move the shorter one. This is the subtle part of two-pointer problems: proving that the pointer you move can't have been part of a better answer.

Three Sum

Find every distinct triple that adds up to zero. Brute force checks all n(n - 1)(n - 2)/6 triples: $O(n^3)$.

```
def three_sum(nums):
    """Every distinct triple adding up to 0: sort, fix the first item,
    then pair_sum on the rest. O(n^2) instead of O(n^3)."""
    nums, out = sorted(nums), []
    for i in range(len(nums) - 2):
        if i and nums[i] == nums[i - 1]:
            continue  # same first number: same triples again
        lo, hi = i + 1, len(nums) - 1
        while lo < hi:
            total = nums[i] + nums[lo] + nums[hi]
            if total < 0:
```

```
                lo += 1
            elif total > 0:
                hi -= 1
            else:
                out.append((nums[i], nums[lo], nums[hi]))
                lo += 1
                while lo < hi and nums[lo] == nums[lo - 1]:
                    lo += 1
    return out
```

Sort first. Then, for each number as the first of the triple, find pairs in the rest that sum to its negative with the opposite-ends technique. That's n passes of O(n): $O(n^2)$. Skipping equal values, both for the first number and after finding a triple, avoids reporting the same triple twice.

Demos and Tests

```
"""The two-pointer solutions on small examples."""

from two_pointers import (
    dedupe_sorted,
    is_palindrome,
    most_water,
    move_zeros,
    pair_sum_sorted,
    three_sum,
)

print(pair_sum_sorted([1, 3, 4, 6, 8, 11], 10))
nums = [1, 1, 2, 3, 3, 3, 7]
print(dedupe_sorted(nums), nums)
print(is_palindrome("Was it a car or a cat I saw?"))
print(most_water([1, 8, 6, 2, 5, 4, 8, 3, 7]))
print(three_sum([-1, 0, 1, 2, -1, -4]))
nums = [0, 1, 0, 3, 12]
move_zeros(nums)
print(nums)
```

```
(2, 3)
4 [1, 2, 3, 7]
True
49
[(-1, -1, 2), (-1, 0, 1)]
[1, 3, 12, 0, 0]
```

```
"""Each two-pointer solution against a brute-force one, on random
inputs."""

import itertools
import random

from two_pointers import (
```

```
    dedupe_sorted,
    is_palindrome,
    most_water,
    move_zeros,
    pair_sum_sorted,
    three_sum,
)

rng = random.Random(2026)


def small_lists(count=2000, low=-6, high=6, longest=9):
    for _ in range(count):
        yield [
            rng.randint(low, high) for _ in range(rng.randint(0, longest))
        ]


def test_pair_sum_sorted():
    for nums in small_lists():
        nums.sort()
        target = rng.randint(-12, 12)
        found = pair_sum_sorted(nums, target)
        pairs = [
            p
            for p in itertools.combinations(range(len(nums)), 2)
            if nums[p[0]] + nums[p[1]] == target
        ]
        assert (found is None) == (not pairs)
        if found:
            assert nums[found[0]] + nums[found[1]] == target


def test_dedupe_and_move_zeros():
    for nums in small_lists():
        s = sorted(nums)
        expected = sorted(set(nums))
        assert dedupe_sorted(s) == len(expected) and s == expected
        moved = list(nums)
        move_zeros(moved)
        nonzero = [x for x in nums if x]
        assert moved == nonzero + [0] * (len(nums) - len(nonzero))


def test_most_water():
    for heights in small_lists(low=0, high=9):
        best = max(
            (
                (j - i) * min(heights[i], heights[j])
                for i, j in itertools.combinations(range(len(heights)), 2)
            ),
            default=0,
        )
        assert most_water(heights) == best


def test_three_sum():
    for nums in small_lists():
        expected = sorted(
            {
```

```
                tuple(sorted(c))
                for c in itertools.combinations(nums, 3)
                if sum(c) == 0
            }
        )
        assert sorted(three_sum(nums)) == expected


def test_is_palindrome():
    assert is_palindrome("A man, a plan, a canal: Panama")
    assert not is_palindrome("race a car") and is_palindrome("")
```

```
.....                                                                    [100%]
5 passed in 0.01s
```

Every function is stress tested against brute force on 2,000 random small lists: every pair for pair sum and the container, every triple (with `itertools.combinations`) for three sum. The small value range, -6 to 6, guarantees plenty of duplicates, which is where `three_sum`'s skipping logic could go wrong.

Timing

```
"""Three-sum: every triple (O(n^3)) against two pointers (O(n^2))."""

import itertools
import random

import bench
from two_pointers import three_sum

rng = random.Random(2026)


def brute(nums):
    return {
        tuple(sorted(c))
        for c in itertools.combinations(nums, 3)
        if sum(c) == 0
    }


def make(n):
    return ([rng.randint(-10_000, 10_000) for _ in range(n)],)


sizes = [100, 200, 400, 800]
results = {
    "all triples": bench.doubling(brute, make, sizes),
    "two pointers": bench.doubling(three_sum, make, sizes),
}
bench.table(results)
```

```
      n     all triples    two pointers
    100         13.1 ms          321 us
    200     113 ms x8.6    1.25 ms x3.9
    400     903 ms x8.0    5.39 ms x4.3
    800     7.54 s x8.4    21.6 ms x4.0
```

The brute force grows eightfold with each doubling: $O(n^3)$. Two pointers grow fourfold: $O(n^2)$. At 800 numbers, it's the difference between 7.5 seconds and about 20 milliseconds.

> **Interview Tip:** When a problem says "sorted", think two pointers or binary search. When it says "in place" or "O(1) extra memory", think a read pointer and a write pointer. When it's about pairs, consider sorting first: O(n log n) to sort is cheap next to $O(n^2)$.

Key Takeaways

- Two pointers replace an $O(n^2)$ scan of pairs with one O(n) pass.
- Opposite-end pointers work on sorted data: each step rules out one item for good.
- Same-direction read and write pointers rewrite a list in place.
- The key to every two-pointer solution is the argument for why the skipped positions can't hold the answer.
- Three sum = sort + a two-pointer pair search for each first item: $O(n^2)$.

Exercises

1. **Sorted squares.** Given a sorted list that may include negative numbers, return the squares in sorted order in O(n). (Hint: the largest square is at one of the ends; fill the answer from the back.)
2. **Trapping rain water.** Bars of given heights stand side by side; how much rain water is trapped between them? Solve it in O(n) time and O(1) extra memory with two pointers.
3. **Subsequence.** Is one string a subsequence of another (its letters appear in order, not necessarily together)?

CHAPTER 26 · ADVANCED

Sliding Window

"Find the longest run of...", "the shortest substring that...", "the best sum of k neighboring...": problems about **contiguous** runs of a list or string have an $O(n^2)$ brute force that tries every start and every end. The **sliding window** pattern is a special case of same-direction two pointers. A window `[left, right]` moves along the sequence: the right edge takes in one new item per step, and the left edge lets items go when the window breaks the problem's rule. Each item enters the window once and leaves it at most once, so the whole scan is O(n), even when there's a `while` loop inside the `for` loop.

There are two kinds. A **fixed-size** window always holds k items: each step adds one on the right and drops one on the left. A **variable-size** window grows and shrinks as needed.

A Fixed-Size Window

```
def max_sum_k(nums, k):
    """The largest sum of k neighboring numbers (a fixed-size window)."""
    window = sum(nums[:k])
    best = window
    for right in range(k, len(nums)):
        window += nums[right] - nums[right - k]  # slide by one
        best = max(best, window)
    return best
```

The largest sum of k neighboring numbers. Summing each window from scratch costs O(k) per window, O(n × k) in total. But two neighboring windows share all but two numbers: sliding by one adds the new number on

the right and subtracts the one that fell off the left. One addition and one subtraction per step: O(n).

Variable-Size Windows

Longest Without Repeats

```
def longest_unique(text):
    """Length of the longest substring with no repeated character."""
    last_seen, left, best = {}, 0, 0
    for right, ch in enumerate(text):
        if last_seen.get(ch, -1) >= left:
            left = last_seen[ch] + 1  # jump past the earlier copy
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

The longest substring with no repeated character. The window holds a substring without repeats. When the new character `ch` already appears inside the window, the window must start after that earlier copy, so `left` jumps straight there; a dictionary of last positions finds it in O(1). The check `>= left` matters: a copy from *before* the window is no longer a repeat.

Shortest With Enough

```
def shortest_with_sum(nums, target):
    """Length of the shortest run of positive numbers with sum >= target,
    or 0. Grow on the right; shrink from the left while still enough."""
    left, total, best = 0, 0, 0
    for right, x in enumerate(nums):
        total += x
        while total >= target:
            if not best or right - left + 1 < best:
                best = right - left + 1
            total -= nums[left]
            left += 1
    return best
```

The shortest run of positive numbers whose sum reaches a target. This is the shape you'll write most often: **grow** on the right until the window is valid, then **shrink** from the left while it stays valid, recording the best window along the way. It depends on the numbers being positive: adding an item can only raise the sum, and dropping one can only lower it. With negative numbers the window loses that guarantee, and you need prefix sums instead (Chapter 27).

The Minimum Window Substring

```
def min_window(text, needed):
    """The shortest substring of text containing every letter of needed
    (with repeats), or ""."""
    need, missing = Counter(needed), len(needed)
    left, best = 0, ""
    for right, ch in enumerate(text):
        if need[ch] > 0:
            missing -= 1
        need[ch] -= 1
        while missing == 0:  # the window has everything: shrink it
            if not best or right - left + 1 < len(best):
                best = text[left : right + 1]
            need[text[left]] += 1
            if need[text[left]] > 0:
                missing += 1
            left += 1
    return best
```

The shortest piece of `text` that contains every letter of `needed`, repeats included. It's the same grow-then-shrink shape, with smarter bookkeeping. `need` counts how many more of each letter the window needs (negative means extras), and `missing` counts the letters still missing in total, so checking "does the window have everything?" is O(1) instead of comparing two counters.

> **Tip:** Every variable-size window solution answers three questions. What does the window hold? What makes it valid? What do you track so you can update that in O(1) as items enter and leave: a sum, a counter, a dictionary of last positions? Answer them before you write the loop.

The Maximum of Every Window

```
def window_maxima(nums, k):
    """The maximum of every window of size k. A deque keeps positions of
    possible maxima with their values decreasing from front to back."""
    q, out = deque(), []
    for right, x in enumerate(nums):
        while q and nums[q[-1]] <= x:
            q.pop()  # smaller and older: can never be a maximum again
        q.append(right)
        if q[0] <= right - k:
            q.popleft()  # fell out of the window
        if right >= k - 1:
            out.append(nums[q[0]])
    return out
```

The maximum of each window of size k needs one more idea, a **monotonic deque**, the cousin of Chapter 8's monotonic stack. The deque holds positions whose values decrease from front to back, so the front is always the window's maximum. When a new number arrives, every smaller number at the back can be thrown away for good: it's both smaller and older than the newcomer, so it can never be a maximum while the newcomer is in the window. The front leaves when it slides out of the window. Each position is added once and removed at most once: O(n), whatever k is.

Demos and Tests

```
"""The sliding-window solutions on small examples."""

from window import (
    longest_unique,
    max_sum_k,
    min_window,
    shortest_with_sum,
    window_maxima,
)

print(max_sum_k([2, 1, 5, 1, 3, 2], 3))
print(longest_unique("abcabcbb"), longest_unique("pwwkew"))
print(shortest_with_sum([2, 3, 1, 2, 4, 3], 7))
print(repr(min_window("ADOBECODEBANC", "ABC")))
print(window_maxima([1, 3, -1, -3, 5, 3, 6, 7], 3))
```

```
9
3 3
2
'BANC'
[3, 3, 5, 5, 6, 7]
```

```
"""Each sliding-window solution against a brute-force one."""

import random

from window import (
    longest_unique,
    max_sum_k,
    min_window,
    shortest_with_sum,
    window_maxima,
)

rng = random.Random(2026)


def test_max_sum_and_maxima():
    for _ in range(2000):
```

```
        nums = [rng.randint(-9, 9) for _ in range(rng.randint(1, 12))]
        k = rng.randint(1, len(nums))
        windows = [nums[i : i + k] for i in range(len(nums) - k + 1)]
        assert max_sum_k(nums, k) == max(map(sum, windows))
        assert window_maxima(nums, k) == [max(w) for w in windows]


def test_longest_unique():
    for _ in range(2000):
        s = "".join(rng.choices("abcd", k=rng.randint(0, 12)))
        best = max(
            (
                j - i
                for i in range(len(s))
                for j in range(i + 1, len(s) + 1)
                if len(set(s[i:j])) == j - i
            ),
            default=0,
        )
        assert longest_unique(s) == best


def test_shortest_with_sum():
    for _ in range(2000):
        nums = [rng.randint(1, 9) for _ in range(rng.randint(0, 12))]
        target = rng.randint(1, 30)
        lengths = [
            j - i
            for i in range(len(nums))
            for j in range(i + 1, len(nums) + 1)
            if sum(nums[i:j]) >= target
        ]
        assert shortest_with_sum(nums, target) == min(lengths, default=0)


def test_min_window():
    from collections import Counter

    for _ in range(1000):
        s = "".join(rng.choices("abc", k=rng.randint(0, 10)))
        t = "".join(rng.choices("abc", k=rng.randint(1, 3)))
        fits = [
            s[i:j]
            for i in range(len(s))
            for j in range(i + 1, len(s) + 1)
            if not Counter(t) - Counter(s[i:j])
        ]
        found = min_window(s, t)
        assert len(found) == min(map(len, fits), default=0)
        assert not fits or not Counter(t) - Counter(found)
```

```
....                                                                 [100%]
4 passed in 0.01s
```

Every function is checked against a brute force that looks at every window or every substring, on thousands of small random inputs. Short strings over a

three- or four-letter alphabet make repeats common, which is where window logic usually breaks.

Timing

```
"""Maximum of every window of size k = n / 10: recomputing each window
against the monotonic deque."""

import random

import bench
from window import window_maxima

rng = random.Random(2026)

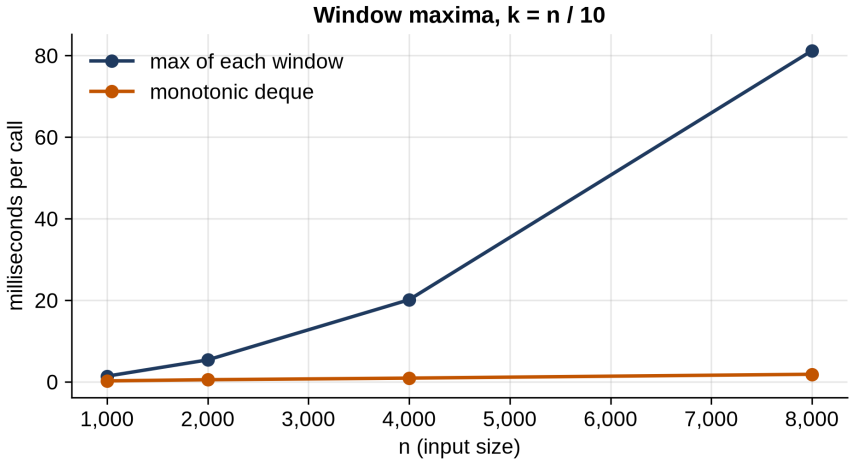

def brute(nums, k):
    return [max(nums[i : i + k]) for i in range(len(nums) - k + 1)]


def make(n):
    return [rng.random() for _ in range(n)], n // 10


sizes = [1_000, 2_000, 4_000, 8_000]
results = {
    "max of each window": bench.doubling(brute, make, sizes),
    "monotonic deque": bench.doubling(window_maxima, make, sizes),
}
bench.table(results)
bench.chart(results, "ch26_window.png", "Window maxima, k = n / 10")
```

```
         n  max of each window  monotonic deque
     1,000             1.39 ms           232 us
     2,000        5.38 ms x3.9      516 us x2.2
     4,000        20.1 ms x3.7      905 us x1.8
     8,000        81.2 ms x4.0     1.85 ms x2.0
```

With k = n / 10, recomputing `max()` over each window does about n × k = n^2 / 10 comparisons, so the time quadruples with each doubling, even though `max()` runs in C. The monotonic deque doubles: O(n). At 8,000 numbers, it's over 40 times faster.



Recomputing each window's maximum grows quadratically; the monotonic deque grows linearly.

Real-World Use: The sliding window is how TCP, the protocol under most internet traffic, controls its flow. The receiver advertises a *window*: how many more bytes it has room for. The sender may have at most that many unacknowledged bytes in flight. As acknowledgments arrive, the window's left edge moves forward and the sender can send more; if the receiver's buffer fills up, it advertises a window of zero and the sender pauses. Rate limiters ("at most 100 requests in any 60 seconds") and moving averages on dashboards are sliding windows too.

Interview Tip: The signal words are "contiguous", "substring", "subarray" and "consecutive", together with "longest", "shortest", "at most k" or "exactly k". If the problem says "subsequence" instead (items in order but not necessarily together), it's usually dynamic programming (Chapter 30), not a window.

Key Takeaways

- A sliding window replaces every start-and-end pair with one pass in which each item enters once and leaves once: O(n).

- A fixed-size window adds one item and drops one per step.
- A variable-size window grows on the right and shrinks on the left while it stays valid (or until it's valid again).
- Keep a running summary of the window (sum, counter, last positions) that updates in O(1).
- Windows on sums need non-negative numbers; with negatives, use prefix sums.
- A monotonic deque gives the maximum (or minimum) of every window in O(n).

Exercises

1. **An anagram in the text.** Does a text contain some rearrangement of a word's letters as a substring? For example, "eidbaooo" contains "ba", an anagram of "ab".
2. **Two kinds of fruit.** A row of fruit trees is given as a list of fruit kinds. You have two baskets, each able to hold only one kind of fruit, and you must pick from consecutive trees. What's the most fruit you can pick?
3. **Longest run of ones.** In a list of 0s and 1s, what's the longest run of 1s you can make if you may flip at most k zeros to ones?

CHAPTER 27 · ADVANCED

Hash Maps and Prefix Sums

Chapter 24's two-sum problem went from $O(n^2)$ to O(n) with one idea: instead of searching the earlier items again for each new one, **remember** them in a hash map, so "have I seen what I need?" costs O(1). That idea solves a surprising share of interview problems. This chapter collects its most useful forms: grouping by a key, counting, set lookups, and its partner, the **prefix sum**, which turns any question about the sum of a range into one subtraction.

Grouping and Counting

```
def group_anagrams(words):
    """Words with the same letters share a key: their sorted letters."""
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())


def first_unique(text):
    """Index of the first character that appears once, or -1."""
    counts = Counter(text)
    return next((i for i, ch in enumerate(text) if counts[ch] == 1), -1)
```

To group anagrams, give every word a **key** that's the same for all words in a group: its letters in sorted order ("eat", "tea" and "ate" all become "aet"). A `defaultdict(list)` collects the words under each key, in one pass. Choosing the key is the whole trick; other problems use a tuple of letter counts, a word's shape, or a normalized form of a record.

To find the first character that appears only once, count first with `Counter` (one pass), then scan again for the first character with a count of 1. Two passes of O(n) beat the $O(n^2)$ of counting each character separately.

Sets for Instant Lookups

```
def longest_consecutive(nums):
    """Length of the longest run of consecutive integers, in any order,
    in O(n): only start counting at a number with no predecessor."""
    present, best = set(nums), 0
    for x in present:
        if x - 1 not in present:
            length = 1
            while x + length in present:
                length += 1
            best = max(best, length)
    return best
```

The longest run of consecutive integers in an unsorted list, such as 1, 2, 3, 4 in `[100, 4, 200, 1, 3, 2]`. Sorting first takes O(n log n). A set does it in O(n): put every number in a set, then only *start* counting at a number whose predecessor is missing, the start of a run. That condition is what keeps it linear. Without it, the loop would count the run 1, 2, 3, 4 from every one of its numbers, $O(n^2)$ on one long run; with it, each run is counted once, from its start, and every number is looked at a constant number of times.

Prefix Sums

```
def prefix_sums(nums):
    """prefix[i] is the sum of the first i numbers, so the sum of
    nums[i:j] is prefix[j] - prefix[i]: any range in one subtraction."""
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    return prefix
```

`prefix[i]` holds the sum of the first i numbers, with `prefix[0] = 0`. Then the sum of any range `nums[i:j]` is `prefix[j] - prefix[i]`: the first j numbers minus the first i. One O(n) pass to build the list, then every range sum in O(1). If you'll ask for many range sums of data that doesn't change, this beats summing each range.

Prefix Sums Meet Hash Maps

The two ideas combine into one of the most useful interview techniques. How many runs of neighboring numbers add up to k, when the numbers can be negative? Chapter 26's sliding window can't help, because with negatives, growing or shrinking the window doesn't move the sum in a predictable direction.

```
def count_subarrays_with_sum(nums, k):
    """How many runs of neighbors add up to k? A run ending here sums to
    k exactly when an earlier prefix sum equals (this prefix - k)."""
    seen = Counter({0: 1})  # the empty prefix
    total = count = 0
    for x in nums:
        total += x
        count += seen[total - k]
        seen[total] += 1
    return count
```

A run ending at the current position sums to k exactly when the running total *before* the run started was `total - k`. So the number of runs ending here is the number of earlier prefix sums equal to `total - k`, which a `Counter` of the prefix sums seen so far answers in O(1). The counter starts with one 0, the empty prefix, for runs that start at the very beginning. It's two sum again, on prefix sums instead of the numbers.

```
"""The hash-map and prefix-sum solutions on small examples."""

from hashing_patterns import (
    count_subarrays_with_sum,
    first_unique,
    group_anagrams,
    longest_consecutive,
    prefix_sums,
)

print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
print(first_unique("swiss"), first_unique("aabb"))
print(longest_consecutive([100, 4, 200, 1, 3, 2]))
sales = [3, 1, 4, 1, 5, 9, 2, 6]
prefix = prefix_sums(sales)
print(prefix, "sum of sales[2:6]:", prefix[6] - prefix[2])
print(count_subarrays_with_sum([1, 2, 3, -3, 1, 2], 3))
```

```
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
1 -1
4
[0, 3, 4, 8, 9, 14, 23, 25, 31] sum of sales[2:6]: 19
6
```

In `[1, 2, 3, -3, 1, 2]`, six runs add up to 3, including `[3, -3, 1, 2]` and `[1, 2, 3, -3]`, which only exist because of the negative number.

Prefix Sums in Two Dimensions

```
"""Prefix sums in two dimensions: the sum of any rectangle of a grid in
four lookups, after one pass to build the table."""

grid = [
    [3, 0, 1, 4],
    [5, 6, 3, 2],
    [1, 2, 0, 1],
]
rows, cols = len(grid), len(grid[0])
# table[r][c]: the sum of the rectangle from (0, 0) up to (r-1, c-1)
table = [[0] * (cols + 1) for _ in range(rows + 1)]
for r in range(rows):
    for c in range(cols):
        table[r + 1][c + 1] = (
            grid[r][c] + table[r][c + 1] + table[r + 1][c] - table[r][c]
        )


def rect_sum(r1, c1, r2, c2):
    """Sum of rows r1..r2 and columns c1..c2, inclusive."""
    return (
        table[r2 + 1][c2 + 1]
        - table[r1][c2 + 1]
        - table[r2 + 1][c1]
        + table[r1][c1]
    )


print(rect_sum(0, 0, 2, 3), rect_sum(1, 1, 2, 2), rect_sum(0, 3, 1, 3))
```

```
28 11 6
```

The same idea works on a grid. `table[r][c]` holds the sum of the whole rectangle above and to the left of cell (r, c). Building it uses **inclusion-exclusion**: the cell, plus the rectangle above, plus the rectangle to the left, minus their overlap, which was added twice. Then the sum of any rectangle takes four lookups, whatever its size.

> **Real-World Use:** This table has two names. Frank Crow introduced it to computer graphics in 1984 as the **summed-area table**, for texture mapping. In 2001, Paul Viola and Michael Jones made it famous in computer vision as the **integral image**: their face detector, fast enough to run in real time, compared the sums of light and dark rectangles at every position and scale of a picture, and the table made each rectangle cost four lookups.

Testing

```
"""The hash-map and prefix-sum solutions against brute force."""

import random

from hashing_patterns import (
    count_subarrays_with_sum,
    first_unique,
    group_anagrams,
    longest_consecutive,
    prefix_sums,
)

rng = random.Random(2026)


def test_group_anagrams():
    groups = group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
    assert sorted(map(sorted, groups)) == [
        ["ate", "eat", "tea"],
        ["bat"],
        ["nat", "tan"],
    ]


def test_first_unique():
    for _ in range(2000):
        s = "".join(rng.choices("abc", k=rng.randint(0, 8)))
        expected = next((i for i, c in enumerate(s) if s.count(c) == 1), -1)
        assert first_unique(s) == expected


def test_longest_consecutive():
    for _ in range(2000):
        nums = [rng.randint(-10, 10) for _ in range(rng.randint(0, 12))]
        best = 0
        for start in nums:
            n = 0
            while start + n in nums:
                n += 1
            best = max(best, n)
        assert longest_consecutive(nums) == best
```

```
def test_prefix_sums_and_subarrays():
    for _ in range(2000):
        nums = [rng.randint(-3, 3) for _ in range(rng.randint(0, 10))]
        prefix = prefix_sums(nums)
        k = rng.randint(-4, 4)
        brute = 0
        for i in range(len(nums)):
            for j in range(i + 1, len(nums) + 1):
                assert prefix[j] - prefix[i] == sum(nums[i:j])
                brute += sum(nums[i:j]) == k
        assert count_subarrays_with_sum(nums, k) == brute
```

```
....                                                                     [100%]
4 passed in 0.01s
```

Timing

```
"""Counting runs with sum k: every run (O(n^2) with running sums)
against prefix sums in a hash map (O(n))."""

import random

import bench
from hashing_patterns import count_subarrays_with_sum

rng = random.Random(2026)


def brute(nums, k):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            count += total == k
    return count


def make(n):
    return [rng.randint(-5, 5) for _ in range(n)], 3


sizes = [500, 1_000, 2_000, 4_000]
results = {
    "every run": bench.doubling(brute, make, sizes),
    "prefix + hash": bench.doubling(count_subarrays_with_sum, make, sizes),
}
bench.table(results)
```

```
        n       every run   prefix + hash
      500         5.14 ms           99 us
    1,000    20.6 ms x4.0     189 us x1.9
    2,000    98.6 ms x4.8     341 us x1.8
    4,000     413 ms x4.2     835 us x2.4
```

The brute force already uses a running total, so it's $O(n^2)$ rather than $O(n^3)$, and it quadruples with each doubling. Prefix sums with a hash map double: at 4,000 numbers, about 500 times faster.

> **Interview Tip:** When a problem asks "how many" or "does any" about pairs or ranges, ask yourself what you'd need to have remembered from earlier positions to answer it at the current one. If that's a value, a count or a position, a dictionary keyed by it usually gives an O(n) solution.

Key Takeaways

- A hash map turns "have I seen this before?" into an O(1) question, so many $O(n^2)$ pair searches become one pass.
- To group items, compute a key that's equal for every item in a group.
- `Counter` and `defaultdict` cover most counting and grouping in a line or two.
- Prefix sums give the sum of any range in O(1) after an O(n) pass; in two dimensions, any rectangle in four lookups.
- Prefix sums in a hash map count ranges with a given sum, even with negative numbers.

Exercises

1. **A nearby duplicate.** Does a list contain two equal values at most k positions apart?
2. **Product of all the others.** For each position, return the product of all the *other* numbers, without using division (which fails when a number is 0). (Hint: the products of everything before each position, and of everything after it.)

3. **Top k words.** Find the k most frequent words in a text, breaking ties alphabetically.

CHAPTER 28 · ADVANCED

BFS and DFS on Grids and Implicit Graphs

Interview problems rarely say "here is a graph". They say "here is a map", "a maze", "a board", "a list of words". The graph is there, but you have to see it: the cells of a grid are vertices, and neighboring cells are joined by edges. Often the graph is never stored at all. Its edges are **implicit**, computed on the fly from a rule, such as "the cells up, down, left and right" or "the words one letter away". Then Chapter 21's BFS and DFS apply unchanged.

A Grid Is a Graph

```
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def neighbors(grid, r, c):
    for dr, dc in STEPS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]):
            yield nr, nc
```

Each cell (r, c) has up to four neighbors. Instead of building adjacency lists, `neighbors` generates them, skipping any that fall off the edge. A grid with R rows and C columns has R × C vertices and fewer than 2 × R × C edges, so a full BFS or DFS of a grid is O(R × C).

Counting Islands with DFS

```
def count_islands(grid):
    """Groups of connected "#" cells. Each new "#" starts a DFS that
    marks its whole island as seen."""
    seen, islands = set(), 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == "#" and (r, c) not in seen:
                islands += 1
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    cell = stack.pop()
                    for n in neighbors(grid, *cell):
                        if grid[n[0]][n[1]] == "#" and n not in seen:
                            seen.add(n)
                            stack.append(n)
    return islands
```

An **island** is a group of `#` cells joined up, down, left or right. Scan the grid; each `#` cell not yet seen starts a new island, and a DFS from it marks the whole island as seen, so it's counted once. This is Chapter 21's `components`, with the edges computed instead of stored. Marking a cell as seen when it's *pushed*, not when it's popped, keeps any cell from entering the stack twice.

> **Real-World Use:** The paint-bucket tool in an image editor does exactly this. Clicking a pixel starts a **flood fill**: a search from that pixel through every neighboring pixel of the same color, all of which get the new color. Counting islands is a flood fill that counts how many times it had to start.

The Shortest Way Through a Maze

```
def shortest_route(grid, start, goal):
    """Fewest steps from start to goal avoiding "#" walls (BFS), and the
    path; (None, []) if the goal can't be reached."""
    parent = {start: None}
    queue = deque([start])
    while queue:
        cell = queue.popleft()
        if cell == goal:
            path = [cell]
            while parent[path[-1]]:
                path.append(parent[path[-1]])
            return len(path) - 1, path[::-1]
        for n in neighbors(grid, *cell):
            if grid[n[0]][n[1]] != "#" and n not in parent:
                parent[n] = cell
```

```
                queue.append(n)
    return None, []
```

BFS finds the route with the fewest steps, avoiding walls. The `parent` dictionary does double duty: it records how each cell was reached, and being in it means the cell has been seen.

```
"""Find the shortest way through a maze and draw it."""

from grids import count_islands, shortest_route, word_ladder

maze = [
    "S..#......",
    ".#.#.####.",
    ".#...#....",
    ".####.#.#.",
    "......#.#G",
]
start, goal = (0, 0), (4, 9)
steps, path = shortest_route(maze, start, goal)
drawn = [list(row) for row in maze]
for r, c in path[1:-1]:
    drawn[r][c] = "*"
print(f"{steps} steps:")
for row in drawn:
    print("".join(row))

islands = ["##..#", "#...#", "..#..", "....#"]
print(count_islands(islands), "islands")
words = ["hot", "dot", "dog", "lot", "log", "cog"]
print(word_ladder("hit", "cog", words), "changes from hit to cog")
```

```
17 steps:
S**#******
.#*#*####*
.#***#...*
.####.#.#*
......#.#G
4 islands
4 changes from hit to cog
```

The `word_ladder` call in the demo is explained below.

Multi-Source BFS

Every minute, rot spreads from each rotten orange to the fresh oranges next to it. How many minutes until none is fresh?

```
def minutes_to_rot(grid):
    """Every minute, rot spreads from each rotten orange ("R") to fresh
    neighbors ("F"). Minutes until none is fresh, or -1. A BFS that
    starts from all rotten oranges at once (multi-source BFS)."""
    rows = [list(row) for row in grid]
    queue = deque(
        (r, c)
        for r, row in enumerate(rows)
        for c, x in enumerate(row)
        if x == "R"
    )
    fresh = sum(row.count("F") for row in rows)
    minutes = 0
    while queue and fresh:
        minutes += 1
        for _ in range(len(queue)):  # one whole level = one minute
            for nr, nc in neighbors(rows, *queue.popleft()):
                if rows[nr][nc] == "F":
                    rows[nr][nc] = "R"
                    fresh -= 1
                    queue.append((nr, nc))
    return -1 if fresh else minutes
```

The trick is to start the BFS from *all* the rotten oranges at once: put every one of them in the queue at the start. BFS then works outward in rings from all of them together, and the ring number is the minute. Processing exactly `len(queue)` cells per round is how the code knows where one ring ends and the next begins. The same pattern finds the distance from every cell to its nearest exit, hospital or charging station: one BFS from all of them, not one BFS each (Exercise 2).

```
"""Multi-source BFS: rot spreads from every rotten orange at once."""

from grids import minutes_to_rot

crate = ["RFF.", "FF.F", ".FFF", "...R"]
print(minutes_to_rot(crate), "minutes")
print(minutes_to_rot(["RF", ".F", "F."]), "(the last one is cut off)")
```

```
3 minutes
-1 (the last one is cut off)
```

Implicit Graphs: The Word Ladder

Change "hit" into "cog" one letter at a time, where every step must be a real word. What's the fewest number of changes?

```
def word_ladder(start, goal, words):
    """Fewest one-letter changes from start to goal, each step a word in
    words. The graph is implicit: neighbors are generated, not stored."""
    words = set(words)
    dist, queue = {start: 0}, deque([start])
    while queue:
        w = queue.popleft()
        if w == goal:
            return dist[w]
        for i in range(len(w)):
            for ch in "abcdefghijklmnopqrstuvwxyz":
                nxt = w[:i] + ch + w[i + 1 :]
                if nxt in words and nxt not in dist:
                    dist[nxt] = dist[w] + 1
                    queue.append(nxt)
    return None
```

The vertices are words; two words are joined when they differ in exactly one letter. Building that graph would mean comparing every pair of words. Instead, `word_ladder` generates the possible neighbors of each word as it's explored, every one-letter change, and keeps those that are real words: 26 × L candidates for a word of L letters, each checked in O(1) with a set. The demo above finds hit → hot → dot → dog → cog, four changes.

Puzzles, game positions, a lock's combinations, a knight's moves on a chessboard (Exercise 3): whenever you can list the moves from a state, BFS finds the fewest moves to a goal.

Why Not Recursion?

DFS on a grid is often written recursively. Here's why this book doesn't:

```
"""Recursive DFS on a big open grid goes thousands of calls deep."""

import sys


def fill_recursive(grid, r, c, seen):
    if not (0 <= r < len(grid) and 0 <= c < len(grid[0])):
        return
    if (r, c) in seen:
        return
    seen.add((r, c))
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        fill_recursive(grid, r + dr, c + dc, seen)


print("recursion limit:", sys.getrecursionlimit())
for side in (20, 100):
    grid = ["." * side] * side
```

```
    try:
        seen = set()
        fill_recursive(grid, 0, 0, seen)
        print(f"{side} x {side}: filled {len(seen):,} cells")
    except RecursionError:
        print(f"{side} x {side}: RecursionError")
```

```
recursion limit: 1000
20 x 20: filled 400 cells
100 x 100: RecursionError
```

On an open 100 × 100 grid, the recursive search snakes through thousands of cells before it ever backs up, so the calls nest thousands deep, and Python's default limit is 1,000. You could raise the limit with `sys.setrecursionlimit`, but then you have to guess how deep the largest input can go, and every nested call holds a frame in memory until the search backs up. An explicit stack needs no guess: it grows as far as the input needs.

Testing

```
"""Grid searches checked against union-find and a slow relaxation."""

import random

from grids import count_islands, minutes_to_rot, shortest_route

rng = random.Random(2026)


def random_grid(rows, cols, chars, weights):
    return [
        "".join(rng.choices(chars, weights, k=cols)) for _ in range(rows)
    ]


def islands_by_union_find(grid):
    parent = {}

    def find(x):
        while parent[x] != x:
            x = parent[x]
        return x

    cells = [
        (r, c)
        for r, row in enumerate(grid)
        for c, x in enumerate(row)
        if x == "#"
    ]
    for cell in cells:
        parent[cell] = cell
    for r, c in cells:
```

```
        for n in ((r + 1, c), (r, c + 1)):
            if n in parent:
                parent[find(n)] = find((r, c))
    return len({find(c) for c in cells})


def test_islands():
    for _ in range(300):
        g = random_grid(rng.randint(1, 8), rng.randint(1, 8), "#.", (1, 1))
        assert count_islands(g) == islands_by_union_find(g)


def test_shortest_route_length():
    for _ in range(300):
        g = random_grid(rng.randint(1, 7), rng.randint(1, 7), "#.", (1, 2))
        g[0] = "." + g[0][1:]
        goal = (len(g) - 1, len(g[0]) - 1)
        steps, path = shortest_route(g, (0, 0), goal)
        # slow check: relax distances until nothing changes
        inf = 10**9
        d = {
            (r, c): inf
            for r in range(len(g))
            for c in range(len(g[0]))
            if g[r][c] != "#"
        }
        d[(0, 0)] = 0
        for _ in range(len(d)):
            for (r, c), v in list(d.items()):
                for n in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                    if n in d and v + 1 < d[n]:
                        d[n] = v + 1
        expected = d.get(goal, inf)
        assert steps == (None if expected == inf else expected)
        if steps is not None:
            assert len(path) == steps + 1 and path[-1] == goal


def test_rotting():
    assert minutes_to_rot(["RFF", "FFF", "FFF"]) == 4
    assert minutes_to_rot(["RF.", ".FF", "F.F"]) == -1
    assert minutes_to_rot(["..", ".."]) == 0
```

```
...                                                                  [100%]
3 passed in 0.01s
```

Islands are checked against a completely different method, union-find (Chapter 19): join each # cell to its # neighbors, then count the groups. Route lengths are checked against relaxing every cell's distance until nothing changes.

Timing

```
"""Shortest route across an open square grid of n cells, corner to
corner. BFS visits each cell once: time grows in proportion to n."""

import bench
from grids import shortest_route


def make(n):
    side = int(n**0.5)
    grid = ["." * side] * side
    return grid, (0, 0), (side - 1, side - 1)


sizes = [10_000, 40_000, 160_000, 640_000]
results = {"BFS": bench.doubling(shortest_route, make, sizes)}
bench.table(results)
```

```
         n             BFS
    10,000         12.3 ms
    40,000    49.4 ms x4.0
   160,000     220 ms x4.5
   640,000     1.09 s x5.0
```

Each row has four times the cells, and the time grows four to five times: roughly linear, as O(R × C) predicts. The drift above four most likely comes from memory: at 640,000 cells, the `parent` dictionary and its tuples outgrow the processor's fast caches, so each access costs a little more. Big O counts steps; the hardware decides what a step costs.

> **Interview Tip:** For grids, say early how you'll mark cells as seen. A `seen` set is clean; overwriting the grid itself (for example, changing `#` to `.`) saves memory but destroys the input, so ask whether that's allowed. And always check the edges: an empty grid, a single cell, a start that's a wall, a goal that can't be reached.

Key Takeaways

- A grid is a graph: cells are vertices, and neighbors are computed from the four (or eight) directions.
- DFS counts and fills connected regions; BFS finds the fewest steps.

- Multi-source BFS starts from every source at once, giving each cell its distance to the nearest one.
- Implicit graphs generate neighbors on the fly; BFS over game states finds the fewest moves.
- On big grids, write DFS with an explicit stack, not recursion.

Exercises

1. **The largest island.** Return the number of cells in the largest island.
2. **Distance to the nearest exit.** In a grid with walls `#` and exits `E`, find each open cell's distance to its nearest exit.
3. **Knight moves.** What's the fewest number of moves for a chess knight to go from one square of an 8 × 8 board to another?

CHAPTER 29 · ADVANCED

Dynamic Programming I: Memoization and Tables

Chapter 5's recursive Fibonacci was exponential because it solved the same smaller problems again and again, and adding a cache made it linear. That fix has a name: **dynamic programming** (DP). It applies whenever a problem has two properties:

- **Optimal substructure:** the answer can be built from the answers to smaller versions of the same problem.
- **Overlapping subproblems:** those smaller problems repeat, so solving each one once and storing the answer saves work.

There are two ways to write a DP solution. **Top-down** (memoization) writes the natural recursion and adds a cache. **Bottom-up** (tabulation) fills a table from the smallest problems up, with a loop and no recursion. Both do the same work; this chapter writes both and times them.

> **Interesting Fact:** The name was chosen to hide what it meant. In his autobiography, Richard Bellman wrote that in the early 1950s his employer, the RAND Corporation, worked for the US Air Force, and the Secretary of Defense above it, Charles Wilson, disliked the very word "research". So Bellman picked a name nobody could object to: "programming" in the sense of planning a schedule, and "dynamic" because it sounded impressive and described the time-varying problems he studied. Historians have pointed out that Wilson only became Secretary of Defense in 1953, after Bellman's work had begun, so the story may be tidier than the facts.

The Five Questions

Every DP solution comes from answering five questions, in this order:

1. **State:** what does one subproblem look like? ("The number of ways to climb the first i stairs.")
2. **Recurrence:** how is a state's answer built from smaller states? (This is the hard part, and the part to say out loud in an interview.)
3. **Base cases:** which states are answered directly?
4. **Order:** in which order must states be computed, so that every state's ingredients are ready first?
5. **Answer:** which state, or combination of states, is the final answer?

Climbing Stairs

```
def climb_ways(n):
    """Ways to climb n stairs taking 1 or 2 at a time. The last step was
    a 1 or a 2, so ways(n) = ways(n - 1) + ways(n - 2)."""
    a, b = 1, 1  # ways(0), ways(1)
    for _ in range(n - 1):
        a, b = b, a + b
    return b if n else 1
```

How many ways are there to climb n stairs taking one or two at a time? The state is the number of stairs. The recurrence comes from thinking about the *last* move: it was either a single step, from stair n - 1, or a double step, from stair n - 2. So ways(n) = ways(n - 1) + ways(n - 2): Fibonacci again. Since

each state only needs the two before it, two variables replace the whole table: O(n) time, O(1) memory.

House Robber

```
def rob(houses):
    """The most money from houses in a row, never two neighbors. Each
    house: skip it (best so far) or take it (best two back + it)."""
    two_back = one_back = 0
    for money in houses:
        two_back, one_back = one_back, max(one_back, two_back + money)
    return one_back
```

Houses stand in a row, each with some money; a robber can't take from two neighbors (the alarms are linked). What's the most they can take? At each house there are two choices. Skip it: the best total is the best up to the previous house. Take it: its money plus the best up to *two* houses back. The answer is the larger. Again only two earlier states are needed.

Greedy shortcuts fail here. Taking every other house from the first gives 5 + 1 = 6 for `[5, 1, 1, 5]`, but the best is 5 + 5 = 10.

Fewest Coins

What's the fewest coins that make a given amount? With US coins, the greedy method of taking the largest coin that fits works. But with coins 1, 3 and 4, greedy makes 6 as 4 + 1 + 1, three coins, when 3 + 3 takes two (Chapter 31 runs this example). DP considers every choice for the last coin:

```
def fewest_coins_memo(coins, amount):
    """Top-down: recursion plus a cache."""

    @cache
    def best(a):
        if a == 0:
            return 0
        options = [best(a - c) + 1 for c in coins if c <= a]
        return min(options, default=math.inf)

    answer = best(amount)
    return -1 if answer == math.inf else answer


def fewest_coins(coins, amount):
    """Bottom-up: fill a table from 0 up to amount. table[a] is the
    fewest coins that make a (inf if none)."""
```

```
    table = [0] + [math.inf] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and table[a - c] + 1 < table[a]:
                table[a] = table[a - c] + 1
    return -1 if table[amount] == math.inf else table[amount]
```

The top-down version is the recurrence written directly: the fewest coins for amount `a` is 1 plus the best for `a - c`, over every coin `c` that fits. `functools.cache` remembers each answer. The bottom-up version fills `table[a]` for every amount from 1 up, so that `table[a - c]` is always ready when it's needed. Both are O(amount × coins). The bottom-up version makes no recursive calls, so it doesn't run into the recursion limit for large amounts; the top-down version would need `sys.setrecursionlimit` for an amount in the thousands.

Watch the table fill:

```
"""Watch the bottom-up table fill: fewest coins for each amount 0..11,
with coins 1, 2 and 5."""

import math

coins, amount = [1, 2, 5], 11
table = [0] + [math.inf] * amount
for a in range(1, amount + 1):
    for c in coins:
        if c <= a and table[a - c] + 1 < table[a]:
            table[a] = table[a - c] + 1
print("amount: " + " ".join(f"{a:>2}" for a in range(amount + 1)))
print("coins:  " + " ".join(f"{t:>2}" for t in table))
```

```
amount:  0  1  2  3  4  5  6  7  8  9 10 11
coins:   0  1  1  2  2  1  2  2  3  3  2  3
```

Each entry is one more than the best of the entries 1, 2 and 5 places to its left. Eleven takes three coins: 5 + 5 + 1.

Counting Combinations

```
def coin_ways(coins, amount):
    """How many combinations of coins make amount (order doesn't
    matter). Looping over coins on the outside counts each combination
    once."""
    ways = [1] + [0] * amount
    for c in coins:
        for a in range(c, amount + 1):
            ways[a] += ways[a - c]
```

```
    return ways[amount]
```

How many different combinations of coins make the amount? The loop order matters. With the coins on the *outside*, each coin is added to the combinations before moving to the next, so 1 + 2 + 2 and 2 + 1 + 2 are counted once, as the same combination. Swap the loops, and the code counts *sequences*, with each order counted separately. Both are useful; know which one the problem asks for.

Paths Through a Grid

```
def min_path_sum(grid):
    """The cheapest path from top-left to bottom-right moving only right
    or down: each cell's best is its cost plus the better of the cells
    above and to the left."""
    rows, cols = len(grid), len(grid[0])
    best = [[0] * cols for _ in range(rows)]
    for r in range(rows):
        for c in range(cols):
            above = best[r - 1][c] if r else math.inf
            left = best[r][c - 1] if c else math.inf
            start = 0 if r == c == 0 else min(above, left)
            best[r][c] = grid[r][c] + start
    return best[-1][-1]
```

The cheapest path from the top-left to the bottom-right of a grid of costs, moving only right or down. A cell can only be entered from above or from the left, so its best cost is its own cost plus the better of those two. Filling the grid row by row, left to right, guarantees both are ready. Two-dimensional states work exactly like one-dimensional ones.

```
"""The DP solutions on small examples."""

from dp_basics import (
    climb_ways,
    coin_ways,
    fewest_coins,
    min_path_sum,
    rob,
)

print([climb_ways(n) for n in range(1, 11)])
print(rob([2, 7, 9, 3, 1]), rob([5, 1, 1, 5]))
print(fewest_coins([1, 5, 10, 25], 63), fewest_coins([2], 3))
print(coin_ways([1, 2, 5], 5))
print(min_path_sum([[1, 3, 1], [1, 5, 1], [4, 2, 1]]))
```

```
[1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
12 10
6 -1
4
7
```

Testing Against Exhaustive Search

```
"""The DP solutions against exhaustive search on small inputs."""

import itertools
import random

from dp_basics import (
    climb_ways,
    coin_ways,
    fewest_coins,
    fewest_coins_memo,
    min_path_sum,
    rob,
)

rng = random.Random(2026)


def test_climb_ways():
    for n in range(12):
        # every sequence of 1s and 2s with sum n
        count = sum(
            1
            for k in range(n + 1)
            for steps in itertools.product((1, 2), repeat=k)
            if sum(steps) == n
        )
        assert climb_ways(n) == count


def test_rob():
    for _ in range(500):
        houses = [rng.randint(0, 9) for _ in range(rng.randint(0, 9))]
        best = 0
        for mask in range(1 << len(houses)):
            if mask & (mask >> 1) == 0:  # no two neighbors
                best = max(
                    best,
                    sum(h for i, h in enumerate(houses) if mask >> i & 1),
                )
        assert rob(houses) == best


def test_coins():
    for _ in range(300):
        coins = sorted(
            set(rng.randint(1, 9) for _ in range(rng.randint(1, 3)))
        )
        amount = rng.randint(0, 20)
```

```
        combos = [
            combo
            for k in range(amount + 1)
            for combo in itertools.combinations_with_replacement(coins, k)
            if sum(combo) == amount
        ]
        fewest = min((len(c) for c in combos), default=-1)
        assert fewest_coins(coins, amount) == fewest
        assert fewest_coins_memo(coins, amount) == fewest
        assert coin_ways(coins, amount) == len(combos)


def test_min_path_sum():
    for _ in range(300):
        rows, cols = rng.randint(1, 4), rng.randint(1, 4)
        grid = [
            [rng.randint(0, 9) for _ in range(cols)] for _ in range(rows)
        ]
        best = min(
            sum(grid[r][c] for r, c in path) for path in paths(rows, cols)
        )
        assert min_path_sum(grid) == best


def paths(rows, cols):
    """Every right/down path from (0, 0) to the bottom-right corner."""
    moves = rows - 1 + cols - 1
    for downs in itertools.combinations(range(moves), rows - 1):
        r = c = 0
        cells = [(0, 0)]
        for m in range(moves):
            if m in downs:
                r += 1
            else:
                c += 1
            cells.append((r, c))
        yield cells
```

```
....                                                                 [100%]
4 passed in 0.01s
```

DP code is easy to get subtly wrong: an off-by-one in a base case, a loop in the wrong order. Every function here is checked against trying *everything* on small inputs: every sequence of 1- and 2-steps, every allowed set of houses (as bit masks with no two neighboring bits set), every combination of coins, every right-and-down path.

Timing

```
"""Fewest coins: plain recursion, recursion with a cache, and a table."""

import math
import sys

import bench
from dp_basics import fewest_coins, fewest_coins_memo

sys.setrecursionlimit(10_000)
COINS = [1, 5, 10, 25]


def plain(amount):
    if amount == 0:
        return 0
    return min(
        (plain(amount - c) + 1 for c in COINS if c <= amount),
        default=math.inf,
    )


sizes = [20, 25, 30, 35]
results = {
    "plain recursion": bench.doubling(plain, lambda n: (n,), sizes),
    "memoized": bench.doubling(
        fewest_coins_memo, lambda n: (COINS, n), sizes
    ),
    "table": bench.doubling(fewest_coins, lambda n: (COINS, n), sizes),
}
bench.table(results)
```

```
         n  plain recursion        memoized           table
        20           253 us         11.3 us         4.15 us
        25     1.17 ms x4.6    14.9 us x1.3    5.45 us x1.3
        30     5.36 ms x4.6    17.5 us x1.2    6.71 us x1.2
        35     24.4 ms x4.6      20 us x1.1    7.52 us x1.1
```

The amounts here grow by 5 per row, not double. Plain recursion multiplies its time by about 4.6 for every 5 cents: exponential. Already at 35 cents it's over a thousand times slower than the memoized version. The memoized and table versions grow linearly, and the table is about two to three times faster than memoization because it skips the function calls and the cache lookups.

> **Tip:** Write the top-down version first: it's the recurrence, word for word, and the cache makes it fast. Switch to bottom-up when recursion depth is a problem, or when you can see that each state only needs the last row or two, so the whole table can shrink to a few variables.

> **Interview Tip:** Signal words for DP: "how many ways", "the minimum or maximum ... such that", "is it possible to", together with choices that affect later choices. If you find yourself drawing a recursion tree with repeated nodes, say so, and add the cache.

Key Takeaways

- DP applies when the answer is built from answers to smaller, repeating subproblems.
- Define the state, the recurrence, the base cases, the order and the final answer, in that order.
- Top-down is recursion plus `functools.cache`; bottom-up fills a table with a loop.
- Both are usually O(states × choices per state); bottom-up avoids the recursion limit and call overhead.
- If each state only needs the last few, keep a few variables instead of the table.

Exercises

1. **Decode ways.** A message of letters was encoded as digits, A = 1, B = 2, ..., Z = 26, and the spaces were lost. In how many ways can a string of digits be decoded? ("226" can be BZ, VF or BBF.)
2. **Paths with obstacles.** How many right-and-down paths cross a grid from the top-left to the bottom-right, when some cells (`#`) are blocked?
3. **The cheapest climb.** Each stair has a cost you pay when you step on it. You may climb one or two stairs at a time and may start on stair 0 or stair 1. What's the cheapest way to get past the top?

CHAPTER 30 · ADVANCED

Dynamic Programming II: Sequences, Strings and Knapsack

The DP problems of Chapter 29 had one-dimensional states: an amount, a stair, a house. This chapter's problems compare two sequences or choose from a list of items under a limit, and their states are pairs: a position in each string, or an item and a remaining capacity. The tables are two-dimensional, but the five questions from Chapter 29 are the same. Along the way, you'll see the DP that powers every `diff` and the spelling suggestions in your phone, and a DP whose $O(n^2)$ table can be replaced by an O(n log n) binary search.

Longest Common Subsequence

A **subsequence** keeps items in order but may skip some: "ACE" is a subsequence of "ABCDE". The **longest common subsequence** (LCS) of two sequences is the longest sequence that's a subsequence of both.

```
def lcs(a, b):
    """The longest common subsequence of a and b (in order, not
    necessarily side by side). table[i][j] is the LCS length of
    a[:i] and b[:j]; walk back through it to recover the letters."""
    table = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                table[i][j] = table[i - 1][j - 1] + 1
            else:
                table[i][j] = max(table[i - 1][j], table[i][j - 1])
```

```
    out, i, j = [], len(a), len(b)
    while i and j:
        if a[i - 1] == b[j - 1]:
            out.append(a[i - 1])
            i, j = i - 1, j - 1
        elif table[i - 1][j] >= table[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return out[::-1]
```

The state is a pair of prefixes: `table[i][j]` is the LCS length of the first i items of `a` and the first j items of `b`. Look at the last item of each. If they're equal, it belongs in the LCS, and the rest is the LCS of the two shorter prefixes: `table[i - 1][j - 1] + 1`. If they differ, at least one of them isn't in the LCS, so drop one or the other and take the better: `max(table[i - 1][j], table[i][j - 1])`. Filling the table is O(n × m).

The table only stores lengths. To recover the subsequence itself, walk back from the bottom-right corner, retracing the choices: a diagonal step for a match (record the item), otherwise a step toward the larger neighbor. Keeping enough information to reconstruct the answer, not just its size, is a common follow-up question.

```
"""The sequence DPs on small examples."""

from dp_sequences import edit_distance, knapsack, lcs, lis

print("".join(lcs("ABCBDAB", "BDCABA")))
before = ["import os", "x = 1", "y = 2", "print(x)"]
after = ["import os", "import sys", "x = 1", "print(x)"]
print(lcs(before, after))  # the lines a diff keeps
print(edit_distance("kitten", "sitting"), edit_distance("flaw", "lawn"))
items = [(1, 1), (3, 4), (4, 5), (5, 7)]  # (weight, value)
print(knapsack(items, 7))
print(lis([10, 9, 2, 5, 3, 7, 101, 18]))
```

```
BCBA
['import os', 'x = 1', 'print(x)']
3 2
9
4
```

The second call compares two versions of a file, line by line. The LCS is exactly the lines a diff keeps unchanged; every other line in `before` was deleted, and every other line in `after` was added.

> **Real-World Use:** Finding the smallest diff between two files is the LCS problem on their lines. Git's default diff algorithm is Eugene Myers' 1986 algorithm, which solves it without filling the whole table: its running time depends on the size of the difference, so it's fast for the usual case of two versions that differ in a few lines. Git also offers the "patience" and "histogram" algorithms, which often produce diffs that are easier for people to read.

Edit Distance

The **edit distance** (or Levenshtein distance) between two strings is the fewest single-letter insertions, deletions and replacements that turn one into the other. "kitten" becomes "sitting" in three: replace k with s, replace e with i, insert g.

```
def edit_distance(a, b):
    """The fewest single-letter inserts, deletes and replacements that
    turn a into b. Only two rows of the table are kept."""
    prev = list(range(len(b) + 1))  # turning "" into b[:j]: j inserts
    for i in range(1, len(a) + 1):
        cur = [i] + [0] * len(b)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                cur[j] = prev[j - 1]
            else:
                cur[j] = 1 + min(prev[j - 1], prev[j], cur[j - 1])
        prev = cur
    return prev[-1]
```

The state is again a pair of prefixes. If the last letters match, they cost nothing: take the distance of the shorter prefixes. Otherwise, the last edit was one of three, and the code tries all three, plus one:

- **replace** the last letter of `a` with the last letter of `b`: `prev[j - 1]`;
- **delete** the last letter of `a`: `prev[j]`;
- **insert** the last letter of `b`: `cur[j - 1]`.

The base cases are the edges of the table: turning a prefix of length i into the empty string takes i deletions, and the empty string into a prefix of length j takes j insertions. Each row only needs the row above it, so the code keeps two rows instead of the whole table: O(n × m) time, O(m) memory. Here's the full table, to see the pattern:

```
"""The full edit-distance table for kitten -> sitting. Each cell is the
distance between a prefix of one word and a prefix of the other."""

a, b = "kitten", "sitting"
table = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
for i in range(len(a) + 1):
    for j in range(len(b) + 1):
        if i == 0 or j == 0:
            table[i][j] = i + j
        elif a[i - 1] == b[j - 1]:
            table[i][j] = table[i - 1][j - 1]
        else:
            table[i][j] = 1 + min(
                table[i - 1][j - 1], table[i - 1][j], table[i][j - 1]
            )
print("      " + "  ".join(b))
for i, row in enumerate(table):
    label = a[i - 1] if i else " "
    print(f"{label}  " + "  ".join(map(str, row)))
```

```
      s  i  t  t  i  n  g
   0  1  2  3  4  5  6  7
k  1  1  2  3  4  5  6  7
i  2  2  1  2  3  4  5  6
t  3  3  2  1  2  3  4  5
t  4  4  3  2  1  2  3  4
e  5  5  4  3  2  2  3  4
n  6  6  5  4  3  3  2  3
```

The answer, 3, is in the bottom-right corner. Moving diagonally down through matching letters (the `t` `t` and `n` stretches) keeps the number the same.

> **Real-World Use:** Spelling suggestions start from edit distance: words a small number of edits away from what you typed are the candidates. DNA and protein alignment in biology uses the same table with scores for each kind of change, an approach published by Needleman and Wunsch in 1970.

The 0/1 Knapsack

A knapsack can carry a limited weight. Each item has a weight and a value, and each can be taken at most once (that's the "0/1": take it or leave it). What's the most value that fits?

```
def knapsack(items, capacity):
    """The most value from (weight, value) items, each taken at most
    once, within capacity. best[c] is the best value using weight c;
    looping c downward keeps each item from being used twice."""
    best = [0] * (capacity + 1)
    for weight, value in items:
        for c in range(capacity, weight - 1, -1):
            best[c] = max(best[c], best[c - weight] + value)
    return best[capacity]
```

The full state is (items considered so far, capacity used), and for each item there are two choices: leave it, or take it and add its value to the best for the remaining capacity. The code compresses the table to one row, `best[c]`, and processes each item by updating the row in place. The capacity loop runs *downward* for a subtle reason: `best[c - weight]` must still hold the value from *before* this item, so that the item can't be taken twice. Run the loop upward, and you've solved a different problem, the **unbounded** knapsack, where each item can be taken any number of times, which is exactly Chapter 29's coin problem.

With items (1, 1), (3, 4), (4, 5) and (5, 7) and capacity 7, the best is the items of weight 3 and 4, worth 9. Greedy by value per kilogram would take the 5-kilogram item (7 / 5 = 1.4 per kilogram) first and end with 8.

The cost is O(n × capacity). That's not polynomial in the size of the input, since the capacity is one number that can be huge: the general knapsack problem is NP-hard, and this DP is only fast when the capacity is reasonably small.

Longest Increasing Subsequence

```
def lis_quadratic(nums):
    """Length of the longest strictly increasing subsequence, O(n^2):
    ends[i] is the longest one ending at nums[i]."""
    ends = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
```

```
                ends[i] = max(ends[i], ends[j] + 1)
    return max(ends, default=0)
```

The longest strictly increasing subsequence of `[10, 9, 2, 5, 3, 7, 101, 18]` has length 4 (for example 2, 3, 7, 18). The textbook DP: `ends[i]` is the longest increasing subsequence that ends at position i, which is one more than the best `ends[j]` over earlier positions with a smaller number. $O(n^2)$.

A cleverer method keeps a different table:

```
def lis(nums):
    """The same in O(n log n): tails[k] is the smallest possible last
    item of an increasing subsequence of length k + 1. It stays sorted,
    so each new number finds its place by binary search."""
    tails = []
    for x in nums:
        k = bisect.bisect_left(tails, x)
        if k == len(tails):
            tails.append(x)
        else:
            tails[k] = x
    return len(tails)
```

`tails[k]` holds the smallest number that can end an increasing subsequence of length k + 1 found so far. A small ending is better than a large one, because more future numbers can extend it. The list `tails` is always sorted, so each new number finds its place with a binary search (Chapter 10): if it's larger than every tail, it extends the longest subsequence; otherwise it replaces the first tail that's at least as big, making that length's ending smaller. The length of `tails` is the answer. O(n log n).

(The list `tails` is not itself an increasing subsequence of the input; only its length is meaningful. Recovering an actual subsequence needs extra bookkeeping of parents.)

Testing

```
"""The sequence DPs against exhaustive search on small inputs."""

import itertools
import random

from dp_sequences import edit_distance, knapsack, lcs, lis, lis_quadratic

rng = random.Random(2026)
```

```
def subsequences(s):
    for k in range(len(s) + 1):
        yield from itertools.combinations(s, k)


def test_lcs():
    for _ in range(500):
        a = "".join(rng.choices("abc", k=rng.randint(0, 6)))
        b = "".join(rng.choices("abc", k=rng.randint(0, 6)))
        common = set(subsequences(a)) & set(subsequences(b))
        found = lcs(a, b)
        assert len(found) == max(map(len, common))
        assert tuple(found) in common


def test_edit_distance():
    def slow(a, b):  # plain recursion: tries everything
        if not a or not b:
            return len(a) + len(b)
        if a[0] == b[0]:
            return slow(a[1:], b[1:])
        return 1 + min(slow(a[1:], b), slow(a, b[1:]), slow(a[1:], b[1:]))

    for _ in range(500):
        a = "".join(rng.choices("ab", k=rng.randint(0, 6)))
        b = "".join(rng.choices("ab", k=rng.randint(0, 6)))
        assert edit_distance(a, b) == slow(a, b)
    assert edit_distance("kitten", "sitting") == 3


def test_knapsack():
    for _ in range(300):
        items = [
            (rng.randint(1, 8), rng.randint(1, 20))
            for _ in range(rng.randint(0, 7))
        ]
        cap = rng.randint(0, 20)
        best = max(
            sum(v for _, v in chosen)
            for chosen in subsequences(items)
            if sum(w for w, _ in chosen) <= cap
        )
        assert knapsack(items, cap) == best


def test_lis():
    for _ in range(500):
        nums = [rng.randint(0, 9) for _ in range(rng.randint(0, 9))]
        best = max(
            (
                len(s)
                for s in subsequences(nums)
                if all(x < y for x, y in zip(s, s[1:]))
            ),
            default=0,
        )
        assert lis(nums) == lis_quadratic(nums) == best
```

```
....                                                                     [100%]
4 passed in 0.01s
```

LCS is checked against every subsequence of both strings; edit distance against plain recursion that tries all three edits everywhere; knapsack against every subset of items; and both LIS versions against every subsequence. Small alphabets ("ab", "abc") make matches common.

Timing

```
"""Longest increasing subsequence: O(n^2) against O(n log n)."""

import random

import bench
from dp_sequences import lis, lis_quadratic

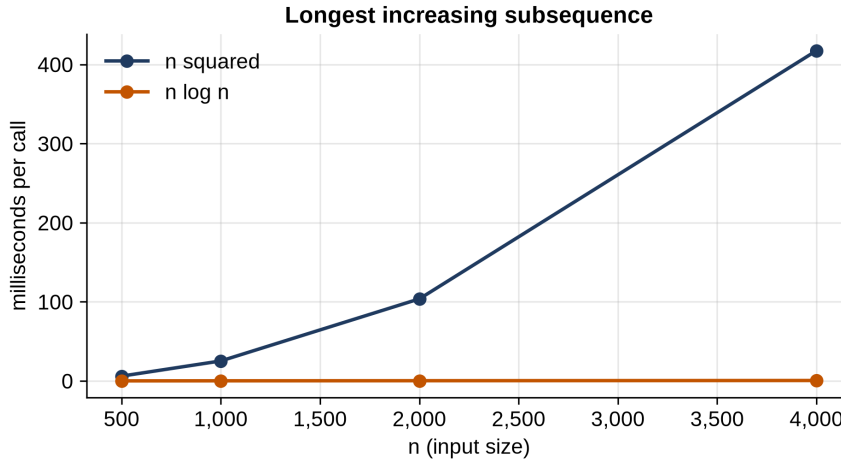
rng = random.Random(2026)


def make(n):
    return ([rng.random() for _ in range(n)],)


sizes = [500, 1_000, 2_000, 4_000]
results = {
    "n squared": bench.doubling(lis_quadratic, make, sizes),
    "n log n": bench.doubling(lis, make, sizes),
}
bench.table(results)
bench.chart(results, "ch30_lis.png", "Longest increasing subsequence")
```

```
         n       n squared         n log n
       500         6.05 ms         47.9 us
     1,000    25.2 ms x4.2     115 us x2.4
     2,000     104 ms x4.1     238 us x2.1
     4,000     417 ms x4.0     511 us x2.1
```

The $O(n^2)$ version quadruples with each doubling; the binary-search version roughly doubles. At 4,000 numbers it's over 800 times faster. Finding a better state is often worth more than optimizing the loops.



The $O(n^2)$ LIS grows four times per doubling; the binary-search version, about twice.

> **Interview Tip:** For two strings, the state is almost always (i, j): a prefix of each. Draw the table for a tiny example, fill a few cells by hand, and the recurrence usually becomes clear from asking "what happened to the last letter?"

Key Takeaways

- DP on two sequences uses a table indexed by a prefix of each: LCS and edit distance are O(n × m).
- Recover the answer, not just its size, by walking back through the table.
- When each row only needs the previous one, keep one or two rows instead of the table.
- In the 0/1 knapsack, the capacity loop runs downward so each item is used at most once; upward gives the unbounded version.
- The longest increasing subsequence drops from $O(n^2)$ to O(n log n) with a sorted list of the best tails and binary search.

Exercises

1. **Maximum run sum.** Find the largest sum of a run of neighboring numbers, which may be negative, in O(n). (Hint: the best run ending at each position either extends the best run ending one step earlier or starts fresh.)
2. **Word break.** Can a string be split into words from a dictionary? "pineapplepenapple" can: "pine apple pen apple" or "pineapple pen apple".
3. **Equal halves.** Can a list of positive numbers be split into two groups with equal sums? (Hint: it's a knapsack question.)

CHAPTER 31 · ADVANCED

Greedy Algorithms and Backtracking

This chapter covers two opposite strategies. A **greedy** algorithm makes the choice that looks best right now and never reconsiders. When it's right, it's the simplest and fastest solution there is, usually a sort and one pass. The catch is that it's often wrong, and it takes a proof or a careful search for counterexamples to know. **Backtracking** is the other extreme: it tries every possible choice, one step at a time, and undoes a choice as soon as it can't lead to a solution. It's always right, but its cost grows exponentially, so it's for small inputs and for problems that ask for *all* the solutions.

Greedy: Scheduling Meetings

One room, many meeting requests, each with a start and an end time. What's the largest number of meetings that can be held without overlaps?

```
def most_meetings(meetings):
    """The most non-overlapping (start, end) meetings in one room: always
    take the meeting that ends first. Ending early leaves the most
    room for everything after."""
    chosen, free_at = [], float("-inf")
    for start, end in sorted(meetings, key=lambda m: m[1]):
        if start >= free_at:
            chosen.append((start, end))
            free_at = end
    return chosen
```

Several greedy rules sound reasonable: the shortest meeting first, the earliest start first, the meeting with the fewest conflicts first. Only one is always right: **the meeting that ends first**. The argument is an **exchange argument**. Take any best schedule. Its first meeting ends no earlier than the meeting that ends first overall, so swapping that one in keeps the schedule valid and the same size, and leaves at least as much time for everything after. Repeat for the rest. So the greedy choice is never worse than the best.

Greedy: Intervals and Rooms

```
def merge_intervals(intervals):
    """Merge overlapping intervals, sorted by start."""
    out = []
    for start, end in sorted(intervals):
        if out and start <= out[-1][1]:
            out[-1][1] = max(out[-1][1], end)
        else:
            out.append([start, end])
    return out


def rooms_needed(meetings):
    """The fewest rooms for all meetings: a heap holds the end times of
    meetings in progress; reuse the room that frees up first."""
    ends = []
    for start, end in sorted(meetings):
        if ends and ends[0] <= start:
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)
```

To merge overlapping intervals, sort them by start; then each interval either overlaps the last merged one (extend it) or starts a new one. Sorting is the usual first step of a greedy algorithm, and it's usually the most expensive one: O(n log n), then O(n) for the pass.

How many rooms are needed to hold *all* the meetings? Go through the meetings by start time, keeping the end times of meetings in progress in a heap (Chapter 13). If the room that frees up first is free by the time the next meeting starts, reuse it; otherwise open a new room. The heap's size at the end is the most meetings ever in progress at once, which is exactly the number of rooms needed.

Greedy: How Far Can You Jump?

```
def can_reach_end(jumps):
    """Each number is the longest jump from that spot. Can we reach the
    last spot? Track the furthest reachable position."""
    furthest = 0
    for i, j in enumerate(jumps):
        if i > furthest:
            return False
        furthest = max(furthest, i + j)
    return True
```

Each number in the list is the longest jump you can make from that position. Can you reach the last position? A search over all jump sequences would work, but one variable is enough: the furthest position reachable so far. If the loop ever arrives at a position beyond it, that position, and everything after, is out of reach.

```
"""Greedy choices on small examples, and one where greedy fails."""

from greedy import (
    can_reach_end,
    greedy_coins,
    merge_intervals,
    most_meetings,
    rooms_needed,
)

meetings = [(9, 10), (9, 12), (10, 11), (11, 13), (12, 14), (13, 15)]
print(most_meetings(meetings))
print(rooms_needed(meetings), "rooms")
print(merge_intervals([(1, 3), (8, 10), (2, 6), (15, 18)]))
print(can_reach_end([2, 3, 1, 1, 4]), can_reach_end([3, 2, 1, 0, 4]))

# US coins: greedy is optimal. Coins 1, 3, 4: it is not.
print(greedy_coins([1, 5, 10, 25], 63), "coins for 63 cents")
print(greedy_coins([1, 3, 4], 6), "coins for 6 with 1, 3, 4 (3 + 3 is 2)")
```

```
[(9, 10), (10, 11), (11, 13), (13, 15)]
2 rooms
[[1, 6], [8, 10], [15, 18]]
True False
6 coins for 63 cents
3 coins for 6 with 1, 3, 4 (3 + 3 is 2)
```

When Greedy Fails

```
def greedy_coins(coins, amount):
    """Take the biggest coin that fits, again and again."""
    count = 0
    for c in sorted(coins, reverse=True):
        count += amount // c
        amount %= c
    return count if amount == 0 else -1
```

The last two lines of the demo show the classic failure. With US coins, taking the largest coin that fits always gives the fewest coins: 63 cents is 25 + 25 + 10 + 1 + 1 + 1, six coins. With coins 1, 3 and 4, greedy makes 6 as 4 + 1 + 1, but 3 + 3 is better. Taking the 4 felt like progress, but it left a remainder that's awkward to make. When an early choice can make later choices worse, greedy breaks, and you need dynamic programming (Chapter 29), which considers every choice for each subproblem.

> **Tip:** Before trusting a greedy idea, try to break it. Write the brute force, which tries every possibility on small inputs, and compare the two on a few thousand random cases, as this chapter's tests do. If they never disagree, look for the exchange argument; if they do, you've found your counterexample in seconds, not in the interview.

> **Interesting Fact:** One of the most widely used greedy algorithms came from a student avoiding an exam. In 1951, David Huffman, a graduate student at MIT, was given the choice of a final exam or a term paper in Robert Fano's information theory course; the paper's problem was to find the most efficient binary code. Huffman's answer, published in 1952, builds the code greedily, always merging the two least frequent symbols, and no code that gives each symbol its own bit pattern can do better. **Huffman coding** is still part of DEFLATE, the compression inside zip, gzip and PNG.

Backtracking: Trying Everything, Carefully

Backtracking builds a solution one choice at a time. At each step it tries each available choice, **recurses** to make the rest of the choices, and then **undoes** the choice before trying the next. The undo step is what makes it

backtracking: one shared list of choices is reused for the whole search, instead of a new copy at every step.

```
def subsets(items):
    out, chosen = [], []

    def go(i):
        if i == len(items):
            out.append(list(chosen))
            return
        go(i + 1)  # leave items[i] out
        chosen.append(items[i])  # put it in
        go(i + 1)
        chosen.pop()  # undo, so the caller sees what it had

    go(0)
    return out


def permutations(items):
    out, chosen, used = [], [], [False] * len(items)

    def go():
        if len(chosen) == len(items):
            out.append(list(chosen))
            return
        for i, x in enumerate(items):
            if not used[i]:
                used[i] = True
                chosen.append(x)
                go()
                chosen.pop()
                used[i] = False

    go()
    return out
```

For subsets, each item has two choices: out or in. For permutations, each position can hold any item not yet used. Notice `out.append(list(chosen))`: it stores a *copy*, because `chosen` keeps changing as the search goes on. Appending `chosen` itself would fill `out` with references to one list, which is empty by the time the search ends. It's the most common backtracking bug.

There are 2^n subsets and n! permutations, so these searches are exponential however they're written; the output alone is that large.

Pruning

Backtracking gets its speed from **pruning**: abandoning a partial solution as soon as it's clear it can't be completed, skipping every solution that would have extended it.

```
def combination_sum(candidates, target):
    """Every combination (repeats allowed) adding up to target."""
    out, chosen = [], []
    candidates = sorted(candidates)

    def go(start, remaining):
        if remaining == 0:
            out.append(list(chosen))
            return
        for i in range(start, len(candidates)):
            c = candidates[i]
            if c > remaining:
                break  # sorted: every later candidate is bigger too
            chosen.append(c)
            go(i, remaining - c)  # i, not i + 1: c may be used again
            chosen.pop()

    go(0, target)
    return out
```

Every combination of candidates, with repeats allowed, that adds up to a target. Because the candidates are sorted, once one is too big for the remaining amount, all later ones are too, so the loop stops (`break`) instead of trying them. And recursing with `i`, not `i + 1`, lets the same candidate be used again, while never going back to an earlier one, so each combination is produced once, in sorted order.

The N-Queens Puzzle

Place n chess queens on an n × n board so that no two attack each other: no two in the same row, column or diagonal.

```
def queens(n):
    """Ways to place n queens on an n x n board, none attacking another.
    One queen per row; sets of used columns and diagonals prune fast."""
    cols, diag, anti = set(), set(), set()

    def place(row):
        if row == n:
            return 1
        count = 0
        for c in range(n):
            if c in cols or row - c in diag or row + c in anti:
```

```
                continue  # attacked: prune this whole branch
            cols.add(c)
            diag.add(row - c)
            anti.add(row + c)
            count += place(row + 1)
            cols.remove(c)
            diag.remove(row - c)
            anti.remove(row + c)
        return count

    return place(0)
```

There's one queen per row, so the search places them row by row, trying each column. Three sets record the attacked columns and diagonals. On one diagonal, `row - col` is the same for every square; on the other, `row + col` is. So checking whether a square is safe takes three O(1) set lookups instead of scanning the board. A square under attack prunes the whole branch below it.

```
"""Backtracking on small examples."""

from backtracking import combination_sum, permutations, queens, subsets

print(["".join(s) for s in subsets("abc")])
print(permutations([1, 2, 3]))
print(combination_sum([2, 3, 5], 8))
print([queens(n) for n in range(1, 11)])
```

```
['', 'c', 'b', 'bc', 'a', 'ac', 'ab', 'abc']
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
[[2, 2, 2, 2], [2, 3, 3], [3, 5]]
[1, 0, 0, 2, 10, 4, 40, 92, 352, 724]
```

On the standard 8 × 8 board there are 92 solutions; there are none for 2 × 2 and 3 × 3.

Testing

```
"""Greedy answers checked by trying everything; backtracking checked
against itertools and known counts."""

import itertools
import random

from backtracking import combination_sum, permutations, queens, subsets
from greedy import (
    can_reach_end,
    merge_intervals,
    most_meetings,
    rooms_needed,
)
```

```
rng = random.Random(2026)


def random_meetings():
    out = []
    for _ in range(rng.randint(0, 7)):
        start = rng.randint(0, 10)
        out.append((start, start + rng.randint(1, 4)))
    return out


def overlap(a, b):
    return a[0] < b[1] and b[0] < a[1]


def test_most_meetings_is_optimal():
    for _ in range(500):
        m = random_meetings()
        best = max(
            (
                len(s)
                for k in range(len(m) + 1)
                for s in itertools.combinations(m, k)
                if not any(
                    overlap(a, b) for a, b in itertools.combinations(s, 2)
                )
            ),
        )
        chosen = most_meetings(m)
        assert len(chosen) == best
        assert not any(
            overlap(a, b) for a, b in itertools.combinations(chosen, 2)
        )


def test_rooms_needed():
    for _ in range(500):
        m = random_meetings()
        busiest = max(
            (sum(s <= t < e for s, e in m) for t in range(20)), default=0
        )
        assert rooms_needed(m) == busiest


def test_merge_and_jumps():
    assert merge_intervals([(1, 3), (2, 6), (8, 10), (9, 12)]) == [
        [1, 6],
        [8, 12],
    ]
    assert can_reach_end([2, 3, 1, 1, 4]) and not can_reach_end(
        [3, 2, 1, 0, 4]
    )


def test_backtracking():
    items = [1, 2, 3, 4]
    assert sorted(map(tuple, subsets(items))) == sorted(
        c for k in range(5) for c in itertools.combinations(items, k)
    )
```

```
    assert permutations(items) == [
        list(p) for p in itertools.permutations(items)
    ]
    assert combination_sum([2, 3, 6, 7], 7) == [[2, 2, 3], [7]]
    assert [queens(n) for n in range(1, 9)] == [1, 0, 0, 2, 10, 4, 40, 92]
```

```
....                                                                     [100%]
4 passed in 0.01s
```

The greedy schedulers are checked against brute force: every subset of up to seven random meetings for the most meetings, and the busiest moment for the number of rooms. Subsets and permutations are checked against `itertools`, and the queens counts against the known values.

Timing

```
"""Counting n-queens placements: backtracking with pruning still grows
exponentially. Each row of the table adds one to n."""

import bench
from backtracking import queens

sizes = [6, 7, 8, 9, 10]
bench.table({"queens(n)": bench.doubling(queens, lambda n: (n,), sizes)})
```

```
         n       queens(n)
         6           90 us
         7     466 us x5.2
         8    1.93 ms x4.1
         9    8.18 ms x4.2
        10    36.9 ms x4.5
```

Each row adds *one* to n, and the time grows four to five times: exponential growth. Without pruning, trying every way to put one queen in each row would mean n^n placements, ten billion for n = 10; pruning cuts that to a search that takes under 40 milliseconds. Pruning doesn't change the exponential shape, but it changes how far you can go before hitting the wall.

> **Real-World Use:** The regular expression engine in Python's `re` module is a backtracking engine. When part of a pattern fails to match, it goes back and tries another way of matching the earlier parts. Some patterns, such as `(a+)+b` on a long string of `a`s with no `b`, give it exponentially many ways to try, which can make a single match take practically forever. Python 3.11 added **atomic groups** `(?>...)` and **possessive quantifiers** such as `a++` to `re`; both tell the engine not to backtrack into a part that has already matched, a form of pruning you write into the pattern.

> **Interview Tip:** "All", "every", "generate", "list each" in a problem statement, with small input sizes (n up to about 10 to 20), point to backtracking. Say the template out loud: choose, recurse, undo. Then say what you'll prune.

Key Takeaways

- Greedy takes the locally best choice and never reconsiders: often a sort plus one pass.
- Greedy is only right with a proof, usually an exchange argument; test it against brute force before trusting it.
- When an early choice can spoil later ones (coin systems, knapsacks), use dynamic programming instead.
- Backtracking tries every choice step by step: choose, recurse, undo. Store copies of solutions, not the shared list.
- Pruning abandons hopeless partial solutions early; it makes exponential searches practical for small inputs.

Exercises

1. **Balanced brackets.** Generate every string of n pairs of balanced brackets, such as `(())()` for n = 3.
2. **A small sudoku.** Solve a 4 × 4 sudoku, in which every row, column and 2 × 2 box must hold each of the digits 1 to 4 once.

PART VII

Putting It Together

CHAPTER 32 · PROJECT

Project: An LRU Cache

A **cache** keeps copies of recently used results close at hand, so they don't have to be fetched or computed again: web pages in a browser, database rows in an application server, function results in a Python program. A cache can't keep everything, so when it's full and something new arrives, it has to **evict** something. The most popular rule is **least recently used** (LRU): throw away the entry that has gone the longest without being used, on the bet that the recent past predicts the near future.

"Design an LRU cache with O(1) get and put" is also one of the best-known interview questions, because no single structure from this book can do it alone. A hash map finds entries in O(1) but has no idea of order. A linked list keeps order and moves nodes in O(1), but finding a node takes O(n). Together, they do everything in O(1). This chapter builds the combination, tests it against a model, uses it as a decorator, sizes it, and times it against the standard library.

The Design

- A **dict** maps each key to its node in the list.
- A **doubly linked list** (Chapter 7) holds the entries in order of use: most recent at the front, least recent at the back.
- **get(key)**: look up the node in the dict. If it's there, unlink it and push it to the front: it was just used.

- **put(key, value)**: if the key is there, update the value and move the node to the front. If not, and the cache is full, unlink the node at the back and delete its key from the dict; then add the new node at the front.

The list has to be *doubly* linked: unlinking a node from the middle needs its predecessor, and only a backward link finds it in O(1). And each node stores its key, not just its value, so that when the back node is evicted, the cache knows which key to delete from the dict.

The Code

```
"""An LRU (least recently used) cache with a fixed capacity.

A dict finds an entry's node in constant time; a doubly linked list
keeps the entries in order of use, most recent at the front. Using an
entry moves its node to the front; when the cache is full, the node at
the back (the least recently used) is evicted. Every operation is O(1).
"""


class _Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=None, value=None):
        self.key, self.value = key, value
        self.prev = self.next = self


class LRUCache:
    def __init__(self, capacity):
        if capacity < 1:
            raise ValueError("capacity must be at least 1")
        self.capacity = capacity
        self._map = {}  # key -> node
        self._sentinel = _Node()  # front: sentinel.next; back: .prev
        self.hits = self.misses = 0

    def __len__(self):
        return len(self._map)

    def _unlink(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def _push_front(self, node):
        first = self._sentinel.next
        node.prev, node.next = self._sentinel, first
        first.prev = node
        self._sentinel.next = node

    def get(self, key, default=None):
        node = self._map.get(key)
        if node is None:
```

```
            self.misses += 1
            return default
        self.hits += 1
        self._unlink(node)  # used just now: move to the front
        self._push_front(node)
        return node.value

    def put(self, key, value):
        node = self._map.get(key)
        if node:
            node.value = value
            self._unlink(node)
        else:
            if len(self._map) == self.capacity:
                oldest = self._sentinel.prev  # the back of the list
                self._unlink(oldest)
                del self._map[oldest.key]
            node = _Node(key, value)
            self._map[key] = node
        self._push_front(node)

    def keys(self):
        """Keys from most to least recently used."""
        node = self._sentinel.next
        while node is not self._sentinel:
            yield node.key
            node = node.next
```

A **sentinel** node, which holds no entry, links the two ends of the list into a circle: its `next` is the front and its `prev` is the back. With the sentinel, the list is never truly empty, so `_unlink` and `_push_front` need no special cases for the first or last node. `__slots__` on the node class saves memory, since a full cache has one node per entry. The `hits` and `misses` counters cost almost nothing and are the first thing you'll want to know about a real cache.

Watching It Work

```
"""An LRU cache of capacity 3, step by step."""

from lru_cache import LRUCache

c = LRUCache(3)
for step in [
    "put a",
    "put b",
    "put c",
    "get a",
    "put d",
    "get b",
    "get c",
    "put e",
]:
    action, key = step.split()
```

```
    if action == "put":
        c.put(key, key.upper())
        result = ""
    else:
        result = f"-> {c.get(key)}"
    print(f"{step} {result:<7} cache (newest first): {list(c.keys())}")
print(f"hits {c.hits}, misses {c.misses}")
```

```
put a         cache (newest first): ['a']
put b         cache (newest first): ['b', 'a']
put c         cache (newest first): ['c', 'b', 'a']
get a -> A    cache (newest first): ['a', 'c', 'b']
put d         cache (newest first): ['d', 'a', 'c']
get b -> None cache (newest first): ['d', 'a', 'c']
get c -> C    cache (newest first): ['c', 'd', 'a']
put e         cache (newest first): ['e', 'c', 'd']
hits 2, misses 1
```

Reading `a` moves it to the front, so when `d` arrives in a full cache, `b`, now the least recently used, is evicted, and the later `get b` misses. Then `get c` saves `c`, and `put e` evicts `a`.

Testing Against a Model

```
"""LRUCache checked against a simple model built on OrderedDict."""

import random
from collections import OrderedDict

import pytest

from lru_cache import LRUCache


class Model:
    """Obviously correct, if not obviously O(1)."""

    def __init__(self, capacity):
        self.capacity, self.d = capacity, OrderedDict()

    def get(self, key):
        if key not in self.d:
            return None
        self.d.move_to_end(key, last=False)  # to the front
        return self.d[key]

    def put(self, key, value):
        self.d[key] = value
        self.d.move_to_end(key, last=False)
        if len(self.d) > self.capacity:
            self.d.popitem(last=True)  # drop the back


def test_eviction_order():
```

```
    c = LRUCache(2)
    c.put("a", 1)
    c.put("b", 2)
    c.get("a")  # now b is the least recently used
    c.put("c", 3)
    assert c.get("b") is None and c.get("a") == 1 and c.get("c") == 3
    assert list(c.keys()) == ["c", "a"] and len(c) == 2
    assert (c.hits, c.misses) == (3, 1)


def test_bad_capacity():
    with pytest.raises(ValueError):
        LRUCache(0)


def test_random_operations_match_model():
    rng = random.Random(2026)
    for capacity in (1, 2, 5, 20):
        c, m = LRUCache(capacity), Model(capacity)
        for _ in range(5000):
            key = rng.randint(0, 30)
            if rng.random() < 0.5:
                assert c.get(key) == m.get(key)
            else:
                value = rng.randint(0, 999)
                c.put(key, value)
                m.put(key, value)
            assert list(c.keys()) == list(m.d)
```

```
...                                                                  [100%]
3 passed in 0.01s
```

The main test runs 5,000 random operations at each of four capacities, including capacity 1, where every new key evicts the previous one, and compares every result and the full order of keys with a **model**: a simple, obviously correct implementation built on `OrderedDict`. Checking the order after every operation catches a broken link immediately, at the operation that caused it, instead of many operations later.

A Memoizing Decorator

Chapter 5 sped up Fibonacci with `functools.cache`, which never forgets anything. A long-running program can't remember every result forever, so the standard library also has `functools.lru_cache(maxsize=...)`, a decorator with an LRU cache inside. Here's the same thing built on our class:

```
"""A memoizing decorator with a bounded cache, like the standard
library's functools.lru_cache, built on our LRUCache."""

import functools

from lru_cache import LRUCache

MISSING = object()


def lru_memoize(capacity):
    def decorate(func):
        cache = LRUCache(capacity)

        @functools.wraps(func)
        def wrapper(*args):
            result = cache.get(args, MISSING)
            if result is MISSING:
                result = func(*args)
                cache.put(args, result)
            return result

        wrapper.cache = cache
        return wrapper

    return decorate


@lru_memoize(capacity=100)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)


@functools.lru_cache(maxsize=100)
def fib_std(n):
    return n if n < 2 else fib_std(n - 1) + fib_std(n - 2)


print(fib(80), fib_std(80))
print("ours:", fib.cache.hits, "hits,", fib.cache.misses, "misses")
print("functools:", fib_std.cache_info())
```

```
23416728348467685 23416728348467685
ours: 78 hits, 81 misses
functools: CacheInfo(hits=78, misses=81, maxsize=100, currsize=81)
```

The arguments tuple is the key. Both decorators compute `fib(80)` with exactly 81 misses, one for each of `fib(0)` to `fib(80)`, and 78 hits. The standard version is written in C, using the same design: a dictionary plus a doubly linked list.

How Big Should the Cache Be?

```
"""How big should a cache be? Requests for 10,000 different pages,
where a few popular pages get most of the traffic (page k is requested
in proportion to 1 / k). Hit rate for different capacities."""

import random

from lru_cache import LRUCache

rng = random.Random(2026)
pages = range(1, 10_001)
weights = [1 / k for k in pages]
requests = rng.choices(pages, weights, k=200_000)

print(f"{'capacity':>9} {'hit rate':>9}")
for capacity in [10, 100, 1_000, 5_000]:
    cache = LRUCache(capacity)
    for page in requests:
        if cache.get(page) is None:
            cache.put(page, True)  # a miss: fetch it, then keep it
    print(f"{capacity:>9,} {cache.hits / len(requests):>9.1%}")
```

```
 capacity  hit rate
       10     13.2%
      100     39.3%
    1,000     67.6%
    5,000     88.8%
```

Real traffic is uneven: a few popular items get most of the requests. This simulation draws 200,000 requests from 10,000 pages, with page k requested in proportion to 1 / k, a pattern known as **Zipf's law** that fits many real request logs roughly. Holding just 1% of the pages (100) answers about 39% of requests; 10% of the pages answers about two thirds. Each tenfold increase in size buys less than the last. Measuring the hit rate on a realistic workload is how cache sizes are chosen in practice.

Timing

```
"""A mixed workload of gets and puts: our LRUCache, an OrderedDict
version, and a dict with no limit (the fastest possible, unbounded)."""

import random
from collections import OrderedDict

import bench
from lru_cache import LRUCache

rng = random.Random(2026)
keys = [rng.randint(0, 2_000) for _ in range(100_000)]


class OrderedLRU:
    def __init__(self, capacity):
        self.capacity, self.d = capacity, OrderedDict()

    def get(self, key, default=None):
        if key in self.d:
            self.d.move_to_end(key)
            return self.d[key]
        return default

    def put(self, key, value):
        self.d[key] = value
        self.d.move_to_end(key)
        if len(self.d) > self.capacity:
            self.d.popitem(last=False)


class Unbounded(dict):
    """A dict with the same get() and put() methods; it never evicts."""

    put = dict.__setitem__


def workload(cache):
    for k in keys:
        if cache.get(k) is None:
            cache.put(k, k)


caches = {
    "LRUCache": lambda: LRUCache(500),
    "OrderedDict LRU": lambda: OrderedLRU(500),
    "plain dict": Unbounded,
}
for name, make in caches.items():
    cache = make()
    seconds = bench.measure(lambda c=cache: workload(c))
    print(f"{name:>16}: {bench.show_time(seconds)} for 100,000 requests")
```

```
       LRUCache: 45.9 ms for 100,000 requests
OrderedDict LRU: 27.4 ms for 100,000 requests
     plain dict: 5.08 ms for 100,000 requests
```

Our cache handles 100,000 mixed requests in about 46 milliseconds, under half a microsecond each. An LRU cache built on `OrderedDict`, whose `move_to_end` and `popitem(last=False)` do the linked-list work in C, is about 1.7 times faster. A plain dict that never evicts anything is about nine times faster than ours, the price of keeping order in pure Python. In an interview, build it yourself; in production, use `functools.lru_cache` or `OrderedDict` (Exercise 1).

> **Real-World Use:** Redis, the in-memory database used as a cache by many large websites, can evict keys by LRU when it runs out of memory, but it deliberately doesn't use the exact design from this chapter. A linked list over millions of keys would cost memory for two pointers per key and work on every read. Instead, Redis keeps a last-access time with each key and, when it must evict, samples a few random keys (five by default, set by `maxmemory-samples`) and evicts the least recently used of the sample. It's an approximation of LRU that's nearly as good in practice, and much cheaper.

> **Interview Tip:** Explain the design before writing code: "a dict for O(1) lookup, a doubly linked list for O(1) reordering and eviction, and each node stores its key so eviction can clean up the dict." Mention the sentinel, which removes edge cases. If time is short, ask whether `OrderedDict` is allowed; it turns the problem into ten lines.

Key Takeaways

- An LRU cache combines a hash map (find in O(1)) with a doubly linked list (reorder and evict in O(1)).
- Nodes store their keys, so evicting from the list can delete from the map.
- A sentinel node removes the special cases at the ends of the list.
- Test a data structure against a simple model, after every operation, on random sequences.

- Choose a cache size by measuring the hit rate on a realistic workload; returns diminish quickly.
- In production, use `functools.lru_cache` or `OrderedDict`; both are the same design, written in C.

Exercises

1. **The short version.** Write an LRU cache with `OrderedDict`, using `move_to_end` and `popitem(last=False)`.
2. **Entries that expire.** Add a time-to-live: `get` treats an entry older than `ttl` seconds as missing. Make the clock a parameter so that tests can move time forward.

CHAPTER 33 · ADVANCED

Is Python Getting Faster?

Every listing in this book was run on four versions of Python: 3.12, 3.13, 3.14 and 3.15. Since 2021, the CPython core developers have worked steadily on speed, and each release's "What's New" document reports gains. This chapter puts the claims to a small test of its own: a suite of the kinds of work this book does, timed on all four versions on the same computer.

What Changed

Each recent release made changes aimed at speed. A few highlights, from the official "What's New" documents:

- **Python 3.11** (2022) was the big jump: its notes say "between 10-60% faster than Python 3.10", 1.25 times faster on average on the standard benchmark suite, mostly from a **specializing interpreter** that rewrites instructions to faster versions for the types it actually sees.
- **Python 3.13** (2024) added two experimental options, both off by default: a build without the global interpreter lock (**free-threaded**), so threads can run Python code on several cores at once, and a **just-in-time (JIT) compiler**.
- **Python 3.14** (2025) made the free-threaded build officially supported and added a new **tail-calling interpreter**, reported as 3 to 5% faster on the standard benchmark suite when CPython is built with a recent Clang. The JIT was included in the official Windows and macOS installers, still off by default.

- **Python 3.15**, released on 9 October 2026, reports the JIT as 7 to 8% faster than the standard interpreter on x86-64 Linux (geometric mean over the benchmark suite).

The Suite

```
"""A small benchmark suite of this book's kinds of work. Run it with
each Python you have to compare them: python3.12 suite.py, and so on.
With --json, prints the results as JSON instead of a table."""

import heapq
import json
import random
import sys
from collections import deque

import bench

rng = random.Random(2026)
NUMBERS = [rng.random() for _ in range(100_000)]
GRID = 200


def append_loop():
    items = []
    for i in range(100_000):
        items.append(i)


def dict_work():
    d = {}
    for i in range(50_000):
        d[i] = i
    return sum(d[i] for i in range(50_000))


def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)


def heap_work():
    h = []
    for x in NUMBERS[:50_000]:
        heapq.heappush(h, x)
    while h:
        heapq.heappop(h)


def bfs_grid():
    seen, queue = {(0, 0)}, deque([(0, 0)])
    while queue:
        r, c = queue.popleft()
        for n in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= n[0] < GRID and 0 <= n[1] < GRID and n not in seen:
                seen.add(n)
```

```
                queue.append(n)


class Node:
    __slots__ = ("value", "next")

    def __init__(self, value, next=None):
        self.value, self.next = value, next


def linked_list():
    head = None
    for i in range(50_000):
        head = Node(i, head)
    total = 0
    while head:
        total += head.value
        head = head.next


def insertion_sort():
    items = NUMBERS[:1_500]
    for i in range(1, len(items)):
        x, j = items[i], i - 1
        while j >= 0 and items[j] > x:
            items[j + 1] = items[j]
            j -= 1
        items[j + 1] = x


TESTS = {
    "list append": append_loop,
    "dict set and get": dict_work,
    "recursion (fib 22)": lambda: fib(22),
    "heap push and pop": heap_work,
    "BFS on a grid": bfs_grid,
    "linked list": linked_list,
    "insertion sort": insertion_sort,
    "sorted() 100,000": lambda: sorted(NUMBERS),
}

results = {name: bench.measure(test) for name, test in TESTS.items()}
if "--json" in sys.argv:
    print(json.dumps(results))
else:
    version = ".".join(map(str, sys.version_info[:3]))
    print(f"Python {version}")
    for name, seconds in results.items():
        print(f"{name:>20}: {bench.show_time(seconds)}")
```

Eight tests, each a kind of work from this book: appending to a list, dictionary inserts and lookups, recursive calls, heap operations, a BFS on a grid, building and walking a linked list of node objects, an insertion sort (pure Python loops and comparisons), and `sorted()` (almost entirely C code). Each test is timed with Chapter 3's `bench.measure`, which keeps the fastest of several rounds.

On its own, the suite prints its times:

```
Python 3.15.0
         list append: 3.78 ms
    dict set and get: 5.35 ms
  recursion (fib 22): 1.9 ms
   heap push and pop: 21.2 ms
       BFS on a grid: 28.9 ms
         linked list: 8.13 ms
      insertion sort: 28.3 ms
    sorted() 100,000: 17.7 ms
```

To compare versions, `tests/record_versions.py` in the book's code runs the suite with each Python in turn, three passes in rotation, and keeps each version's fastest time for each test, so a burst of background activity on the computer can't favor one version. The results are saved in `versions.json`, and this short program compares them:

```
"""Compare the Python versions: each test's time relative to Python
3.12 (below 1.00 is faster), from the results in versions.json."""

import json
from statistics import geometric_mean

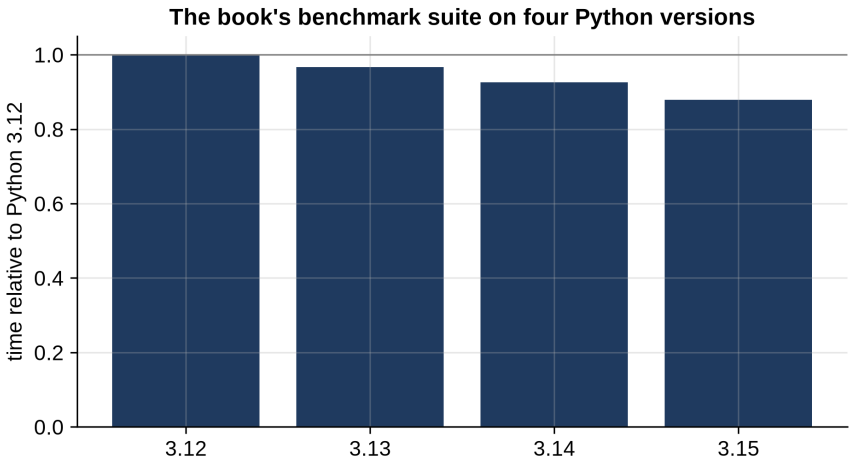
import matplotlib.pyplot as plt

with open("versions.json") as f:
    results = json.load(f)
versions = list(results)
base = results[versions[0]]

print(f"{'':>20}" + "".join(f"{v:>8}" for v in versions))
ratios = {v: [] for v in versions}
for test in base:
    row = f"{test:>20}"
    for v in versions:
        ratio = results[v][test] / base[test]
        ratios[v].append(ratio)
        row += f"{ratio:>8.2f}"
    print(row)
overall = {v: geometric_mean(ratios[v]) for v in versions}
print(f"{'overall':>20}" + "".join(f"{overall[v]:>8.2f}" for v in versions))

fig, ax = plt.subplots()
ax.bar(versions, [overall[v] for v in versions])
ax.axhline(1, color="gray", linewidth=0.8)
ax.set_ylabel("time relative to Python 3.12")
ax.set_title("The book's benchmark suite on four Python versions")
fig.tight_layout()
fig.savefig("ch33_versions.png")
```

```
                      3.12    3.13    3.14    3.15
        list append   1.00    0.86    1.06    0.94
   dict set and get   1.00    0.95    0.92    0.99
recursion (fib 22)    1.00    0.87    0.79    0.79
 heap push and pop    1.00    0.97    1.04    1.02
     BFS on a grid    1.00    1.10    0.92    0.87
       linked list    1.00    0.85    0.78    0.68
    insertion sort    1.00    1.21    0.86    0.75
  sorted() 100,000    1.00    0.97    1.12    1.08
           overall    1.00    0.97    0.93    0.88
```



The suite's overall time on each version, relative to Python 3.12.

What the Numbers Say

Each number is a test's time relative to Python 3.12: 0.88 means it took 88% as long, 12% faster. The "overall" row is the **geometric mean** of the ratios, the right way to average ratios: a test that takes half as long and one that takes twice as long average to 1.00, no change, as they should.

- **The trend is real but modest.** Overall, 3.13 is about 3% faster than 3.12, 3.14 about 7%, and 3.15 about 12%. That's without the JIT or free threading, with the default build of each version.
- **Pure Python code gains the most.** The linked list (creating objects and following attributes) takes 32% less time on 3.15, the insertion sort 25% less, and recursion 21% less. This is exactly the code the interpreter improvements target: many small Python-level operations.

- **Code that runs in C gains nothing.** `sorted()` spends nearly all its time in C comparisons, and the heap test mostly calls the C functions in `heapq`, so interpreter improvements can't help them. The heap test is within a few percent of 3.12 on every version, and `sorted()` is even 8 to 12% slower on 3.14 and 3.15 here, more likely an effect of the different builds (see below) than of any change to the sorting code.
- **Not every step is forward.** On 3.13, the insertion sort was 21% *slower* and the BFS 10% slower; both improved well past 3.12 on 3.14 and 3.15. A single test on a single computer can move by several percent for reasons that have nothing to do with the language, such as how the code happens to line up in the processor's caches.

> **Tip:** Treat any single benchmark number with suspicion, including this book's. Ratios under about 5% can come from the build, the computer and the moment as much as from the language. The trends that hold across many tests, such as "pure Python code is getting faster, C code isn't", are the ones to trust.

How the Pythons Were Built

The comparison is fair only if the builds are comparable, and here they aren't quite the same. Python 3.12.3 is the version that came with the operating system (Ubuntu), compiled with GCC. Python 3.13.5, 3.14.8 and 3.15.0 were installed with `uv`, a Python package and project manager, which downloads prebuilt, optimized builds of CPython, compiled with Clang; the 3.14 and 3.15 builds use the tail-calling interpreter. All four are standard builds with the global interpreter lock, and the JIT, which the 3.14 and 3.15 builds include, was left off, its default. Appendix C has the details of the computer.

So part of the difference between 3.12 and the rest may come from the compiler, not from Python. If you want to know what upgrading will do for *your* program, the honest answer is: measure it. Run the suite, or better, your own program's benchmarks, with each version you're considering, as this chapter did. The JIT can be tried on 3.14 or 3.15 by setting the environment variable `PYTHON_JIT=1` before starting Python, if your build includes it.

What This Means for You

The improvements in recent versions are welcome, and they're free: upgrade and your pure Python code runs faster. But compare their size with the differences measured throughout this book:

- choosing a set instead of a list for membership tests: thousands of times faster (Chapter 1);
- a hash map instead of a nested loop for two sum: over a thousand times faster at 4,000 numbers (Chapter 24);
- a better algorithm for the longest increasing subsequence: over 800 times faster at 4,000 numbers (Chapter 30).

A new Python version gives you 5 or 10%. A better data structure or algorithm can give you a factor of a hundred or a thousand, and the factor grows with the input. Upgrade Python, by all means. But the choices this book teaches are where speed really comes from.

Key Takeaways

- On this book's suite, Python 3.15 is about 12% faster than 3.12 overall, with the default build and no JIT.
- Pure Python code (object creation, attribute access, loops, recursion) gained the most; code that runs in C gained nothing.
- Individual results move by several percent between versions and builds; trust trends across many tests, not single numbers.
- Average ratios with the geometric mean.
- Measure your own program on the versions you're considering.
- Version upgrades give percentages; better algorithms and data structures give multiples.

CHAPTER 34 · INTERMEDIATE

Next Steps

You've built a dynamic array, linked lists, stacks and queues, two kinds of hash table, binary search, a shelf of sorting algorithms from bubble sort to radix sort, a heap, binary search trees and an AVL tree, a trie, union-find, graphs with BFS, DFS, topological sort, Dijkstra, Bellman-Ford, Kruskal and Prim, and an LRU cache. You've tested every one of them against a simpler, trusted version on thousands of random inputs, and timed them until the Big O on the page matched the numbers on your screen. And you've learned the patterns that turn most interview problems from a blank page into a familiar shape.

This last chapter is a map of where to go next.

Keep the Habits

The structures matter less than three habits that run through every chapter:

- **Measure.** Before believing that something is fast or slow, time it, with the fastest of several runs, on inputs that double in size. The ratio column tells you the growth rate; the chart shows you where the crossover is.
- **Test against something simpler.** For every clever piece of code, there's a slow, obviously correct version: a built-in, a brute force, a model. Compare them on thousands of small random inputs, and keep the inputs small so that a failure is easy to debug.
- **Know what runs in C.** In Python, the biggest speedups often come from moving a loop into a built-in: `sorted`, `sum`, `set`, `dict`, `heapq`, `bisect`,

`collections`. An O(n log n) sort in C beats an O(n) loop in Python for surprisingly large n, and the only way to know where the crossover is is to measure.

Structures to Learn Next

Each of these builds directly on something in this book.

- **Fenwick trees and segment trees** (from prefix sums, Chapter 27). Prefix sums answer range sums in O(1), but a single change to the data means rebuilding them in O(n). These trees support both updating an item and querying a range in O(log n).
- **B-trees** (from Chapter 16's BST and Chapter 17's balancing). Instead of two children per node, a B-tree node holds hundreds of keys, sized to match a disk page, so a lookup among millions of rows reads only three or four pages. Most databases store their tables and indexes this way.
- **Skip lists** (from Chapter 7's linked lists). Linked lists with extra "express lanes" at random heights give O(log n) search on average, with simpler code than a balanced tree.
- **Bloom filters** (from Chapter 9's hashing). A bit array and several hash functions answer "is this item in the set?" with "definitely not" or "probably", using a small fraction of the memory of a real set. Databases use them to skip disk reads for keys that can't be there.
- **String algorithms.** Searching for a pattern in a text with Knuth-Morris-Pratt or with rolling hashes (Rabin-Karp), both O(n + m); and suffix arrays, which make any substring search a binary search (Chapter 10).
- **A* search** (from Chapter 22's Dijkstra). Dijkstra's algorithm, guided by an estimate of the remaining distance, so it explores toward the goal first. It's the standard pathfinding algorithm in games and robotics.
- **Network flow and matching** (from Chapter 21's graph search). How much can flow through a network of pipes or links with capacities, and how to pair people with jobs; both are solved by repeated BFS.

Practice

Interview skill comes from practice, and the most efficient practice is deliberate:

- **Practice by pattern.** Work through several problems of one pattern in a row, such as a week of sliding windows, until recognizing the signals in Chapter 24's table becomes automatic. Then mix the patterns, as a real interview does.
- **Time yourself.** Give yourself 30 to 40 minutes per problem, the length of a typical interview question. If you're stuck at the end, study a solution, then solve the problem again from scratch two or three days later.
- **Talk out loud.** Practice explaining your approach and its complexity before writing code. A friend, a recording or a rubber duck all work.
- **Test your solutions the way this book does.** Write the brute force, and stress test your solution against it. You'll find bugs that the examples in a problem statement don't reveal.

Read the Source

The real-world boxes in this book point to the source code of CPython, Linux, Git and others. Reading code written by experienced programmers is one of the best ways to improve. Good places to start in CPython, all readable without knowing C deeply:

- `Lib/heapq.py` and `Lib/bisect.py`: the Python versions of the modules from Chapters 10 and 13, short and carefully commented.
- `Lib/functools.py`: the pure Python version of `lru_cache`, with its own linked list.
- `Lib/graphlib.py`: the topological sorter from Chapter 21.
- `Objects/listsort.txt`: Tim Peters' description of the sorting algorithm behind `sorted()`, one of the best design documents in any open-source project.
- `Objects/dictobject.c`: the comments at the top explain how Python's dictionary works, from the probing scheme of Chapter 9 to the compact layout.

Use Your Code

All the code from this book is in the repository at **github.com/aadhavanshines/data-structures-and-algorithms-in-python-hands-on**, with its tests and benchmarks, ready to run on Python 3.12 to 3.15. Change it, break it, extend it: add `delete` to the trie, make the hash table shrink, run the benchmarks on your own computer and see whether your numbers tell the same story as the book's.

Thank you for building all of this with me. Good luck in your interviews, and enjoy the next fast program you write, because now you'll know why it's fast.

Appendix A: Exercise Solutions

Here are worked solutions to every exercise. Each one was run on Python 3.12, 3.13, 3.14 and 3.15, and the output shown is the output it produced. Most solutions also check themselves on a few examples; the bigger ones are tested in the book's repository against a brute force, as in the chapters. There's usually more than one good answer: if yours gives the same results and you can explain its cost, it's right. The same files are in the repository as `solution_1.py`, `solution_2.py` and so on, in each chapter's folder.

Chapter 2: Setting Up

Exercise 1: Smallest and largest.

```
"""Exercise 2.1: the smallest and largest items in one pass, comparing
items in pairs: about 3 comparisons per 2 items instead of 4."""

import random


def min_and_max(numbers):
    if not numbers:
        raise ValueError("min_and_max() of an empty list")
    lo = hi = numbers[0]
    for i in range(1, len(numbers) - 1, 2):
        a, b = numbers[i], numbers[i + 1]
        if a > b:
            a, b = b, a  # 1 comparison: now a <= b
        lo = min(lo, a)  # the smaller can only be a new minimum
        hi = max(hi, b)  # the larger can only be a new maximum
    if len(numbers) % 2 == 0:  # one item left over at the end
        lo, hi = min(lo, numbers[-1]), max(hi, numbers[-1])
    return lo, hi
```

```
rng = random.Random(2026)
for _ in range(1000):
    nums = [rng.randint(-100, 100) for _ in range(rng.randint(1, 50))]
    assert min_and_max(nums) == (min(nums), max(nums))
print(min_and_max([3, -1, 4, 1, -5, 9, 2]))
print("1,000 random lists: all agree with min() and max()")
```

```
(-5, 9)
1,000 random lists: all agree with min() and max()
```

Exercise 2: A bug the tests catch.

```
"""Exercise 2.2: largest() with best = 0 is wrong whenever every number
is negative. The random test finds such a list quickly, because many
random lists are short, and a list of one negative number is enough."""

import random


def largest_buggy(numbers):
    best = 0  # the bug: 0 may be bigger than every item
    for x in numbers:
        if x > best:
            best = x
    return best


print(largest_buggy([-5, -2, -9]), "instead of", max([-5, -2, -9]))

rng = random.Random(2026)
for attempt in range(1, 1001):
    nums = [rng.randint(-100, 100) for _ in range(rng.randint(1, 50))]
    if largest_buggy(nums) != max(nums):
        print(f"random list number {attempt} exposes it: {nums}")
        break
```

```
0 instead of -2
random list number 14 exposes it: [-77]
```

Chapter 3: Measuring Speed: Big O You Can See

Exercise 1: Count the steps.

```
"""Exercise 3.1: an inner loop that runs i times for each i makes
0 + 1 + ... + (n - 1) = n(n - 1)/2 steps: O(n^2), like all pairs."""


def steps(n):
    count = 0
    for i in range(n):
        for _ in range(i):
```

```
            count += 1
    return count


for n in [10, 100, 1_000]:
    formula = n * (n - 1) // 2
    print(f"n = {n:>5,}: {steps(n):>7,} steps; n(n - 1)/2 = {formula:,}")
```

```
n =    10:      45 steps; n(n - 1)/2 = 45
n =   100:   4,950 steps; n(n - 1)/2 = 4,950
n = 1,000: 499,500 steps; n(n - 1)/2 = 499,500
```

Exercise 2: Read a benchmark.

```
"""Exercise 3.2: classify two functions from their timings, then
predict their time at n = 64,000 (three more doublings)."""

sizes = [1_000, 2_000, 4_000, 8_000]
timings = {"first": [0.5, 2.1, 8.3, 33], "second": [1.0, 2.2, 4.6, 9.8]}

for name, ms in timings.items():
    ratios = [b / a for a, b in zip(ms, ms[1:])]
    average = sum(ratios) / len(ratios)
    kind = "O(n^2)" if average > 3 else "O(n) or O(n log n)"
    predicted = ms[-1] * average**3  # 8,000 -> 64,000 is 3 doublings
    shown = ", ".join(f"x{r:.1f}" for r in ratios)
    print(f"{name}: ratios {shown}, so {kind};")
    print(f"   at 64,000 about {predicted / 1000:.1f} seconds")
```

```
first: ratios x4.2, x4.0, x4.0, so O(n^2);
   at 64,000 about 2.2 seconds
second: ratios x2.2, x2.1, x2.1, so O(n) or O(n log n);
   at 64,000 about 0.1 seconds
```

Ratios near 4 per doubling mean $O(n^2)$; ratios a little above 2 mean O(n) or O(n log n), and timings alone can't tell those two apart. Each doubling multiplies the time by the ratio again, so three more doublings multiply it by the ratio cubed.

Chapter 4: Python's Built-ins Under the Hood

Exercise 1: Membership in a loop.

```
"""Exercise 4.1: items of a that are also in b, two ways, counting the
work each does."""

import random

rng = random.Random(2026)
a = rng.sample(range(10_000), 1_000)
```

```
b = rng.sample(range(10_000), 1_000)


def common_list(a, b):
    out, comparisons = [], 0
    for x in a:
        for y in b:  # what `x in b` does for a list
            comparisons += 1
            if x == y:
                out.append(x)
                break
    return out, comparisons


def common_set(a, b):
    b_set = set(b)  # one pass over b to build it
    return [x for x in a if x in b_set], len(a)  # one lookup per item


in_list, list_work = common_list(a, b)
in_set, set_work = common_set(a, b)
assert in_list == in_set
print(f"{len(in_list)} common items")
print(f"list: {list_work:,} comparisons; set: {set_work:,} lookups")
```

```
94 common items
list: 957,365 comparisons; set: 1,000 lookups
```

Exercise 2: When does the list grow?

```
"""Exercise 4.2: where a list grows, measured with sys.getsizeof and
predicted with the rule from CPython's listobject.c."""

import sys

SLOT = 8  # bytes per reference
EMPTY = sys.getsizeof([])


def predicted(needed):
    return (needed + (needed >> 3) + 6) & ~3


items, capacity, growths = [], 0, []
for n in range(1, 201):
    items.append(None)
    measured = (sys.getsizeof(items) - EMPTY) // SLOT
    if measured != capacity:
        growths.append((n, measured, predicted(n)))
        capacity = measured
for n, measured, rule in growths:
    print(f"at length {n:>3}: capacity {measured:>3} (rule says {rule})")
```

```
at length   1: capacity   4 (rule says 4)
at length   5: capacity   8 (rule says 8)
at length   9: capacity  16 (rule says 16)
at length  17: capacity  24 (rule says 24)
at length  25: capacity  32 (rule says 32)
at length  33: capacity  40 (rule says 40)
at length  41: capacity  52 (rule says 52)
at length  53: capacity  64 (rule says 64)
at length  65: capacity  76 (rule says 76)
at length  77: capacity  92 (rule says 92)
at length  93: capacity 108 (rule says 108)
at length 109: capacity 128 (rule says 128)
at length 129: capacity 148 (rule says 148)
at length 149: capacity 172 (rule says 172)
at length 173: capacity 200 (rule says 200)
```

Chapter 5: Recursion and the Call Stack

Exercise 1: Power.

```
"""Exercise 5.1: fast power. Halving n at each step means about
2 log2(n) multiplications instead of n - 1."""

multiplications = 0


def power(x, n):
    global multiplications
    if n == 0:
        return 1
    if n % 2 == 0:
        half = power(x, n // 2)
        multiplications += 1
        return half * half
    multiplications += 1
    return x * power(x, n - 1)


for n in [10, 1_000, 1_000_000]:
    multiplications = 0
    assert power(3, n) == 3**n
    print(f"n = {n:>9,}: {multiplications} multiplications")
```

```
n =        10: 5 multiplications
n =     1,000: 15 multiplications
n = 1,000,000: 26 multiplications
```

Exercise 2: Count the ways.

```
"""Exercise 5.2: ways to make an amount from coins 1, 2 and 5. Either
use the biggest remaining coin type at least once, or never use it."""

import itertools
from functools import cache

COINS = (1, 2, 5)


@cache
def ways(amount, kinds=len(COINS)):
    if amount == 0:
        return 1  # one way: no more coins
    if amount < 0 or kinds == 0:
        return 0
    coin = COINS[kinds - 1]
    return ways(amount - coin, kinds) + ways(amount, kinds - 1)


def brute(amount):
    return sum(
        1
        for a, b, c in itertools.product(range(amount + 1), repeat=3)
        if a * 1 + b * 2 + c * 5 == amount
    )


print([ways(a) for a in range(11)])
assert [ways(a) for a in range(11)] == [brute(a) for a in range(11)]
print(ways(1000))
```

```
[1, 1, 2, 2, 3, 4, 5, 6, 7, 8, 10]
50401
```

Chapter 6: A Dynamic Array

Exercise 1: extend.

```
"""Exercise 6.1: add extend(), which appends every item of an iterable,
growing the storage at most once when the length is known."""

from dynamic_array import DynamicArray


class ExtendableArray(DynamicArray):
    def extend(self, items):
        items = list(items)  # so we know how many there are
        needed = self._n + len(items)
        if needed > self._capacity:
            capacity = self._capacity
            while capacity < needed:
                capacity = max(capacity + 1, int(capacity * self._growth))
```

```
            self._resize(capacity)  # one resize, not one per doubling
        for x in items:
            self._data[self._n] = x
            self._n += 1


a = ExtendableArray()
a.append(0)
a.extend(range(1, 10))
assert list(a) == list(range(10))
print(a, "capacity", a._capacity, "copies", a.copies)
```

```
DynamicArray([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]) capacity 16 copies 1
```

Exercise 2: Why a quarter?

```
"""Exercise 6.2: why shrink at a quarter full, not at half full?
Count the copies when a program appends and pops at the boundary."""


def copies(shrink_at, steps=1000):
    """Start full at 8 slots, then alternate append and pop."""
    n, capacity, total = 8, 8, 0
    for step in range(steps):
        if step % 2 == 0:  # append
            if n == capacity:
                total += n
                capacity *= 2
            n += 1
        else:  # pop
            n -= 1
            if n <= capacity * shrink_at:
                total += n
                capacity //= 2
    return total


print("shrink at half full:   ", copies(0.5), "copies")
print("shrink at quarter full:", copies(0.25), "copies")
```

```
shrink at half full:    8000 copies
shrink at quarter full: 8 copies
```

Shrinking at half full makes the array thrash: one append at the boundary doubles it, the next pop halves it again, and every step copies all the items. Shrinking at a quarter full leaves room on both sides of every resize, so the cost stays amortized O(1).

Exercise 3: remove.

```
"""Exercise 6.3: add remove(value), which deletes the first item equal
to value and raises ValueError if there is none, like list.remove()."""

from dynamic_array import DynamicArray


class RemovableArray(DynamicArray):
    def remove(self, value):
        for i in range(self._n):
            if self._data[i] == value:
                self.pop(i)  # pop() shifts the later items left
                return
        raise ValueError(f"{value!r} is not in the array")


a = RemovableArray()
for x in [3, 1, 4, 1, 5]:
    a.append(x)
a.remove(1)
print(a)
try:
    a.remove(9)
except ValueError as error:
    print("ValueError:", error)
```

```
DynamicArray([3, 4, 1, 5])
ValueError: 9 is not in the array
```

Chapter 7: Linked Lists

Exercise 1: k-th from the end.

```
"""Exercise 7.1: remove the k-th node from the end in one pass,
using two pointers k nodes apart."""

from linked_list import LinkedList, Node


def remove_kth_from_end(head, k):
    dummy = Node(None, head)
    lead = trail = dummy
    for _ in range(k):
        lead = lead.next  # lead is now k nodes ahead of trail
    while lead.next:
        lead, trail = lead.next, trail.next
    trail.next = trail.next.next  # trail is just before the target
    return dummy.next


def values(head):
    while head:
        yield head.value
```

```
        head = head.next


for k in (1, 2, 5):
    head = LinkedList([1, 2, 3, 4, 5]).head
    print(k, list(values(remove_kth_from_end(head, k))))
```

```
1 [1, 2, 3, 4]
2 [1, 2, 3, 5]
5 [2, 3, 4, 5]
```

Exercise 2: Palindrome.

```
"""Exercise 7.2: is a linked list a palindrome? Constant extra memory:
find the middle, reverse the second half, compare, restore."""

from linked_list import LinkedList


def reverse_chain(node):
    before = None
    while node:
        node.next, before, node = before, node, node.next
    return before


def is_palindrome(items):
    head = LinkedList(items).head
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    second = reverse_chain(slow)
    a, b, same = head, second, True
    while b:
        if a.value != b.value:
            same = False
            break
        a, b = a.next, b.next
    reverse_chain(second)  # put the list back as it was
    return same


for word in ["racecar", "abba", "abca", "", "x"]:
    print(repr(word), is_palindrome(word))
```

```
'racecar' True
'abba' True
'abca' False
'' True
'x' True
```

Exercise 3: Where does the cycle start?

```
"""Exercise 7.3: where does a cycle start? After the pointers meet,
restart one from the head; moving both one step at a time, they meet
again exactly at the first node of the cycle."""

from linked_list import LinkedList


def cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            slow = head
            while slow is not fast:
                slow, fast = slow.next, fast.next
            return slow
    return None


chain = LinkedList(range(10))
print(cycle_start(chain.head))
node = chain.find(6)
chain.tail.next = node  # 9 now points back to 6
print(cycle_start(chain.head).value)
```

```
None
6
```

Chapter 8: Stacks and Queues

Exercise 1: Min stack.

```
"""Exercise 8.1: a stack that also reports its minimum in constant
time. A second stack holds the minimum at each height."""


class MinStack:
    def __init__(self):
        self._items, self._mins = [], []

    def push(self, value):
        self._items.append(value)
        current = self._mins[-1] if self._mins else value
        self._mins.append(min(value, current))

    def pop(self):
        self._mins.pop()
        return self._items.pop()

    def min(self):
        return self._mins[-1]
```

```
s = MinStack()
for x in [5, 3, 7, 2, 8]:
    s.push(x)
    print(f"push {x}: min {s.min()}")
for _ in range(3):
    print(f"pop {s.pop()}: min {s.min()}")
```

```
push 5: min 5
push 3: min 3
push 7: min 3
push 2: min 2
push 8: min 2
pop 8: min 2
pop 2: min 3
pop 7: min 3
```

Exercise 2: Reverse Polish notation.

```
"""Exercise 8.2: evaluate an expression in reverse Polish notation,
where each operator follows its two numbers: "3 4 + 2 *" is (3+4)*2."""

import operator

OPS = {
    "+": operator.add,
    "-": operator.sub,
    "*": operator.mul,
    "/": operator.truediv,
}


def rpn(expression):
    stack = []
    for token in expression.split():
        if token in OPS:
            right, left = stack.pop(), stack.pop()  # the order matters
            stack.append(OPS[token](left, right))
        else:
            stack.append(float(token))
    if len(stack) != 1:
        raise ValueError("not a complete expression")
    return stack[0]


for e in ["3 4 + 2 *", "5 1 2 + 4 * + 3 -", "10 4 /"]:
    print(f"{e} = {rpn(e):g}")
```

```
3 4 + 2 * = 14
5 1 2 + 4 * + 3 - = 14
10 4 / = 2.5
```

Exercise 3: Next greater element.

```
"""Exercise 8.3: next greater element. For each number, the first
larger number to its right, or -1. A monotonic stack, as for days
until warmer."""


def next_greater(numbers):
    answer = [-1] * len(numbers)
    waiting = []
    for i, x in enumerate(numbers):
        while waiting and numbers[waiting[-1]] < x:
            answer[waiting.pop()] = x
        waiting.append(i)
    return answer


print(next_greater([4, 5, 2, 25, 7, 6, 30]))
print(next_greater([5, 4, 3]))
```

```
[5, 25, 25, 30, 30, 30, -1]
[-1, -1, -1]
```

Chapter 9: Hash Tables

Exercise 1: A hashable Point.

```
"""Exercise 9.1: a Point class usable as a dict key. Equal points must
have equal hashes; hashing a tuple of the fields does it."""


class Point:
    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        if not isinstance(other, Point):
            return NotImplemented
        return (self.x, self.y) == (other.x, other.y)

    def __hash__(self):
        return hash((self.x, self.y))

    def __repr__(self):
        return f"Point({self.x}, {self.y})"


names = {Point(0, 0): "origin", Point(3, 4): "P"}
print(names[Point(3, 4)])  # a different object, but equal: found
print(Point(1, 2) in {Point(1, 2)}, len({Point(1, 2), Point(1, 2)}))
```

```
P
True 1
```

Exercise 2: Word counts.

```
"""Exercise 9.2: count words with your own ChainedMap, then list the
three most common."""

from chained_map import ChainedMap

text = """the quick brown fox jumps over the lazy dog the fox
the dog sleeps and the fox runs"""

counts = ChainedMap()
for word in text.split():
    counts[word] = counts[word] + 1 if word in counts else 1
top = sorted(counts, key=lambda w: (-counts[w], w))[:3]
for word in top:
    print(word, counts[word])
```

```
the 5
fox 3
dog 2
```

Exercise 3: Why tombstones?

```
"""Exercise 9.3: why tombstones? Delete by emptying the slot instead,
and a key stored past it can no longer be found."""

from open_map import EMPTY, OpenMap


class BrokenOpenMap(OpenMap):
    def __delitem__(self, key):
        i, found = self._find(key)
        if not found:
            raise KeyError(key)
        self._slots[i] = EMPTY  # wrong: breaks the chain of probes
        self._n -= 1


for make in (OpenMap, BrokenOpenMap):
    m = make()
    for k in [0, 8, 16]:  # all start at slot 0: stored in slots 0, 1, 2
        m[k] = "here"
    del m[8]
    print(f"{make.__name__}: 16 in map? {16 in m}")
```

```
OpenMap: 16 in map? True
BrokenOpenMap: 16 in map? False
```

Key 16 was placed past the deleted key because its probe sequence ran through that slot. Emptying the slot ends the probe sequence early, so the search stops before reaching 16. A tombstone tells searches to keep going while still letting inserts reuse the slot.

Chapter 10: Binary Search

Exercise 1: First and last.

```
"""Exercise 10.1: the first and last position of a target in a sorted
list, with two binary searches: [first, last] or [-1, -1]."""

from binary_search import lower_bound, upper_bound


def first_and_last(items, target):
    first = lower_bound(items, target)
    if first == len(items) or items[first] != target:
        return [-1, -1]
    return [first, upper_bound(items, target) - 1]


items = [5, 7, 7, 8, 8, 8, 10]
for t in [8, 7, 6, 10]:
    print(t, first_and_last(items, t))
```

```
8 [3, 5]
7 [1, 2]
6 [-1, -1]
10 [6, 6]
```

Exercise 2: Rotated list.

```
"""Exercise 10.2: search a sorted list that was rotated, such as
[4, 5, 6, 7, 0, 1, 2]. One half around mid is always in order."""


def search_rotated(items, target):
    lo, hi = 0, len(items) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if items[mid] == target:
            return mid
        if items[lo] <= items[mid]:  # the left half is in order
            if items[lo] <= target < items[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:  # the right half is in order
            if items[mid] < target <= items[hi]:
                lo = mid + 1
            else:
```

```
                hi = mid - 1
    return -1


items = [4, 5, 6, 7, 0, 1, 2]
print([search_rotated(items, t) for t in [0, 4, 7, 2, 3]])
```

```
[4, 0, 3, 6, -1]
```

Exercise 3: Integer square root.

```
"""Exercise 10.3: integer square root by binary search on the answer:
the largest r with r * r <= n, even for huge n."""

import math


def isqrt(n):
    lo, hi = 0, n + 1  # r is in lo..hi-1
    while hi - lo > 1:
        mid = (lo + hi) // 2
        if mid * mid <= n:
            lo = mid
        else:
            hi = mid
    return lo


for n in [0, 1, 15, 16, 10**30 + 7]:
    assert isqrt(n) == math.isqrt(n)
    print(n, isqrt(n))
```

```
0 0
1 1
15 3
16 4
1000000000000000000000000000007 1000000000000000
```

Chapter 11: Simple Sorts

Exercise 1: Count inversions.

```
"""Exercise 11.1: count inversions, the pairs i < j with
items[i] > items[j]. Each step of insertion sort's inner loop fixes
exactly one, so counting the shifts counts them."""


def inversions(items):
    items, count = list(items), 0
    for i in range(1, len(items)):
        x, j = items[i], i - 1
        while j >= 0 and items[j] > x:
```

```
            items[j + 1] = items[j]
            j -= 1
            count += 1
        items[j + 1] = x
    return count


print(
    inversions([1, 2, 3]),
    inversions([3, 2, 1]),
    inversions([2, 4, 1, 3, 5]),
)
```

```
0 3 3
```

Exercise 2: Two keys.

```
"""Exercise 11.2: sort by several keys with a stable sort: sort by the
least important key first, then by the most important."""

people = [
    ("Ada", "Lisbon", 30),
    ("Bo", "Oslo", 25),
    ("Cy", "Lisbon", 25),
    ("Di", "Oslo", 30),
]
by_age_then_city = sorted(people, key=lambda p: p[1])  # city first
by_age_then_city.sort(key=lambda p: p[2])  # stable: cities stay ordered
for p in by_age_then_city:
    print(p)
print(by_age_then_city == sorted(people, key=lambda p: (p[2], p[1])))
```

```
('Cy', 'Lisbon', 25)
('Bo', 'Oslo', 25)
('Ada', 'Lisbon', 30)
('Di', 'Oslo', 30)
True
```

Chapter 12: Merge Sort and Quicksort

Exercise 1: Count inversions fast.

```
"""Exercise 12.1: count inversions in n log n time. While merging,
taking an item from the right half jumps it over every item still
waiting in the left half: that many inversions at once."""


def sort_and_count(items):
    if len(items) <= 1:
        return list(items), 0
    mid = len(items) // 2
    left, a = sort_and_count(items[:mid])
```

```
    right, b = sort_and_count(items[mid:])
    out, i, j, count = [], 0, 0, a + b
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
            count += len(left) - i
    return out + left[i:] + right[j:], count


print(sort_and_count([2, 4, 1, 3, 5])[1])
print(sort_and_count(list(range(1000, 0, -1)))[1])  # 1000 * 999 / 2
```

```
3
499500
```

Exercise 2: Quickselect.

```
"""Exercise 12.2: quickselect, the k-th smallest item (k = 0 is the
smallest) in linear time on average: partition, then keep only the
side that holds position k."""

import random

from quicksort import partition


def quickselect(items, k):
    items = list(items)
    lo, hi = 0, len(items) - 1
    while True:
        r = random.randint(lo, hi)
        items[r], items[hi] = items[hi], items[r]
        p = partition(items, lo, hi)
        if p == k:
            return items[p]
        if k < p:
            hi = p - 1
        else:
            lo = p + 1


random.seed(2026)
data = [7, 2, 9, 4, 1, 8, 3]
print([quickselect(data, k) for k in range(len(data))])
```

```
[1, 2, 3, 4, 7, 8, 9]
```

Exercise 3: Sort colors.

```
"""Exercise 12.3: sort a list of 0s, 1s and 2s in one pass (the Dutch
national flag problem): quicksort3's three-way partition with pivot 1."""


def sort_colors(items):
    lt, i, gt = 0, 0, len(items) - 1
    while i <= gt:
        if items[i] == 0:
            items[lt], items[i] = items[i], items[lt]
            lt, i = lt + 1, i + 1
        elif items[i] == 2:
            items[i], items[gt] = items[gt], items[i]
            gt -= 1
        else:
            i += 1


colors = [2, 0, 2, 1, 1, 0, 1, 2, 0]
sort_colors(colors)
print(colors)
```

```
[0, 0, 0, 1, 1, 1, 2, 2, 2]
```

Chapter 13: Heaps and Priority Queues

Exercise 1: Heapsort.

```
"""Exercise 13.1: heapsort with MinHeap: heapify, then pop n times."""

import random

from binary_heap import MinHeap


def heapsort(items):
    h = MinHeap.heapify(items)
    return [h.pop() for _ in range(len(items))]


rng = random.Random(2026)
data = [rng.randint(0, 99) for _ in range(12)]
print(heapsort(data))
assert heapsort(data) == sorted(data)
```

```
[13, 15, 28, 40, 53, 64, 65, 71, 73, 76, 79, 82]
```

Exercise 2: k closest points.

```
"""Exercise 13.2: the k points closest to the origin. Keep a max-heap of
the k best so far; a closer point replaces the farthest of them.
Takes n log k time and k memory, however long the input."""

import heapq


def k_closest(points, k):
    heap = []  # (-distance squared, point): the farthest is on top
    for x, y in points:
        d = x * x + y * y
        if len(heap) < k:
            heapq.heappush(heap, (-d, (x, y)))
        elif -heap[0][0] > d:
            heapq.heapreplace(heap, (-d, (x, y)))
    return sorted(p for _, p in heap)


points = [(3, 3), (5, -1), (-2, 4), (1, 1), (0, -2), (6, 6)]
print(k_closest(points, 3))
```

```
[(0, -2), (1, 1), (3, 3)]
```

Exercise 3: Merge k sorted lists.

```
"""Exercise 13.3: merge k sorted lists with a heap of the next item from
each list: n log k comparisons for n items in all."""

import heapq


def merge_k(lists):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
    heapq.heapify(heap)
    out = []
    while heap:
        value, i, j = heapq.heappop(heap)
        out.append(value)
        if j + 1 < len(lists[i]):
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out


print(merge_k([[1, 4, 9], [2, 2, 8], [], [0, 10]]))
```

```
[0, 1, 2, 2, 4, 8, 9, 10]
```

Chapter 14: Sorting Without Comparing

Exercise 1: Sort by age.

```
"""Exercise 14.1: sort people by age (0 to 120) with a stable counting
sort; people of the same age keep their original order."""

from linear_sorts import counting_sort_by

people = [("Ana", 34), ("Ben", 7), ("Caz", 34), ("Dee", 71), ("Eli", 7)]
for name, age in counting_sort_by(people, lambda p: p[1], 120):
    print(f"{age:>3} {name}")
```

```
  7 Ben
  7 Eli
 34 Ana
 34 Caz
 71 Dee
```

Exercise 2: Bucket sort.

```
"""Exercise 14.2: bucket sort for floats spread evenly in [0, 1):
put each number in one of n buckets by its value, sort each small
bucket, and join them."""

import random


def bucket_sort(numbers):
    n = len(numbers)
    buckets = [[] for _ in range(n)]
    for x in numbers:
        buckets[int(x * n)].append(x)
    out = []
    for b in buckets:
        out.extend(sorted(b))  # each bucket holds about one number
    return out


rng = random.Random(2026)
data = [rng.random() for _ in range(10_000)]
assert bucket_sort(data) == sorted(data)
print([f"{x:.6f}" for x in bucket_sort(data)[:4]])
```

```
['0.000048', '0.000214', '0.000275', '0.000347']
```

Chapter 15: Binary Trees

Exercise 1: Mirror.

```
"""Exercise 15.1: mirror a tree, swapping every left and right child."""

from tree import from_levels, levels


def mirror(node):
    if node:
        node.left, node.right = mirror(node.right), mirror(node.left)
    return node


t = from_levels([1, 2, 3, 4, 5, None, 6])
print(levels(t))
print(levels(mirror(t)))
```

```
[[1], [2, 3], [4, 5, 6]]
[[1], [3, 2], [6, 5, 4]]
```

Exercise 2: Diameter.

```
"""Exercise 15.2: the diameter of a tree, the number of edges on the
longest path between any two nodes. Each node's best path through it
is its left height + its right height + 2."""

from tree import from_levels


def diameter(root):
    best = 0

    def height(node):
        nonlocal best
        if node is None:
            return -1
        left, right = height(node.left), height(node.right)
        best = max(best, left + right + 2)
        return 1 + max(left, right)

    height(root)
    return best


print(diameter(from_levels([1, 2, 3, 4, 5])))  # 4-2-1-3 or 5-2-1-3
print(diameter(from_levels([1, 2, None, 3, 4, 5, None, None, 6])))
```

```
3
4
```

Exercise 3: Lowest common ancestor.

```
"""Exercise 15.3: the lowest common ancestor of two values: the deepest
node that has both below it (or is one of them)."""

from tree import from_levels


def lca(node, a, b):
    if node is None or node.value in (a, b):
        return node
    left, right = lca(node.left, a, b), lca(node.right, a, b)
    if left and right:  # one on each side: this node joins them
        return node
    return left or right


t = from_levels([3, 5, 1, 6, 2, 0, 8, None, None, 7, 4])
for a, b in [(5, 1), (6, 4), (7, 8), (5, 4)]:
    print(a, b, "->", lca(t, a, b).value)
```

```
5 1 -> 3
6 4 -> 5
7 8 -> 3
5 4 -> 5
```

Chapter 16: Binary Search Trees

Exercise 1: Is it a BST?

```
"""Exercise 16.1: is a tree a valid BST? Every node must lie strictly
between the bounds set by its ancestors, not only beat its parent."""

from bst import BST, Node


def is_bst(node, low=None, high=None):
    if node is None:
        return True
    if (low is not None and node.key <= low) or (
        high is not None and node.key >= high
    ):
        return False
    return is_bst(node.left, low, node.key) and is_bst(
        node.right, node.key, high
    )


t = BST()
for k in [50, 30, 70, 20, 40]:
    t.put(k, None)
print(is_bst(t.root))
t.root.left.right = Node(55, None)  # 55 is right of 30 but left of 50
print(is_bst(t.root))
```

```
True
False
```

Exercise 2: Range query.

```
"""Exercise 16.2: the keys between low and high, in order, visiting
only the parts of the tree that can hold them."""

from bst import BST


def keys_between(node, low, high):
    if node is None:
        return
    if low < node.key:  # smaller keys may be on the left
        yield from keys_between(node.left, low, high)
    if low <= node.key <= high:
        yield node.key
    if node.key < high:  # bigger keys may be on the right
        yield from keys_between(node.right, low, high)


t = BST()
for k in [50, 30, 70, 20, 40, 60, 80, 35, 65]:
    t.put(k, None)
print(list(keys_between(t.root, 33, 66)))
```

```
[35, 40, 50, 60, 65]
```

Exercise 3: k-th smallest.

```
"""Exercise 16.3: the k-th smallest key (k = 1 is the smallest): walk
in order and stop after k keys."""

from itertools import islice

from bst import BST

t = BST()
for k in [50, 30, 70, 20, 40, 60, 80]:
    t.put(k, None)
for k in (1, 3, 7):
    print(k, next(islice(iter(t), k - 1, None)))
```

```
1 20
3 40
7 80
```

Chapter 17: Balanced Trees: AVL

Exercise 1: Counting rotations.

```
"""Exercise 17.1: count the rotations while inserting 1,000 keys in
sorted order and in random order."""

import random

import avl

rotations = 0
original_right, original_left = avl.rotate_right, avl.rotate_left


def counted(rotate):
    def wrapper(node):
        global rotations
        rotations += 1
        return rotate(node)

    return wrapper


avl.rotate_right = counted(original_right)
avl.rotate_left = counted(original_left)
rng = random.Random(2026)
for name, keys in [
    ("sorted", range(1000)),
    ("random", rng.sample(range(1000), 1000)),
]:
    rotations = 0
    t = avl.AVLTree()
    for k in keys:
        t.put(k, None)
    print(f"{name}: {rotations} rotations, height {t.height()}")
```

```
sorted: 990 rotations, height 9
random: 741 rotations, height 11
```

Exercise 2: The skinniest AVL tree.

```
"""Exercise 17.2: the smallest number of nodes an AVL tree of height h
can have: N(h) = N(h - 1) + N(h - 2) + 1. It grows like Fibonacci,
which is why the height stays logarithmic."""

import math

fewest = {-1: 0, 0: 1}
for h in range(1, 31):
    fewest[h] = fewest[h - 1] + fewest[h - 2] + 1
for h in [5, 10, 20, 30]:
    n = fewest[h]
    print(
        f"height {h:>2} needs at least {n:>9,} nodes; "
        f"log2 of that is {math.log2(n):.1f}"
```

```
    )
```

```
height  5 needs at least        20 nodes; log2 of that is 4.3
height 10 needs at least       232 nodes; log2 of that is 7.9
height 20 needs at least    28,656 nodes; log2 of that is 14.8
height 30 needs at least 3,524,577 nodes; log2 of that is 21.7
```

Chapter 18: Tries

Exercise 1: Longest common prefix.

```
"""Exercise 18.1: the longest prefix shared by every word: walk down
the trie while there is exactly one child and no word ends."""

from trie import Trie


def longest_common_prefix(words):
    node, prefix = Trie(words).root, ""
    while len(node.children) == 1 and not node.is_word:
        ch, node = next(iter(node.children.items()))
        prefix += ch
    return prefix


print(longest_common_prefix(["flower", "flow", "flight"]))
print(longest_common_prefix(["interview", "internet", "interval"]))
print(repr(longest_common_prefix(["dog", "car"])))
```

```
fl
inter
''
```

Exercise 2: Wildcards.

```
"""Exercise 18.2: search with wildcards, where "." matches any one
letter: at a ".", try every child."""

from trie import Trie


def matches(node, pattern):
    if not pattern:
        return node.is_word
    ch, rest = pattern[0], pattern[1:]
    if ch == ".":
        return any(matches(child, rest) for child in node.children.values())
    child = node.children.get(ch)
    return child is not None and matches(child, rest)


t = Trie(["bad", "dad", "mad", "made"])
```

```
for p in ["pad", "bad", ".ad", "b..", "ma.e", "...."]:
    print(p, matches(t.root, p))
```

```
pad False
bad True
.ad True
b.. True
ma.e True
.... True
```

Chapter 19: Union-Find

Exercise 1: The redundant cable.

```
"""Exercise 19.1: in a network that started as a tree, one extra cable
made a loop. Find the first cable that joins two already-connected
computers."""

from union_find import UnionFind


def redundant(n, cables):
    uf = UnionFind(n)
    for a, b in cables:
        if not uf.union(a, b):
            return a, b
    return None


print(redundant(5, [(0, 1), (1, 2), (3, 4), (2, 3), (1, 4)]))
```

```
(1, 4)
```

Exercise 2: When did everyone connect?

```
"""Exercise 19.2: friendships form over time, as (time, a, b) records.
When did everyone first belong to one group?"""

from union_find import UnionFind


def all_connected_at(n, records):
    uf = UnionFind(n)
    for time, a, b in sorted(records):
        uf.union(a, b)
        if uf.sets == 1:
            return time
    return None


records = [(20, 0, 1), (35, 3, 4), (5, 1, 2), (61, 2, 3), (50, 0, 2)]
print(all_connected_at(5, records))
```

```
print(all_connected_at(6, records))
```

```
61
None
```

Chapter 20: Graphs

Exercise 1: Degrees.

```
"""Exercise 20.1: in-degree and out-degree of every vertex in a
directed graph."""

from graph import Graph

g = Graph.from_edges(
    [("a", "b"), ("a", "c"), ("b", "c"), ("c", "a"), ("d", "c")],
    directed=True,
)
indegree = dict.fromkeys(g.vertices(), 0)
for _, v, _ in g.edges():
    indegree[v] += 1
for v in sorted(g.vertices()):
    print(f"{v}: out {g.degree(v)}, in {indegree[v]}")
```

```
a: out 2, in 1
b: out 1, in 1
c: out 1, in 3
d: out 1, in 0
```

Exercise 2: The town judge.

```
"""Exercise 20.2: the town judge. Everyone trusts the judge; the judge
trusts nobody. In a directed "trusts" graph that is the one vertex
with in-degree n - 1 and out-degree 0."""


def judge(n, trusts):
    score = [0] * (n + 1)  # in-degree minus out-degree
    for a, b in trusts:
        score[a] -= 1
        score[b] += 1
    for person in range(1, n + 1):
        if score[person] == n - 1:
            return person
    return -1


print(judge(3, [(1, 3), (2, 3)]))
print(judge(3, [(1, 3), (2, 3), (3, 1)]))
```

```
3
-1
```

Exercise 3: Reverse the edges.

```
"""Exercise 20.3: reverse every edge of a directed graph (its
transpose), as used to find who links to a page."""

from graph import Graph

links = Graph.from_edges(
    [("home", "about"), ("home", "blog"), ("blog", "about")],
    directed=True,
)
reverse = Graph(directed=True)
for v in links.vertices():
    reverse.add_vertex(v)
for u, v, w in links.edges():
    reverse.add_edge(v, u, w)
for page in sorted(reverse.vertices()):
    print(f"{page} is linked from {sorted(reverse.neighbors(page))}")
```

```
about is linked from ['blog', 'home']
blog is linked from ['home']
home is linked from []
```

Chapter 21: Breadth-First and Depth-First Search

Exercise 1: A cycle in an undirected graph.

```
"""Exercise 21.1: does an undirected graph have a cycle? In a DFS, a
visited neighbor that isn't the vertex we came from closes a loop."""


def has_cycle(graph):
    seen = set()
    for start in graph:
        if start in seen:
            continue
        stack = [(start, None)]
        while stack:
            v, came_from = stack.pop()
            if v in seen:
                return True
            seen.add(v)
            for w in graph[v]:
                if w != came_from:
                    stack.append((w, v))
    return False


path = {1: [2], 2: [1, 3], 3: [2]}
loop = {1: [2, 3], 2: [1, 3], 3: [2, 1]}
```

```
print(has_cycle(path), has_cycle(loop))
```

```
False True
```

Exercise 2: Two teams.

```
"""Exercise 21.2: split people into two teams so no two who dislike
each other share a team, or report that it can't be done."""

from search import two_colors

dislikes = [("Ann", "Bob"), ("Bob", "Cal"), ("Cal", "Dee"), ("Dee", "Eva")]
graph = {}
for a, b in dislikes:
    graph.setdefault(a, []).append(b)
    graph.setdefault(b, []).append(a)
colors = two_colors(graph)
for team in (0, 1):
    print(
        f"team {team + 1}:",
        sorted(p for p, c in colors.items() if c == team),
    )
graph["Eva"].append("Ann")  # a five-person loop
graph["Ann"].append("Eva")
print(two_colors(graph))
```

```
team 1: ['Ann', 'Cal', 'Eva']
team 2: ['Bob', 'Dee']
None
```

Chapter 22: Shortest Paths

Exercise 1: Network delay.

```
"""Exercise 22.1: a signal leaves one server and travels along timed
links. How long until every server has it? The largest of the
shortest distances, or -1 if some server is never reached."""

from shortest import dijkstra


def time_to_reach_all(n, links, start):
    graph = {v: {} for v in range(1, n + 1)}
    for a, b, ms in links:
        graph[a][b] = ms
    dist, _ = dijkstra(graph, start)
    return max(dist.values()) if len(dist) == n else -1


links = [(2, 1, 1), (2, 3, 1), (3, 4, 1)]
print(time_to_reach_all(4, links, 2))
print(time_to_reach_all(4, links, 1))
```

```
2
-1
```

Exercise 2: Cheapest flight with at most k stops.

```
"""Exercise 22.2: the cheapest flight with at most k stops. Bellman-Ford
limited to k + 1 rounds, each round relaxing from the previous round's
prices only, so a route never uses more flights than rounds."""

import math


def cheapest(flights, start, goal, k):
    price = {start: 0}
    for _ in range(k + 1):
        new = dict(price)
        for a, b, cost in flights:
            if a in price and price[a] + cost < new.get(b, math.inf):
                new[b] = price[a] + cost
        price = new
    return price.get(goal, -1)


flights = [
    ("A", "B", 100),
    ("B", "C", 100),
    ("A", "C", 500),
    ("C", "D", 100),
    ("B", "D", 600),
]
for k in (0, 1, 2):
    print(f"at most {k} stops: {cheapest(flights, 'A', 'D', k)}")
```

```
at most 0 stops: -1
at most 1 stops: 600
at most 2 stops: 300
```

Chapter 23: Minimum Spanning Trees

Exercise 1: Connect the points.

```
"""Exercise 23.1: connect points on a grid, where joining two costs
their Manhattan distance |x1 - x2| + |y1 - y2|. Every pair can be
joined, so this is a dense graph: Prim with a plain array, in V^2."""

import math


def connect_points(points):
    n = len(points)
    best = [math.inf] * n  # cheapest edge into the tree for each point
    best[0], total, in_tree = 0, 0, [False] * n
```

```
    for _ in range(n):
        u = min(
            (i for i in range(n) if not in_tree[i]), key=best.__getitem__
        )
        in_tree[u] = True
        total += best[u]
        for v in range(n):
            if not in_tree[v]:
                d = abs(points[u][0] - points[v][0]) + abs(
                    points[u][1] - points[v][1]
                )
                best[v] = min(best[v], d)
    return total


print(connect_points([(0, 0), (2, 2), (3, 10), (5, 2), (7, 0)]))
```

```
20
```

Exercise 2: Maximum spanning tree.

```
"""Exercise 23.2: a maximum spanning tree, keeping the heaviest edges,
for example the strongest links of a network. Kruskal with the
weights negated."""

from mst import kruskal

edges = [(4, 0, 1), (8, 0, 2), (2, 1, 2), (6, 1, 3), (3, 2, 3)]
tree = kruskal(4, [(-w, u, v) for w, u, v in edges])
print(
    [(-w, u, v) for w, u, v in tree], "total", -sum(w for w, _, _ in tree)
)
```

```
[(8, 0, 2), (6, 1, 3), (4, 0, 1)] total 18
```

Chapter 24: Solving a Problem in an Interview

Exercise 1: Two sum on a stream.

```
"""Exercise 24.1: two sum on a stream. Numbers arrive one at a time;
after each, say whether any two seen so far add up to the target.
A set of the numbers seen answers each step in O(1)."""


def stream_two_sum(numbers, target):
    seen, found = set(), False
    for x in numbers:
        found = found or (target - x) in seen  # once true, stays true
        seen.add(x)
        yield x, found
```

```
for x, found in stream_two_sum([3, 8, 1, 6, 5], 11):
    print(f"after {x}: {'yes' if found else 'no'}")
```

```
after 3: no
after 8: yes
after 1: yes
after 6: yes
after 5: yes
```

Chapter 25: Two Pointers

Exercise 1: Sorted squares.

```
"""Exercise 25.1: square every number of a sorted list (negatives
included) and return the squares sorted, in linear time. The biggest
square is at one of the two ends, so fill the answer from the back."""


def sorted_squares(nums):
    out = [0] * len(nums)
    lo, hi = 0, len(nums) - 1
    for k in range(len(nums) - 1, -1, -1):
        if abs(nums[lo]) > abs(nums[hi]):
            out[k] = nums[lo] ** 2
            lo += 1
        else:
            out[k] = nums[hi] ** 2
            hi -= 1
    return out


print(sorted_squares([-7, -3, 2, 3, 11]))
```

```
[4, 9, 9, 49, 121]
```

Exercise 2: Trapping rain water.

```
"""Exercise 25.2: trapping rain water. Water above a bar is limited by
the shorter of the tallest bars to its left and right. Two pointers
keep both maxima and always settle the side with the lower one."""


def trapped(heights):
    lo, hi = 0, len(heights) - 1
    left_max = right_max = water = 0
    while lo < hi:
        if heights[lo] < heights[hi]:
            left_max = max(left_max, heights[lo])
            water += left_max - heights[lo]
            lo += 1
        else:
            right_max = max(right_max, heights[hi])
```

```
            water += right_max - heights[hi]
            hi -= 1
    return water


print(trapped([0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]))
print(trapped([4, 2, 0, 3, 2, 5]))
```

```
6
9
```

Exercise 3: Subsequence.

```
"""Exercise 25.3: is `small` a subsequence of `big` (its letters appear
in order, not necessarily side by side)? One pointer in each string."""


def is_subsequence(small, big):
    i = 0
    for ch in big:
        if i < len(small) and small[i] == ch:
            i += 1
    return i == len(small)


print(is_subsequence("ace", "abcde"), is_subsequence("aec", "abcde"))
```

```
True False
```

Chapter 26: Sliding Window

Exercise 1: An anagram in the text.

```
"""Exercise 26.1: does text contain an anagram of word? A window of
len(word) letters slides along, its letter counts updated as it moves."""

from collections import Counter


def has_anagram(word, text):
    k = len(word)
    target, window = Counter(word), Counter(text[:k])
    if window == target:
        return True
    for right in range(k, len(text)):
        window[text[right]] += 1
        window[text[right - k]] -= 1
        if window[text[right - k]] == 0:
            del window[text[right - k]]  # so == compares fairly
        if window == target:
            return True
    return False
```

```
print(has_anagram("ab", "eidbaooo"), has_anagram("ab", "eidboaoo"))
```

```
True False
```

Exercise 2: Two kinds of fruit.

```
"""Exercise 26.2: the longest run of fruit trees you can pick from with
two baskets, each holding one kind: the longest window with at most two
distinct values."""

from collections import Counter


def longest_two_kinds(trees):
    counts, left, best = Counter(), 0, 0
    for right, kind in enumerate(trees):
        counts[kind] += 1
        while len(counts) > 2:
            counts[trees[left]] -= 1
            if counts[trees[left]] == 0:
                del counts[trees[left]]
            left += 1
        best = max(best, right - left + 1)
    return best


print(longest_two_kinds([1, 2, 1]), longest_two_kinds([1, 2, 3, 2, 2]))
```

```
3 4
```

Exercise 3: Longest run of ones.

```
"""Exercise 26.3: the longest run of 1s if you may flip at most k 0s:
the longest window holding at most k zeros."""


def longest_ones(bits, k):
    left = zeros = best = 0
    for right, b in enumerate(bits):
        zeros += b == 0
        while zeros > k:
            zeros -= bits[left] == 0
            left += 1
        best = max(best, right - left + 1)
    return best


print(longest_ones([1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0], 2))
```

```
6
```

Chapter 27: Hash Maps and Prefix Sums

Exercise 1: A nearby duplicate.

```
"""Exercise 27.1: is any value repeated within k positions? Remember
each value's last position."""


def nearby_duplicate(nums, k):
    last = {}
    for i, x in enumerate(nums):
        if x in last and i - last[x] <= k:
            return True
        last[x] = i
    return False


print(nearby_duplicate([1, 2, 3, 1], 3), nearby_duplicate([1, 2, 3, 1], 2))
```

```
True False
```

Exercise 2: Product of all the others.

```
"""Exercise 27.2: for each position, the product of all the other
numbers, without division: prefix products times suffix products."""


def product_except_self(nums):
    out = [1] * len(nums)
    left = 1
    for i in range(len(nums)):
        out[i] = left  # product of everything before i
        left *= nums[i]
    right = 1
    for i in range(len(nums) - 1, -1, -1):
        out[i] *= right  # times product of everything after i
        right *= nums[i]
    return out


print(product_except_self([1, 2, 3, 4]))
print(product_except_self([2, 0, 5]))
```

```
[24, 12, 8, 6]
[0, 10, 0]
```

Exercise 3: Top k words.

```
"""Exercise 27.3: the k most frequent words, ties broken alphabetically:
Counter, then a heap of size k (heapq.nsmallest with a key)."""

import heapq
from collections import Counter
```

```
text = "to be or not to be that is the question to be"
counts = Counter(text.split())
top = heapq.nsmallest(3, counts, key=lambda w: (-counts[w], w))
print([(w, counts[w]) for w in top])
```

```
[('be', 3), ('to', 3), ('is', 1)]
```

Chapter 28: BFS and DFS on Grids and Implicit Graphs

Exercise 1: The largest island.

```
"""Exercise 28.1: the area of the largest island."""

from grids import neighbors


def largest_island(grid):
    seen, best = set(), 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] != "#" or (r, c) in seen:
                continue
            seen.add((r, c))
            stack, area = [(r, c)], 0
            while stack:
                cell = stack.pop()
                area += 1
                for n in neighbors(grid, *cell):
                    if grid[n[0]][n[1]] == "#" and n not in seen:
                        seen.add(n)
                        stack.append(n)
            best = max(best, area)
    return best


print(largest_island(["##..#", "#...#", "..###", "....#"]))
```

```
6
```

Exercise 2: Distance to the nearest exit.

```
"""Exercise 28.2: the distance from every open cell to the nearest exit
"E", walls "#": one multi-source BFS from all exits at once."""

from collections import deque

from grids import neighbors

grid = ["E..#", "..#.", "....", "#..E"]
```

```
dist = {
    (r, c): 0
    for r, row in enumerate(grid)
    for c, x in enumerate(row)
    if x == "E"
}
queue = deque(dist)
while queue:
    cell = queue.popleft()
    for n in neighbors(grid, *cell):
        if grid[n[0]][n[1]] == "." and n not in dist:
            dist[n] = dist[cell] + 1
            queue.append(n)
for r, row in enumerate(grid):
    print(
        " ".join(
            row[c] if row[c] in "#E" else str(dist.get((r, c), "?"))
            for c in range(len(row))
        )
    )
```

```
E 1 2 #
1 2 # 2
2 3 2 1
# 2 1 E
```

Exercise 3: Knight moves.

```
"""Exercise 28.3: the fewest moves for a chess knight between two
squares of an 8 x 8 board: BFS on the implicit graph of knight moves."""

from collections import deque

MOVES = [
    (1, 2),
    (2, 1),
    (2, -1),
    (1, -2),
    (-1, -2),
    (-2, -1),
    (-2, 1),
    (-1, 2),
]


def knight_moves(start, goal):
    dist, queue = {start: 0}, deque([start])
    while queue:
        r, c = queue.popleft()
        if (r, c) == goal:
            return dist[r, c]
        for dr, dc in MOVES:
            n = (r + dr, c + dc)
            if 0 <= n[0] < 8 and 0 <= n[1] < 8 and n not in dist:
                dist[n] = dist[r, c] + 1
                queue.append(n)
```

```
print(knight_moves((0, 0), (7, 7)), knight_moves((0, 0), (1, 1)))
```

```
6 4
```

Chapter 29: Dynamic Programming I: Memoization and Tables

Exercise 1: Decode ways.

```
"""Exercise 29.1: ways to decode a string of digits where A=1 ... Z=26.
Like the stairs: the last letter used one digit or two."""


def decode_ways(digits):
    two_back, one_back = 1, 1  # ways for "" and for the first digit
    if not digits or digits[0] == "0":
        return 0
    for i in range(1, len(digits)):
        ways = one_back if digits[i] != "0" else 0
        if 10 <= int(digits[i - 1 : i + 1]) <= 26:
            ways += two_back
        two_back, one_back = one_back, ways
    return one_back


for d in ["12", "226", "06", "11106"]:
    print(d, decode_ways(d))
```

```
12 2
226 3
06 0
11106 2
```

Exercise 2: Paths with obstacles.

```
"""Exercise 29.2: paths through a grid moving right or down, with
blocked cells ("#"). Each cell's count is the sum of the counts above
it and to its left."""


def unique_paths(grid):
    cols = len(grid[0])
    row = [0] * cols
    row[0] = 1
    for line in grid:
        for c in range(cols):
            if line[c] == "#":
                row[c] = 0
            elif c:
                row[c] += row[c - 1]  # row[c] still holds the cell above
```

```
    return row[-1]


print(unique_paths(["...", ".#.", "..."]), unique_paths(["...."] * 4))
```

```
2 20
```

Exercise 3: The cheapest climb.

```
"""Exercise 29.3: the cheapest way up a staircase where each step has a
cost and you may climb one or two at a time, starting on step 0 or 1."""


def min_cost_climb(cost):
    two_back = one_back = 0  # cheapest way to stand on the step
    for i in range(2, len(cost) + 1):
        two_back, one_back = (
            one_back,
            min(one_back + cost[i - 1], two_back + cost[i - 2]),
        )
    return one_back


print(min_cost_climb([10, 15, 20]))
print(min_cost_climb([1, 100, 1, 1, 1, 100, 1, 1, 100, 1]))
```

```
15
6
```

Chapter 30: Dynamic Programming II: Sequences, Strings and Knapsack

Exercise 1: Maximum run sum.

```
"""Exercise 30.1: the maximum sum of a run of neighbors (Kadane's
algorithm): the best run ending here either extends the best run
ending one step back or starts fresh."""


def max_run_sum(nums):
    best = here = nums[0]
    for x in nums[1:]:
        here = max(x, here + x)
        best = max(best, here)
    return best


print(max_run_sum([-2, 1, -3, 4, -1, 2, 1, -5, 4]))
print(max_run_sum([-3, -1, -2]))
```

```
6
-1
```

Exercise 2: Word break.

```
"""Exercise 30.2: can text be split into dictionary words?
ok[i] means text[:i] can be split."""


def word_break(text, words):
    words = set(words)
    ok = [True] + [False] * len(text)
    for i in range(1, len(text) + 1):
        ok[i] = any(ok[j] and text[j:i] in words for j in range(i))
    return ok[-1]


words = ["apple", "pen", "pine", "pineapple"]
print(word_break("pineapplepenapple", words), word_break("pinepen", words))
print(word_break("applepie", words))
```

```
True True
False
```

Exercise 3: Equal halves.

```
"""Exercise 30.3: can a list be split into two parts with equal sums?
A knapsack question: can some items add up to exactly half?"""


def can_split(nums):
    total = sum(nums)
    if total % 2:
        return False
    reachable = {0}
    for x in nums:
        reachable |= {s + x for s in reachable}
    return total // 2 in reachable


print(can_split([1, 5, 11, 5]), can_split([1, 2, 3, 5]))
```

```
True False
```

Chapter 31: Greedy Algorithms and Backtracking

Exercise 1: Balanced brackets.

```
"""Exercise 31.1: every string of n pairs of balanced brackets. Add "("
while some are left; add ")" only while it closes an open one."""


def brackets(n):
    out = []

    def go(text, opened, closed):
        if len(text) == 2 * n:
            out.append(text)
            return
        if opened < n:
            go(text + "(", opened + 1, closed)
        if closed < opened:
            go(text + ")", opened, closed + 1)

    go("", 0, 0)
    return out


print(brackets(3))
print([len(brackets(n)) for n in range(1, 9)])
```

```
['((()))', '(()())', '(())()', '()(())', '()()()']
[1, 2, 5, 14, 42, 132, 429, 1430]
```

Exercise 2: A small sudoku.

```
"""Exercise 31.2: solve a 4 x 4 sudoku (digits 1-4; every row, column
and 2 x 2 box holds each digit once) by backtracking. 0 is empty."""


def solve(board):
    for r in range(4):
        for c in range(4):
            if board[r][c] == 0:
                for d in range(1, 5):
                    if fits(board, r, c, d):
                        board[r][c] = d
                        if solve(board):
                            return True
                        board[r][c] = 0  # undo and try the next digit
                return False  # nothing fits here: backtrack
    return True  # no empty cell left


def fits(board, r, c, d):
    box_r, box_c = r // 2 * 2, c // 2 * 2
    box = [
        board[i][j]
        for i in range(box_r, box_r + 2)
        for j in range(box_c, box_c + 2)
```

```
    ]
    column = [board[i][c] for i in range(4)]
    return d not in board[r] and d not in column and d not in box


board = [[1, 0, 0, 0], [0, 0, 3, 0], [0, 4, 0, 0], [0, 0, 0, 2]]
solve(board)
for row in board:
    print(*row)
```

```
1 3 2 4
4 2 3 1
2 4 1 3
3 1 4 2
```

Chapter 32: Project: An LRU Cache

Exercise 1: The short version.

```
"""Exercise 32.1: an LRU cache in a few lines with OrderedDict, whose
move_to_end() and popitem(last=False) do the linked-list work."""

from collections import OrderedDict


class SmallLRU(OrderedDict):
    def __init__(self, capacity):
        super().__init__()
        self.capacity = capacity

    def get(self, key, default=None):
        if key not in self:
            return default
        self.move_to_end(key)  # the end is the most recent
        return self[key]

    def put(self, key, value):
        self[key] = value
        self.move_to_end(key)
        if len(self) > self.capacity:
            self.popitem(last=False)  # the front is the oldest


c = SmallLRU(2)
c.put("a", 1)
c.put("b", 2)
c.get("a")
c.put("c", 3)
print(list(c.items()))
```

```
[('a', 1), ('c', 3)]
```

Exercise 2: Entries that expire.

```
"""Exercise 32.2: entries that expire. Each entry stores when it was
written; get() treats anything older than ttl seconds as missing. The
clock is a parameter, so the test can move time forward."""

from lru_cache import LRUCache


class TTLCache(LRUCache):
    def __init__(self, capacity, ttl, clock):
        super().__init__(capacity)
        self.ttl, self.clock = ttl, clock

    def put(self, key, value):
        super().put(key, (self.clock(), value))

    def get(self, key, default=None):
        entry = super().get(key)
        if entry is None or self.clock() - entry[0] > self.ttl:
            return default
        return entry[1]


now = [0]
c = TTLCache(10, ttl=60, clock=lambda: now[0])
c.put("price", 9.99)
now[0] = 30
print(c.get("price"))
now[0] = 61
print(c.get("price"))
```

```
9.99
None
```

Appendix B: Complexity Cheat Sheet

The costs of the structures and algorithms in this book, with a measured time from the book's benchmarks where there is one. The measurements come from one computer (Appendix C) on Python 3.15; yours will differ, but the ratios between rows, and how each row grows, should be similar. "Amortized" means averaged over a long sequence of operations; individual operations can occasionally cost more.

Python's Built-In Structures

Operation	Cost	Measured (Chapter)
`list.append(x)`, `list.pop()`	O(1) amortized	about 0.11 µs for the pair, at any size (4)
`items[i]`, `items[i] = x`	O(1)	about 0.12 µs (4)
`list.insert(0, x)`, `list.pop(0)`	O(n)	446 µs for the pair at 160,000 items (4)
`x in items` (list)	O(n)	11.6 ms at 1,000,000 items (10)
`x in s`, `s.add(x)` (set)	O(1) average	about 0.13 µs for add and remove, at any size (4)
`d[key]`, `d[key] = value`, `del d[key]`	O(1) average	about 0.11 µs per get, at any size up to 1,000,000 (4)

Operation	Cost	Measured (Chapter)
`deque.append`, `appendleft`, `pop`, `popleft`	O(1)	about 0.11 µs at the front, at any size (7)
`heapq.heappush`, `heappop`	O(log n)	7.5 ms for 80,000 pushes (13)
`heapq.heapify`	O(n)	5.8 ms for 80,000 items (13)
`bisect.bisect_left`	O(log n)	0.30 µs at 1,000,000 items (10)
`sorted()`, `list.sort()`	O(n log n); O(n) on sorted or nearly sorted data	17.5 ms for 80,000 numbers (12)
Building a string with `+=` in a loop	O(n) total in CPython's best case, $O(n^2)$ when the string has another reference; `"".join` is always O(n)	1.55 ms against 77.7 ms for 40,000 words; `join`, 0.20 ms (4)

The Structures You Built

Structure	Main operations	Cost	Measured (Chapter)
Dynamic array	append	O(1) amortized	13.1 ms for 80,000 appends, doubling (6)
Dynamic array, growing by a constant	append	O(n) per append	1.08 s for 80,000 appends (6)
Singly linked list	add or remove at the front	O(1)	0.31 µs (7)
Linked list	find, index	O(n)	(7)
Stack (list)	push, pop	O(1) amortized	(8)
Queue: `RingQueue`, `TwoStackQueue`	enqueue, dequeue	O(1) (amortized for two stacks)	35 ms and 20 ms for 80,000 of each (8)
Hash table, chaining	get, set, delete	O(1) average	0.27 µs per get at 1,000,000 keys (9)
Hash table, open addressing	get, set, delete	O(1) average	0.36 µs per get at 1,000,000 keys (9)

Structure	Main operations	Cost	Measured (Chapter)
Binary heap	push, pop	O(log n)	38 ms for 80,000 pushes (13)
Binary heap	heapify	O(n)	34 ms for 80,000 items (13)
Binary search tree	search, insert, delete	O(height): O(log n) random, O(n) sorted keys	0.53 µs random, 148 µs sorted, at 4,000 keys (16)
AVL tree	search, insert, delete	O(log n) always	21 ms to insert 4,000 sorted keys, against 589 ms for a plain BST (17)
Trie	insert, lookup, prefix check	O(length of the word)	0.28 µs per prefix check at any size (18)
Union-find, with path compression and union by size	find, union	nearly O(1) amortized	3.3 ms for a chain of 8,000 unions, then 8,000 finds (19)
LRU cache (dict + doubly linked list)	get, put	O(1)	46 ms for 100,000 requests (32)

Sorting

Algorithm	Best	Average	Worst	Extra memory	Stable	Measured at 80,000 (Chapter)
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	(11)
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	(11)
Bubble sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	(11)
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	179 ms (12)
Quicksort, random pivot	O(n log n)	O(n log n)	$O(n^2)$, very unlikely	O(log n)	No	130 ms (12)

Algorithm	Best	Average	Worst	Extra memory	Stable	Measured at 80,000 (Chapter)
Heapsort	O(n log n)	O(n log n)	O(n log n)	O(1) in place (O(n) in Exercise 13.1)	No	(13)
Counting sort	O(n + k)	O(n + k)	O(n + k)	O(n + k)	Yes	25 ms (14)
Radix sort	O(d(n + b))	O(d(n + b))	O(d(n + b))	O(n + b)	Yes	357 ms (14)
`sorted()` (Timsort/powersort)	O(n)	O(n log n)	O(n log n)	O(n)	Yes	17.5 ms (12)

Here k is the range of the keys, d the number of digits and b the base.

Searching and Graphs

Algorithm	Cost	Notes (Chapter)
Binary search	O(log n)	sorted data; also "search the answer" (10)
BFS, DFS	O(V + E)	BFS: fewest edges; use `deque` as the queue (21)
Topological sort (Kahn)	O(V + E)	detects cycles; `graphlib` in the standard library (21)
Dijkstra with a heap	O(E log V)	non-negative weights only (22)
Bellman-Ford	O(V × E)	negative weights; detects negative cycles (22)
Floyd-Warshall	$O(V^3)$	all pairs; used as the test reference (22)
Kruskal	O(E log E)	sort the edges, union-find (23)
Prim with a heap	O(E log V)	grows one tree (23)
Edge check: adjacency lists, sets, matrix	O(degree), O(1), O(1)	0.18, 0.09 and 0.10 µs (20)

Interview Patterns

Pattern	Typical cost	Signals (Chapter)
Hash map lookups (two sum)	O(n)	pairs, "seen before", counting (24, 27)
Two pointers	O(n), or O(n log n) with a sort	sorted input, pairs, in place (25)
Sliding window	O(n)	contiguous, substring, longest or shortest run (26)
Prefix sums	O(n) to build, O(1) per range	range sums; with a hash map, counting runs (27)
Monotonic stack or deque	O(n)	next greater, window maximum (8, 26)
BFS on grids and implicit graphs	O(cells) or O(states)	fewest steps, spreading, flood fill (28)
Dynamic programming	O(states × choices)	how many ways, best total, overlapping subproblems (29, 30)
Greedy	usually O(n log n) for the sort	intervals, scheduling; needs a proof (31)
Backtracking	exponential, cut by pruning	all solutions, small n (31)

Input Size and the Complexity You Need

With a few seconds to spare, in Python:

Largest n	Fast enough
about 10 to 20	$O(2^n)$, O(n!)
about 500	$O(n^3)$
about 5,000	$O(n^2)$
100,000 to 1,000,000	O(n log n), O(n)
beyond	O(log n), O(1)

Appendix C: How the Benchmarks Were Run

Every timing in this book comes from running the benchmark listing printed with it. This appendix describes the computer, the method, and how to run the benchmarks yourself.

The Computer

- A cloud virtual machine with 4 cores of an Intel Xeon processor at 2.80 GHz and 15 GB of memory
- Ubuntu 24.04 LTS
- Python 3.15.0 for every benchmark table and chart, except Chapter 33, which compares Python 3.12.3, 3.13.5, 3.14.8 and 3.15.0 (how each was built is described in that chapter)

A cloud machine shares its hardware with other machines, which adds noise to timings. The method below is designed to reduce it, and the book only draws conclusions from patterns that are much larger than the noise: growth ratios close to 2 or 4, and differences of several times between structures.

The Method

All timings use the `bench` module from Chapter 3 (`code/common/bench.py`), which every chapter folder has a copy of:

- `measure` times an operation with `time.perf_counter`, repeating it until the total takes at least 20 milliseconds, so even operations of a tenth of a microsecond are timed accurately. It does this five times and keeps the **fastest** round, because noise from other programs only ever makes a run slower. Garbage collection is turned off while timing, so its pauses don't land at random in one measurement.
- When an operation changes its input, such as a sort, a `setup` function makes fresh input for every call, and only the operation itself is timed.
- `doubling` times an operation at a series of input sizes. It **interleaves** the rounds: each round times every size once, and each size keeps its fastest time across the rounds. A burst of background activity therefore affects one round of every size a little, instead of every round of one size.
- `table` prints the times with the ratio to the previous size, and `chart` draws them.

Recording and Checking

The book's repository contains three tools for this, in the `tests` folder:

- `record_benchmarks.py` runs every benchmark listing once with Python 3.15 and saves both its printed table (the `.out` file shown in the book) and its chart, so the table and the chart always come from the same run.
- `run_listings.py` checks every listing in the book on all four versions of Python. Ordinary listings must print exactly their recorded output. Benchmarks run in a quick mode (`BENCH_QUICK=1`), which checks that they work without waiting for exact timings, since their numbers differ on every run.
- `record_versions.py` runs Chapter 33's suite on each version of Python in turn, three passes in rotation, and keeps each version's fastest time for each test.

Running the Benchmarks Yourself

From the chapter's folder in the repository, with your virtual environment active (Chapter 2):

```
$ python bench_append.py
```

Expect different numbers: a faster or slower computer changes every time in the table. The ratios between rows and the shape of each chart should be close to the book's. If they're not, run the benchmark again with your other programs closed; if they're still different, you've found something interesting about your computer or your version of Python, and the benchmark is the tool to explore it.

Appendix D: Glossary

Adjacency list. A graph representation that stores, for each vertex, the list (or set) of its neighbors (Chapter 20).

Adjacency matrix. A graph representation as a V × V table, with a mark where an edge exists (Chapter 20).

Algorithm. A precise, step-by-step method for solving a problem (Chapter 1).

Amortized cost. The cost of an operation averaged over a long sequence of operations, such as `append` on a dynamic array: O(1) amortized, although an occasional append copies everything (Chapter 6).

AVL tree. A binary search tree that keeps the heights of every node's two subtrees within one of each other, using rotations, so its height stays O(log n) (Chapter 17).

Backtracking. Building a solution one choice at a time, undoing each choice to try the next; pruning skips choices that can't succeed (Chapter 31).

Balanced tree. A tree whose height stays O(log n) as items are added and removed (Chapter 17).

Base case. The input a recursive function answers directly, without recursing (Chapter 5).

BFS (breadth-first search). Exploring a graph in rings around the start, using a queue; finds paths with the fewest edges (Chapters 21 and 28).

Big O notation. A description of how an algorithm's cost grows with the input size, ignoring constant factors: O(1), O(log n), O(n), O(n log n), $O(n^2)$ (Chapter 3).

Binary heap. A complete binary tree, stored in a list, in which every parent is no larger than its children (a min-heap); the smallest item is always at the root (Chapter 13).

Binary search. Finding an item in sorted data by halving the range at each step: O(log n) (Chapter 10).

Binary search tree (BST). A binary tree in which every node's left subtree holds smaller keys and its right subtree larger ones (Chapter 16).

Bipartite graph. A graph whose vertices split into two groups with every edge going between the groups (Chapter 21).

Brute force. A solution that tries every possibility; slow, but a reliable reference for testing (Chapter 24).

Cache. A store of recent results kept for fast reuse; an LRU cache evicts the least recently used entry when full (Chapter 32).

Collision. Two keys whose hashes lead to the same slot of a hash table (Chapter 9).

Component (connected component). A group of vertices that are all reachable from each other, and from no vertex outside the group (Chapters 19 and 21).

Cycle. A path in a graph that returns to its starting vertex (Chapters 20 and 21).

DAG (directed acyclic graph). A directed graph with no cycles; exactly the graphs that have a topological order (Chapters 20 and 21).

DFS (depth-first search). Exploring a graph by following one path as far as possible before backing up, using a stack (Chapters 21 and 28).

Doubly linked list. A linked list whose nodes link both to the next node and to the previous one (Chapter 7).

Dynamic array. An array that grows by allocating a bigger block and copying, with spare capacity; Python's `list` is one (Chapter 6).

Dynamic programming (DP). Solving a problem by building on stored answers to smaller, overlapping subproblems (Chapters 29 and 30).

Edge. A connection between two vertices of a graph; may be directed and may have a weight (Chapter 20).

Exchange argument. A proof that a greedy choice is safe: any best solution can be changed to include it without getting worse (Chapter 31).

Geometric mean. The n-th root of the product of n numbers; the right way to average ratios (Chapter 33).

Graph. A set of vertices joined by edges (Chapter 20).

Greedy algorithm. An algorithm that always takes the choice that looks best right now and never reconsiders (Chapter 31).

Hash function. A function that turns a key into a number, used to choose a slot in a hash table (Chapter 9).

Hash table. A structure that stores items in slots chosen by the hash of their key, for O(1) average lookups; Python's `dict` and `set` are hash tables (Chapter 9).

Height (of a tree). The number of edges on the longest path from the root down to a leaf (Chapter 15).

In-degree. The number of edges pointing into a vertex (Chapter 21).

Implicit graph. A graph whose edges are computed by a rule when needed rather than stored (Chapter 28).

Invariant. A condition that a structure or loop keeps true at every step, such as a binary search's range always containing the answer if it exists (Chapters 10 and 17).

Linked list. A sequence of nodes, each holding an item and a link to the next node (Chapter 7).

Load factor. The number of items in a hash table divided by the number of slots (Chapter 9).

Memoization. Caching a function's results by its arguments, so each subproblem is solved once (Chapters 5 and 29).

Minimum spanning tree (MST). The cheapest set of edges that connects all the vertices of a weighted graph (Chapter 23).

Monotonic stack or deque. A stack or deque kept in increasing or decreasing order, by discarding items that can never matter again (Chapters 8 and 26).

Open addressing. Handling collisions by probing other slots of the same table, rather than chaining lists (Chapter 9).

Prefix sum. The running total of a list; the sum of any range is the difference of two prefix sums (Chapter 27).

Priority queue. A collection that always hands out its smallest (or largest) item next; usually built on a heap (Chapter 13).

Pruning. Abandoning a branch of a search as soon as it can't lead to a solution (Chapter 31).

Queue. A first-in, first-out (FIFO) collection (Chapter 8).

Recursion. A function that solves a problem by calling itself on smaller inputs (Chapter 5).

Relaxation. Improving a vertex's best known distance by going through a neighbor (Chapter 22).

Rotation. A local rearrangement of three nodes and their subtrees that rebalances a binary search tree while keeping its order (Chapter 17).

Sentinel. A dummy node that removes special cases at the ends of a linked list (Chapters 7 and 32).

Sliding window. A range of a sequence that moves along it, adding items on one side and dropping them on the other (Chapter 26).

Stable sort. A sort that keeps items with equal keys in their original order (Chapter 11).

Stack. A last-in, first-out (LIFO) collection (Chapter 8).

Stress test. Comparing a solution with a brute force on many random inputs (Chapter 24).

Tombstone. A marker left in an open-addressing hash table where an item was deleted, so that searches continue past it (Chapter 9).

Topological order. An ordering of a directed graph's vertices in which every edge points forward (Chapter 21).

Trie. A tree of characters in which each path from the root spells a prefix; finds words and prefixes in time proportional to their length (Chapter 18).

Two pointers. Two indexes that move through a sequence in one pass, replacing a search over all pairs (Chapter 25).

Union-find (disjoint set union). A structure that tracks which items are in the same group, with nearly O(1) union and find (Chapter 19).

Vertex (node). A point in a graph (Chapter 20).

Appendix E: Sources

The Real-World Use and Interesting Fact boxes rely on the following public sources, checked in October 2026. Where possible, the primary source, the project's own source code or documentation, was read directly; file paths refer to each project's source repository.

Chapter	Fact	Source
3	`timeit`'s documentation: "the min() of the result is probably the only number you should be interested in"	CPython `Doc/library/timeit.rst`
4, 6	CPython over-allocates lists: `new_allocated = (newsize + (newsize >> 3) + 6) & ~3`, growth pattern 0, 4, 8, 16, 24, 32, 40, 52, 64, 76 (matches the measurement in Chapter 4)	CPython `Objects/listobject.c`
4	Dict insertion order became an official part of the language in Python 3.7	CPython `Doc/whatsnew/3.7.rst`
4	PEP 8: do not rely on CPython's in-place string concatenation; use `''.join()` in performance-sensitive code	PEP 8, python/peps repository
6	Java's `ArrayList` grows by half its capacity (`oldCapacity >> 1`)	OpenJDK `java/util/ArrayList.java`

Chapter	Fact	Source
6	Go slices double while small, then move toward 1.25x growth after a threshold of 256	Go `src/runtime/slice.go`, `nextslicecap`
7, 8	`collections.deque` is a doubly linked list of blocks of 64 slots	CPython `Modules/_collectionsmodule.c`, `BLOCKLEN 64`
7	Linux's `include/linux/list.h`: "Circular doubly linked list implementation"; an initialized list head is an empty list	Linux source
8	Linux `kfifo`: a generic kernel FIFO built on a ring buffer (sizes rounded to powers of 2)	LWN coverage of the kfifo rewrite
9	CPython's dict uses open addressing (not chaining) with a perturbed probe sequence, a compact layout (an index table plus an entries array, which keeps insertion order), and resizes at two thirds full	CPython `Objects/dictobject.c` comments, `USABLE_FRACTION`
9	Java's `HashMap` uses chaining with load factor 0.75 and turns a bucket into a balanced tree when it holds 8 or more entries	OpenJDK `java/util/HashMap.java`, `TREEIFY_THRESHOLD = 8`, `DEFAULT_LOAD_FACTOR = 0.75f`
9	Hashes of `str` and `bytes` are salted with a random value per process, enabled by default since Python 3.3, to defend against denial of service through inputs chosen to make dict insertion quadratic	CPython `Doc/reference/datamodel.rst` (`object.__hash__` note); `Doc/whatsnew/3.3.rst`
9	Python 3.6 reimplemented dict "to use a more compact representation", 20% to 25% less memory than 3.5	CPython `Doc/whatsnew/3.6.rst`

Chapter	Fact	Source
9	Hash flooding: Klink and Wälde, 28C3 (December 2011), demonstrated against PHP, Python, Java, Ruby and V8; advisory oCERT-2011-003	LWN article 474365; oCERT advisory
10	`git bisect` uses binary search to find the commit that introduced a bug	Git `Documentation/git-bisect.adoc`
10	Joshua Bloch (2006) showed that the binary search in many libraries, including Java's, overflowed in `(low + high) / 2` for arrays of about 2^30 or more elements	Google Research blog, "Extra, Extra - Read All About It: Nearly All Binary Searches and Mergesorts are Broken", 2 June 2006
10	Bloch's 2006 report: the bug in *Programming Pearls* went unnoticed for two decades; Java's own binary search had the same overflow	Google Research blog
11, 12	CPython's sort extends short runs to a minimum length with binary insertion sort and reverses descending runs in place	CPython `Objects/listsort.txt`
12	Tim Peters rewrote `list.sort()` for Python 2.3 (the algorithm now called Timsort); since Python 3.11 it uses the "powersort" merge strategy of Munro and Wild	CPython `Doc/whatsnew/2.3.rst`; `Objects/listsort.txt`; bpo-34561 commit
12	V8 (Chrome's JavaScript engine) switched `Array.prototype.sort` to TimSort in V8 v7.0 / Chrome 70 (September 2018), making it stable	V8 blog "Getting things sorted in V8", via the v8.dev repository on GitHub
12	Go's `sort` package uses pattern-defeating quicksort (pdqsort) since Go 1.19	Go `src/sort/zsortfunc.go`; Go 1.19 release notes in the golang/website repository

Chapter	Fact	Source
12	Tony Hoare devised quicksort in 1959-60 as a visiting student in Moscow, to sort words for machine translation; published in Communications of the ACM in 1961	Stanford and other histories
12	Dijkstra posed the Dutch national flag problem in *A Discipline of Programming* (1976)	NIST DADS; Wikipedia
13	asyncio's event loop keeps scheduled callbacks in a heap (`heapq.heappush(self._scheduled, timer)`)	CPython `Lib/asyncio/base_events.py`
14	Punched-card sorters sorted one column per pass; multi-column keys were sorted least significant column first (LSD radix sort)	Wikipedia "Punched card sorter"; Columbia University computing history pages; IBM 83 sorter manual
17	Linux uses red-black trees for I/O scheduler requests, high-resolution timers, virtual memory areas and epoll	Linux `Documentation/core-api/rbtree.rst`
17	SQLite is "an external (disk-based) database using BTrees"; PostgreSQL's `CREATE INDEX` makes B-tree indexes by default	SQLite `src/btree.c`; PostgreSQL `doc/src/sgml/indices.sgml`
17	The AVL tree (Adelson-Velsky and Landis, 1962) was the first self-balancing binary search tree	NIST Dictionary of Algorithms and Data Structures; Wikipedia
18	Edward Fredkin named the trie in 1960, from the middle of "retrieval"; René de la Briandais described the idea in 1959	Fredkin, "Trie Memory", CACM 3(9), 1960

Chapter	Fact	Source
19	Tarjan (1975) proved union-find with path compression and union by rank runs in nearly constant time per operation (inverse Ackermann)	Tarjan, JACM 22(2) 1975
21	`graphlib.TopologicalSorter` has been in the standard library since Python 3.9	CPython `Doc/library/graphlib.rst`
22	Dijkstra designed his shortest-path algorithm in about twenty minutes in 1956, at a café in Amsterdam with his fiancée, without pencil and paper, for a demonstration of the ARMAC computer on a map of Dutch cities; published 1959	Interview with Dijkstra (Philip Frana), Communications of the ACM 2010
22	OSPF routers (RFC 2328) compute shortest-path trees with Dijkstra's algorithm; RIP (RFC 2453) is a distance-vector protocol from the Bellman-Ford family	RFC 2328, RFC 2453
23	Borůvka (1926, for electrifying Moravia), Jarník (1930), Kruskal (1956) and Prim (1957, telephone networks)	Published histories of the problem, such as Graham and Hell, "On the History of the Minimum Spanning Tree Problem", *Annals of the History of Computing*, 1985
26	TCP flow control is a sliding window: the receiver advertises how many bytes it can still accept, and the sender keeps no more than that many unacknowledged bytes in flight; a zero window pauses the sender	IBM networking documentation; University of Washington CSE 461 lecture notes; RFC 9293 consolidated the TCP specification in 2022

Chapter	Fact	Source
27	Summed-area tables: Frank Crow, "Summed-Area Tables for Texture Mapping" (SIGGRAPH 1984); renamed the "integral image" by Viola and Jones, whose 2001 face detector used it to sum rectangles with four lookups	Wikipedia "Summed-area table"; ImageJ wiki "Integral Image Filters"
29	Richard Bellman chose the name "dynamic programming" to hide mathematical research from Secretary of Defense Charles Wilson	Bellman, *Eye of the Hurricane* (1984), as quoted in several histories of the field
30	Git's default diff algorithm is Myers'; patience and histogram are options	Git `Documentation/diff-algorithm-option.adoc`
30	Myers' diff algorithm, "An O(ND) Difference Algorithm and Its Variations" (Algorithmica, 1986): running time grows with the size D of the difference; Needleman and Wunsch published their sequence-alignment DP in 1970 (*Journal of Molecular Biology*)	Myers, *Algorithmica* 1(2), 1986; Needleman and Wunsch, *Journal of Molecular Biology* 48(3), 1970; Git `Documentation/diff-algorithm-option.adoc` for Git's options
31	Huffman found his coding method in a 1951 term paper for Robert Fano's course at MIT (students could write the paper instead of taking the final exam); published 1952. DEFLATE (zip, gzip, PNG) combines LZ77 with Huffman coding	Huffman's 1952 paper; RFC 1951
31	Python 3.11 added atomic groups and possessive quantifiers to `re`	CPython `Doc/whatsnew/3.11.rst`

Chapter	Fact	Source
32	Redis evicts keys with an approximation of LRU, sampling a few keys (`maxmemory-samples`), because true LRU costs more memory	Redis docs, `content/develop/reference/eviction/index.md`
33	Python 3.11: "between 10-60% faster than Python 3.10", 1.25x on average. Python 3.13: experimental free-threaded build (PEP 703) and experimental JIT (PEP 744), both off by default. Python 3.14: free-threaded build officially supported; new tail-calling interpreter 3-5% faster; JIT in Windows and macOS binaries. Python 3.15 (released 9 October 2026): JIT 7-8% faster than the standard interpreter on x86-64 Linux (geometric mean)	CPython `Doc/whatsnew/3.11.rst`, `3.14.rst`, `3.15.rst`; `3.13.rst`

About the Author

Aadhavan Muthurengan is an automation and artificial intelligence professional with more than 15 years of experience in the automotive industry.

Throughout a career focused on product development and validation, Aadhavan has designed and built a wide range of in-house tools that automate complex engineering workflows, speed up testing, and help teams deliver reliable, high-quality results. Tools like these live or die by their data structures: a log search that has to stay fast as the logs grow, a scheduler that always knows what runs next, a test system that must never give a wrong answer. This book teaches the habits that work demands: build it so you understand it, test it against something you trust, and measure before you believe.

Thank you for reading *Data Structures and Algorithms in Python, Hands-On*. May your code be correct and your algorithms fast.

More Books by the Author

If you enjoyed this book, you may like these, also by Aadhavan Muthurengan. All are available on Amazon in paperback and Kindle editions.



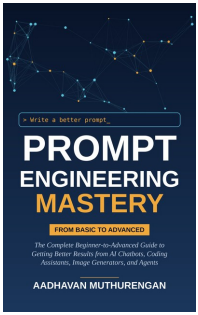
Python Programming Hands-On: Learn Python from Basic to Advanced by Writing Real Programs, with Exercises, Solutions and Every Example Tested. The natural companion to this book. Python from your first line of code to tested, packaged programs, a database app, a web API and automation scripts, with every example run before publication.



Vibe Coding for Beginners: Build Real Apps with Claude Code, from Your First Prompt to Tested, Deployed Software. Build real web and mobile apps by working with an AI coding assistant, from the first prompt to tested, deployed software.



Building AI Agents with Claude: A Beginner's Guide to Agents That Use Tools, Remember, and Get Real Work Done, from Your First Agent to Production. Build AI agents that use tools, remember context and complete real tasks, from a first agent to production.



Prompt Engineering Mastery: The Complete Beginner-to-Advanced Guide to Getting Better Results from AI Chatbots, Coding Assistants, Image Generators, and Agents. Practical techniques for getting reliable, high-quality results from AI tools, from everyday chat to coding assistants and agents.

Python Data Structures and Algorithms: A Comprehensive Guide. A guide to the data structures and algorithms behind efficient Python programs.